

Java 8,11,17, 21,25

(avec JDBC)

Table des matières

I - Présentation du langage JAVA.....	7
1. Historique et évolution.....	7
2. Machine virtuelle JAVA.....	9
3. Principales particularités du langage JAVA.....	11
4. Structuration des API Java (JavaSE / JavaEE).....	13
5. IDE (Environnement de développement intégré).....	15
6. Maven (ou gradle) quasi indispensable.....	15
II - Eléments de base du langage Java.....	17
1. Compilation , exécution et IDE.....	17
2. Types de données.....	19
3. Classes, instances et références.....	23

4. Ramasse miettes (G.C.).....	30
5. Opérateurs du langage Java.....	32
6. Boucles & instructions de Java.....	35
7. Chaînes de caractères.....	36
8. Tableaux.....	39
9. Méthode et variables de classes.....	42
III - Structures fondamentales(collection, generic).....	45
1. Packages et archives (.jar).....	45
2. Collections (depuis le jdk 1.2).....	52
3. Generics (depuis Java 5).....	56
4. Eléments de Java >=5 (jdk 1.5 , 1.6 et 1.7).....	62
IV - Héritage , polymorphisme , interface.....	64
1. Généralisation / Héritage.....	64
2. Polymorphisme.....	69
3. Classes abstraites.....	71
4. Interfaces.....	73
V - Eléments structurants de java.....	76
1. Propriétés du système et arguments du main.....	76
2. Gestion des exceptions.....	77
3. Présentation des API de log.....	81
4. Classes imbriquées (depuis jdk 1.1).....	84
5. Modules java 9+ (essentiel).....	85
VI - Prog. Fonctionnelle (lambda, stream, ...).....	91
1. Java multi-paradigme "objet + fonctionnel".....	91
2. Lambda expressions et prog. fonctionnelle.....	93
3. Streams (java 8 et plus).....	101
VII - Classes utilitaires , aspects divers.....	105
1. Quelques anciennes classes utilitaires.....	105
2. Internationalisation.....	107
3. Mise en forme du texte (java.text).....	108
4. Optional , DateTime ,	109
VIII - Entrées/sorties (io) – fichiers.....	114

1. Essentiel de java.io.....	114
2. Spécificités à connaître et détails intéressants.....	118
3. try with resources (AutoClosable).....	118
4. NIO , NIO2.....	120
XI - Introspection et Sérialisation.....	121
1. Introspection (java.lang.reflect).....	121
2. Sérialisation (et persistance élémentaire).....	125
XII - JDBC (accès aux bases de données).....	128
1. JDBC : Présentation et structure.....	128
2. Paramétrage et établissement d'une connexion.....	129
3. Principaux objets de l'api JDBC.....	131
4. Lancer un ordre sql.....	131
5. Effectuer une requête (select).....	132
6. Balayer les lignes du résultat.....	132
7. Accès à la structure de la base (MetaData).....	133
8. Gestion des transactions (tout ou rien).....	134
9. Préparer et lancer n fois un ordre Sql paramétrable.....	134
10. Appels de procédures stockées.....	134
11. fermer proprement les connexions.....	135
12. Récupérer la valeur d'une clef auto-incrémentée.....	135
13. Fonctionnalités à partir de la version 2 de JDBC.....	136
14. Mémento SQL + Mise en oeuvre MySQL.....	136
XIII - Threads (java).....	138
1. Concept de threads.....	138
2. Gestion des threads avec java.....	139
3. Synchronisation des threads.....	141
XIV - Annexe - Api pour IHM/GUI (swing, ...).....	144
1. Eléments de base sur AWT/SWING et événements.....	144
2. Gestion des événements.....	151
XV - Annexe – Prog Objet et UML (aperçu).....	156
1. Concepts objets fondamentaux.....	156
2. Aperçu sur UML (Unified Modeling Language).....	161

XVI - Annexe – nouveautés v12-v17 et v18-v21.....165

1. Switch expressions.....	165
2. Pattern Matching instanceof.....	167
3. TextBloc.....	168
4. Record (classe de données simplifiée pour DTO).....	170
5. Sealed classes.....	176
6. Pattern matching sur switch/case of object.....	178
7. utilitaire jpackage.....	179
8. Autres apports des versions récentes (12,...,17).....	180
9. Nouveau type de licence depuis java 17.....	180
10. Pattern and record matching.....	181
11. Virtual Thread.....	183
12. Structured Concurrency (preview java 21).....	187
13. Foreign Function and Memory (preview java 21).....	190
14. Vector api (incubator java 21).....	191

XVII - Entrées/sorties (nio,nio2) – Path, Files,193

1. NIO2 (new IO).....	193
2. Principales classes et interfaces de nio2.....	194
3. Gestion des chemins (Path).....	195
4. Classe utilitaire Files (" <i>helper</i> " avec 50 méthodes statiques).....	199
5. Parcours des éléments d'un répertoire.....	201
6. Parcours d'une hiérarchie de répertoires (visiteur).....	203
7. FileSystem (par défaut et "personnalisé").....	203
8. Lecture et écriture dans un fichier.....	204

XVIII - Annexe – Quelques API de java.....206

1. Process Api.....	206
---------------------	-----

XIX - Annexe – Tests unitaires (essentiel JUnit 5).....210

1. Tests unitaires avec JUnit 5.....	210
--------------------------------------	-----

XX - Annexe – Tests avancés (JaCoCo, Mockito,...).....228

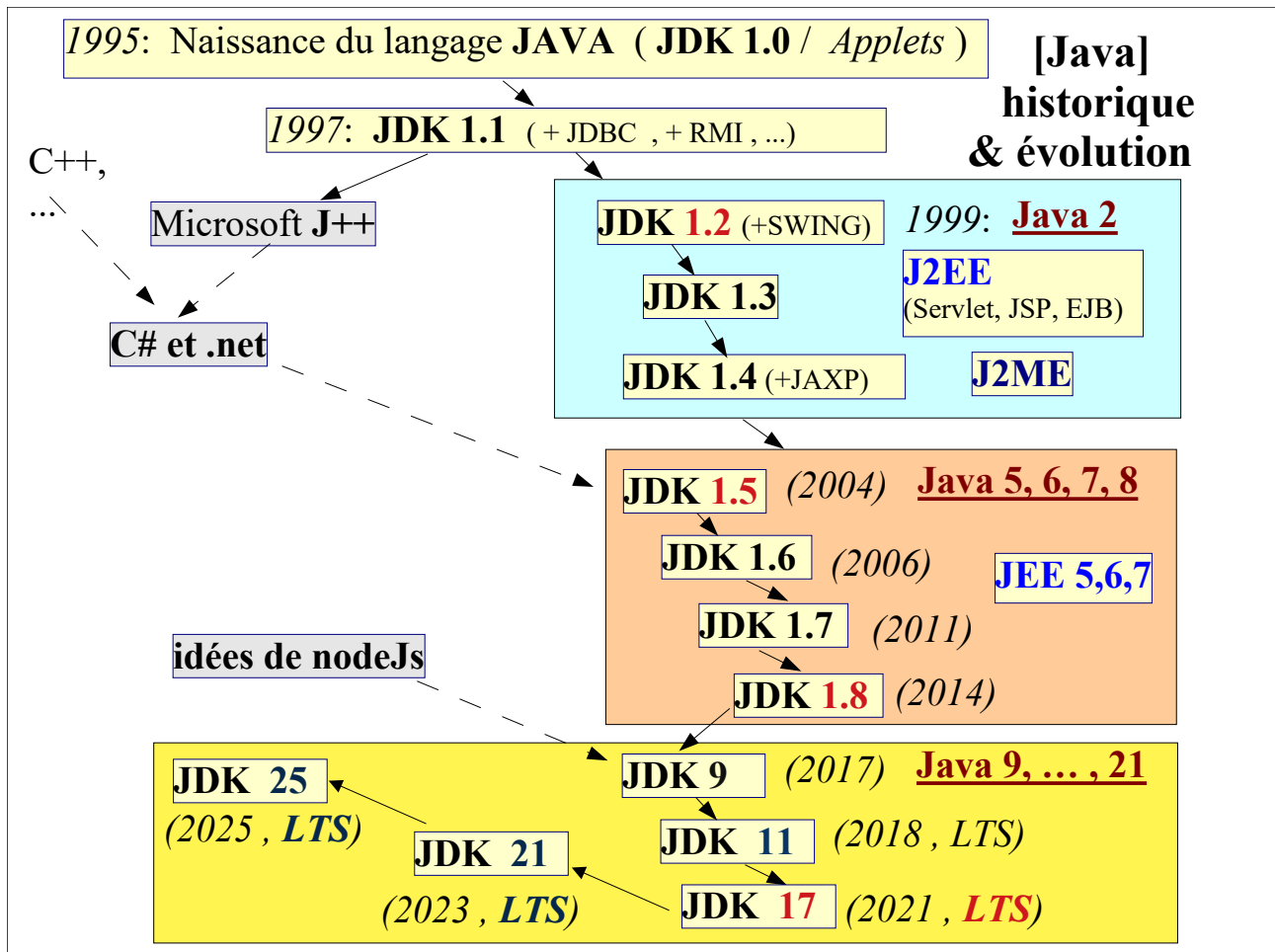
1. Analyse de la couverture des tests.....	228
2. Positionnement et intérêts des "mocks".....	230
3. Mockito.....	231
4. Présentation de JMeter.....	239
5. Configuration et démarrage du produit JMeter.....	241

6. Configuration élémentaire d'un plan de test JMeter.....	242
7. Très important: tester tout d'abord le test !!!.....	246
8. Gestion correcte des sessions HTTP (cookies).....	247
9. Successions de requêtes http liées entre elles.....	247
10. Configurations diverses.....	250
XXI - Annexe – Monitoring JVM (strict essentiel).....	253
1. JConsole.....	253
2. Processus d'optimisation (tuning).....	255
XXII - Annexe – Annotations Java.....	256
1. Annotations : Présentation et intérêts.....	256
2. Annotations et méta-annotations prédéfinies.....	256
3. Création de nouvelles annotations.....	258
4. Insertion d'annotations au sein d'un code source.....	259
5. Accès aux annotations (de rétention RUNTIME) via l'introspection de java >=5.....	259
6. Analyse et traitement des annotations de rétention « SOURCE ».....	261
XXIII - Annexe – Structure globale appli.....	267
1. Architecture généralement recommandée.....	267
XXIV - Annexe – Lombok (très utile).....	268
XXV - Annexe – énoncés des TP.....	269
1. TP (prise en main du jdk).....	269
2. TP (première classe simple, conventions JavaBean).....	269
3. TP (Tableaux, String, ...).....	270
4. TP static – constante ,	271
5. TP (Collections & Generics).....	271
6. TP POO (Héritage, Polymorphisme, interface).....	272
7. TP - Premier traitement d'exception.....	278
8. TP - Second traitement d'exception.....	279
9. TP (logs , test unitaires) à éventuellement différer.....	280
10. TP tris et lambda, stream.....	280
11. TP facultatif (LocalDate & LocalTime).....	282
12. TP (Gestion des fichiers) :.....	282
13. TP (Accès aux bases de données via JDBC):.....	283

14. TP (Gestion élémentaire des threads) :.....	284
15. TP (syntaxes java récentes):.....	285
16. TP (debug et monitoring de JVM):.....	285

I - Présentation du langage JAVA

1. Historique et évolution



JDK signifie *Java Development Kit*.

Les JDK 1.2 et 1.5 ont apportées de grandes nouveautés qui ont modifié le langage en profondeur.

Le terme plate-forme **Java 2** désigne toutes les versions de *Java* à partir du *JDK 1.2* et englobe également une extension pour les serveurs d'applications : *J2EE* (*Java 2 Enterprise Edition*).

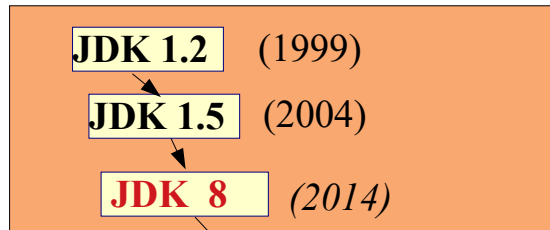
Bien que différents et incompatibles les langages Java et C# comportent beaucoup de points communs (syntaxe et architecture assez proches).

La société **SUN MicroSystem** qui a inventé le langage **JAVA** est le propriétaire officiel du langage et décide de son évolution (en tenant compte des avis de ses partenaires).

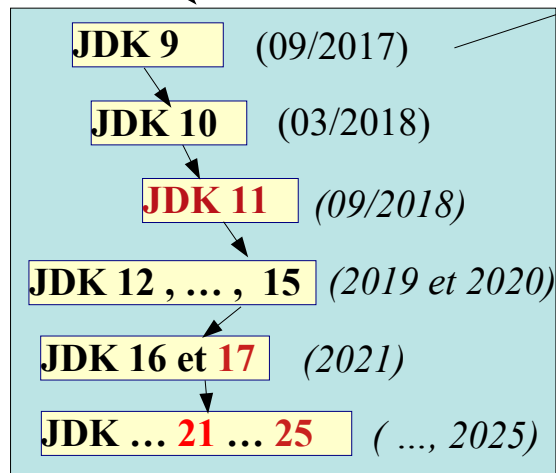
L'entreprise "SUN" a été rachetée par "Oracle".

Oracle est maintenant le **nouveau propriétaire** de "**Java**".

La version **8** du **jdk** a apporté quelques grandes nouveautés syntaxiques (**lambda expressions**, **streams**, ...) et propose **java-fx** à la place de **swing**.



Evolution récente du langage java



Depuis V9 (2017) , **une nouvelle version tous les 6 mois** avec quelquefois très peu de nouveautés !

NB: Les versions **11, 17 , 21** et **25** (stables) sont **LTS** (Long Time Support) et donc conseillées .

Attention : le mode "**--enable-preview**" est **éphémère**

(remis en cause tous les 6 mois et exploitable qu'avec une version bien trop précise du JDK et/ou d'un IDE (eclipse ou autre)).

La version 21 (de fin 2023) a apporté la nouveauté "Threads virtuels"

La version 25 (64bits seulement, de fin 2025) a apporté que de toutes petites améliorations .

NB : à partir du jdk 1.11 , java-fx est devenu une extension facultative (désormais plus incluse dans le jdk) .

NB : a coté du jdk officiel (de l'entreprise Oracle) , il existe également le "**openJdk**" entièrement "open source" (un tout petit peu moins complet que le jdk d'Oracle mais sans partie potentiellement payante en production) .

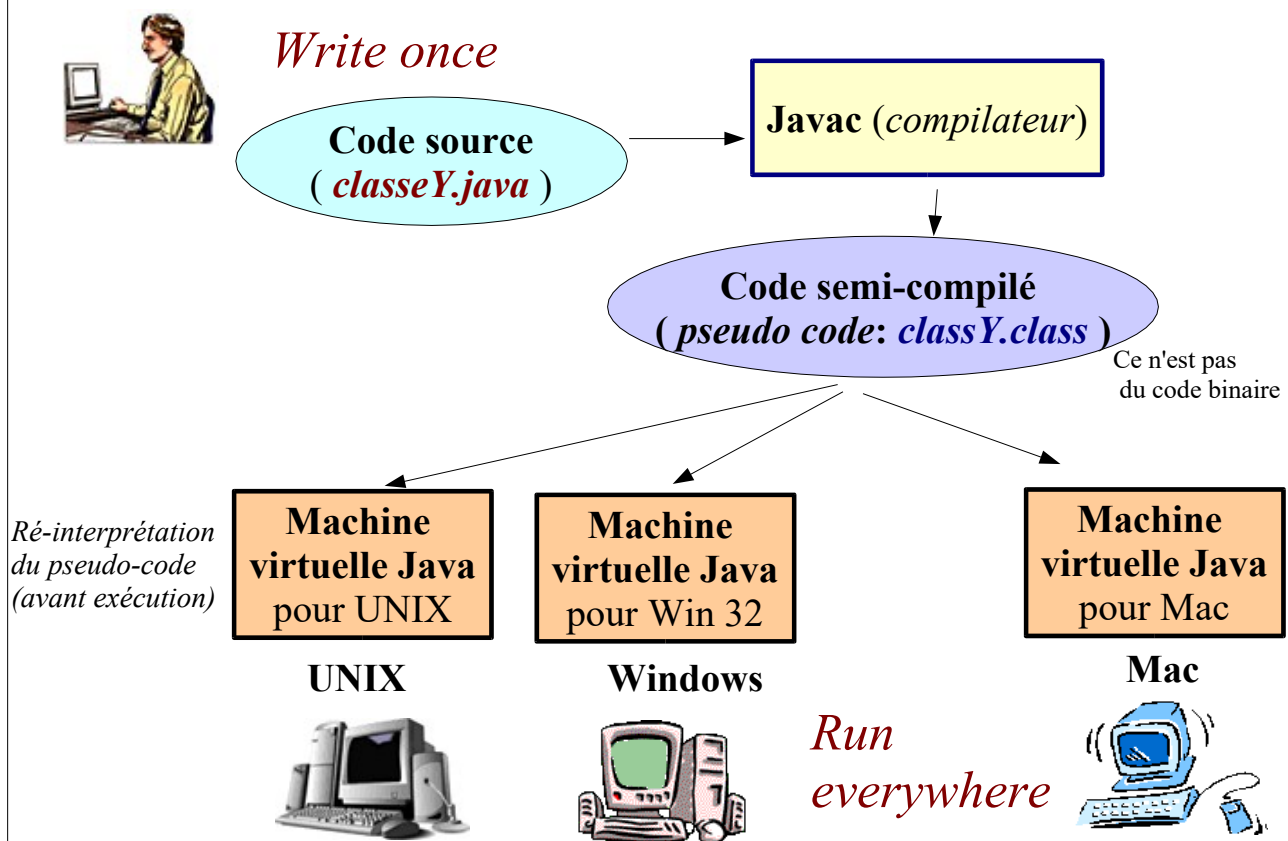
2. Machine virtuelle JAVA

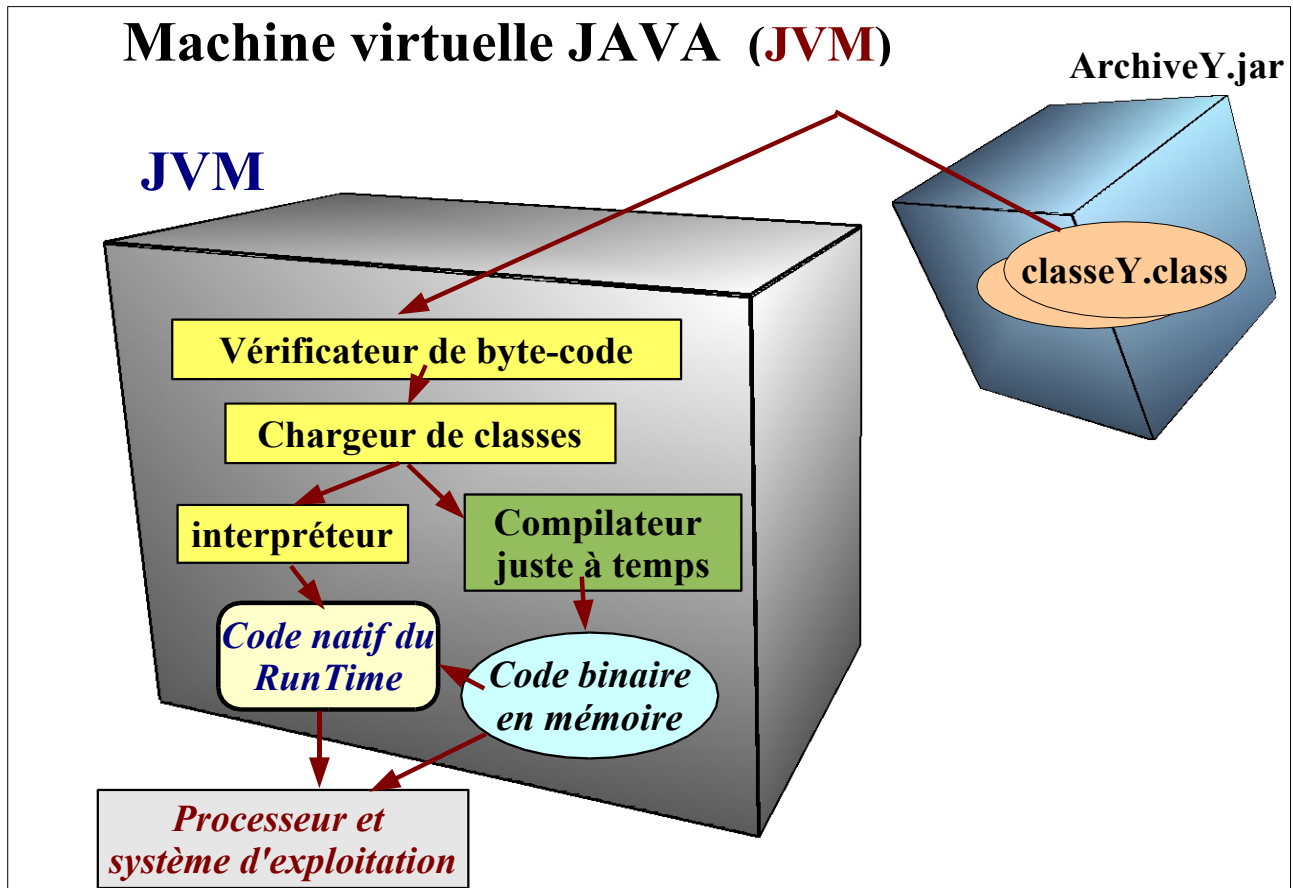
La principale particularité du langage *JAVA* est d'être **multi-plateforme**:

- 1) Le **développeur écrit du code source** (*classeY.java*) **puis le compile sur n'importe quelle sorte de machine** (Windows , Unix , Linux ,).
- 2) Cette **compilation partielle** ne génère pas un code binaire compréhensible que par un certain type de machine mais génère un **pseudo-code portable** appelé "*byte-code*" : *classeY.class* .
- 3) Ce pseudo-code portable est ensuite packagé dans des archives (.jar) puis distribué à travers le réseau vers différentes sortes de machines.
- 4) N'étant pas dédié à un type précis de machine, **ce pseudo code portable a besoin d'être ré-interprété** par une application spécifique appelée "*machine virtuelle java*" .

A chaque type de plate-forme, correspond une version spécifique de la machine virtuelle java (généralement packagée dans le *JRE* = *Java Runtime Environnement*) .

Java: langage *partiellement compilé* et *interprété* (via JVM)



**NB:**

- Il n'y a pas d'édition de liens à effectuer préalablement (les classes sont chargées au fur et à mesure des besoins du programme lancé au sein de la machine virtuelle) .
- Le **compilateur juste à temps** permet d'**améliorer sensiblement la vitesse d'exécution** d'une application java et est systématiquement activé (sauf option contraire).

Principales contraintes liées aux mécanismes de JAVA:

- L'initialisation de la machine virtuelle, le chargement des classes en mémoire ainsi que la compilation juste à temps sont des opérations assez lourdes qui occasionnent des **temps de démarrage relativement longs**.
- L'ensemble des constituants d'une application Java chargée au sein d'une machine virtuelle **occupe une assez grande place en mémoire vive**.

==>

Ceci explique que **Java est beaucoup utilisé coté serveur** (là où une grande consommation mémoire et un temps élevé de démarrage sont moins gênants) .

L'utilisation de java coté client (*ex*: applet , interfaces graphiques Swing, java web start, ...) nécessite des ordinateurs relativement puissants (rapides et bien dotés en mémoire vive).

3. Principales particularités du langage JAVA

Langage générique <i>[large palette d'applications]</i>	<u>Le langage Java peut servir à créer différentes sortes d'entités:</u> ♦ Applications autonomes (ne nécessitant qu'une JVM) ♦ JavaBean = composants quelconques (pour composer une interface graphique ou pour effectuer des traitements "métier"). ♦ Servlet = composant permettant de générer des pages WEB (<i>Un servlet s'exécute coté serveur</i>) ♦ Applet (mini-application s'exécutant coté client, l'affichage s'effectue dans une sous fenêtre du navigateur internet) ♦ Application native android , ...
Complètement orienté objet	Java est un langage résolument orienté objet . Les fonctions globales n'existent pas en java , <i>toutes les fonctions sont obligatoirement intégrées dans des objets</i> . Offrant un support à tous les principaux concepts objets (classe, instance, encapsulation, polymorphisme , héritage , ...) , Java permet de <u>très bien structurer les programmes</u> (==> bonne modularité).
Relativement simple	La gestion des références (qui sont plus simples à manipuler que les pointeurs) est en grande partie automatisée: <i>un ramasse miette libère automatiquement les blocs mémoires devenus inutiles</i>. Bien que <i>syntactiquement proche du C++</i> , java ne s'est inspiré que des éléments fondamentaux et simples de ce langage .
Souple, expressif et bien adapté au couches hautes	Souple , modulaire et étant doté de beaucoup d' API prédéfinies , Java est un langage de prédilection pour les développements "3-tiers" liés à l'informatique de gestion et à internet.

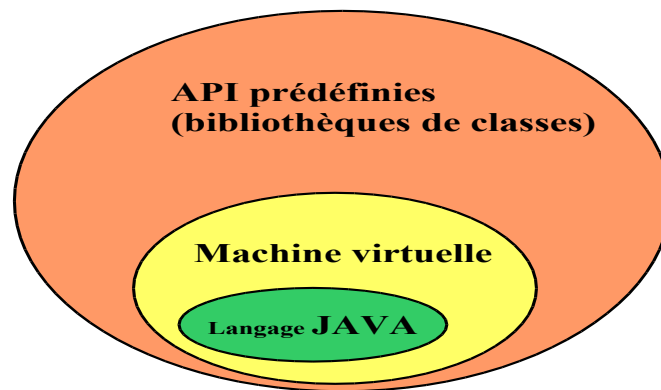
Vitesse d'exécution correcte et interfaçage possible avec le langage C	Par contre, JAVA n'est (pour l'instant) pas du tout approprié pour coder des couches de bas niveau: Les langages C et C++ restent incontournables pour coder des traitements pointus devant s'exécuter très rapidement sur une plate-forme spécifique (ex: informatique industrielle, calcul scientifique, ..) Java peut s'interfacer avec du code C ou C++ via JNI (<i>Java Native Interface</i>) et des DLL locales ou via le protocole SOAP des services WEB.
Robuste et fiable	Langage fortement typé ==> <u>beaucoup d'erreurs détectées dès la compilation.</u> Gestion des exceptions bien structurée (<i>try/catch + pile d'appels ==> "Debug" simple et rapide</i>).
Introspection	Certains <u>mécanismes prédéfinis du langage JAVA permettent de récupérer automatiquement une description d'une classe Java</u> (<i>liste des attributs et des fonctions internes, types des paramètres, ...</i>) <u>même si celle-ci n'est disponible que sous la forme compilée</u> (<i>sans code source</i>) . Ceci constitue un gros point fort (vis à vis du C++) et <u>permet d'automatiser certains traitements</u> (persistance des données, proxy dynamique, ...)
Prise en charge (en standard) du multi-threading	==> Java permet d'écrire du <u>code ré-entrant</u> que l'on peut écrire de façon <u>portable</u> (sans être dépendant d'un système d'exploitation).

4. Structuration des API Java (JavaSE / JavaEE)

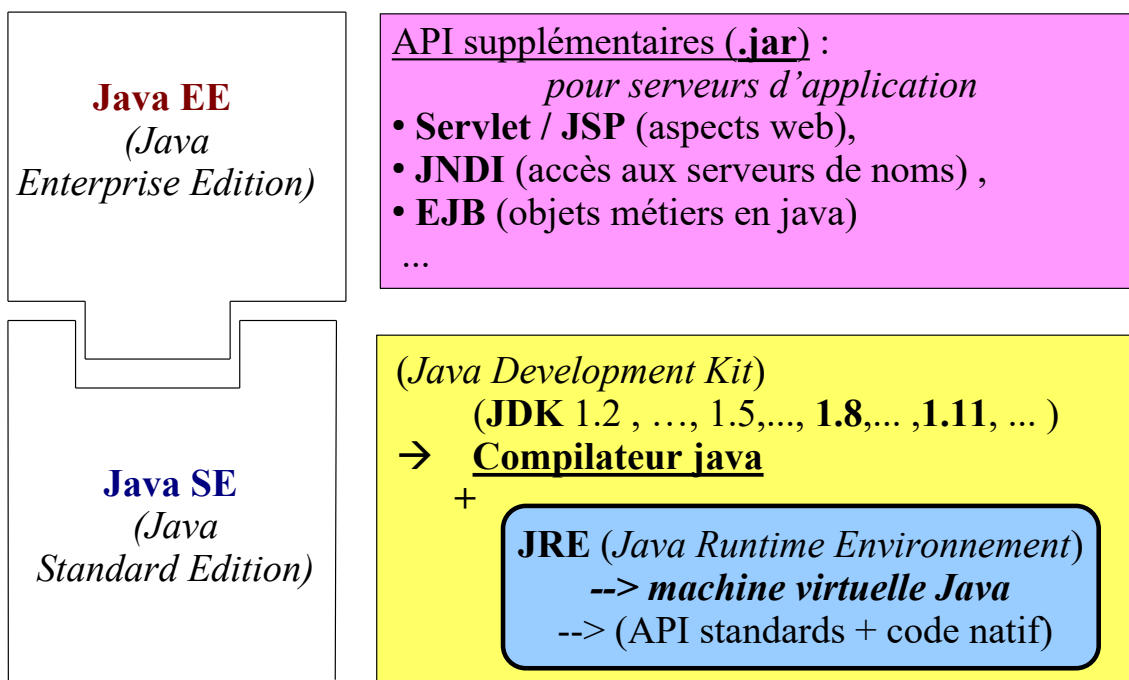
Le langage **JAVA** est associé à une machine virtuelle qui le rend portable sur une large palette de plate-formes (UNIX, Windows ,) et est accompagné d'un immense ensemble d'API (bibliothèques de classes) qui sont:

- **standards** (officialisées par SUN/Oracle , JEE , ...)
- **portables** (utilisables sur une multitude de systèmes [Linux , Windows, ...])
- **très souvent gratuites** (Open source ou ...)

Java est donc bien plus qu'un langage informatique , c'est une véritable plate-forme virtuelle:



(plate forme Java) JavaSE / JavaEE et JavaME



NB: il existe aussi **JavaME (Micro Edition)** = **JavaSE simplifié/allégé** pour les **cartes à puce** (ex : carte bancaire) .

4.1. Principales API de JAVA

Api	intégration	fonctionnalités
JDBC	API intégré dans J2SE mais <i>Driver JDBC à récupérer</i>	Accès générique aux bases de données relationnelles (==> requêtes SQL)
AWT	J2SE (depuis JDK 1.0)	Bases du graphisme et du multi-fenêtrage
SWING	J2SE (depuis JDK 1.2)	Graphisme 2D et contrôles graphiques 100% java
javaFx	J8SE (depuis JDK 1.8)	Api graphique plus moderne
SERVLET / JSP	J2EE	génération de pages HTML (nécessite un conteneur Web du type Tomcat)
RMI (<i>Remote Method Invocation</i>)	J2SE (depuis JDK 1.1)	Appels de fonctions à travers le réseau
EJB (Enterprise Java Bean)	J2EE	Objets "métier" en java (nécessite serveur JEE ex: WebSphere_AS, JBoss_AS,)
JPA 1 et 2	J5EE	Java Persistance Api (ORM / hibernate)
DI , CDI	J6EE	Injection de dépendances
JNDI	J2SE (depuis JDK)	Accès à des serveurs de noms (LDAP, ...)
JMS	J2EE	Interface java pour MiddleWare orienté message asynchrone.
JAXP (Java Api for Xml Processing)	J2SE (depuis JDK 1.4)	Parsing XML (SAX & DOM) + transformations XSLT
JAX-WS, JAX-RS		api officielles pour Web-services (SOAP et REST)
...		

NB : à coté des spécifications officielles "JavaEE" , il existe un très bon framework "**Spring**" (standard pas officiel mais standard de fait --> écosystème java très complet) .

5. IDE (Environnement de développement intégré)

Le **JDK (JRE + compilateur en mode texte)** ne suffit pas pour programmer de façon confortable.

Un *environnement de développement graphique* intégrant au minimum un *éditeur* et une *gestion automatisée des compilations* permet d'être efficace durant les phases de programmation .

Eclipse (*Open Source*) , et **IntelliJ** (*de JetBrains*) sont actuellement les deux **IDE Java** qui sont les plus connus et utilisés .

- **Eclipse** est complètement open source mais est quelquefois lent si trop de plugins.
- **IntelliJ** existe en 2 versions : "**community**" (gratuite) et "**ultimate**" (payante)

NetBeans (*de Sun/Oracle*) existe également mais est un peu moins utilisé.

NB : L'IDE "**Visual Studio code**" (à la base prévu pour les langages javascript et typescript) , peut également assez bien prendre en charge un projet java via une **extension (plugin)** pour "**java**" .

6. Maven (ou gradle) quasi indispensable

Dans le cadre d'un projet d'entreprise sérieux, **un outil de gestion de version** (CVS,**SVN** ou **GIT**) permet de stocker/centraliser le code de tous les développeurs dans un **référentiel partagé en commun** .

En outre le produit "**Maven**" est très souvent utilisé pour **gérer les librairies java (".jar")** et leurs inter-dépendances .

NB : en structurant un projet java sur la technologie "maven" (et en plaçant le code source dans un référentiel GIT) , on fait un peu comme tout le monde mais surtout , un tel projet peut être partagé (dans le cadre d'un travail d'équipe) par plusieurs développeurs qui peuvent éventuellement utiliser des IDE différents.

Autrement dit, un projet java au format "maven" (avec l'encodage "UTF-8" pour les caractères accentués) est utilisable partout (windows, linux, mac, ...) et avec un IDE quelconque (Eclipse, IntelliJ ,) .

Création d'un nouveau projet maven sous **eclipse :**

File/new/project.../maven/maven project...

choisir "**Create a simple project (skip archetype selection)**"

group id : *tp* ou *com.xyz* **artifactId** : *tpJava* ou autre

Création d'un nouveau projet maven sous *intelliJ*:

File/new/project... / Maven , Next (ne pas choisir d'archetype) , **renseigner le nom du projet** (et éventuellement le groupId)

pom.xml

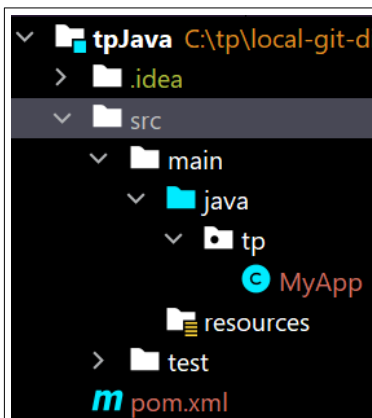
```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>tp</groupId>
  <artifactId>basesJava</artifactId>
  <version>0.0.1-SNAPSHOT</version>

  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <maven.compiler.release>17</maven.compiler.release>
    <maven.compiler.target>17</maven.compiler.target>
    <maven.compiler.source>17</maven.compiler.source>
  </properties>

  <build>
    <finalName>${project.artifactId}</finalName>
  </build>
</project>
```

NB :

- *src/main/java* est un *répertoire d'organisation du code* quasi imposé par la technologie *maven*
src/main/resources sert à placer d'éventuels **fichiers de configuration** (*.properties*, ...)
src/test/java sert à placer les tests unitaires (selon JUnit).
- Au sein de *src/main/java* , il faut créer un ou plusieurs **packages** (ex : *com.xyz* ou *tp*)
- Au sein du package *tp* (dans *src/main/java/tp*) , on peut créer une classe java principale (ex : *MyApp.java* avec la méthode *main()*)

***ProjetXXX***

```
src/main/java
└─ package1 ── ClasseX.java

bin (ou target/classes)
└─ package1 ── ClasseX.class
```

II - Eléments de base du langage Java

1. Compilation , exécution et IDE

1.1. Structuration élémentaire d'un programme java

Un programme Java doit au minimum comporter une classe représentant le point de démarrage de l'application et intégrant la méthode **main()** :

tp/MyApp.java

```
package tp;

public class MyApp {

    public static void main(String args[]) // fonction principale d'une application Java
    {
        System.out.println("Hello World\n");
        // Fin du main
    } // Fin de la classe
```

NB: Le langage Java impose les choses suivantes:

- Le **nom d'une classe** publique doit **obligatoirement correspondre au nom du fichier**.
- Le **nom du package** (contenant la classe) **doit être le même que le nom du répertoire** comportant le fichier lié à la classe.

Conventions importantes:

- Nom de package (répertoire) entièrement en minuscules (*ex: tp*)
- **Nom de classe commençant toujours par une majuscule** (*ex: MyApp*)
- Nom de méthode (fonction) commençant toujours par une minuscules (*ex: println()*)
- Nom de constante entièrement en majuscules (*ex: Math.PI*)

Remarque: on peut utiliser un éditeur de texte quelconque (vi , nano, notepad++ , ...)

Exécution directe du code sous eclipse : click droit puis "**run as .../ java application**"

Exécution directe du code sous intelliJ: "**Run**" (triangle vert)

1.2. Compilation et exécution avec le jdk

Après s'être placé à la racine d'un projet (potentiellement basé sur maven),...

lancerCompilationAvecJdk.bat

```
set JAVA_HOME=C:\Program Files\Java\jdk-17
REM set JAVA_HOME=C:\Program Files\eclipse\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_17.0.2.v20220201-1208\jre
set PATH=%JAVA_HOME%\bin
cd "%~dp0"
cd src/main/java
javac -d ../..../bin tp/MyApp.java
pause
```

l'option **-d** permet de choisir le répertoire où sera placé le code compilé (ex : "bin" ou autre)

on démarre ensuite l'interprétation de cette application par la machine virtuelle **java** :

lancerAppliAvecJdk.bat

```
set JAVA_HOME=C:\Program Files\Java\jdk-17
REM set JAVA_HOME=C:\Program Files\eclipse\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_17.0.2.v20220201-1208\jre
set PATH=%JAVA_HOME%\bin
cd "%~dp0"
cd bin
java tp.MyApp
pause
```

tp.MyApp correspond ici au nom complet de la classe java qui comporte la méthode *main()* .

1.3. Compilation (build) via maven

Avec maven installé (en ligne de commande) :

mvn compile ou bien **mvn install**
ou bien **mvn clean package**

→ le code construit est placé dans le répertoire **target** (et sous répertoire **classes**)

Déclenchement d'un build maven sous IntelliJ :

développer le panneau "**maven**" (coté droit) , développer "**Lifecycle**" , choisir "**package**" et "**Run**"

Déclenchement d'un build maven sous eclipse :

se placer sur le projet (project explorer) , click droit puis "**Run as**" / "**maven install**" ou "**maven build...**" puis goals : "**clean package**"

2. Types de données

2.1. Vue d'ensemble sur les différents types de données de Java

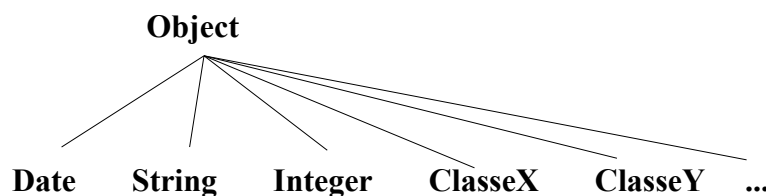
- types élémentaires (non orientés objet et entièrement orthographiés en minuscule) :

int , short, long , boolean, byte , char, float , double

- types "objet" (classes prédéfinies ou pas) :

String , ArrayList, Date et tous les autres types de données possibles.

Remarque: tous les classes de java dérivent du type générique **Object** :



Un "*ArrayList*" est une *Collection* particulière de références sur des "*Object*" quelconques.

2.2. Types élémentaires (int, double , ...)

Les types de données élémentaires de JAVA sont toujours manipulés par valeur.

Type	Contenu	Valeur défaut	Taille (bits)	Valeur min. Valeur max
boolean	true or false	false	1 (*)	(*) 1 bit virtuel , 1 octet sous utilisé
char	caractère Unicode	\u0000	16	\u0000 à \uFFFF
byte	entier signé	0	8	-128 à 127
short	entier signé	0	16	-32 768 à 32 767
int	entier signé	0	32	-2 147 483 648 à 2 147 483 647
long	entier signé	0	64	-9 223 372 036 854 775 808 à 9 223 372 036 854 775 807
float	flottant IEEE 754	0.0	32	1.40239846 ^E -45 à 3.40282347 ^E +38
double	flottant IEEE 754	0.0	64	4.94...E-324 à 1.797...E+308

Remarque:

- Les **types élémentaires** sont directement (ou quasi-directement) mis en relation avec des types de données nativement gérés par le micro processeur . ils occupent peu de place en mémoire (comparés à des types "objet") et sont gérés de façon efficace ==> **bonnes performances**.
- Dans le cas (assez rare) , où certains calculs dépasseraient la valeur limite **Long.MAX_VALUE** ($2^{63}-1$) , on pourrait utiliser la classe *java.math.BigInteger* (sans limitation mais à utiliser via des méthodes spéciales *.add()*, *.multiply()* , ...). Il existe également *java.math.BigDecimal* lorsque double ne serait pas assez précis.

NB:

- JAVA ne considère pas que **false** est équivalent à 0 .
- Le mot clef *unsigned* n'existe pas en Java (contrairement au C/C++)
- 'A' ou '\u0041' est une valeur littérale de type **char**
- 3.14159f est une valeur littérale de type **float**
- 3.14159 ou 3.14159d est une valeur littérale de type **double**
- 0xa25c est une valeur littérale de type **int** exprimée en **hexadécimal**

Les types **float** et **double** peuvent acquérir des valeurs spéciales définies dans *java.lang.Float* et *java.lang.Double*. Ces valeurs spéciales sont des constantes (POSITIVE_INFINITY, NEGATIVE_INFINITY, NaN signifiant *Not A Number* ,...) qui sont généralement le résultat d'une opération en virgule flottante.

L'arithmétique sur les réels en virgules flottantes ne produit jamais d'exception, même dans le cas d'une division par 0.0

Conversions entre types:

int i=4 ,j , k;

double x=3.5 , y, z ;

boolean b=true , bb;

String ch;

ch=**String.valueOf**(i); j=**Integer.parseInt**(ch); System.out.println("j="+j);

ch=**String.valueOf**(x); y=**Double.parseDouble**(ch); System.out.println("y="+y);

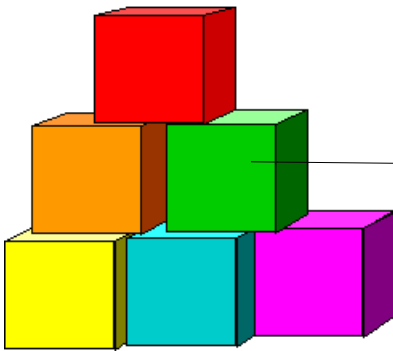
ch=**String.valueOf**(b); bb=**Boolean.valueOf**(ch).**booleanValue**();

k = (**int**) x; // i va récupérer la partie entière de x

i=2; j=3; k=i/j; // k=0 [division entre nombres entiers]

z = ((**double**) i) / ((**double**) j); // z =0.6666666666

2.3. Types "Objets" et notion de référence



Un **objet** est une *brique de base* dans un programme.

Un **objet** est une entité qui rend un certain service (ex: mémorisation, affichage de données , ...).

Un objet est une entité qui regroupe:

- ♦ des données internes appelées **attributs** (et parfois propriétés)
- ♦ des traitements (fonctions) internes appelées **méthodes**

Tous les éléments du langage JAVA qui ne sont pas de types primitifs sont des objets manipulés par référence. C'est le cas des tableaux et des chaînes de caractères.

Une *référence* est un *pointeur caché* sur un objet créé dynamiquement en mémoire.

```
ClasseY ref = null; // déclaration d'une variable ref qui pourra référencer un
                  // futur objet de type ClasseY
```

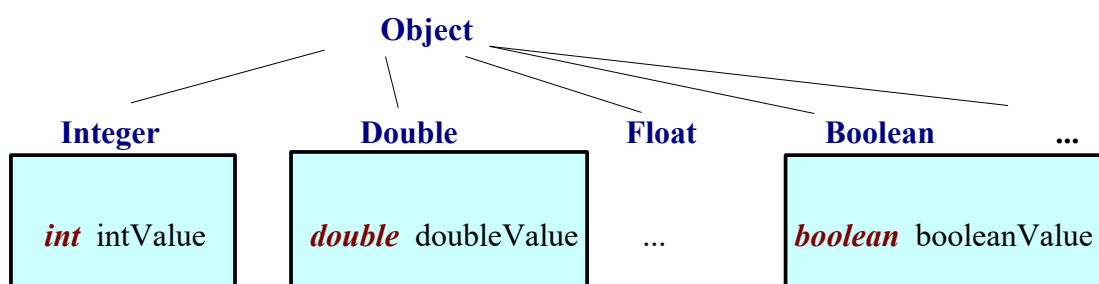
```
ref = new ClasseY(); // création d'un nouvel objet (instance) de type ClasseY .
```

2.4. Wrapper class

Il est quelquefois pratique de pouvoir manipuler un simple nombre entier comme un objet .

Ceci permet par exemple d'insérer une valeur (au départ "non objet") dans une collection d'objets.

A cet effet , le langage Java comporte des classes dites "*Wrapper*" qui correspondent à des enveloppes "*Objet*" permettant d'incorporer des valeurs élémentaires (*de types primitifs*).



Exemple: pour placer un entier nombre de valeur 5 dans une Collection d'objet , il faut créer un objet de la classe **Integer** (commençant par un **I** majuscule) qui va lui même incorporer la valeur 5 :

```
java.util.List<Integer> liste = new java.util.ArrayList<>();  
liste.add (new Integer(5));
```

Inversement pour récupérer la valeur du premier élément de la liste , il faut extraire la valeur primitive de son enveloppe objet via la méthode prédéfinie *intValue()* :

```
Integer objVal = (Integer) liste.elementAt(0);  
int a = objVal.intValue() ;
```

2.5. Boxing / unboxing (depuis Java 5)

Depuis le JDK 1.5 , les conversions entre les types primitifs (int, double, ...) et les types "Wrapper" (enrobages "objet") correspondants (Integer, Double , ...) sont devenus implicites et automatiques:

```
java.util.List<Integer> liste = new java.util.ArrayList<>();  
liste.add (5); // BOXING (incorporation automatique : new Integer(5))  
Integer objVal = (Integer) liste.elementAt(0);  
int a = objVal ; // UNBOXING (extraction automatique via intValue() )
```

2.6. Caractéristiques des classes enveloppes ("wrapper class") :

- Les instances des classes **Integer, Long, Float, Double, Boolean** (et également **String**) sont **immuables (immutable)**. Cela signifie que pour changer la valeur, on ne peut pas modifier l'instance courante et qu'il faut reconstruire un nouvel objet qui remplacera l'ancien .

```
Integer objI = new Integer(5) ;  
objI.setValue(6) ; //n'existe pas !  
objI = new Integer(6) ; //nouvel objet (avec nouvelle valeur) remplaçant l'ancien.
```

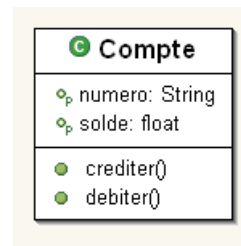
- Une variable de type int, double, ... ne peut jamais avoir la valeur null (0 mais pas null)
- Une variable de type Integer, Double , ... peut avoir la valeur null** (quelquefois par défaut si attribut d'une classe).
La valeur **null** signifie quelquefois "*pas saisie*" dans interface graphique .
La valeur **null** signifie quelquefois "*inconnue*" dans colonne d'une table (null au sens SQL) .

3. Classes, instances et références

3.1. Classe d'objet (concepts & syntaxe)

Une **classe** correspond plus ou moins à un **type d'objet**. C'est une **entité qui décrit une structure de données et une liste de méthodes (fonctions internes) opérant sur ces mêmes données**:

Exemple : représentation UML d'une classe Compte:



Code Java associé (fichier Compte.java):

```

package ex;

public class Compte {

    // Attributs (données internes):
    public String numero;
    public float solde;

    // Méthodes (fonctions internes):
    public void debiter(float montant) { solde = solde - montant; }
    public void crediter(float montant) { solde = solde + montant; }
}
  
```

3.2. Instances manipulées via des références

Pour utiliser une classe, il faut (à l'extérieur de la définition de celle-ci) :

- Déclarer une variable de type "référence sur un objet de la classe considérée"
- créer un objet ou une instance (exemplaire) de la classe.

```

Compte c1 = null; // c1 est ici une référence sur un objet de type Compte
...                // qui n'existe pas encore.
c1 = new Compte(); // c1 référence maintenant un nouvel exemplaire de la classe Compte.
  
```

Remarque importante:

En JAVA, **tous les objets (instances d'une classe)** sont toujours **manipulés par référence** (pointeur caché) et **alloués dynamiquement en mémoire** (directement via le mot clef **new** ou indirectement via d'autres mécanismes).

Accès aux attributs et méthodes publiques:

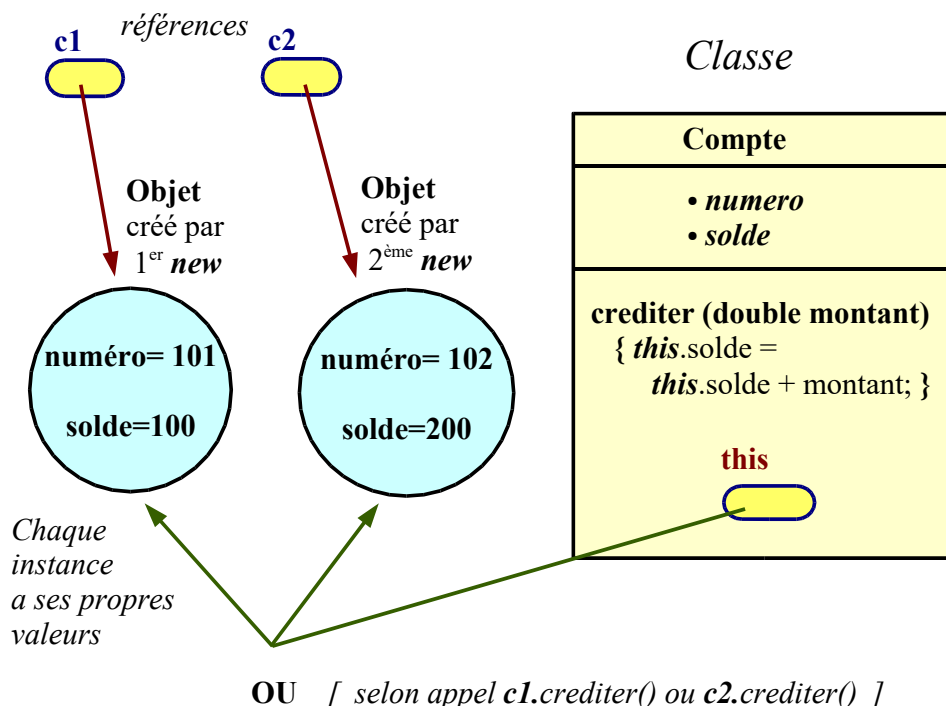
```
public class MyApp {  
  
    public static void main(String[] args) {  
        Compte c1, c2;  
        c1=new Compte();  
        c1.numero = "101"; c1.solde = 100;  
        c2 = new Compte();  
        c2.numero = "102"; c2.solde = 200;  
        c1.crediter(10); c2.debiter(10);  
        System.out.println("Le compte num = " + c1.numero + " a un solde de " + c1.solde );  
        System.out.println("Le compte num = " + c2.numero + " a un solde de " + c2.solde );  
    }  
}
```

NB:

L'écriture ***c1.crediter(10)*** revient à appeler la méthode "*crediter*" depuis l'objet référencé par ***c1***. La méthode en question est alors invoquée avec une référence interne implicite dénommée ***this***. ***this*** référence systématiquement l'objet (courant) à partir duquel la méthode a été appelée.

Le code interne de la méthode *crediter* est implicitement converti en

```
{ this.solde = this.solde + montant ; }
```



Au moment où **c1.crediter(10)** est appelé la référence spéciale **this** est automatiquement initialisée en y recopiant la valeur (adresse mémoire) de c1. Donc this et c1 pointent vers le même objet. Lorsque le code de la méthode (fonction) *créditer* s'exécute, il manipule les valeurs internes (numéro, solde) de l'objet couramment référencé par this.

Repère Syntaxique:

Appel classique de fonction en langage "C" (non orienté objet):

```
res = fonctionDeTraitement ( structureDeDonnées);
```

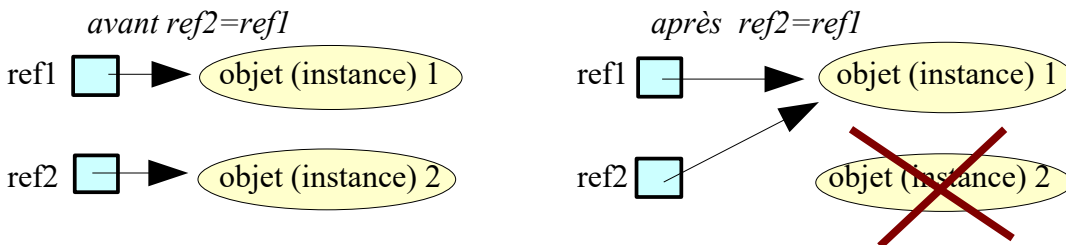
Appel classique de méthode (fonction interne) sur un objet Java:

```
res = ObjetAvecSesPropresDonnées.méthodeDeTraitement();
```

3.3. Copies de références et tests d'égalités

Soient 2 références **ref1** et **ref2** pointant initialement sur 2 objets différents (*instances construites via new*). Alors l'affectation **ref2=ref1** va faire en sorte que:

- les 2 références ref1 et ref2 vont pointées sur le même objet
- l'ancien objet 2 (jusqu'à référencé par ref2) ne sera plus référencé et sera automatiquement détruit par le ramasse-miettes.



```
if( ref1 == ref2 )
```

//teste simplement le fait que ref1 et ref2 référencent bien le même objet.

Pour tester si deux objets distincts ont des valeurs internes identiques, il faut utiliser la méthode spéciale **equals** (déclarée au niveau de **Object** et recodée dans les sous classes) :

```
if( obj1.equals(obj2) ) ...
```

```
if( chaine1.equals("abc") )...
```

La méthode *clone()* lorsqu'elle existe, permet de déclencher une copie en profondeur:

```
ref3 = ref.clone(); // ref3 référence une nouvelle instance, résultant du clonage
```

3.4. Constructeurs

*Lorsqu'un nouvel objet est créé, ses valeurs internes peuvent alors être automatiquement initialisées au moyen d'une fonction particulière appelée **constructeur**.*

Le **constructeur** d'une classe est une **méthode très spéciale** qui doit absolument porter le **même nom de la classe** et qui n'a **pas de type de retour** (*pas de void*).

Exemple:

```
public class Compte {
...
public Compte (String num, double solde_initial)
    { this.numero = num; this.solde = solde_initial ; }
...}
```

Utilisation du constructeur:

```
Compte c = new Compte("103",150.0); // nouveau compte de numéro "103" et de solde 150.0
```

NB: Une classe peut comporter **plusieurs versions** de son constructeur (**fonction surchargée**):

```
public Compte (String num, double s) { this.numero = num; this.solde = s; }
public Compte() { this.numero = "0"; this.solde = 0; }
public Compte( Compte c) { this.numero = c.numero; this.solde = c.solde; }
```

La version à utiliser sera choisie en fonction des arguments présents sur la ligne de code effectuant l'appel:

```
c1 = new Compte();
c2 = new Compte("104",120);          c3 = new Compte(c2);
```

NB: Un constructeur peut en interne s'appuyer sur une des autres versions pour développer son code. On utilise pour cela une *syntaxe* faisant intervenir le mot clé **this** avec des parenthèses:

```
public Compte (String num, double s) { this.numero = num; this.solde = s; }
public Compte() { this("0",0.0); }
public Compte( Compte c) { this(c.numero, c.solde); }
```

Remarque importante:

- ♦ Si aucun constructeur n'a été programmé au niveau d'une classe, le langage Java en fabrique un automatiquement sans argument: *refObjX = new ClasseX();*
- ♦ Si les seuls constructeurs qui ont été programmés au niveau d'une certaine classe ont tous au moins un paramètre alors on pourra créer une nouvelle instance via *refObjX = new ClasseX(valParam1, ...);* mais on ne pourra plus écrire *refObjX = new ClasseX();*

==> **Il est donc très fortement conseillé de programmer soi même** (avant de l'oublier) **le constructeur par défaut (sans argument).**

3.5. Accesseurs (get/set)

La plupart des classes *java* doivent être programmées dans les règles de l'art :

- les attributs (données internes) doivent normalement être déclarés privés (via le mot clef "**private**"). ils ne pourront alors pas être directement modifié depuis une méthode d'une autre classe . On parle de **protection des données**.
- Des **méthodes publiques** `getXxx()` et `setXxx(...)` permettant respectivement de **recupérer** et de **modifier indirectement** la valeur de l'attribut `xxx`.
- Un constructeur par défaut (sans argument) doit être explicité

Remarque importante:

Toute classe java qui respecte les règles précédemment évoquées constitue un "**Java Bean**" (quelquefois appelé **POJO** : Plain Old Java Object ou Plain Ordinary Java Object) .

Un **JavaBean** est un composant de base qui peut être facilement réutilisé même si l'on ne dispose pas de son code source.

Nouvelle version de la classe Compte (respectant les conventions d'un **JavaBean**):

```
public class Compte {

// Attributs privés:
    private String numero;
    private float solde;

// Accesseurs:
    public String getNumero() { return numero;}
    public void setNumero(String numero) { this.numero = numero; }

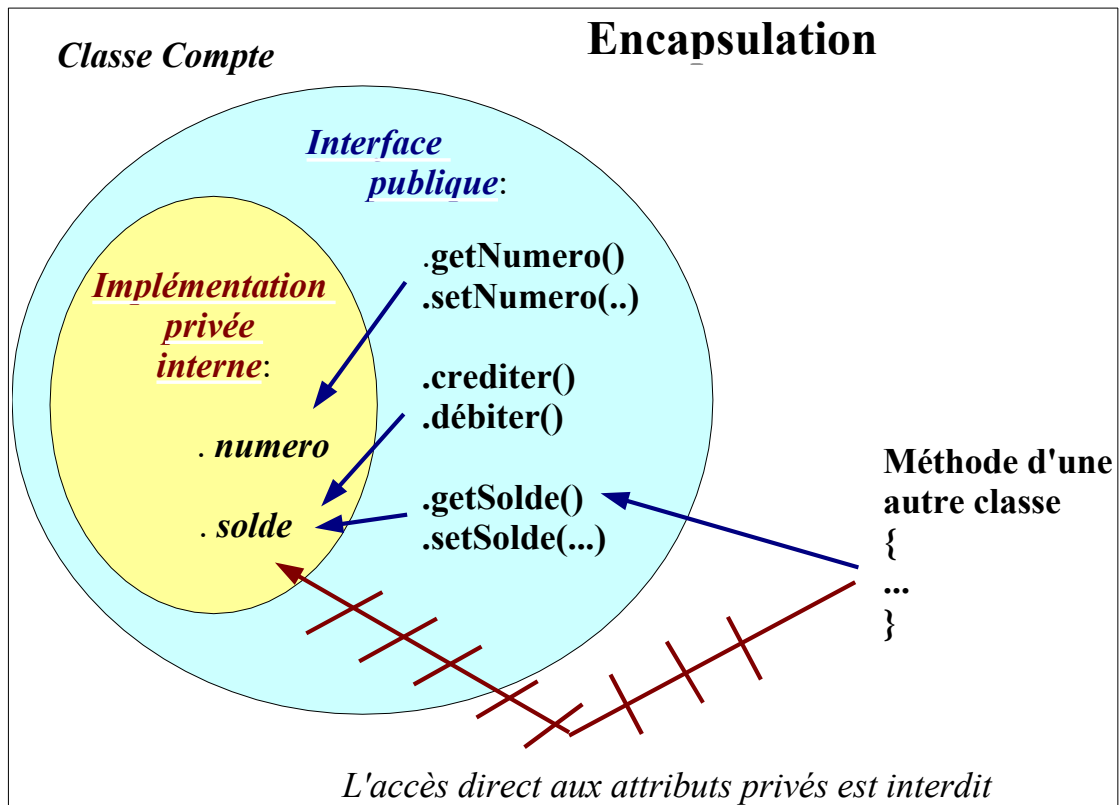
    public float getSolde() { return solde; }
    public void setSolde(float solde) { this.solde = solde; }

// Méthodes :
    public Compte() { numero="0"; solde=0; } // constructeur par défaut

    public void debiter(float montant) { solde = solde - montant; }
    public void crediter(float montant) { solde = solde + montant; }

}
```

```
...
c1.setNumero("102"); c1.setSolde(200);
System.out.println("Le compte num = " + c1.getNumero() + " a un solde de " + c1.getSolde() );
```



Principaux intérêts de l'encapsulation :

- ◆ Les méthodes de la classe courante sont les seules à pouvoir manipuler les attributs privés internes. La cohérence des données est ainsi plus simple à assurer. Du code externe (issu d'une autre classe) sera obligé de passer par les méthodes **getXxx()** et **setXxx()** pour récupérer et modifier indirectement les valeurs --> un contrôle est alors possible:

```
void setAge(int nouvel_age)
{ if(nouvel_age >= 0 && nouvel_age < 150) this.age = nouvel_age; }
```
- ◆ Ceci permet d'effectuer des **éventuelles évolutions** (version 2, ...) au niveau de la représentation interne des données **sans remettre en cause toutes les utilisations externes** d'une certaine classe. C'est un des points clefs de la **modularité**.
- ◆ D'autre part, un **objet** ne doit normalement pas être vu comme une simple structure de donnée (que l'on manipule directement de l'extérieur) mais comme une **entité relativement autonome qui sait se gérer elle même** (grâce à ses méthodes internes).
- ◆ D'un point de vue conceptuel, appeler une méthode **getXxx()** sur un objet revient à lui envoyer un **message** (ou *requête*) du genre "**retourne moi la valeur courante de ta propriété Xxx**".

DTO , JavaBean , POJO et Record

Dans beaucoup d'applications java , on a souvent besoin de manipuler deux types d'objets de données :

- les **entités persistantes** (avec valeurs stockées et récupérées en base de données)
- les **DTO (Data Transfert Object)** alias VO (Value Object) pour communication entre couches logicielles ou via le réseau (avec conversions Java <—> JSON ou XML)

NB :

- Des entités ou **DTOs** sont toujours programmables comme des JavaBeans/POJO .
- On éventuellement s'appuyer sur l'extension lombok (@Getter @Setter ...) pour simplifier le code des JavaBeans/POJO.
- Depuis java 16,17 un nouveau mot clef **record** permet de coder de manière très compacte des classes de données spéciales qui sont complètement immuables/immutables .

Ces "record"s , présentés en détails dans l'annexe "nouveauautés des versions 12-17"
sont maintenant utilisables pour mettre en œuvre des **DTOs**

4. Ramasse miettes (G.C.)

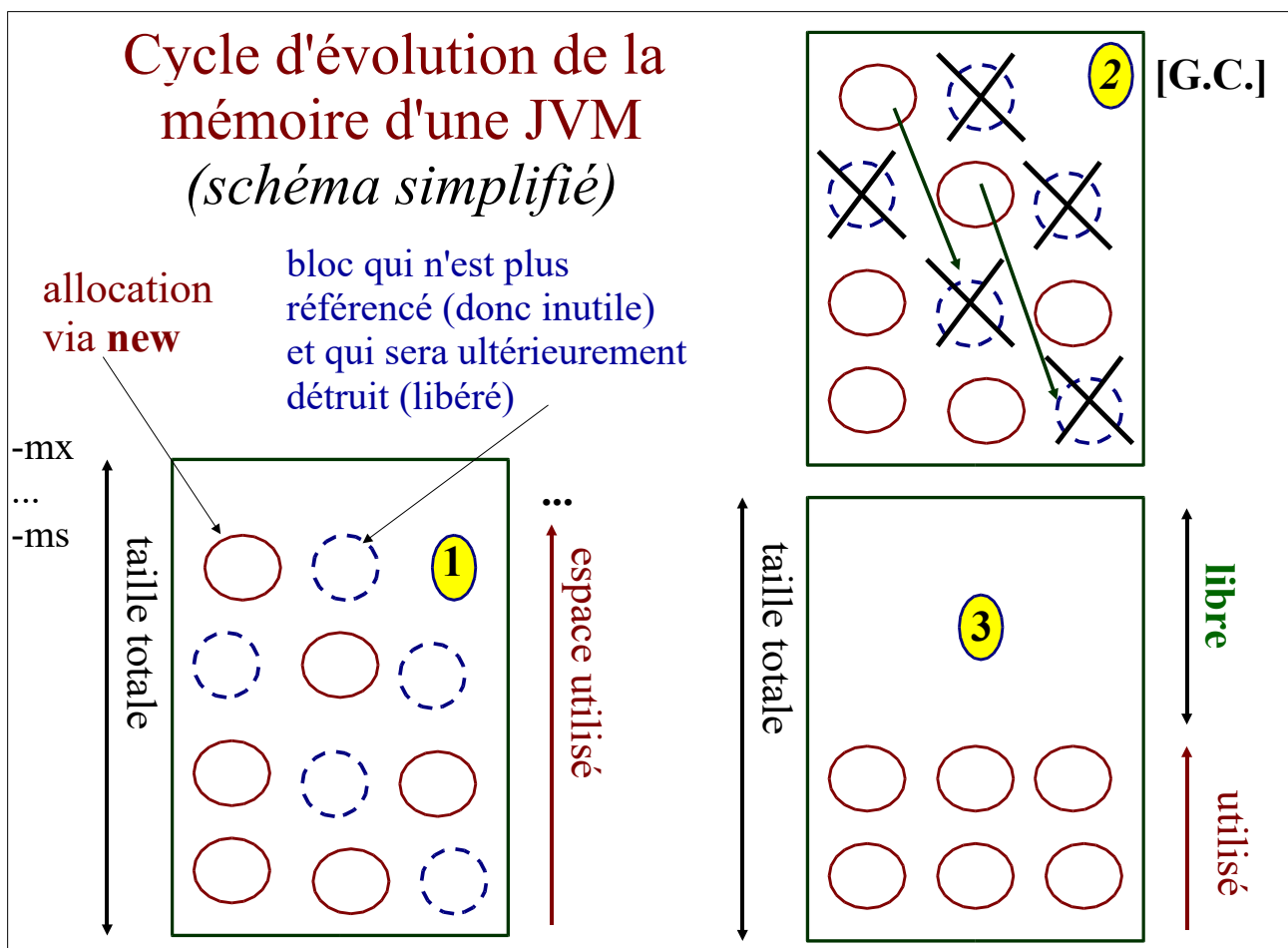
4.1. Destruction automatique des objets

Contrairement aux langages C/C++, le langage JAVA ne comporte *pas de mot clef delete* (ni *free*) pour libérer la mémoire dynamiquement allouée par les précédents appels à *new*.

Les objets JAVA sont automatiquement détruits par un mécanisme interne de la machine virtuelle dès qu'ils ne sont plus référencés. Le programmeur n'a donc plus à gérer explicitement la désallocation de la mémoire: il n'a plus qu'à oublier les objets dont il n'a plus besoin.

Le **ramasse-miettes JAVA** (appelé *Garbage Collector en anglais*) est exécuté par un thread de basse priorité tournant en tâche de fond.

Règle importante: **un objet JAVA sera habituellement détruit lorsque toutes les références sur celui-ci (variables locales, membres d'autres objets, ...) auront disparues.**



=> Une défragmentation de la mémoire est ainsi régulièrement effectuée.

NB1: pour optimiser la gestion de la mémoire, on peut explicitement indiquer à JAVA que l'on a plus besoin d'un objet en donnant la valeur **null** à une référence sur celui-ci:

```
Compte c= new Compte();
c.solde = 120.5;
...
c = null; // ce petit coup de main donné au ramasse miettes n'a d'intérêt que si c peut exister longtemps
           // si la référence c est une variable locale, elle sera rapidement détruite en fin de fonction.
...
```

NB2: **System.gc()** permet de demander au ramasse miettes de s'exécuter tout de suite.

Cependant, ce ramasse-miettes se met à travailler que s'il y a au moins n octets à libérer en mémoire. On est donc jamais complètement sûr du moment où les objets sont détruits.

4.2. Éventuelles finalisation d'objets (obsolète depuis java 9)

Symétriquement aux constructeurs, les **finaliseurs** sont des *méthodes qui sont automatiquement appelées juste avant qu'un objet d'une certaine classe soit détruit*.

De la même façon que les destructeurs du langage C++, **les finaliseurs du langage JAVA ne servent pas à détruire l'objet lui même mais servent à déclencher une cascade d'autres actions à répercuter (fichiers à fermer, déconnexions ...) juste avant que celui-ci ne soit détruit.**

Le finaliseur d'une classe JAVA est une méthode qui porte obligatoirement le nom "**finalize**".

Exemple:

```
...
protected void finalize() throws IOException
{ if(...)
    fic.close();
}
```

Remarque très importante:

- Etant donné qu'on ne contrôle pas du tout le moment exact où un objet java sera détruit, **il faut absolument appeler nous même (dans le cadre d'une bonne programmation) les instructions de fermeture .close() dans le bon ordre : ordre inverse des ouvertures / connexions** .
- Une méthode finalize() appelant une méthode close() ne doit être considérée que comme un mécanisme permettant de remédier à des oublis .
- **En plus d'un problème d'instabilité/fiabilité , la présence de finalize() ralentit le fonctionnement du ramasse-miettes . C'est pourquoi finalize() est officiellement "deprecated" (déconseillée) depuis java 9 . on peut encore utiliser finalize() sur de très vieilles applications .**

5. Opérateurs du langage Java

5.1. Liste des opérateurs du langage JAVA:

opérateurs	Utilisations	Sémantiques
+	expr + expr	Addition
-	expr - expr / -expr	Soustraction / Opposé
*	expr * expr	Multiplication
/	expr / expr	Division
%	expr % expr	Modulo (reste de la division entière)
^	expr ^ expr	Ou exclusif ($1 \wedge 1 ==> 0$)
&	expr & expr	Et bit à bit
	expr expr	Ou inclusif bit à bit
!	! expr	négation logique
=	lvalue = expr	Affectation
<	expr < expr	inférieur
>	expr > expr	supérieur
<=	expr <= expr	inférieur ou égal
>=	expr >= expr	supérieur ou égal
++	++expr / expr++	incrémementation (pré/post)
--	--expr / expr--	décrémementation (pré/post)
<<	expr << expr	décalage à gauche
>>	expr >> expr	décalage à droite
>>>	expr >>> expr	décalage à droite (avec introduction de 0)
instanceof	if(obj instanceof classeXXX) ...	test appartenance à une classe
==	expr == expr	(test) égalité (res. booléen)
!=	expr != expr	(test) différence
&&	expr && expr	Et logique
	expr expr	Ou logique
+=	lvalue += expr	équivalent à lvalue = lvalue + expr
-=	lvalue -= expr	équivalent à lvalue = lvalue - expr
*=	lvalue *= expr	équivalent à lvalue = lvalue * expr
/=	lvalue /= expr	équivalent à lvalue = lvalue / expr
%=	lvalue %= expr	équivalent à lvalue = lvalue %expr
^=	lvalue ^= expr	équivalent à lvalue = lvalue ^ expr
&=	lvalue &= expr	équivalent à lvalue =lvalue & expr
=	lvalue = expr	équivalent à lvalue = lvalue expr
<<=	lvalue <<= expr	équivalent à lvalue = lvalue<< expr
>>=	lvalue >>= expr	équivalent à lvalue = lvalue>> expr
()	(type) expr	conversion de type / casting

NB: `y=Math.abs(x++)` s'exécute avec la valeur courante de `x` (avant incrémentation).

Si `x` valait initialement 5 ==> `y` = 5 et `x`=6

`y=Math.abs(++x)` s'exécute avec la valeur préalablement incrémentée de `x`

Si `x` valait initialement 5 ==> `y` = 6 et `x`=6

Les opérateurs "bit à bit" opèrent sur les représentations binaires des nombres entiers:

a = 5 ($= 1*2^2 + 0*2^1 + 1*2^0$) se code en binaire comme **0...0000101**

b = 3 ($= 1*2^1 + 1*2^0$) se code en binaire comme **0...0000011**

c = a & b donne donc **0...0000001** soit **c=1**

c = a | b donne donc **0...0000111** soit **c=7**

c = a ^ b donne donc **0...0000110** soit **c=4**

c = a << 3 donne donc **0...0101000** soit **c=5 * 2³ = 40**

Le résultat d'un ET logique est globalement faux si la première expression est fausse ==> la seconde expression ne sera donc évaluée que si la première est vraie :

if(`ch !=null && ch.equals("ok")`) ...

Attention à ne pas confondre l'affectation (=) avec un test d'égalité (==)

5.2. Priorités des opérateurs

Niveau de priorité	opérateurs	associativité
1	! - unaire ++ -- (type)	droite-->gauche
2	* / %	gauche-->droite
3	+ -	gauche-->droite
4	<< >> >>>	gauche-->droite
5	< > <= >= instanceof	gauche-->droite
6	== !=	gauche-->droite
7	& bit à bit ou booléen	gauche-->droite
8	^ bit à bit ou booléen	gauche-->droite
9	bit à bit ou booléen	gauche-->droite
10	&& booléen	gauche-->droite
11	booléen	gauche-->droite
12	?: (Expression conditionnelle si?alors:sinon)	droite-->gauche
13	= *= /= += -= %= <<= >>= &= ^= >>>=	droite-->gauche

NB: l'**associativité** désigne l'ordre d'enchaînement lorsqu'il n'y pas de parenthèse:

32 / 2 / 4 s'exécute comme **(32/2) / 4** soit de gauche à droite

a = b = c s'exécute comme **(a = (b=c))** soit de droite à gauche

5.3. Opérateur "instanceof"

```
java.util.ArrayList<Object> liste = new java.util.ArrayList<>();  
liste.add(new String("abc"));  
liste.add(new Integer(5));
```

...

```
java.util.Iterator<Object> it = liste.iterator();  
Object element = it.next(); // it.next(); retourne une chose vague de type Object  
...  
if( element instanceof String) // si l'élément est de type String (?)  
{  
    // Demander l'environnement java de considérer l'élément  
    // comme étant de type précis "String" (conversion - casting) :  
    String chElement = (String) element;  
  
    // Appeler une méthode (fonction interne) spécifique à la classe "String"  
    // (n'existant pas au niveau de la classe générique "Object"):  
    System.out.println("Le premier caractère est " + chElement.charAt(0) );  
}
```

6. Boucles & instructions de Java

6.1. tests conditionnels (if)

```
if ( x > 0)
    y=x; /* une seule instruction qui n'a pas besoin d'être englobée par des {} */
else y=-x;
```

```
if( a == b) /* jamais de then , ce mot clé n'existe pas , la sémantique est implicite */
    { s = a*a ; p = 4*a ; }
else
    { s = a*b ; p = 2*(a+b) ; }
```

6.2. branchements à choix multiples (switch/case)

```
switch(n) /* n doit être d'un type primitif exact (ex: int, char mais pas double) */
          /* depuis le jdk 1.7 , le switch/case fonctionne également avec des "String" */
{
    case 0:
        System.out.println("Choix 0"); break;
    case 5:
    case 10:
        System.out.println("Choix 5 ou 10"); break;
    ...
    default: System.out.println("Choix par défaut");
}
```

NB: l'instruction switch/case a été améliorée dans les versions récentes de java. L'annexe sur les nouveautés de java 12 à 17 sera intéressante à étudier ultérieurement pour approfondir le sujet .

6.3. Boucle while (tant que ...)

```
int i = 100;
while (i > 0) /* tant que */
{
    ... ; i-- ;
}
```

```
do /* faire au moins une fois , ... */
{
    ... } while (i != 0); /* répéter tant que */
```

6.4. Boucle for (pour i allant de ... à ...par pas de ...)

```
for(int i=0 /* valeur initiale du compteur */ ; i<n /* teste pour continuer */ ; i++ /* ou i = i+ 1 */)
{ System.out.println(i);
  /* instruction(s) exécutées avant l'incrémentation (i++) */ }
```

L'instruction **break** sert à déclencher une **sortie anticipée de boucle** :

```
for(i=0;i<n;i++) { if(tab[i] == valRecherchee)
                  { indice=i; break; }
                }
```

NB: l'instruction **continue** permet **passer directement à l'itération suivante** en sautant toutes les instructions situées entre le mot clef continue et l'accolade fermante de la boucle.

7. Chaînes de caractères

7.1. char, String, StringBuilder

Le langage Java comporte 3 types de données permettant de gérer les caractères et les chaînes de caractères:

Types	caractéristiques	valeurs littérales
char	caractère UNICODE (codé sur 2 octets)	'A' , '\u0041'
String	Chaîne de caractères (instance représentant la valeur globale de toute la chaîne).	"abc"
StringBuilder StringBuffer	Buffer de caractères servant à construire efficacement une chaîne via de multiples concaténations .	à convertir en <i>String</i>

L'opérateur + permet d'effectuer des **concaténations** entre 2 objets de type **String**.

```
String ch1="ile" , ch2=" de " , ch3="Ré" , ch4=null;
ch4 = ch1 + ch2 + ch3; // concaténation ==> "ile de Ré"
```

Principales méthodes de la classe String :

.charAt(i) avec i entre 0 et n-1	retourne le i ^{ème} caractère de la chaîne
.length()	retourne la longueur de la chaîne
.equals(autreChaine)	test si la chaîne courante à la même valeur qu'une autre
.substring(firstPos,lastPos+1);	retourne une nouvelle instance = sous partie de la chaîne.

Exemple:

```
String chFileName = "ficA.txt";
int n = chFileName.length(); // 8 caractères
String chExt = chFileName.substring(n-3,n);
```

```
String ch="";
for(int i=0;i<64;i++)
    ch=ch + "";
//Cette version n'est pas bien (performante) car chaque itération de la boucle for
//implique la création d'un nouvel objet de type String (résultat de la concaténation)
//et la destruction (différée) de l'ancienne chaîne référencée par ch .
```

```
//StringBuffer buffer = new StringBuffer(64); // Bien (si plusieurs threads) !!!
StringBuilder buffer = new StringBuilder(64); // très souvent encore mieux !!!
for(int i=0;i<64;i++)
    buffer.append("");
String ch = buffer.toString();
```

Quelques autres méthodes disponibles sur la classe String:

indexOf (String chMotif ...)	retourne la première position où la sous-chaîne est trouvée
toLowerCase ()	retourne une nouvelle instance en <i>minuscules</i>
toUpperCase ()	retourne une nouvelle instance en <i>MAJUSCULES</i>
String.valueOf (...)	convertit une valeur de type <i>int</i> , <i>double</i> , ... en <i>String</i>
split (String separateur)	construit un tableau des sous chaines séparées par un séparateur

```
String s = "un;deux;trois" ;
String[] parties = s.split(";"); //parties[0] vaut "un", parties[1] vaut "deux" et parties[2] vaut "trois"
for(int i=0 ; i < parties.length ; i++) System.out.println(parties[i]) ;
```

7.2. Expressions régulières

Depuis la version 1.4 du JDK, la classe **String** comporte certaines méthodes permettant de gérer efficacement les **expressions régulières**.

Rappels sur les expressions régulières:

.	caractère quelconque
.* , c*	* signifie 0 ou n fois le caractère précédent (.* ==> chaîne quelconque)
[0-9] , [a-z]	un caractère au sein d'une plage possible

Gestion des expressions régulières depuis la classe String:

```
String ligne="1969";
String grep_regex="[0-9]*";
if(ligne.matches(grep_regex))
    ....
```

```
String newLigne = ligne.replaceAll(replace_regex,replace_text);
```

7.3. TextBlocs

NB : depuis java 15,16,17 , on peut englober via des *triples doubles quotes* le longues chaînes de caractères (réparties sur plusieurs lignes , avec à l'intérieur d'éventuels caractères ") appelées *TextBlocs* :

```
String myJsonString =
""""
{
"taille" : 166 ,
"nom" : "toto"
}
""";
```

8. Tableaux

8.1. Syntaxe liée aux tableaux

Au sein du langage Java, Les **tableaux** sont manipulés comme des **objets assez particuliers**:

- En tant qu'objets, ils sont créés dynamiquement (via new) puis manipulés au moyen de références.
- En tant que tableaux, on peut utiliser l'**opérateur []** pour directement accéder au **i^{ème}** élément.

Syntaxe:

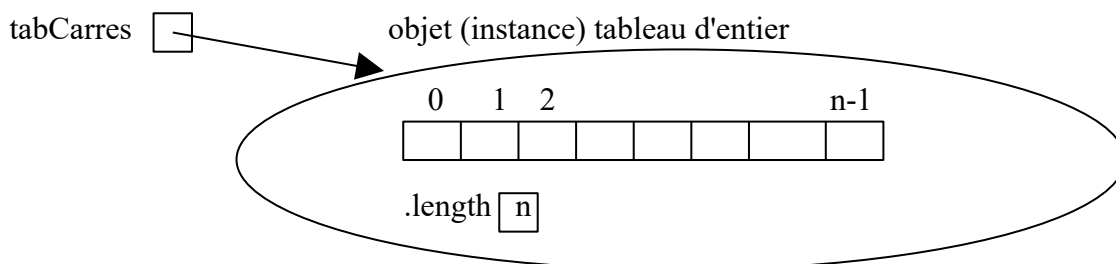
```
TypeElt [ ] refSurTableau=null; // ou bien TypeElt refSurTableau[]=null;
...
refSurTableau = new TypeElt[tailleDuTableau];
...
refSurTableau[positionElt] = valeur;
```

Exemple:

```
byte[] buffer; // ou bien byte buffer[]; --- déclaration d'une référence sur un futur tableau
buffer =new byte[1024]; // construction d'un tableau de 1024 éléments (taille fixe)
buffer[7]=0; // mettre la valeur 0 dans la (7+1)ème case du tableau.
```

// NB: tout tableau a un champ **length** correspondant à sa taille:

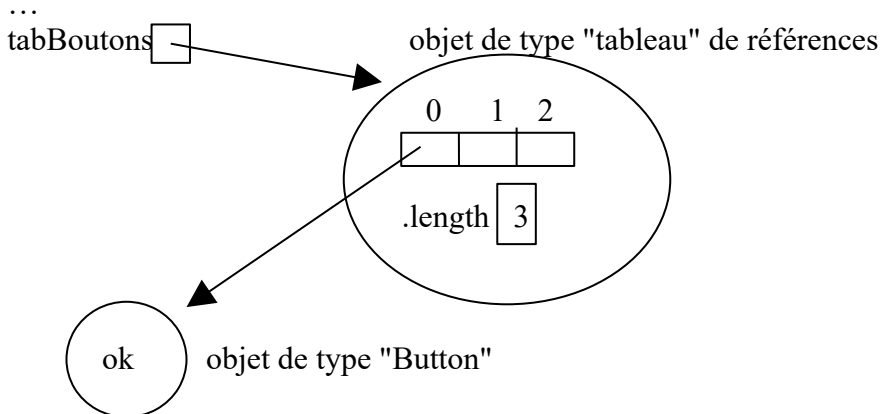
```
int[] tabCarres = new int[20]; // déclaration et construction possible sur la même ligne
for(int i= 0; i<tabCarres.length; i++)
    tabCarres[i] = i * i;
```



```
int tableau_en_dur[] = { 1 , 2 , 4 , 8, 16 }; // initialiseur statique
```

tableau de références sur des objets:

```
Button[] tabBoutons = new Button[3]; // tableaux de 3 références non initialisées.
tabBoutons[0] = new Button("ok"); // la première case du tableau référence un bouton
```



Nb:

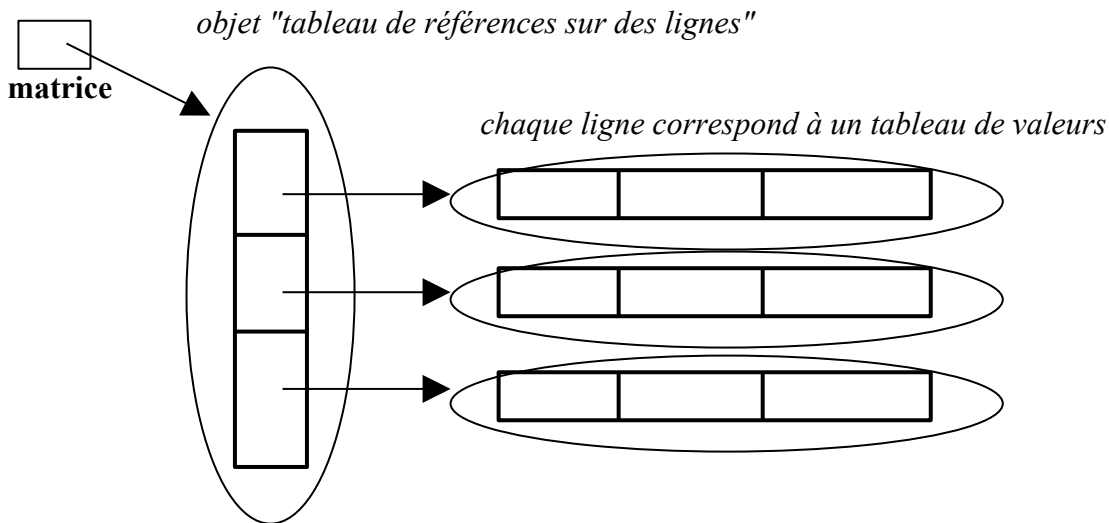
Toute tentative d'accès à un élément dont l'index est hors de l'intervalle $[0, \text{tableau.length}-1]$ provoquera la levée d'une exception de type *ArrayIndexOutOfBoundsException*.

Remarques importantes:

- ♦ Un **tableau ordinaire** a une **taille** qui est **fixée de façon rigide** au moment de sa création en mémoire (lors du *new*). A l'inverse une collection de type **ArrayList** (du package *java.util*) a le mérite d'être **redimensionnable**.
- ♦ Un **tableau ordinaire** peut comporter des éléments de types élémentaires [*non orientés objet*] (int , double , boolean, ...) et **tous les éléments d'un tableau sont issus d'un même et unique type**.
La classe **ArrayList** (du package *java.util*) et toutes les autres collections **ne peuvent comporter que des références sur des objets** (String , Integer mais pas int).

8.2. Tableaux multi-dimensionnels

Java implémente , les tableaux à plusieurs dimensions sous forme de tableau de références sur des sous tableaux.



On peut alors soit:

- allouer toutes les dimensions au départ (`double[][][] matrice = new double[3][3];`)
- allouer que les premières dimensions (`double[][][] triangle = new double[12][[]];`)
pour allouer ultérieurement les autres dimensions:
`for(i=0;i<12;i++) triangle[i] = new double[i+1];`

On peut également initialiser en dur des tableaux multi dimensionnels:

```
int[][] triangle2 = { {1} , { 1, 2 } , { 1 , 2 , 4 } , { 1, 2,4,8} };
```

```
String[][] param_info = { { "param1", "type1" , "libelle1" },  
                           { "param2", "type2" , "libelle2" } }
```

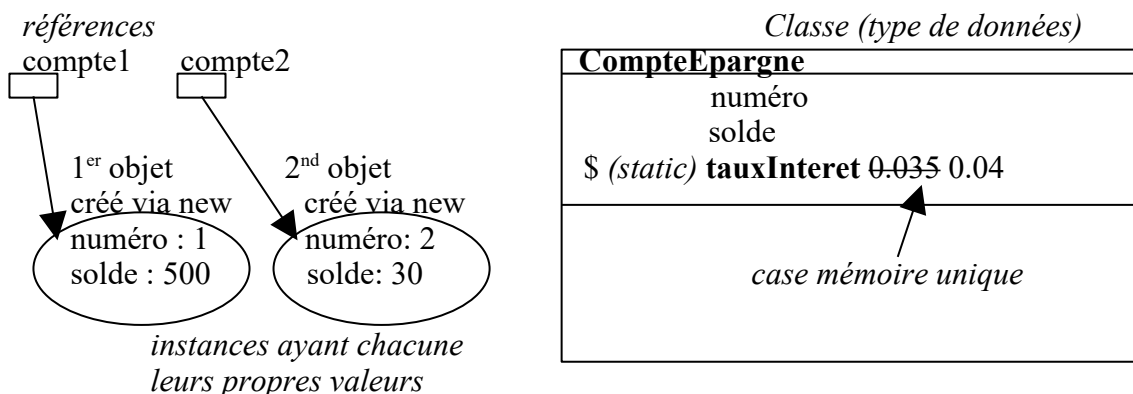
9. Méthode et variables de classes

9.1. Variables de classe

Le mot clé **static** permet de définir des attributs particuliers, appelés "variable de classe", qui seront **partagés par toutes les instances d'une même classe**. On aura alors affaire à une **case mémoire unique** qui sera liée à la classe plutôt qu'à chacune des différentes instances.

exemple:

```
public class CompteEpargne {
    private String numero;
    private double solde;
    private static double tauxInteret = 0.035;
    ...
    public CompteEpargne(String num,double soldeInitial)
    { numero = num; solde= soldeInitial; }
    ...
    public static double getTauxInteret() { return tauxInteret; }
    public static void setTauxInteret(double taux) { tauxInteret = taux; }
}
```



```
compte1.setNuméro(1);

compte1.tauxInteret = 0.04; // possible (depuis même package) mais très déconseillé
compte1.setTauxInteret(0.04); // possible mais très déconseillé(ambigu).
CompteEpargne.tauxInteret = 0.04; // c'est un peu mieux(possible depuis même package)
CompteEpargne.setTauxInteret(0.04); // c'est encore mieux
```

Autre exemple:

```
public class CXxx {
    /* public */ static int nb_instances = 0; // compteur d'instances
    ...
    public CXxx () { nb_instances++; ... } // constructeur(s)
    protected void finalize() { nb_instances--; ...}
}
```

Une **variable de classe** (*statique*) peut naturellement être **préfixée par un nom de classe**:

```
System.out.println("Nombre d'objets créés: " + CXxx.nb_instances );
```

Les **variables de classes publiques** forment le plus proche équivalent JAVA des **variables globales du langage C/C++** .

9.2. Constantes

Au sein du langage JAVA, une **constante** ne peut se définir que sous la forme d'une **variable de classe déclarée finale** (*ne pouvant plus changer de valeur*):

```
public class Cercle
{
public static final double PI = 3.141592653589... ;
... }
```

On pourra ainsi écrire $2 * \text{Cercle.PI} * r$.

NB: La classe **Math** contient déjà une constante dénommée **PI**.
Par convention, toutes les constantes sont déclarées en MAJUSCULES.

9.3. Méthodes de classe

Le mot clé **static** permet également de définir une **méthode de classe** .

Une telle **méthode statique** a la particularité de **pouvoir être invoquée depuis un nom de classe** et non pas seulement depuis une instance particulière de la classe.

Exemples:

```
public class CompteEpargne {
...
private static double tauxInteret = 0.035;
...
public static double getTauxInteret() { return tauxInteret; }
public static void setTauxInteret(double taux) { tauxInteret = taux; }
}
```

```
CompteEpargne c1,c2;
c1=new CompteEpargne("101",100);
c2=new CompteEpargne ("102",200);
double tauxInteretCourant = CompteEpargne.getTauxInteret();
```

```
// Appel de la méthode de classe "Math.sqrt"
double distance = Math.sqrt(x*x+y*y);
```

NB:

- Les méthodes statiques sont en JAVA ce qui se rapproche le plus des fonctions globales du langage C.
- Etant donné qu'une méthode statique est généralement appelée depuis une classe et non pas à partir d'une instance particulière, le mot clé **this** ne peut pas être utilisé au sein du code interne d'une méthode statique.
- La plus classique des méthodes statiques est la fameuse méthode principale **main()**.

Exemple:

```
public class MyApp {  
  
    public static void main(String[] args)  
    {  
        System.out.println("Démarrage du programme \n");  
    }  
}
```

9.4. Initialiseurs statiques

Un constructeur ordinaire est appelé au moment où un nouvel objet est construit et ne peut initialiser que des variables d'instances (attributs qui ne sont pas "*static*").

Les variables de classes sont quant à elles construites lors du chargement de la classe en mémoire. Elles peuvent être initialisées en utilisant ce qu'on appelle un "*initialiseur statique*". Il s'agit d'un *bloc de code (entre {}) directement préfixé par le mot clé static*.

Exemple:

```
public class CXxx {  
    ...  
    private static int tabCoeff[] = new int[4];  
  
    ...  
    static {  
        tabCoeff[0]=2; tabCoeff[1]=4; tabCoeff[2]=1; tabCoeff[3]=2;  
    }  
    ...  
}
```

III - Structures fondamentales(collection, generic)

1. Packages et archives (.jar)

1.1. correspondance entre nom de package et un chemin relatif

En Java, *chaque classe compilée est stockée dans un fichier à part. Le nom de ce fichier doit être le même que celui de la classe* (ex: MaClasse.class).

Les classes sont elles mêmes regroupées au sein de "packages".

Le nom d'un package peut être composé et doit être apparenté à une structure arborescente de répertoires et de sous répertoires :

Ainsi le package

com.worldcompany.utils

correspond au répertoire dont le chemin relatif est

com/worldcompany/utils .

Le nom complet d'une classe java est "nom_composé_du_package.NomClasse"

Grâce à ceci, une **machine virtuelle JAVA** est ainsi capable de situer (*relativement à partir d'une base de départ*) le **fichier d'une certaine classe d'un certain package**.

1.2. Archive Java (.jar)

Un programme ou une API Java est généralement constitué d'un grand nombre de classes organisées au sein de différents packages.

De façon à faciliter le déploiement vers d'autres machines, toute cette arborescence est en général packagée au sein d'une archive (fichier ".ZIP" ou ".JAR") .

Un fichier **.JAR (Java ARchive)** est un fichier au format **ZIP** (que l'on peut par exemple créer ou consulter avec WinZip ou 7Zip ou bien l'utilitaire [bin\jar.exe](#) du jdk).

Construction de "my_appli.jar" pour démarrer une application via un simple double-click:

Soit **pom.xml** une configuration **maven** (située à la racine d'un projet) et permettant de construire automatiquement une archive "**myAppli.jar**" en fonction de tout le contenu du projet :

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>org.mycontrib.xyz</groupId>
  <artifactId>myAppli</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <packaging>jar</packaging>
```

```
<properties>...</properties>
<dependencies>...</dependencies>
<build>
  <finalName>${project.artifactId}</finalName>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-jar-plugin</artifactId>
      <configuration>
        <archive>
          <manifest>
            <mainClass>org.mycontrib.xyz.App</mainClass>
          </manifest>
        </archive>
      </configuration>
    </plugin>
  </plugins>
</build>
</project>
```

mvn package permet de générer une archive "**myAppli.jar**" comportant le fichier *Manifest* suivant:

META-INF/Manifest.mf

```
Manifest-Version: 1.0
Main-class:   org.mycontrib.xyz.App
```

L'indication "*Main-class:* " permet de préciser la classe principale du programme à lancer suite à un double-click sur le "*.jar auto-démarrable*".

NB: Le démarrage de l'application java via un double click ne fonctionne que dans le contexte suivant :

- ordinateur windows
- java installé et configuré dans le PATH
- le programme java lancé ne s'arrête pas aussitôt (il comporte une fenêtre graphique pour que l'on puisse le voir , ex : *JOptionPane.showMessageDialog(null,"Bienvenue MyApp");*)

Pour un lancement/démarrage plus universel (sans les contraintes précédentes) , on aura besoin d'écrire un script de ce genre :

```
set JAVA_HOME=C:\Program Files\java\jdk17
REM set JAVA_HOME=C:\Program Files\eclipse\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_17.0.2.v20220201-1208\jre
set PATH=%JAVA_HOME%\bin
cd "%~dp0"
cd target
java -jar myAppli.jar
pause
```

1.3. javadoc

NB: Javadoc tient compte des éventuels **commentaires spéciaux** (au format ci-après) pour générer quelques éléments de la documentation:

```
/**
 * @author didier
 *
 * Cette classe sert a ...
 */
....
```

mvn javadoc:javadoc (ou une ligne de commande équivalente sans maven) permet de **générer automatiquement une documentation HTML** sur les classes du projet .
Le point d'entrée de cette documentation est **target/site/apidocs/index.html** .

The screenshot displays the Javadoc HTML output for the `Bagage` class. The left sidebar contains navigation links (OVERVIEW, PACKAGE, CLASS, USE, TREE, INDEX, HELP) and a summary of the class: Package `tp`, Class `Bagage` (inherited from `java.lang.Object` and implementing `Transportable`). The main content area is divided into two sections: **Field Summary** and **Method Summary**.

Field Summary:

Modifier and Type	Field	Description
static final Double	<code>Poids_MOYEN</code>	
static final Double	<code>VOLUME_MOYEN</code>	

Method Summary:

Modifier and Type	Method	Description
double	<code>getPoids()</code>	
double	<code>getVolume()</code>	
void	<code>setPoids(Double poids)</code>	
void	<code>setVolume(Double volume)</code>	
String	<code>toString()</code>	

NB: L'aide en ligne sur les éléments prédéfinis du langage java est également générée à partir de javadoc et se consulte de la manière suivante:

<https://docs.oracle.com/en/java/javase/17/docs/api/>

The screenshot shows the Oracle Java API documentation page. The navigation bar includes links for **All Modules**, **Java SE**, **JDK**, and **Other Modules**. Below this is a table listing modules and their descriptions.

Module	Description
<code>java.base</code>	Defines the foundational APIs of the Java SE Platform.
<code>java.compiler</code>	Defines the Language Model, Annotation Processing, and Java Compiler APIs.

Below the modules table, there is a section titled **Packages** with a sub-section **Exports**. This section contains a table listing packages and their descriptions.

Package	Description
<code>java.io</code>	Provides for system input and output through data streams, serialization and the file system.
<code>java.lang</code>	Provides classes that are fundamental to the design of the Java programming language.

1. Choix d'un module et d'un package (ex: java.lang ou java.text)
2. Choix d'une interface ou d'une classe
3. Visualisation des détails (héritages , attributs , méthodes,)

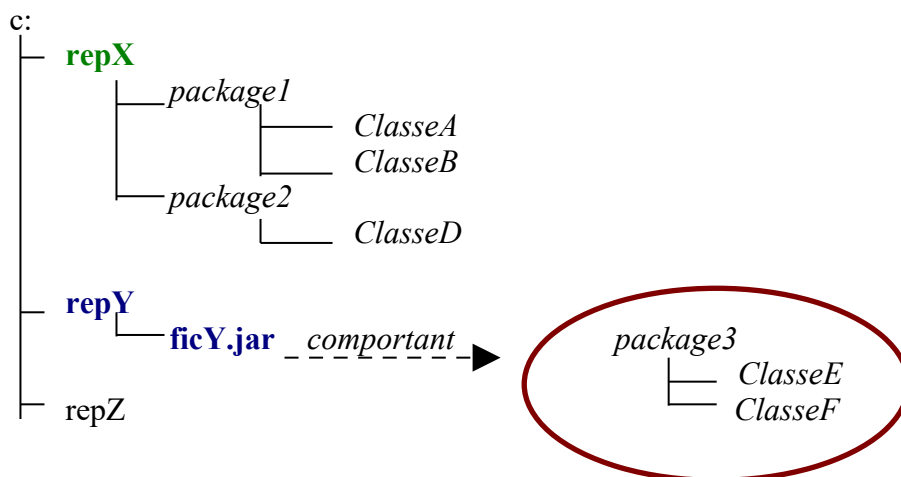
Module java.base Package java.text Class SimpleDateFormat java.lang.Object java.text.Format java.text.DateFormat java.text.SimpleDateFormat All Implemented Interfaces: Serializable, Cloneable	Constructor Summary Constructors Constructor SimpleDateFormat() SimpleDateFormat(String pattern) SimpleDateFormat(String pattern, DateFormatSymbols formatSymbols) SimpleDateFormat(String pattern, Locale locale)
---	--

1.4. CLASSPATH (liste des endroits où rechercher les classes)

- ♦ La variable d'environnement **CLASSPATH** est constituée d'*une liste de chemins menant à des paquets de classes* que l'on peut avoir besoin de charger en mémoire.
- ♦ Chaque **chemin** (séparé par ';' sous windows ou par ':' sous unix) est *soit* un répertoire du système de fichier *soit* une archive (fichier '.zip' ou '.jar') .
- ♦ *Sous chaque partie référencée par le CLASSPATH on doit pouvoir trouver en relatif les sous répertoires associés à une arborescence des packages* ainsi que les fichiers de classes (.class)

Exemple:

```
set CLASSPATH=%CLASSPATH%;c:\repX;c:\repY\ficY.jar
```



Remarque:

Les classes prédéfinies des API standards de Java sont placées dans certaines archives que la machine virtuelle consulte systématiquement :

- ... \jre\lib\rt.jar (pour java 1 à 8)
- ./jmods/java.base.jmod , java.xyz.jmod (à partir de java 9)

L'API Java est elle même constituée de plusieurs packages:

Nom du package	Contenu
java.applet	Classes pour coder les applets
java.awt	(Abstract Window Toolkit) Gestion des fenêtres, GUI
javax.swing	Classes graphique 100% java (JButton, ...)
java.sql	Accès aux bases de données via JDBC
java.io	Entrées/sorties
java.lang	Base du langage JAVA (types de base, ...)
java.net	Communication réseau (Socket, UrlConnection,...)
java.util	Classes utilitaires (Vector, Date, ...)
...	...

Les packages ne servent pas uniquement à trouver les classes à charger.

Les deux grands autres intérêts des packages sont les suivants:

- **Eviter des conflits de noms entre classes développées par différents développeurs.**
Le nom complet d'une classe est en effet quelque chose du type *nompaket.NomClasse* (ex: *java.util.Date* est différent de *java.sql.Date*)
- Délimiter un espace sur lequel certains mots clés du langage (public, private, protected) auront une incidence sur la visibilité des choses.

Les principales règles liées à la visibilité des entités de JAVA sont les suivantes:

- Un package est accessible si les répertoires et fichiers associés le sont.
- *Toute classe d'un package est accessible depuis toutes les autres classes du même package.*
- *Seules les classes déclarées publiques dans un package sont accessibles depuis les autres packages.*
- *Un membre d'une classe (variable ou méthode) est accessible depuis une autre classe du même package si et seulement si il n'est pas déclaré privé.* Les champs privés ne sont accessibles qu'à l'intérieur de la classe en question.
- *Les membres d'une classe sont accessibles depuis un package différent à partir du moment où la classe est accessible (public) et que ses membres sont déclarés publics, ou bien si ses membres sont déclarés protégés(protected) et que l'on y accède depuis une sous classe.*

1.5. Instruction package

L'instruction **package** ne peut apparaître que sur la première ligne (différente d'un commentaire) d'un fichier source.

Cette instruction indique le nom du package auquel appartient le code du fichier source.

```
package com.worldcompany.utils;
```

NB: Si l'instruction package est omise (ce qui n'est pas du tout conseillé), le code du fichier source appartiendra au package par défaut qui n'a pas de nom.

1.6. Instruction import

La directive **import** ressemble de loin à l'instruction `#include` des langages C et C++ .
Vue d'un peu plus près , elle est bien loin d'y ressembler.

La directive **import** indique simplement à JAVA que **l'on pourra utiliser des noms abrégés (sans préfixe correspondant au package) pour nommer certaines classes.**

Exemple :

```
import java.awt.*;
```

Grâce à l'instruction précédente on pourra écrire :

```
bouton =new Button("Ok");
```

au lieu d'écrire :

```
bouton = new java.awt.Button("Ok");
```

Remarques importantes:

- L'instruction ***import java.lang.*;*** n'est pas indispensable puisqu'elle est systématiquement et implicitement prise en compte (c'est un cas particulier).
- Le caractère `*` souvent utilisé en fin des directives "**import**" signifie "n'importe quelle classe du package" mais ne signifie pas "n'importe quelle classe et n'importe quel sous package".

```
import java.awt.*;  
import java.awt.event.*; // les 2 lignes sont nécessaires.
```

- Beaucoup de programmeurs consciencieux préfèrent indiquer **une longue série de directives import** en n'utilisant pas le caractère `*` mais **en spécifiant à chaque fois le nom exact d'une des classes utilisées.** [Ceci n'est intéressant qu'au niveau du code source. Le résultat de la compilation est exactement le même]:

```
package ex;  
  
import java.util.ArrayList;  
import java.util.Iterator;  
import java.util.Date;    // On visualise tout de suite toutes les classes utilisées par ce fichier de code  
...
```

1.7. Résumé sur les modificateurs d'accès:

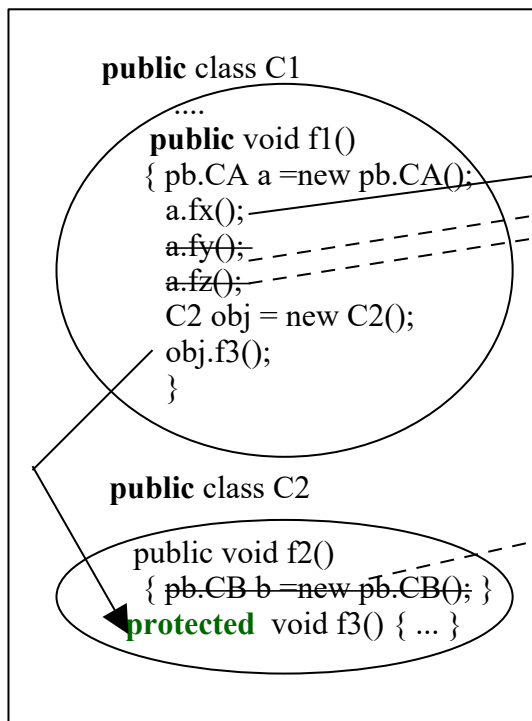
Accès depuis le package courant:

modificateur d'accès (m)	héritage possible d'une classe déclarée <i>m</i> pour une sous-classe du même package	membre déclaré <i>m</i> accessible depuis le code d'une autre classe du même package
par défaut (rien d'indiquer)	oui	oui
public	oui	oui
protected	oui	oui
private	non	non

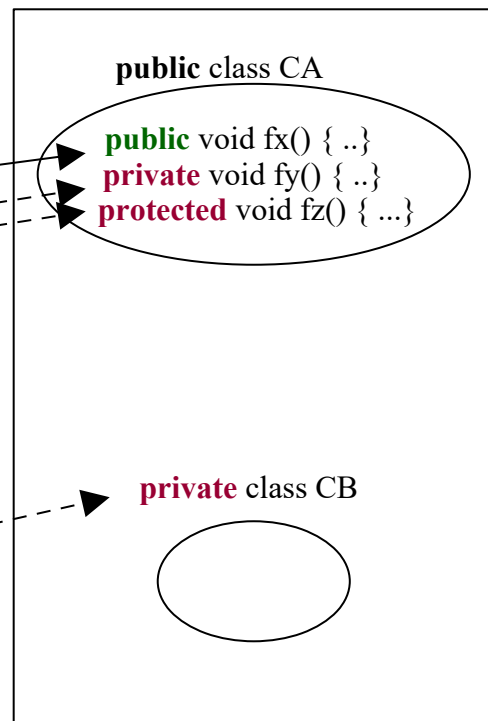
Accès depuis un autre package:

modificateur d'accès (m)	accès (ou héritage) possible vers une classe déclarée <i>m</i> pour une classe d'un autre package	membre déclaré <i>m</i> accessible depuis le code d'une autre classe d'un autre package
par défaut (rien d'indiquer)	non	non
public	oui	oui
protected	oui	non (sauf si depuis sous-classe)
private	non	non

package pa



package pb



2. Collections (depuis le jdk 1.2)

Implémentations → Interfaces ↓	Hash Table	Resizable Array	Balanced Tree	Linked List	Vielles classes du jdk1.1 et réadaptées sur les collections du jdk1.2
Set	HashSet		TreeSet		
List		ArrayList		LinkedList	Vector
Map	HashMap		TreeMap		Hashtable

NB: Les interfaces **Set**, **List** (et des parties internes de **Map**) **héritent** de l'interface **Collection**

Set ==> Ensemble d'éléments (pas de duplication possible)

List ==> Liste ordonnées d'éléments (doublons éventuels) + Accès via index

Map ==> Table d'association [ensemble de couples (clef,valeur)]

NB: Les éléments internes des **TreeSet** et **TreeMap** sont **ordonnés**.

----> Les éléments internes des **HashSet** et **HashMap** ne sont **pas ordonnés** (ordre variable au cours du temps suite à des ajouts ou suppressions) .

2.1. Interface Collection

principales méthodes	détails
boolean add (Object o)	retourne false si l'élément y était déjà ou si ...
boolean addAll (Collection c)	
void clear ()	vide tout
boolean contains (Object o)	objet contenu dans la collection ?
boolean isEmpty ()	
Iterator iterator ()	retourne un itérateur pour parcourir la collection
boolean remove (Object o)	
int size ()	Nombre d'éléments de la collection
Object[] toArray ()	
...	

Nb: un **itérateur** s'utilise de la façon suivante:

```

Iterator<Object> it = MaCollection.iterator();
while(it.hasNext())
{
    Object obj = it.next();
    obj. ....
}

```

NB: Depuis le jdk 1.2 , il est conseillé d'utiliser **Iterator** à la place de **Enumeration**.

NB: La classe **Vector** est *synchronisée* (accès concurrents possibles depuis différents threads).

----> **ArrayList** et les autres collections ne sont **pas synchronisés**.

Cependant la méthode statique ***Collections.synchronizedList(liste)*** permet d'obtenir une version synchronisée lorsque c'est nécessaire .

L'obtention d'un itérateur et le parcours associé doit quelquefois être effectué au sein d'un bloc {...} encadré par *synchronized(liste)* de façon à garantir une cohérence au niveau de l'itérateur lors d'éventuels accès concurrents (plusieurs threads).

Exemple:

```
List list = Collections.synchronizedList(new ArrayList());
...
synchronized(list) {
    Iterator i = list.iterator(); // Must be in synchronized block
    while (i.hasNext())
        f1(i.next());
}
```

2.2. Interface List

L'interface **List** hérite de l'interface **Collection** et offre en plus les méthodes suivantes:

<i>principales méthodes</i>	<i>détails</i>
Object get (int index)	retourne l'objet à la position indiquée
int indexOf (Object o)	retourne -1 si l'objet de fait pas partie de la liste
int lastIndexOf (Object o)	idem pour dernière occurrence trouvée
ListIterator listIterator ()	
Object set (int index, Object o)	remplace l'élément en position index
List subList (int first, int last)	du premier index inclus au dernier exclus

NB:

L'interface **ListIterator** hérite de **Iterator** et offre en plus :

- un parcours dans le sens inverse (via **HasPrevious()** et **previous()**)
- un parcours basé sur les index (via **nextIndex()** et **previousIndex()**).

Exemple :

```
List<String> liste = new ArrayList<>();
liste.add("janvier");
liste.add("fevrier");
liste.add("mars");

//parcours avec code proche d'un parcours de tableau ordinaire:
int n = liste.size();
System.out.println("n="+n);
for(int i=0;i<n;i++) {
    Object obj = liste.get(i);
    System.out.println("obj="+obj);
}
```

```
//parcours avec un itérateur (années 2000-2005):
//Iterator it = liste.iterator();
Iterator<String> it = liste.iterator();
while(it.hasNext()) {
    String valeur = /*(String)*/ it.next();
    System.out.println("valeur="+valeur);
}

//parcours moderne avec boucle for() au sens forEach():
for(String sVal : liste) {
    System.out.println("sVal="+sVal);
}
```

2.3. Interface Map

L'interface **Map** représente les fonctionnalités d'une **association** (ensemble de couple Key/Value)

<i>principales méthodes</i>	<i>détails</i>
boolean containsKey (Object key)	
boolean containsValue (Object v)	
void clear ()	vide tout
Object get (Object key)	
boolean isEmpty ()	
Object put (Object key,Object val)	
boolean remove (Object o)	
int size ()	Nombre d'éléments de la collection
Set entrySet ()	vue sous la forme d'un ensemble d'éléments de type Map.Entry ==> (get/set Key/Value ())
Collection values ()	
...	

Exemple :

```
Map<Integer,String> mapMois = new HashMap<>();
mapMois.put(1, "janvier");
//...
mapMois.put(12, "decembre");

String nomDuMoisNumero12 = mapMois.get(12);
System.out.println("nomDuMoisNumero12="+nomDuMoisNumero12);//affiche decembre
```

2.4. Tri

La classe **Collections** (avec un *s* à la fin) comporte une méthode statique **sort** permettant de trier les éléments de la liste passée en paramètre:

```
java.util.Collections.sort(liste);
```

NB: L'ordre de tri est par défaut piloté par l'interface **java.lang.Comparable** implémentée par des éléments tels que ceux des types *Integer*, *String*, *Double*,

NB: Un vecteur étant considéré comme une liste (depuis le jdk 1.2), on peut le trier via le mécanisme précédent.

NB: une seconde version de **Collections.sort()** admet un second paramètre de type **java.util.Comparator** de façon à contrôler finement l'ordre de tri (lorsque c'est nécessaire).

L'interface **Comparator** nous impose de programmer une méthode

```
public int compare(...o1,...o2){
    //si o1 est plus petit que o2 alors return -1 (ou valeur négative);
    //si o1 est égal à o2 alors return 0;
    //si o1 est plus grand que o2 alors return +1 (ou valeur positive);
}
```

```
List<Double> listeNumerique = new ArrayList<>();
```

```
listeNumerique.add(12.0);
```

```
listeNumerique.add(4.0);
```

```
listeNumerique.add(8.0);
```

```
Collections.sort(listeNumerique); //les Integer,Long,Deouble,String se comparent entre eux automatiquement
```

```
public class CompareurDePersonne implements java.util.Comparator<Personne>{
    @Override
    public int compare(Personne o1, Personne o2) {
        //return o1.getTaille() - o2.getTaille();
        if(o1.getNom() != null)
            return o1.getNom().compareTo(o2.getNom());
        else return -1;
    }
}
```

```
List<Personne> listePersonnes = new ArrayList<>();
```

```
listePersonnes.add(new Personne("jean","Bon","jb@xy.com",177));
```

```
listePersonnes.add(new Personne("axelle","Aire","aa@xy.com",167)); //...
```

```
CompareurDePersonne compareurDePersonne = new CompareurDePersonne();
```

```
Collections.sort(listePersonnes,compareurDePersonne);
```

```
for(Personne p : listePersonnes) {
```

```
    System.out.println("\t" + p); //"\t" pour tabulation , "\n" pour saut de ligne
```

```
}
```

3. Generics (depuis Java 5)

3.1. Incohérence de type sur les éléments d'une collection

Soit une classe utilisant une Collection d'objet String et parcourue grâce à un itérateur :

```
import java.util.*;

public class Ex1
{
    private void testCollection() {
        List list = new ArrayList();
        list.add(new String("Hello world!"));
        list.add(new String("Good bye!"));
        list.add(new Integer(95));
        printCollection(list);
    }

    private void printCollection(Collection c) {
        Iterator i = c.iterator();
        while(i.hasNext()) {
            String item = (String) i.next();
            System.out.println("Item: "+item);
        }
    }

    public static void main(String argv[]) {
        Ex1 e = new Ex1(); e.testCollection();
    }
}
```

← // incohérence
// pas détectée
// à la compilation

Bien que comportant une incohérence (le troisième élément de la *Collection* est un *Integer*), ce code passe bien à la compilation .

Le problème vient du fait qu'il est nécessaire de faire un CAST sur le résultat de la méthode *i.next()*. Seule l'exécution de ce programme provoquera une **ClassCastException**.

3.2. Classes génériques fortement typées

Les classes génériques java permettent d'utiliser une collection avec un type donné et de vérifier les erreurs de Cast. En compilant l'exemple ci-après avec l'option *-source 1.5*, on obtient une erreur lorsque l'on essaye d'ajouter un objet *Integer* dans notre *Collection* de *String*.

```
import java.util.*;

public class Ex2 {

    private void testCollection() {
        List<String> list = new ArrayList<String>();
        list.add(new String("Hello world!"));
        list.add(new String("Good bye!"));
        list.add(new Integer(95)); // ERREUR
        printCollection(list);
    }

    private void printCollection(Collection<String> c) {
        Iterator<String> i = c.iterator();
        while(i.hasNext()) {
            String item = i.next();    // plus de cast à expliciter
            System.out.println("Item: "+item);
        }
    }

    public static void main(String argv[]) {
        Ex2 e = new Ex2();
        e.testCollection();
    }
}
```

Prédéfinis (en mieux) dans java.util:

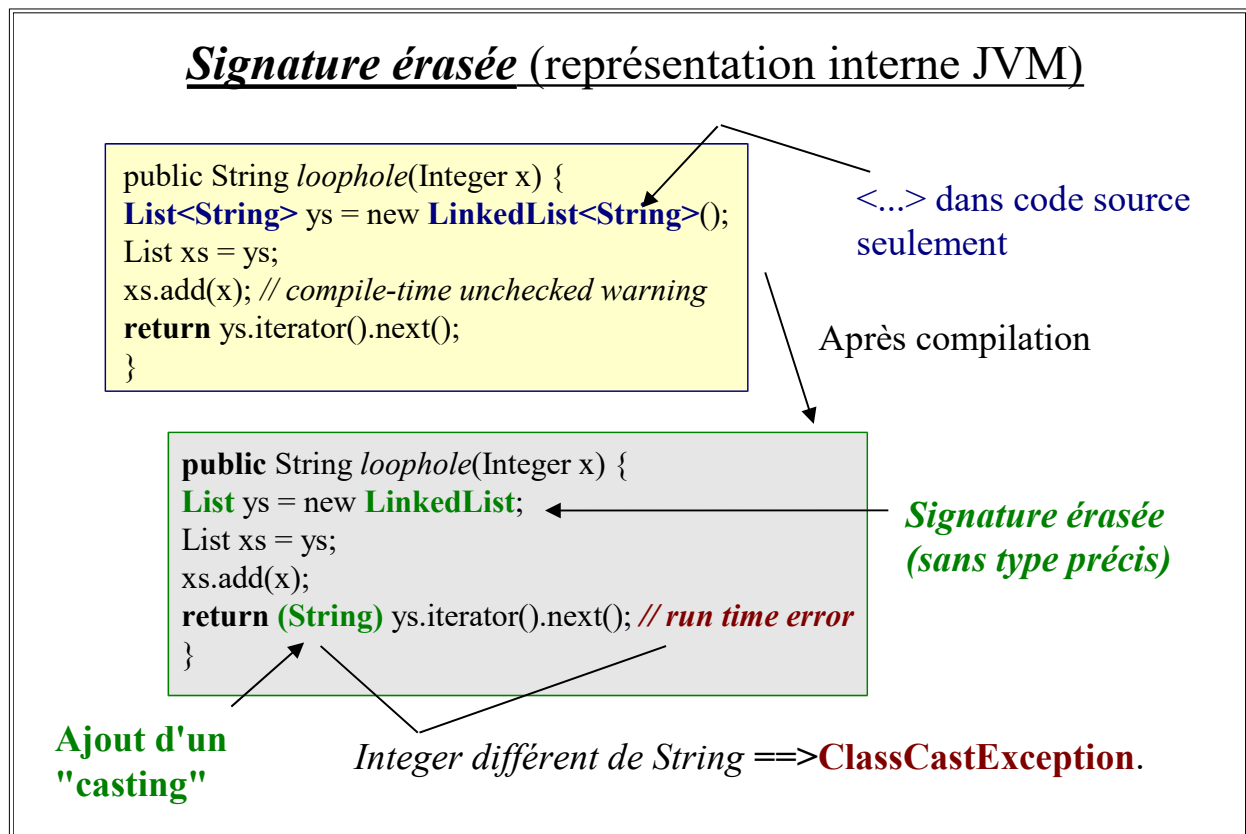
```
public interface List<E> {
    void add(E x);
    Iterator<E> iterator();
    ...
}

public interface Iterator<E> {
    E next();
    boolean hasNext();
    ...
}
```

3.3. Java Generics vs. C++ Templates

Même si les classes génériques java ressemblent aux templates C++, elles sont différentes. Les classes "Generics" JAVA font simplement une **vérification** de type à la compilation et éliminent l'utilisation explicite des CAST.

Le C++ construit quant à lui différentes classes "assez statiques (ex: " vector<int> , vector<double> ,) à partir d'un unique modèle générique de code appelé "template" (ex: Vector<T>) .



Wildcards de java >=5 .Wildcards (?) de Java 5**? = wildcard** (any type)**Collection<?> = Collection of Unknown .**

```
void printCollection(Collection<?> c) {
    for (Object e : c) {
        System.out.println(e);
    }
}
```

```
public void drawAll(List<? extends Shape> shapes) {
    // liste de chose héritant de Shape --> appel possible avec List<Circle> ou ...
    for (Shape s: shapes) {s.draw(this); // ok : opération en lecture/affichage
    }
}

public void addRectangle(List<? extends Shape> shapes) {
    shapes.add(0, new Rectangle()); // compile-time error!
    // même si Rectangle hérite de Shape , car opération impossible
    // si ? Remplace Circle héritant de shape
}
```

On peut programmer de nouvelles classes génériques comme le montre l'exemple suivant:

```
package tp;

import java.util.ArrayList;
import java.util.List;

/* LIFO = Last In , First Out ,
   mieux que FINO = First In, Never Out*/

public class MyStack<T> {
    private List<T> listeInterne = new ArrayList<T>();

    public void push(T objVal) {
        listeInterne.add(objVal);
    }

    public T pop() {
        T objVal = null;
        if(!listeInterne.isEmpty()){
            int lastIndex = listeInterne.size() - 1;
            objVal = listeInterne.remove(lastIndex);
        }
        return objVal;
    }
}
```

test/utilisation:

```
MyStack<Integer> pileEntiers = new MyStack<Integer>();
pileEntiers.push(2); pileEntiers.push(1);
System.out.println(pileEntiers.pop());
System.out.println(pileEntiers.pop());
```

```
MyStack<String> pileChaines = new MyStack<String>();
pileChaines.push("abc"); pileChaines.push("def");
System.out.println(pileChaines.pop());
System.out.println(pileChaines.pop());
```

3.4. Diamond operator (simplification depuis jdk 1.7)

```
MyStack<Integer> pileEntiers = new MyStack<>();
MyStack<String> pileChaines = new MyStack<>();
```

Le compilateur (devenu plus intelligent) est capable (par inférence) de déduire le type précis à instancier en se basant sur le type précis de l'interface .

3.5. Nouvelle boucle for (sémantique "for each")

La nouvelle version de la boucle **for** permet de parcourir des collections et des tableaux sans utiliser explicitement d'itérateur ni d'index.

Nouvelle syntaxe :

for (FormalParameter : Expression) Statement

L'expression doit être un tableau ou l'instance d'une nouvelle interface *java.lang.Iterable*, nécessaire pour l'utilisation de la nouvelle boucle.

L'interface *java.util.Collection* implémente dorénavant *Iterable*.

Exemple avec une collection :

```
public void oldFor(Collection c) {
    for(Iterator i = c.iterator(); i.hasNext(); ) {
        String str = (String) i.next();
        System.out.println(str);
    } }
```

Peut être ré-écrit avec ici une utilisation conjointe d'une classe générique java :

```
public void newFor(Collection<String> c) {
    for(String str : c) {
        System.out.println(str);
    } }
```

Exemple avec un tableau :

```
public int sumArray(int array[]) {
    int sum = 0;
    for(int i=0;i<array.length;i++) {
        sum += array[i];
    }
    return sum; }
```

peut être ré-écrit :

```
public int sumArray(int array[]) {
    int sum = 0;
    for(int i : array) {
        sum += i;
    }
    return sum; }
```

Cette nouvelle version de la boucle for ne remplacera pas toujours l'ancienne. L'index *i* étant parfois nécessaire dans le traitement.

NB: La nouvelle boucle **for** de **JAVA 5** ressemble fortement à la boucle *foreach* de **C#**.

La nouvelle boucle aurait pu s'écrire : **foreach**(int i **in** array)

Sun a préféré ne pas créer de nouveau mots clés par soucis de compatibilité.

4. Éléments de Java >=5 (jdk 1.5 , 1.6 et 1.7)

4.1. Enumérations

```
public class Cxx {
    public enum Jour { DIMANCHE , LUNDI, MARDI, ..., SAMEDI };

    private Jour jour = Jour.DIMANCHE; // valeur par défaut ici.

    public Jour getJour(){ return this.jour; }
    public void setJour(Jour jour) { this.jour = jour; }
}
```

Le mot clef **enum** est maintenant réservé. Son utilisation nécessite l'option de compilation **-source 1.5** (ou 1.6 , 1.7) du compilateur javac .

- **Jour** est ainsi considéré comme un **type** de données . Il s'agit d'une classe implémentant implicitement les interfaces **Serializable** et **Comparable**. Cette classe Java d'énumération héritant implicitement de **Object** surcharge de façon adéquat les méthodes *toString()* , *equals()* et *hashCode()* .
- Les éléments internes de l'énumération sont codés comme des constantes entières statiques (valeurs = 0 ,1 ,) ==> méthode **ordinal()**
- Toute classe d'énumération (comme ici Jour) comporte une méthode statique **.values()** retournant le tableau des valeurs énumérables ainsi qu'une méthode **valueOf (String name)** retournant la valeur d'une constante selon son nom sous forme de chaîne de caractères.

exemple:

```
for (Jour j : Jour.values() )
    System.out.println(j + " [" + j.ordinal() + "]" );
```

...

```
objX.setJour(Jour.LUNDI);
Jour j = objX.getJour();
switch(j){
    //NB: bizarrement pas de case Jour.LUNDI mais case LUNDI
    case LUNDI: System.out.println("Lundi c'est ravioli"); break;
    case MARDI: System.out.println("Mardi c'est brocoli"); break;
}
```

4.2. Fonctions à nombre d'arguments variable

```
void argtest(Object ... args) {
    for (int i=0;i <args.length; i++) {
        System.out.println(args[i]);
    }
}

// invocation de la méthode :
argtest("test", "data");
```

4.3. Sorties formatées (printf)

Les fonctions à nombres d'arguments variables ont permis l'implémentation de *printf* (identique à celle du langage C).

```
System.out.printf("%s %3d", name, age);
```

%s ==> string

%d ==> entier décimal (base 10)

%f ==> floating point

%g ==> floating point (quelque soit la précision : double ou float)

... voir l'aide en ligne sur *printf* pour approfondir la syntaxe.

4.4. import static (depuis Java 5)

```
import static packageX.interfaceX.* ;
```

permet d'utiliser tous les éléments statiques de l'interfaceX (constantes , méthodes statiques ,) sans avoir à les préfixer :

On peut alors se contenter d'écrire *CONSTANTE1* au lieu de *InterfaceX.CONSTANTE1* .

Exemples:

```
import static java.lang.Math.*;  
import static java.lang.System.*;
```

- on peut écrire **PI** au lieu de *Math.PI*
- **sin(x)** à la place de *Math.sin(x)*
- **out.println("ok")** à la place de *System.out.println("ok")*

4.5. Saisie clavier via java.util.Scanner (java >=5)

Exemple de code java (toute version) permettant la lecture d'un entier au clavier :

```
BufferedReader br = new BufferedReader(new InputStreamReader(System.in));  
String str = br.readLine();  
int n = Integer.parseInt(str);
```

La classe **java.util.Scanner** disponible depuis java 5 permet le même traitement avec moins de code :

```
Scanner reader = new Scanner(System.in) ;  
int n = reader.nextInt() ;  
// Double d = reader.nextDouble() ;  
// String ch = reader.next() ;  
// String ligne = reader.nextLine() ;
```

reader.close(); //à ne pas appeler trop tôt .

Pour gérer des entrée plus complexe, on peut utiliser la classe *java.util.Formatter*, qui inclus des algorithmes à base de patterns.

IV - Héritage , polymorphisme , interface

1. Généralisation / Héritage

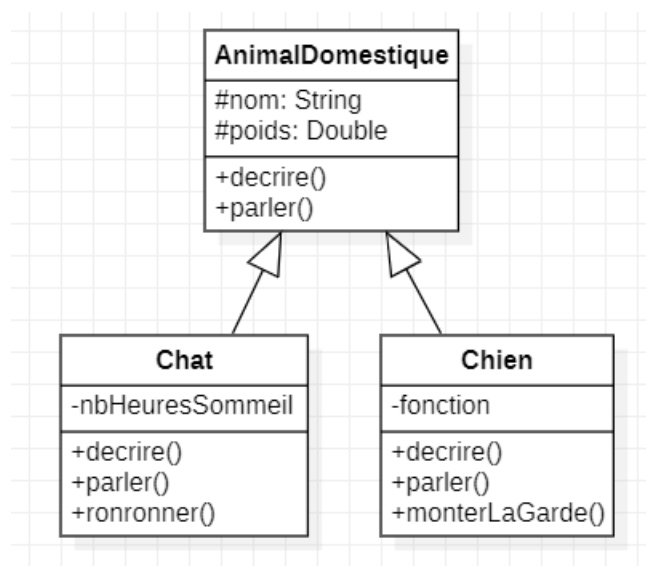
Lorsque l'on souhaite définir une **nouvelle classe dont une partie est déjà implémentée par une classe de base existante** et qui d'autre part doit comporter de **nouvelles spécificités**, il suffit alors de la faire **hériter** de la classe de base.

Dans le schéma conceptuel objet, l'**héritage correspond à un ajout de spécifications**. On parle de **spécialisation** (inversement vu comme une **généralisation**). Pour que l'héritage ait une signification correcte, on doit absolument pouvoir mentionner :

classe dérivée (sous classe) **est une sorte de** *classe de base (super classe)*

Remarque:

- Les relations d'héritage sont souvent représentées sous la forme d'un arbre.
- En UML, la relation **est une sorte de** est représentée par une **flèche** allant de la *classe dérivée (sous classe)* vers la *classe de base (super classe)*.



1.1. Syntaxe de l'héritage en JAVA:

```

public class NomClasseDérivée extends NomSurClasse
{
    ... // Attributs et méthodes supplémentaires
    ...// Opérations (Méthodes) redéfinies (même signature, nouveau code)
}
  
```

NB:

- Quand une classe entière est déclarée avec le mot clé **final**, elle ne peut plus être étendue (on ne peut plus en hériter). Ceci permet au compilateur d'effectuer quelques optimisations. `java.lang.System` constitue un exemple de classe finale. Depuis les versions 16,17 de java , le nouveau mot clef "**sealed**" (signifiant *scellé*) permet d'indiquer que seulement certaines classes auront le droit d'hériter d'une classe bien particulière (via le mot clef **permits**).
L'annexe sur les nouveautés de java 12à17 permet d'approfondir si besoin le le sujet.
- Si on ne définit pas de super-classe au moyen de la clause **extends**, le langage JAVA fournit une **sur-classe par défaut** appelé **Object**.
- La classe **Object** est ainsi une classe très spéciale dont toutes les autres classes JAVA héritent directement ou indirectement. La classe **Object** constitue *le sommet de l'arbre d'héritage* en JAVA.

Nouveau mot clef partiellement lié à la notion d'héritage: **protected** (protégé)

Tout comme `private` et `public` , le qualificatif **protected** permet de définir la **visibilité des membres d'une classe**. Les membres protégés seront accessibles depuis les méthodes de la classe courante ainsi que depuis toutes les méthodes des classes dérivées appartenant éventuellement à d'autres package (sous (sous) classes) mais resteront inaccessibles depuis l'extérieur (autres classes et autre package).

Exemple:

```
package tp;

public class AnimalDomestique {
    protected String nom;
    protected Double poids;//en kg

    public void decrirer(){
        System.out.println("AnimalDomestique de nom="+this.nom + " et de poids="+this.poids);
    }
    public void parler(){
        System.out.println("...");
    }

    public AnimalDomestique(String nom, Double poids) {
        this.nom = nom;    this.poids = poids;
    }

    public AnimalDomestique(){
        this("?",null);
    }

    public String getNom() { return nom; }
    public void setNom(String nom) { this.nom = nom; }
    public Double getPoids() { return poids; }
```

```

    public void setPoids(Double poids) {    this.poids = poids;    }
}

```

```

package tp;

public class Chat extends AnimalDomestique{

    private int nbHeuresSommeil;

    @Override
    public void parler() { System.out.println("miaou miaou"); }

    @Override
    public void decrire() {
        System.out.println("Je suis un chat qui dort " + nbHeuresSommeil + " heures par jour");
        System.out.print("\t"); super.decrire();
    }

    public void ronronner() { System.out.println("ronron"); }

    public Chat(String nom, Double poids, int nbHeuresSommeil) {
        super(nom, poids);
        this.nbHeuresSommeil = nbHeuresSommeil;
    }
    public int getNbHeuresSommeil() { return nbHeuresSommeil; }
    public void setNbHeuresSommeil(int nbHeuresSommeil) { this.nbHeuresSommeil =
nbHeuresSommeil;}
    ...
}

```

1.2. Utilisation d'une sous classe

```

public class MyApp {
    public static void main(String[] args) {
        AnimalDomestique a = null; //référence sur animal quelconque
        Chat c =null; // référence sur chat quelconque
        c=new Chat(); // instantiation de la classe dérivée

        double p = c.getPoids(); // appel direct d'une méthode héritée

        c.ronronner(); // appel de l'une des méthodes supplémentaires

        c.parler(); // appel de la nouvelle version de parler (spécifique aux Chats)

        a= c; // a référence un chat qui est un animal domestique particulier.
        Double p2 = a.getPoids();

        /* a.ronronner(); */ // fonction uniquement valable
        ((Chat) a).ronronner(); //sur le type de donnée Chat

        a.decrire(); // tient compte de la nature exacte de l'objet référencé (polymorphisme).
    }
}

```

```
}
}
```

1.3. Liaison entre une méthode redéfinie et la version héritée

Le mot clé **super** peut servir à appeler, au sein d'une méthode que l'on redéfinit, le code de la version liée à une super-classe.

Exemple:

```
public class Chien extends AnimalDomestique {
    private String fonction; //ex: "garder troupeau" , "guider aveugle" , "tenir compagnie"

    @Override
    public void decrire(){
        System.out.println("Je suis un chien qui a cette principale fonction: " + fonction );
        System.out.print("\t"); super.decrire(); //appel de decrire() de AnimalDomestique
    }
}
```

NB:

- Au sein d'une méthode redéfinie qui n'est pas un constructeur, on peut utiliser le mot clé **super** sur une ligne quelconque (pas obligatoirement la première)
- La construction suivante est illégale : **super.super.m1()**;
- **@Override** (*facultatif mais conseillé*) sert à vérifier que la méthode `decrire()` existe bien dans la super classe héritée "AnimalDomestique" avec la même signature (mêmes éventuels paramètres et même type de valeur de retour)

1.4. Constructeur d'une sous classe

Le constructeur d'une sous-classe peut appeler celui de sa super-classe en utilisant le mot clef **super**.

Exemple:

```
public class AnimalDomestique { ...
public AnimalDomestique(String nom, Double poids) {
    this.nom = nom;
    this.poids = poids;
}
}
```

```
public class Chien extends AnimalDomestique {
    private String fonction; //ex: "garder troupeau" , "guider aveugle" , "tenir compagnie"

    public Chien(String nom, Double poids, String fonction) {
        super(nom, poids);
        this.fonction = fonction;
    }
}
```

NB:

- De java 1 à 21 , le mot clef **super** ne pouvait être utilisé qu'en tant que *première ligne de code d'un constructeur* d'une sous-classe. Cette limitation n'a été levée qu'au sein de java 25.
- Si l'appel à **super** n'est pas explicitement mentionné, il est alors implicitement introduit sans

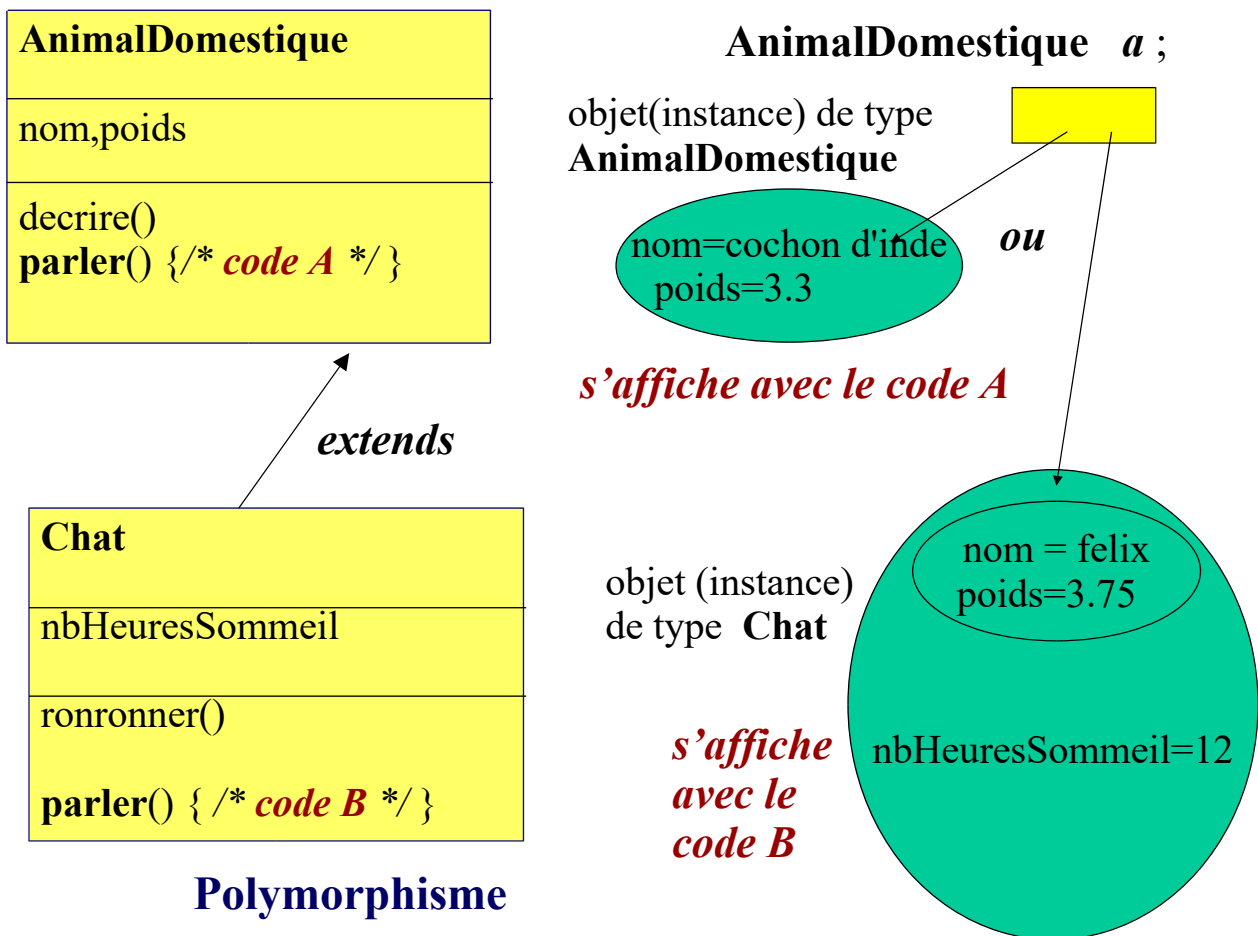
argument. Ce qui a pour effet d'appeler le constructeur par défaut (sans argument) de la surclasse. Il y a donc un ***enchaînement automatique des constructeurs*** : Toute construction d'un objet d'une classe dérivée implique l'appel (implicite ou explicite) du constructeur de la classe d'au dessus qui appelle à son tour le constructeur de sa surclasse, ...

1.5. Redéfinition de méthode & opérations abstraites

Précisions sur quelques notions fondamentales:

- **Toute nouvelle classe java correspond à un type de données.**
- **Le type de données associé à une sous classe est compatible (en tant que cas particulier) avec le type de données générique lié à la super classe.**
- Une fonctionnalité que l'on peut attendre d'un type d'objet correspond à une opération. Une opération est toujours mise en correspondance avec un nom de fonction. **Une opération a une signature précise** (type de retour et paramètres d'entrée bien précisés).
- **Une opération peut éventuellement être codée différemment dans diverses (sous) classes.**
- **La version exacte d'une opération au niveau d'une certaine classe est appelée méthode.** Une méthode est donc associée à du code bien précis.

2. Polymorphisme



Polymorphisme

- Lorsque la référence **a** pointe sur un objet de type **AnimalDomestique** (suite à une instruction du genre **a = new AnimalDomestique()** ou indirectement via une copie de référence), un appel à l'opération **parler()** déclenche le **code A** défini au niveau de la classe de base **AnimalDomestique**.
- Lorsque cette même référence **a** pointe sur un objet de type **Chat** (via **a = new Chat()**), un appel à l'opération **parler()** déclenche le **code B** défini au niveau de la classe **Chat**.
- Ainsi, une même ligne de code **a.afficher()**; peut déclencher plusieurs traitements légèrement différents suivant la nature exacte de l'objet qu'il y a au bout de la référence à un instant précis.

```
AnimalDomestique a =null;
a = new AnimalDomestique("cochon d'inde",2.9);
a.parler(); // ...
a = new Chat("felix",3.9);    //ou bien Chat c = new Chat(...); a=c ;
a.parler(); //miaou miaou
a = new Chien("medor",9.9);
a.parler(); //waouf waouf
```

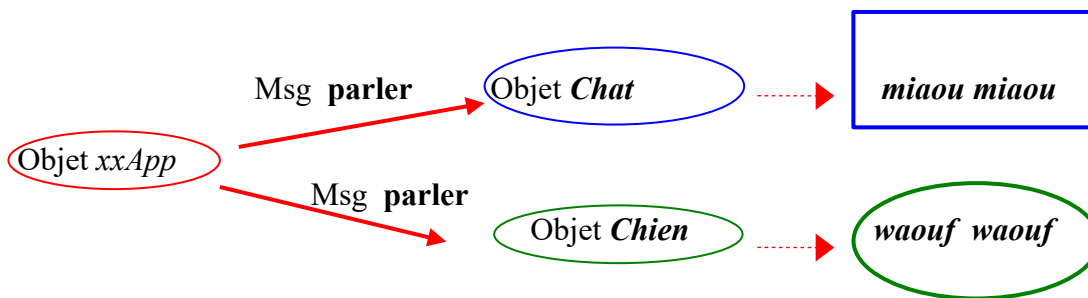
2.1. Liaison dynamique (formalisation du polymorphisme)

Lorsque l'on définit une référence dont le type est une classe de base (générique), cette variable "référence" peut très bien référencer une instance d'une sous classe ($a = c$ de l'exemple ci-dessus).

Conceptuellement, Appeler une opération **parler** à partir une référence **refObj** (instruction **refObj.parler();**) consiste à envoyer un message dénommé **parler** (ou *parle*) à l'objet référencé par **refObj**.

D'après le principe du **polymorphisme** que vérifie implicitement JAVA, on a alors le comportement suivant:

- Un même message envoyé vers divers objets peut déclencher des actions qui peuvent prendre plusieurs formes différentes.
- L'action (code exécuté) par l'objet (instance) recevant le message dépend de sa classe précise (celle qui a été utilisée pour le créer à coup de *new*).



NB: Lorsque le compilateur compile une ligne de code du type **refObj.parler();** il ne connaît pas encore la version exacte de *parler()* qui sera appelée car **refObj** peut référencer des instances issues d'une vaste panoplie de classes. C'est au moment de l'exécution du programme qu'est effectué le choix de la version de la méthode qui sera lancée. On parle de **liaison dynamique**.

3. Classes abstraites

- Une **classe abstraite** est une classe intermédiaire dans la partie haute d'un arbre d'héritage et qui permet de répertorier un ensemble d'opérations qui seront différemment codées dans les indispensables sous classes concrètes.
- Une **opération (méthode) abstraite** (déclarée avec le mot clé **abstract**) est une méthode sans code qui doit absolument être redéfinie dans les sous (sous) classes.
- Une classe contenant au moins une méthode abstraite est elle même abstraite et ne peut pas être directement instanciée (via le mot clef new).
- On peut tout de même déclarer une référence de type abstrait pour ensuite:
 1. référencer des instances issues de sous classes concrètes.
 2. appeler des opérations depuis la référence générique (polymorphisme).

Exemple:

```
public abstract class AnimalDomestique //classe abstraite
{
    private String nom; // véritable attribut
    ...
    public abstract void parler(); // opération (méthode) abstraite sans code
    public String getNom() { return nom; } // fonction non abstraite (avec code)
    public void setNom(String nom) { this.nom = nom; } // fonction non abstraite (avec code)
}
```

```
public class Chat extends AnimalDomestique // Chat = classe concrète
{
    ...
    public void parler() // méthode redéfinie
        { System.out.println("miaou miaou"); }
}
```

```
public class Chien extends AnimalDomestique // Chien = classe concrète
{
    ...
    public void parler() // méthode redéfinie
        { System.out.println("waouf waouf"); }
}
```

AnimalDomestique a; // variable générique pouvant référencer tout type d'objet graphique

~~**a = new AnimalDomestique();**~~ // new direct impossible car la classe est abstraite.

...

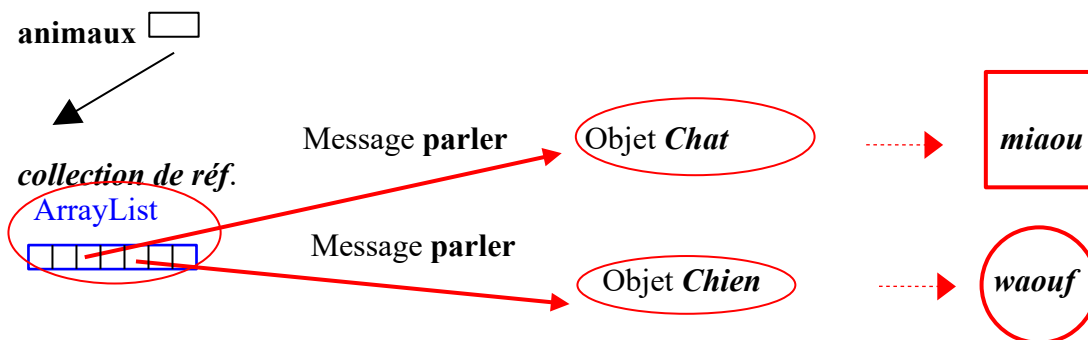
Chat **chat1** = **new Chat**("felix",3.9);

```

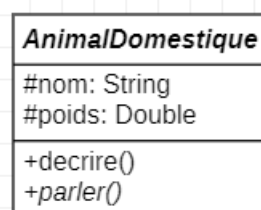
Chien chien1 = new Chien("medor",9.9);
...
a = chat1;
a.dessiner(g); // fait parler un chat (polymorphisme) , miaou miaou
a = chien1;
a.dessiner(g); // fait parler un chien (polymorphisme) , waouf waouf

List<AnimalDomestique> animaux = new ArrayList<>();
animaux.add(chat1);
animaux.add(chien1);
animaux.add(new Chat("malo", 5.2));
...
for(AnimalDomestique a : animaux)
    a.parler(); // fait parler chaque animal, ... (polymorphisme)
...

```



- *Bien que partiellement programmée, une classe abstraite est tout de même vue comme un type de données.*
- Au sein des diagrammes de classes UML , les opérations et classes abstraites sont représentées en caractères italiques:

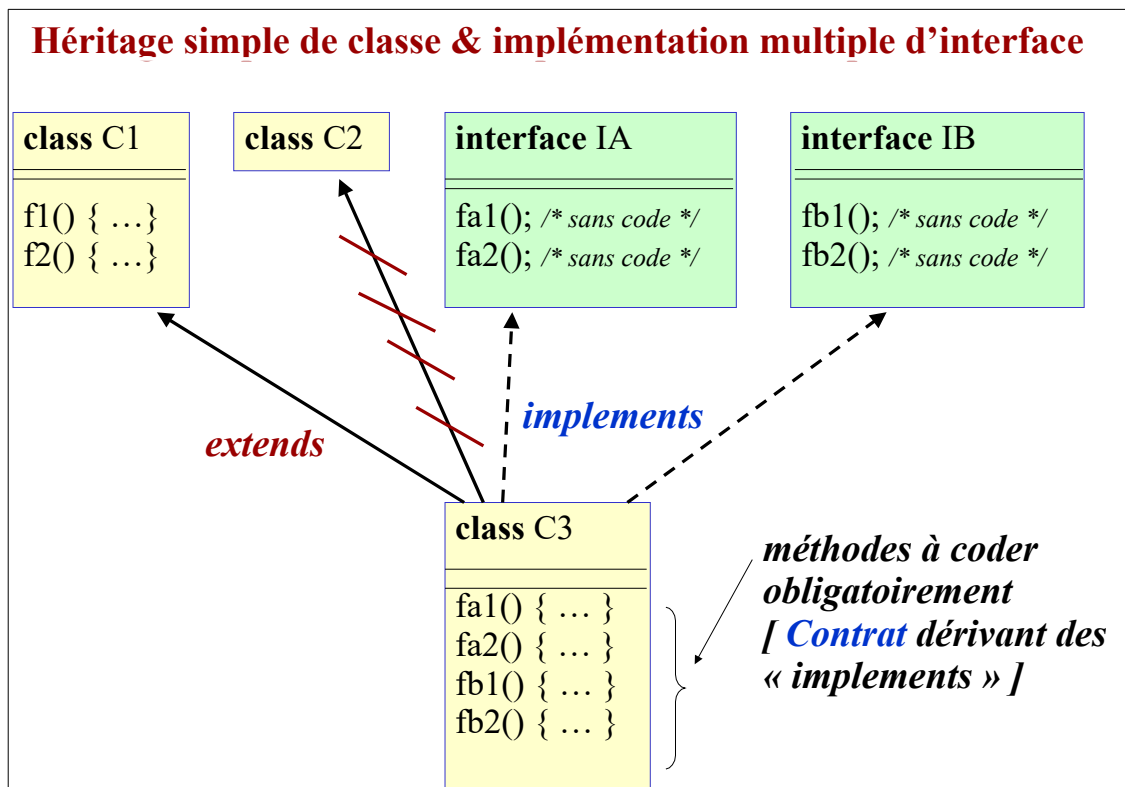


4. Interfaces

4.1. concept d'interface et syntaxe java

Une interface est une pseudo-classe qui ne comporte que des déclarations de méthodes sans code. Une interface ressemble beaucoup à une classe abstraite. Toutes les méthodes d'une interface sont implicitement abstraites.

Une classe JAVA ne peut avoir qu'une seule super-classe. L'héritage multiple d'implémentation n'est pas supporté par JAVA. Le langage *JAVA supporte par contre un héritage multiple d'interface*. Une classe JAVA peut hériter d'une classe concrète et d'un nombre quelconque d'interfaces (On dit qu'une classe étend une super-classe et implémente certaines interfaces).



Syntaxe----> `public class C3 extends C1 implements IA , IB { ... }`

Exemple:

```

public interface Localisable
{ // méthodes implicitement abstraites:
    public Point getCenter();
    public int getWidth();
    public int getHeight();
    // constante(s) directement visible(s) (sans préfixe) depuis les classes qui
    // implémenteront cette interface :
    public static final String DEFAULT_UNIT = "mm";
}
  
```

Nb: une interface publique se programme dans un fichier à part (ayant le même nom que l'interface).

Classe implémentant une interface:

```

public class CercleLocalisable extends Cercle implements Localisable
{ String unite ;
...
    public CercleLocalisable () { unite = DEFAULT_UNIT; ...}
    public Point getCenter() { return new Point(xc,yc); }
    public int getWidth() { return 2*rayon ; }
    public int getHeight() { return 2*rayon ; }
... }

```

NB: Une classe implémentant une interface est obligée de coder toutes les méthodes abstraites qui en découlent. Le compilateur Java détecte les oublis et génère des messages d'erreurs .

On peut déclarer une référence générique de type interface XXX, pour ensuite:

- Référencer une instance d'une classe implémentant cette interface.
- Appeler à partir de cette référence, l'une des méthodes redéfinies qui ont été déclarées au niveau de l'interface (*polymorphisme*).

Exemple:

```

List<Localisable> locObjects = new ArrayList<>(); // collection de choses localisables
CercleLocalisable c1 = new CercleLocalisable();
RectangleLocalisable r1 = new RectangleLocalisable();
... locObjects.add(c1);    locObjects.add(r1); ...
for(Localisable obj : locObjects)
    { ... obj.getCenter(); ...} // polymorphisme

```

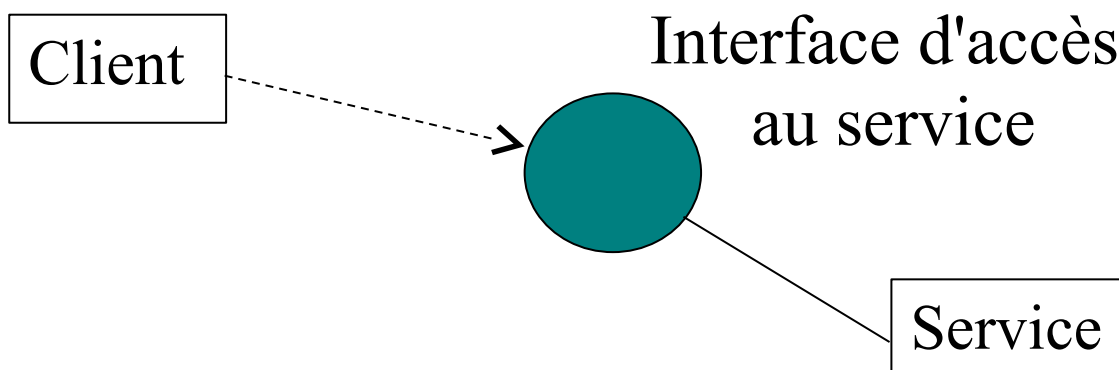
Héritage entre interfaces:

Une interface peut hériter d'une ou plusieurs interfaces existantes.

exemple: public **interface** Dynamique *extends* Deplacable, Zoomable {}

4.2. Sémantique (signification) des interfaces:

- ♦ Une **interface** peut être considérée comme un **contrat** car chaque classe qui choisira d'implémenter (réaliser) l'interface sera obligée de programmer à son niveau toutes les opérations décrites dans l'interface .
- ♦ Chacune de ces opérations devra être convenablement codée de façon à rendre le **service effectif** qu'un client est en droit d'attendre lorsqu'il appelle une des méthodes de l'interface.



Grâce à une interface, un morceau de code d'une API prédéfinie pourra activer des traitements au sein de nos propres classes sans préjuger de quoi que ce soit :

Notre classe maison pourra avoir un nom quelconque et pourra hériter de n'importe quelle classe. la seule chose imposée est d'implémenter les fonctions de l'interface.

Interface = contrat (liste de méthodes à coder quelque part)

Interface = type de données abstrait

- c'est une des clefs à bien maîtriser pour écrire des programmes modulaires.

La **logique événementielle** (qui sera vue dans un chapitre ultérieur) constituera un exemple concret d'utilisation des interfaces .

V - Éléments structurants de java

1. Propriétés du système et arguments du main

JAVA ne supporte pas directement les **variables d'environnement** du système car il est censé être indépendant de la plate-forme. **JAVA** propose en remplacement la notion de "**liste de propriété du système**":

Récupération d'une propriété au sein du programme:

```
String ModeDebug = System.getProperty("monAppli.Debug");
```

L'option **-D** de l'interpréteur **JAVA** permet de fixer une propriété sur la ligne de commande:

```
java -DmonAppli.Debug=true packagexxx.monAppli
```

Quelques propriétés prédéfinies:

user.home	home directory
java.home	répertoire d'installation de java
java.class.path	path pour trouver et charger les fichiers .class
os.name	nom du système d'exploitation hôte
line.separator	ex: "\n" sous Unix
path.separator	Ex: ":" sous Unix , ";" sous PC
user.name	nom de l'utilisateur courant
user.dir	répertoire courant de l'utilisateur

Arguments de la ligne de commande et valeur en retour :

```
public class MyEcho {
    public static void main(String argv[])
    {
        int argc = argv.length; // Nb: argv[0] correspond au premier argument
        for(int i=0;i<argc;i++)
            System.out.print(argv[i]+" ");
        if(...)
            System.exit(0); // code de retour dont la signification dépend de l'OS.
        ... }
}
```

Lignes typiques de commandes à placer dans un fichier .bat :

```
set JAVA_HOME=C:\Program Files\java\jdk17
set PATH=%JAVA_HOMEPATH%
set CLASSPATH=%CLASSPATHjava -classpath c:\rep\autreApi.jar -Dprop1=val1 -Dprop2=val2 package_p.MonAppli arg1
arg2 .... argn > ficRes.txt 2> ficErr.txt
```

2. Gestion des exceptions

- ♦ Une **exception** est une espèce de **signal** qui indique qu'un **événement exceptionnel (erreur, condition non vérifiée) est advenu**. Les exceptions de JAVA seront en fait des *instances des classes d'exception*.
- ♦ **Lancer ou lever une exception** consiste à **indiquer que quelque chose d'exceptionnel vient de se passer**.
- ♦ **Capter une exception** consiste à indiquer que l'on va la **gérer** (essayer de rattraper le coup ou afficher un message d'erreur significatif).

Les exceptions se propagent en remontant les appels de fonctions qui ont été effectués.

Si une exception n'est pas gérée par le bloc qui l'a lancée, on dit que le bloc est négligeant.

L'exception se propage alors vers le bloc (ou fonction) directement englobant. Si aucun des niveaux ne gère l'exception, l'interpréteur JAVA arrête alors le programme après avoir affiché un message d'erreur.

Intérêt du mécanisme:

Lorsqu'une fonction de bas niveau s'aperçoit qu'il y a un problème (fichier non existant, connexion impossible, ...) elle ne connaît généralement pas le contexte global de l'application et ne sait pas s'il faut:

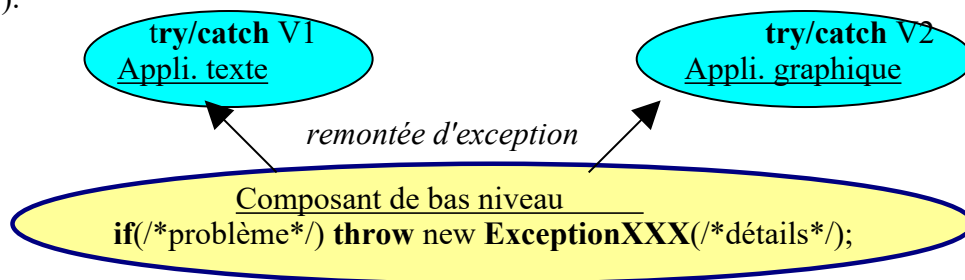
- Afficher un message d'erreur en mode texte (sur la console).
- Afficher un message en mode graphique dans une boîte de dialogue.
- Retourner l'erreur vers le client si l'erreur s'est produite coté serveur.
- Enregistrer le problème dans un fichier le log.

Tout traitement d'erreur en "dur" n'est donc jamais complètement satisfaisant dans tous les cas de figure et restreint donc le panel d'utilisation du composant de bas niveau.

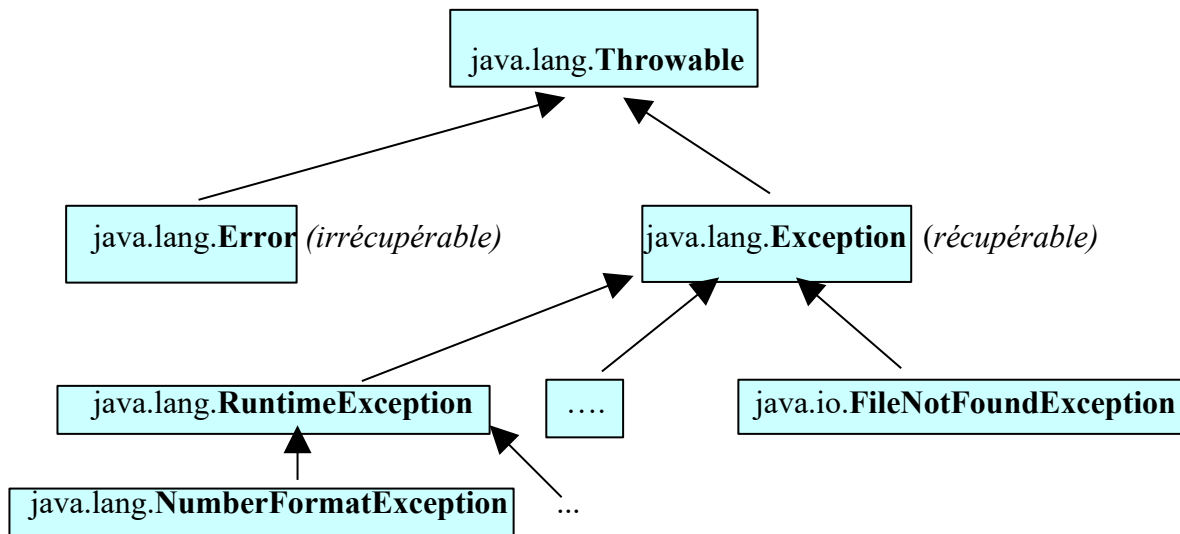
L'autre solution qui consiste à retourner systématiquement un code d'erreur (éventuellement du genre ERROR_SUCCESS) n'est pas non plus très élégant car il oblige le code appelant à systématiquement tester (à coup de if/else ou switch) la valeur de retour de la fonction de bas niveau pour savoir si tout s'est bien passé.

Le mécanisme de traitement des exceptions apporte la meilleure solution car il permet de remonter simplement une indication d'exception de la couche N (de bas niveau) vers la couche N+1 (de haut niveau). Cette remontée d'exception n'est déclenchée qu'en cas de problème (rien ne se passe quand tout se passe bien).

La couche N+1 (qui reçoit l'exception) connaît généralement mieux le contexte global de l'application et va pouvoir déclencher un traitement approprié (en mode texte, en mode graphique, ...).



2.1. Classes d'exceptions:



Remarques:

- Tout type d'exception dérivant de **Exception** peut être rattrapé au moyen d'un bloc try/catch
- Toute exception dont le type (ou sous type) n'est pas **RuntimeException** doit obligatoirement être gérée à un certain niveau pour que le code puisse être compilé.
- Par contre une exception dérivant de **RuntimeException** n'a pas à être systématiquement traitée (le try/catch est alors facultatif, on parle alors en terme de "**unchecked exception**")

2.2. Levée (lancement) d'une exception:

Le mot clé **throw** (conceptuellement équivalent à un return) permet de quitter une fonction dans un cas d'erreur en lançant une exception qu'il faudra alors rattraper au dehors.

if(/*problème*/)

```
throw new MyException("paramètre , contexte de l'exception");
```

MyException peut être l'une des nombreuses classes prédéfinies d'exception de JAVA (EOFException, ...) ou bien une nouvelle classe dérivant de `java.lang.Exception`.

2.3. Déclaration d'exceptions:

Une fonction (méthode) JAVA doit déclarer la liste des types d'exceptions qu'elle est susceptible de lever (ou de laisser remonter):

exemples:

```
public void open_file() throws IOException { ... }
```

```
public void mafonction(param1,...) throws MyException1, MyException2 { ... }
```

NB:

- Une fonction qui ne gère pas en interne certaines exceptions est dite **négligente** et ne pourra être compilée que si les exceptions non gérées à ce niveau sont déclarées après le mot clef **throws**.
- La partie **throws** d'un prototype de méthode permet d'**indiquer à un futur (autre) programmeur** quelles sont **les exceptions qui pourront éventuellement être déclenchées** lorsque ce code de bas niveau sera utilisé. Cette information est très utile pour savoir quels sont les "try/catch" nécessaires.

2.4. Gestion d'exceptions:

Les combinaisons possibles sont **try/finally** ou **try/catch** ou **try/catch/finally**

```
try {
... // Instructions sous contrôle. Comportement normal si tout se passe bien.
... // On sort définitivement de ce bloc en cas d'exception.
... // Si on tombe sur une instruction de type return on exécute alors le bloc finally
... // avant de sortir.
}

catch(NullPointerException e1)
{
... // Gestion de l'exception e1 qui est une instance de NullPointerException
... // ou d'une sous classe.
}
catch(NullPointerException e2)
{
System.err.print("Exception: " + e2.getMessage());
}
catch(Exception ex /* n'importe quelle autre exception */)
{
ex.printStackTrace(); // très pratique pour le debug
}

finally
{
... // Code exécuté dans tous les cas de figure (après fin normal des instructions,
... // après exception traitée ou pas, après un return du bloc try ).
}
```

NB: En cas d'exception , il est souvent utile de générer des lignes au sein de certains fichiers de logs.

NB2: Depuis le jdk 1.7 il est possible d'écrire

```
catch(NullPointerException e1, NullPointerException e2){ ..... }
```

2.5. Niveau intermédiaire – alternatives de traitement (exceptions)

```
... class MyApp {
...main(...)
{ Cxxx objX ...
  try{ objX.f1()}
  catch(Exception ex)
  {
    ex.printStackTrace();
  }
}
```

Remontée
d'exception

```
... class Cyyy {
...f1a(...) throws MyException
{...
  if(...)
    throw new MyException(...);
  ...
}
```

```
... class Cxxx {
...f1(...)
{ Cyyy objY ...
  try{ objY.f1a()}
  catch(MyException myex)
  {
    myex.printStackTrace();
    ... }
}

OU BIEN

... f1(...) throws MyException
{
  Cyyy objY ...
  objY.f1a();
}

OU BIEN ...

... catch(MyException myex)
{ myex.printStackTrace();
  throw new OtherException(...);}
}
```

Le niveau intermédiaire peut au choix:

- **traiter lui même l'exception** (via try/catch)
- **laisser remonter l'exception telle quelle sans la gérer** (sans try/catch mais throws)
- **traiter l'exception et en propager une nouvelle version** (throw new OtherException ...)

NB: La troisième solution est la plus appropriée dans un environnement n-tiers , car chaque niveau intermédiaire peut alors :

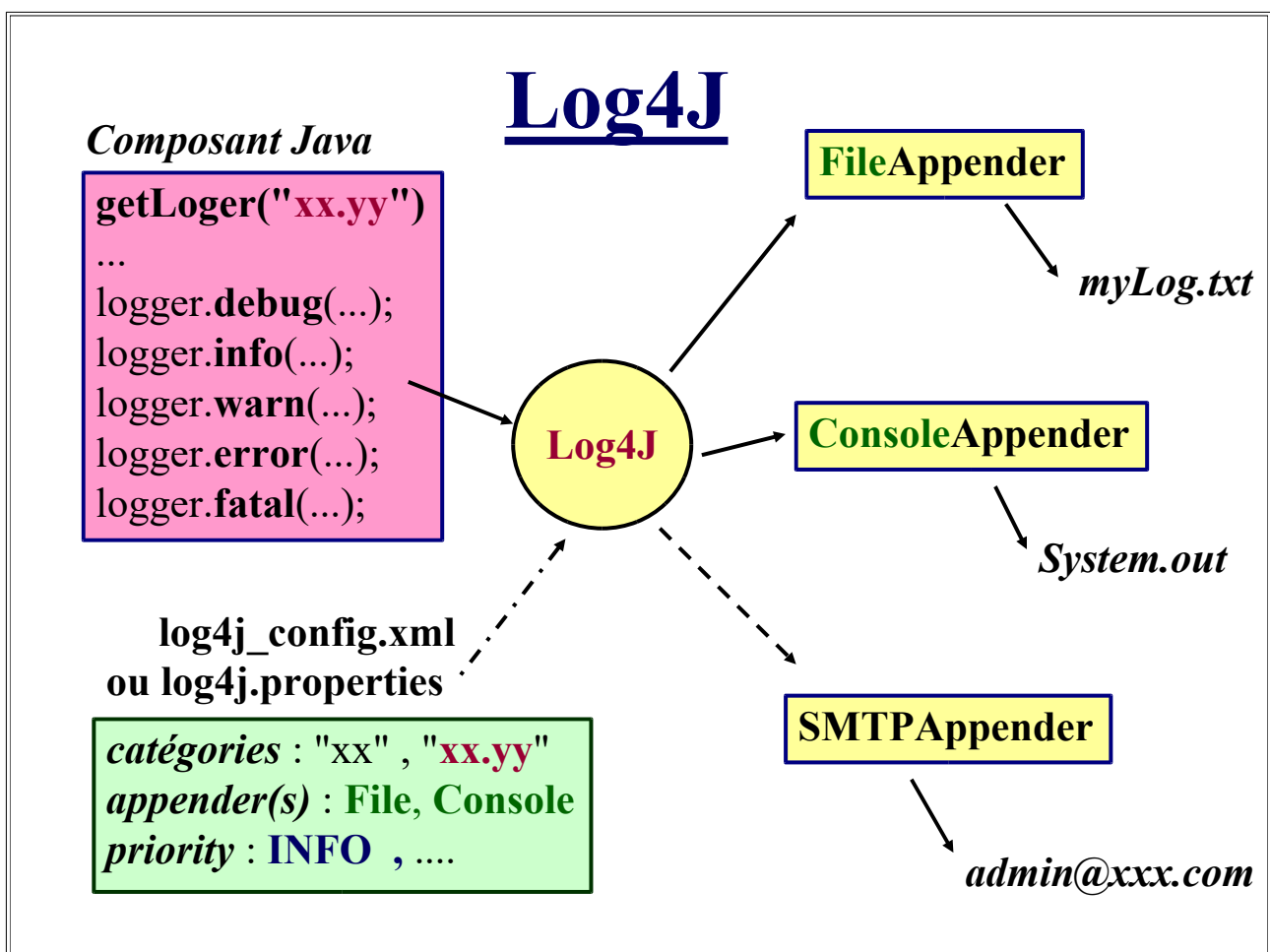
- générer des logs sur le problème vu localement (avec détails techniques)
- remonter une exception dans le format attendu par le niveau appelant

3. Présentation des API de log

3.1. Différentes API disponibles

- Historiquement , **Log4J** fut la première API de log efficace dans le mode Java .
- Le **JDK 1.4** a par la suite introduit un équivalent se voulant standard => *package* **java.util.logging**
- La communauté open source "*Apache*" a ensuite proposé une petite API chapeautant les 2 premières. L' API **org.apache.commons.logging** utilise en interne Log4J si présent ou bien **java.util.logging** du JDK1.4 sinon.
- Encore quelques années plus tard, est apparue une nouvelle Api "**slf4j**" (simple log façade for java) qui (comme commons-logging) est également une petite API de haut niveau délégrant à des sous API (ex: log4j , ...) . **slf4j** impose cependant un choix statique dès la construction de l'application et se montre plus fiable et plus performante que commons-logging. Pour faire "comme tous le monde" à partir de 2010 , il est grandement conseillé d'utiliser "**slf4j**" . Slf4j est par exemple utilisé en interne par les versions récentes de Spring et d'Hibernate .

3.2. Concepts communs



Via l'une de ces API , le développeur génère des logs (avec message et niveau de gravité) sans avoir

à connaître le fichier de destination ni les règles de filtrage.

Via un fichier de paramétrage (.xml ou .properties) , un administrateur peu effectuer quelques réglages fins:

- Niveau des lignes de logs à récupérer (ERROR, WARNING , INFO ,)
- Destination(s) (fichier texte , fichier xml, console , ...)

Principal avantage ==> pas de if(DEBUG) dans le code et donc pas de perte de temps CPU pour tenir compte des paramètres.

Will Output Messages Of Level		DEBUG	INFO	WARN	ERROR	FATAL
Logger Level	DEBUG					
	INFO					
	WARN					
	ERROR					
	FATAL					
		ALL				
		OFF				

NB : il existe maintenant (aversion récente) , le niveau **TRACE** encore plus élevé que DEBUG.

3.3. Génération des lignes de logs (slf4j)

NB: **log4j-slf4j2-impl** (ou des équivalents) doivent être ajouté au "CLASSPATH"

```
package tp;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
//@Slf4j
public class Essai_Slf4j {
    //ligne de code quelquefois généré par @Slf4j de lombok :
    private final Logger log= LoggerFactory.getLogger(Essai_Slf4j.class);

    public void doJob()
    {
        int x=1;          int y=0;
        try {
            int z=x/y;
        }
        catch(Exception ex) {
            log.error("error msg",ex); // log.fatal("Fatal Error");
        }
        log.warn("my warning");
        log.info("my info");
        log.debug("my debug");
        log.trace("my trace");
    }
}
```

...

3.4. Configurations de la sortie des logs (via slf4j et log4j2)

src/main/resources/log4j2.properties

```
# Extra logging related to initialization of Log4j
# Set to debug or trace if log4j initialization is failing
status = warn
# Name of the configuration
name = ConsoleLogConfigDemo

# Console appender configuration
appender.console.type = Console
appender.console.name = consoleLogger
appender.console.layout.type = PatternLayout
appender.console.layout.pattern = %d{yyyy-MM-dd HH:mm:ss} %-5p %c{1}:%L - %m%n

# RollingFileAppender name, pattern, path and rollover policy
appender.rolling.type = RollingFile
appender.rolling.name = fileLogger
appender.rolling.fileName= mylogs.log
appender.rolling.filePattern= mylogs_%d{yyyyMMdd}.log.gz
appender.rolling.layout.type = PatternLayout
appender.rolling.layout.pattern = %d{yyyy-MM-dd HH:mm:ss.SSS} %level [%t] [%l] - %msg%n
appender.rolling.policies.type = Policies

# Root logger level
#rootLogger.level = debug
rootLogger.level = warn
# Root logger referring to console appender
rootLogger.appenderRef.stdout.ref = consoleLogger

logger.xyz.name = tp.xyz
logger.xyz.level = debug
logger.xyz.appenderRef.stdout.ref = consoleLogger
logger.xyz.additivity = false
```

avec

```
<dependency>
  <groupId>org.apache.logging.log4j</groupId>
  <artifactId>log4j-slf4j2-impl</artifactId>
  <version>2.25.3</version>
</dependency>
```

4. Classes imbriquées (depuis jdk 1.1)

Les classes imbriquées (*Inner classes*) ont été introduites dans le JDK 1.1 pour pouvoir gérer relativement facilement les événements (voir chapitre AWT).

Exemples commentés:

Ce premier exemple montre que le niveau imbriqué peut directement accéder aux membres du niveau englobant:

```
public class ClasseContenante {
    public int x=1;
    private int y=2; // attribut privé de la classe de premier niveau

    private class ClasseImbrique {
        public int a=10;
        public int x=100; // attribut de la classe imbriquée

        public void aff()
        {
            System.out.println("x du niveau imbrique : " + x);
            //this correspond ici a l'objet de la classe imbriquée:
            System.out.println("this.a = " + this.a);
            //Le niveau imbriqué a directement accès aux membres de la
            //classe englobante (même s'ils y sont déclarés privés)
            System.out.println("y = " + y);
            affContenant(); // appel direct sur le niveau englobant
        }
    } // fin classe imbriquée

    private void affContenant() {
        System.out.println("---> fonction de la classe englobante :");
        System.out.println("x du niveau englobant : " + x);
    }

    public void essai() { ClasseImbrique obj = new ClasseImbrique();
                        obj.aff(); }
}
```

Dans ce second exemple très classique, la classe imbriquée n'a pas de nom et l'indication `java.awt.event.ActionListener()` qui précède le bloc entre `{ }` désigne ici le fait que la classe imbriquée (hérite ou) implémente l'interface `ActionListener` du package `java.awt.event` :

```
....
jButtonOK.addActionListener ( new
    /* début du code de la classe imbriquée (et anonyme) implémentant */
    java.awt.event.ActionListener()
    {
        public void actionPerformed(ActionEvent e)
        {
            jButtonOK_actionPerformed(e);
        }
    }
    /* fin du code de la classe imbriquée */
    );
....
```

====> classes de type `XXX$1.class`, `XXX$2.class` ... , une fois compilées.

5. Modules java 9+ (essentiel)

5.1. Problèmes et limitations en java 8

En Java 8 et antérieur, à l'exécution d'une application, la JVM recherche les classes utilisées par l'application dans :

- Les classes prédéfinies de la plate-forme Java : stockées dans l'**énorme** fichier **monolithique** `rt.jar` (*jusqu'à 53 Mo en Java 8 !*).
- Le **classpath** paramétré au lancement (un ensemble de chemins relatifs ou absolus vers des `.jars` ou des répertoires contenant des arborescences en package contenant des fichiers `“.class”`)

Selon d'éventuelles différences de contextes (ordinateurs différents, "jdk" différents, ...) , **le classpath n'est pas forcément le même à la compilation et à l'exécution** et cela peut engendrer `"java.lang.NoClassDefFoundError"` !!!!

En Java 8 et antérieur, à l'exécution d'une application, le **"classloader"** de la JVM a le comportement suivant :

- il charge les classes sur demande lors d'un `new` ou d'un appel à une méthode statique.
- il charge les classes linéairement (selon l'ordre des jars dans le classpath qui est donc très important) et s'arrête à la première classe correspondant au nom complet demandé.
- aucune vérification n'est faite à l'exécution sur l'existence de plusieurs occurrences d'une même classe.
- aucune vérification n'est faite au démarrage de l'application sur la présence de tous les jars / classes nécessaires au bon fonctionnement de l'application.
- le classpath est plat sans aucune notion de dépendances entre jar.

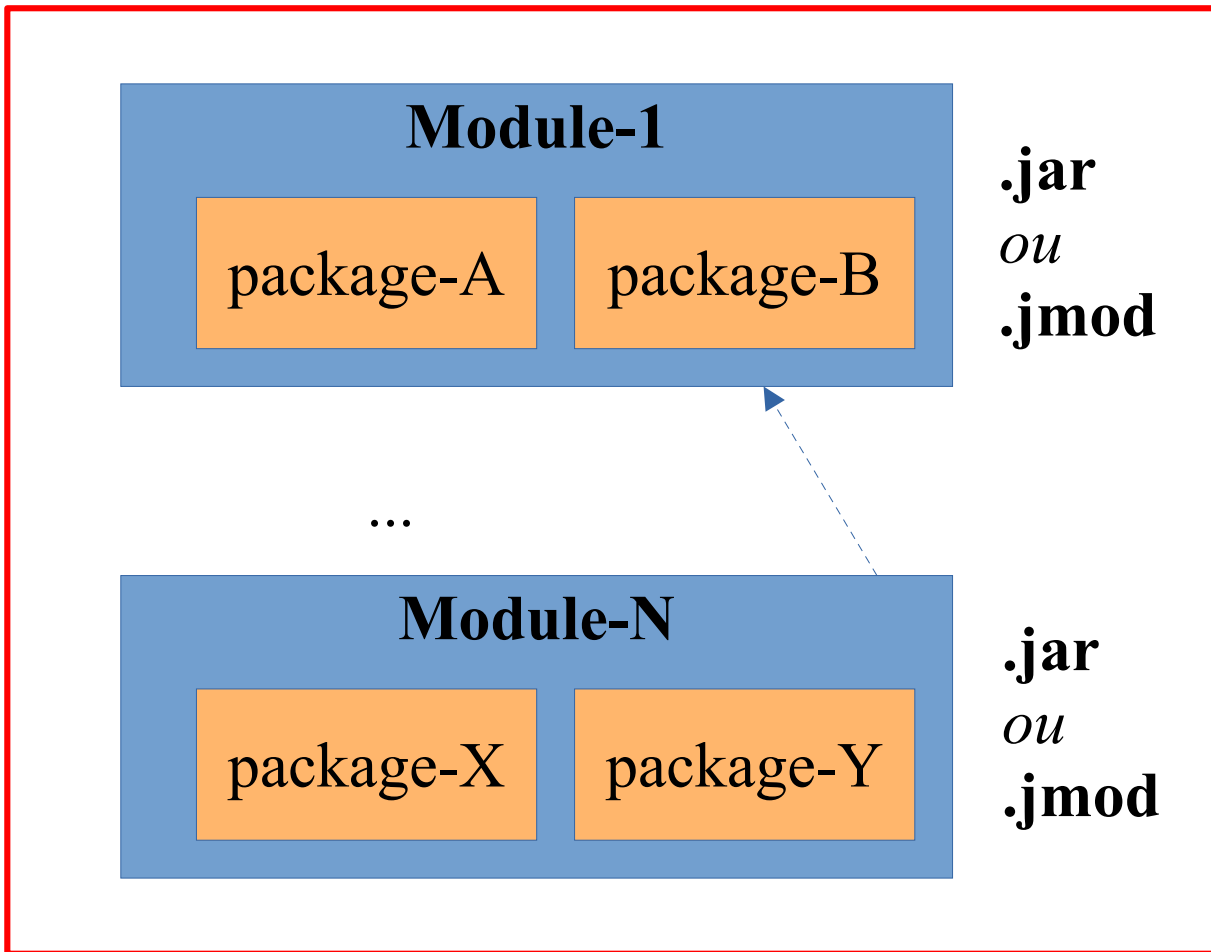
5.2. Projet JIGSAW (modularisation de la jvm depuis java >=9)

Principaux **apports** de la **modularisation** de la JVM (depuis V9) :

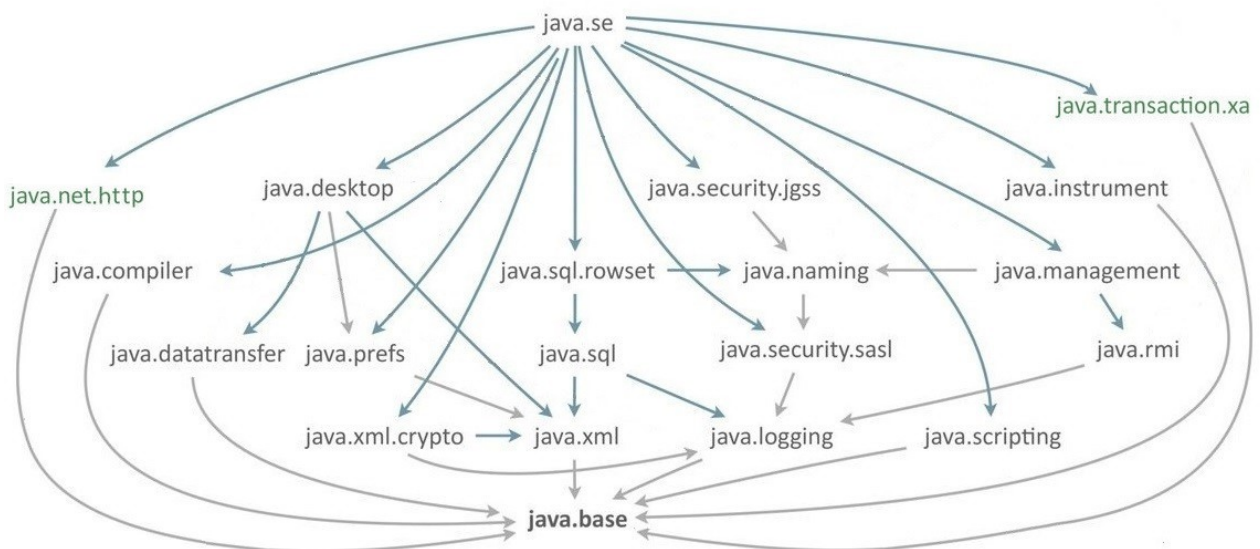
- **un classpath sous forme d'arbre de dépendances**
- **une vérification de la présence de tous les modules nécessaires à notre application au démarrage**, sans quoi l'application ne démarre pas.
- **un renforcement de la sécurité**, seuls les packages exportés explicitement par un module sont visibles par un autre
- la JVM elle-même est **modulaire** (*l'énorme fichier `rt.jar` n'existe plus depuis java 9*)
(NB : la taille du JRE a sensiblement diminué à partir de java 11)

NB : Un module "java >=9" est un **.jar** comportant le fichier ***module-info.java*** (compilé en *module-info.class*) à la racine du module .

Appli ou JVM modulaire depuis java 9



Exemple de graphe de dépendances entre modules d'un jdk récent :



Modules du jdk 11 :

Java > jdk-11.0.4 > jmods

Nom	Modifié le	Type	Taille
 java.base.jmod	19/08/2019 20:57	Fichier JMOD	18 021 Ko
 java.compiler.jmod	19/08/2019 20:57	Fichier JMOD	109 Ko
 java.datatransfer.jmod	19/08/2019 20:57	Fichier JMOD	50 Ko
 java.desktop.jmod	19/08/2019 20:57	Fichier JMOD	11 799 Ko
 java.instrument.jmod	19/08/2019 20:57	Fichier JMOD	116 Ko
 java.logging.jmod	19/08/2019 20:57	Fichier JMOD	110 Ko
 java.management.jmod	19/08/2019 20:57	Fichier JMOD	867 Ko
 java.management.rmi.jmod	19/08/2019 20:57	Fichier JMOD	87 Ko
 java.naming.jmod	19/08/2019 20:57	Fichier JMOD	436 Ko
 java.net.http.jmod	19/08/2019 20:57	Fichier JMOD	680 Ko
 java.prefs.jmod	19/08/2019 20:57	Fichier JMOD	53 Ko
 java.rmi.jmod	19/08/2019 20:57	Fichier JMOD	369 Ko
 java.scripting.jmod	19/08/2019 20:57	Fichier JMOD	44 Ko
 java.se.jmod	19/08/2019 20:57	Fichier JMOD	3 Ko
 java.security.jgss.jmod	19/08/2019 20:57	Fichier JMOD	638 Ko
 java.security.sasl.jmod	19/08/2019 20:57	Fichier JMOD	79 Ko
 java.smartcardio.jmod	19/08/2019 20:57	Fichier JMOD	57 Ko
 java.sql.jmod	19/08/2019 20:57	Fichier JMOD	74 Ko
 java.sql.rowset.jmod	19/08/2019 20:57	Fichier JMOD	184 Ko
 java.transaction.xa.jmod	19/08/2019 20:57	Fichier JMOD	4 Ko
 java.xml.crypto.jmod	19/08/2019 20:57	Fichier JMOD	635 Ko
 java.xml.jmod	19/08/2019 20:57	Fichier JMOD	4 380 Ko
 jdk.accessibility.jmod	19/08/2019 20:57	Fichier JMOD	484 Ko
 jdk.aot.jmod	19/08/2019 20:57	Fichier JMOD	274 Ko
 jdk.attach.jmod	19/08/2019 20:57	Fichier JMOD	38 Ko
 jdk.charsets.jmod	19/08/2019 20:57	Fichier JMOD	1 451 Ko
 jdk.compiler.jmod	19/08/2019 20:57	Fichier JMOD	6 904 Ko
⋮			
 jdk.crypto.cryptoki.jmod	19/08/2019 20:57	Fichier JMOD	305 Ko
 jdk.crypto.ec.jmod	19/08/2019 20:57	Fichier JMOD	148 Ko
 jdk.crypto.mscapi.jmod	19/08/2019 20:57	Fichier JMOD	61 Ko
 jdk.dynalink.jmod	19/08/2019 20:57	Fichier JMOD	159 Ko
 jdk.editpad.jmod	19/08/2019 20:57	Fichier JMOD	7 Ko
 jdk.hotspot.agent.jmod	19/08/2019 20:57	Fichier JMOD	2 298 Ko
 jdk.httpserver.jmod	19/08/2019 20:57	Fichier JMOD	98 Ko
 jdk.internal.ed.jmod	19/08/2019 20:57	Fichier JMOD	8 Ko
 jdk.internal.jvmstat.jmod	19/08/2019 20:57	Fichier JMOD	88 Ko
 jdk.internal.le.jmod	19/08/2019 20:57	Fichier JMOD	181 Ko
 jdk.internal.opt.jmod	19/08/2019 20:57	Fichier JMOD	80 Ko
 jdk.internal.vm.ci.jmod	19/08/2019 20:57	Fichier JMOD	399 Ko
 jdk.internal.vm.compiler.jmod	19/08/2019 20:57	Fichier JMOD	6 025 Ko
 jdk.internal.vm.compiler.manag...	19/08/2019 20:57	Fichier JMOD	12 Ko
 jdk.jartool.jmod	19/08/2019 20:57	Fichier JMOD	195 Ko
 jdk.javadoc.jmod	19/08/2019 20:57	Fichier JMOD	1 594 Ko
 jdk.jcmd.jmod	19/08/2019 20:57	Fichier JMOD	143 Ko
 jdk.jconsole.jmod	19/08/2019 20:57	Fichier JMOD	456 Ko
 jdk.jdeps.jmod	19/08/2019 20:57	Fichier JMOD	710 Ko
 jdk.jdi.jmod	19/08/2019 20:57	Fichier JMOD	841 Ko
 jdk.jdp.agent.jmod	19/08/2019 20:57	Fichier JMOD	121 Ko
 jdk.jfr.jmod	19/08/2019 20:57	Fichier JMOD	414 Ko
 jdk.jlink.jmod	19/08/2019 20:57	Fichier JMOD	388 Ko
 jdk.jshell.jmod	19/08/2019 20:57	Fichier JMOD	626 Ko
 jdk.jsobject.jmod	19/08/2019 20:57	Fichier JMOD	6 Ko
 jdk.jstatd.jmod	19/08/2019 20:57	Fichier JMOD	33 Ko
⋮			
 jdk.localedata.jmod	19/08/2019 20:57	Fichier JMOD	9 327 Ko
 jdk.management.agent.jmod	19/08/2019 20:57	Fichier JMOD	80 Ko
 jdk.management.jfr.jmod	19/08/2019 20:57	Fichier JMOD	34 Ko
 jdk.management.jmod	19/08/2019 20:57	Fichier JMOD	68 Ko
 jdk.naming.dns.jmod	19/08/2019 20:57	Fichier JMOD	57 Ko
 jdk.naming.rmi.jmod	19/08/2019 20:57	Fichier JMOD	19 Ko
 jdk.net.jmod	19/08/2019 20:57	Fichier JMOD	21 Ko
 jdk.pack.jmod	19/08/2019 20:57	Fichier JMOD	131 Ko
 jdk.rmic.jmod	19/08/2019 20:57	Fichier JMOD	516 Ko
 jdk.scripting.nashorn.jmod	19/08/2019 20:57	Fichier JMOD	2 143 Ko
 jdk.scripting.nashorn.shell.jmod	19/08/2019 20:57	Fichier JMOD	56 Ko
 jdk.sctp.jmod	19/08/2019 20:57	Fichier JMOD	23 Ko
 jdk.security.auth.jmod	19/08/2019 20:57	Fichier JMOD	73 Ko
 jdk.security.jgss.jmod	19/08/2019 20:57	Fichier JMOD	24 Ko
 jdk.unsupported.desktop.jmod	19/08/2019 20:57	Fichier JMOD	14 Ko
 jdk.unsupported.jmod	19/08/2019 20:57	Fichier JMOD	18 Ko
 jdk.xml.dom.jmod	19/08/2019 20:57	Fichier JMOD	42 Ko
 jdk.zipfs.jmod	19/08/2019 20:57	Fichier JMOD	87 Ko

5.3. option "--list-modules" de java

print_list_modules.bat

```
set JAVA_HOME=D:\Prog\java\jdk-14.0.1_windows-x64_bin\jdk-14.0.1
"%JAVA_HOME%\bin\java" --list-modules
pause
```

--> affiche la liste des modules systèmes de la JRE (et du JDK) , soit par exemple :

```
java.base@14.0.1
java.compiler@14.0.1
java.datatransfer@14.0.1
java.desktop@14.0.1
...
jdk.xml.dom@14.0.1
jdk.zipfs@14.0.1
```

5.4. option "--module-path" de java

L'option **--module-path** d'une ligne de commande java permet de spécifier le "module path". C'est une **liste de un ou plusieurs répertoires qui comportent des modules** .

Exemple :

prepare_modules.bat

```
cd /d "%~dp0"
REM cd ..
REM mvn package
REM cd /scripts
copy ..\mod_xx\target\mod_xx.jar .\my_modules
copy ..\mod_yy\target\mod_yy.jar .\my_modules
pause
```

launch_app_withmodules.bat

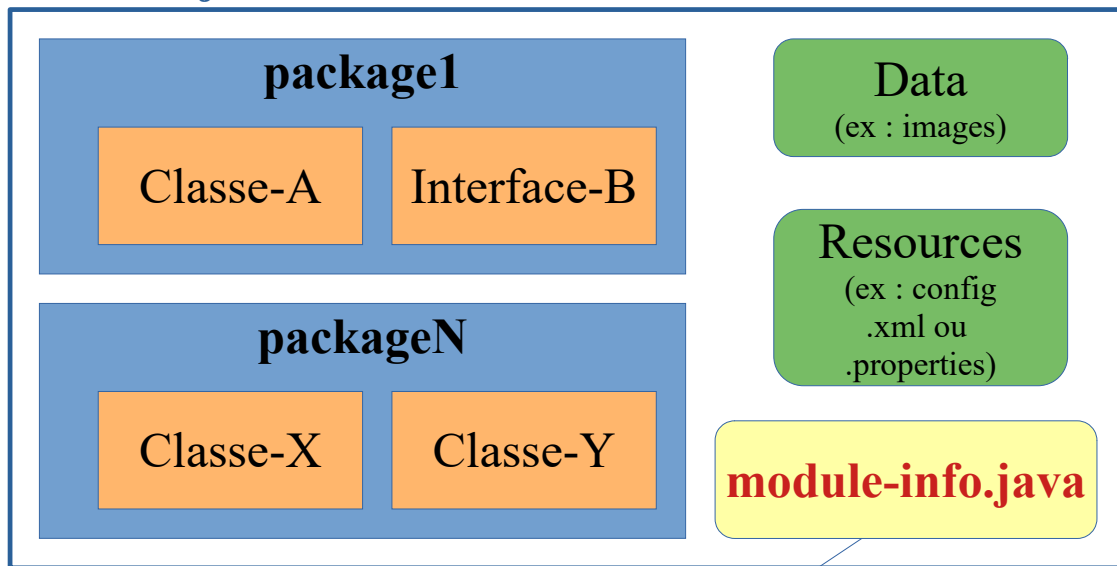
```
cd /d "%~dp0"
REM java --module-path modules_directory -m mainModuleName/mainClassFullName
java --module-path .\my_modules -m tp.module.modyy/tp.mod.mod_yy.app.MyApp
pause
```

5.5. Structure d'un module

Un module java (packagé en un ".jar" éventuellement renommé) est **l'association** de :

- un **paquet de packages java** (un sous package précis "xxx.yyy.zzz" ne peut être rangé que dans un seul module)
- un **paquet d'éventuelles ressources** (fichiers annexes : configuration_xml , images, ...)
- un fichier de configuration **module-info.java**

Module java 9+



*NB: si module-info.java pas spécifié , alors module automatique (exports * , requires *)*

```
module m2 {
    exports package1;
    requires m1;
}
```

5.6. Types de modules

Types de modules	Caractéristiques
System Modules	modules prédéfinis livrés avec jre/jdk (listés via java --list-modules)
Application Modules	modules applicatifs (.jar fabriqués avec module-info.class)
Automatic Modules	unofficial module (.jar fabriqués sans module-info.class) (with names based on ".jar" name) chargés depuis le --module-path . <i>Attention : sans module-info.class , ces modules ont par défaut un accès complet en lecture vers tous les autre modules chargés .</i>
Unnamed Module	" module fourre tout sans nom " comportant toutes les classes et packages chargés depuis des ".jar" via le --classpath (et pas le nouveau --module-path) -> pour récupérer tout les éléments codés en java <= 8 .

VI - Prog. Fonctionnelle (lambda, stream, ...)

1. Java multi-paradigme "objet + fonctionnel"

1.1. Evolution importante dès java 8

Les premières versions de java (1.0 à 1.7) étaient centrées sur le paradigme orienté objet .

A partir de la version 8 , le langage java s'est fortement inspiré du langage scala et est devenu un langage multi-paradigme "objet" et "fonctionnel" .

Sans renier ses fondements orientés objets, le langage java a introduit à partir de la version 8 un **nouveau style complémentaire de programmation** appelée "**programmation fonctionnelle**" .

1.2. Programmation fonctionnelle

Au sein d'un contexte de programmation purement orienté objet , les objets comportent des données dont les valeurs (à l'instant t) représentent un certain état .

En appelant une méthode sur un objet , il y a souvent un changement de valeur(s) effectué et donc un changement d'état .

Exemple :

```
p1 = new Personne("jean", "Bon", 40) ;// où 40 signifie ici 40 ans
p1.incrementerAge() ; //premier changement d'état (40 ans → 41 ans)
p1.incrementerAge() ; //second changement d'état (41 ans → 42 ans)
System.out.println("nouvel age de p1=" + p1.getAge()) ; //affiche 42
```

A l'inverse en appliquant le style de programmation fonctionnelle , on manipule avant tout des fonctions pures (sans contexte , sans état contextuel) qui permettent d'appliquer des transformations (ou des calculs) .

Exemple :

```
Personne p1 = new Personne("jean","Bon",40);
System.out.println("p1="+p1);
Function<Personne,Personne> avecUnAnDePlus ;
avecUnAnDePlus = (Personne p) -> new Personne(p.getPrenom(),
                                p.getNom(),
                                p.getAge() + 1);
Personne p1Bis = avecUnAnDePlus.apply(avecUnAnDePlus.apply(p1));
//NB: l'objet (de données) p1 n'est pas modifié.
//la double application de la fonction de transformation avecUnAnDePlus
//a permis de créer un nouvel objet p1Bis ayant 2 ans de plus que p1.
System.out.println("p1Bis="+p1Bis);//42 ans
```

NB : La syntaxe java suivante

```
avecUnAnDePlus = (Personne p) -> new Personne(p.getPrenom(),  
                                                p.getNom(),  
                                                p.getAge() + 1);
```

est nouvelle depuis java 8 et correspond à une *lambda expression* .

Ça correspond ici à une fonction admettant en entrée un objet de type *Personne* et retournant en retour un autre objet *Personne* .

1.3. Programmation fonctionnelle accrochée à un squelette objet

Etant donné que java est un langage avant tout orienté objet , les nouveaux éléments ayant un style "fonctionnel" doivent s' accrocher à un squelette de code un peu orienté objet pour pouvoir exister , être compilé et être exécuté par une machine virtuelle java.

2. Lambda expressions et prog. fonctionnelle

2.1. Interfaces fonctionnelles , "lambda expression" et références

Une **interface fonctionnelle** (que l'on peut facultativement explicitement marquer avec la nouvelle annotation `@FunctionalInterface` du `jdk >= 1.8`) est **une interface qui ne comporte qu'une seule méthode ordinaire** (sans "static" ni "default").

Son rôle est de permettre une gestion simple d'une certaine méthode/fonction abstraite .

(*SAM*= Single Abstract Method → interface de type "SAM")

Exemple (V1) :

```
package tp.langage.v8.sam;

/* L'annotation @FunctionalInterface (du jdk >= 1.8) est facultative
   elle permet au compilateur de vérifier que l'interface comporte une seule méthode (ordinaire) */
@FunctionalInterface
public interface SamPredicate<E> {
    boolean test(E e); //retourne true si l'entité e (de type E) satisfait certains critères
}
```

Exemple (V2) :

```
package tp.langage.v8.sam;

@FunctionalInterface
public interface SamPredicate<E> {
    boolean test(E e); //retourne true si l'entité e (de type E) satisfait certains critères
    default String getAuthor() { return "developpeur fou"; } //méthode par défaut
    // (alias "extension method" alias "defender method" ). Les méthodes par défaut ont permis
    // d'ajouter de nouvelles fonctionnalités à java8 sans remettre en question les interfaces java7
    static void methodeStatiqueAutoriseeSurInterfaceDepuisJava8(String msg){
        System.out.println(msg);
    }
    //les méthodes statiques au sein des interfaces permettent d'éviter la programmation de
    // nombreuses mini classes utilitaires périphériques
}
```

NB : `java.util.function.Predicate` (depuis java 8) correspond à un équivalent prédéfini de la version 1 de l'interface fonctionnelle `SamPredicate` .

Classe "Person" (pour la compréhension de l'exemple) :

```
...
public class Person {
    String firstname; //+get/set
    String lastname; //+get/set
    int age; //+get/set

    public Person(String firstname, String lastname, int age) { ...}
    public Person(){ }
    public String toString() {return "Person ("+  firstname + "," + lastname + ", age=" + age + ")"; }

    public static int compareP1P2ByAge(Person p1, Person p2) {
        if (p1.getAge() > p2.getAge()) {
            return 1;
        } else if (p1.getAge() < p2.getAge()) {
            return -1;
        } else {
            return 0;
        }
    }

    public int compareThisP2ByAge(Person p2) {
        if (this.getAge() > p2.getAge()) {
            return 1;
        } else if (this.getAge() < p2.getAge()) {
            return -1;
        } else {
            return 0;
        }
    }
}
```

Syntaxe lourde (java 7) avec classes anonymes imbriquées :

```
...
public class FilterSortListJava7TestApp {
    ...
    public static List<Person> extractSubListBySamPredicate(List<Person> pList,
                                                            final SamPredicate<Person> samPredicate) {
        final List<Person> sublist = new ArrayList<Person>();
        for (Person p : pList) {
            if (samPredicate.test(p)) {
                sublist.add(p);
            }
        }
        return sublist;
    }
    ...
    public static void mainWithInnerAnonymousSamPredicateImplementations() {
        List<Person> pList = StaticPersonList.initList();

        System.out.println("person sublist with age from 20 to 25:");
        List<Person> subList1 = extractSubListBySamPredicate(pList,
            new SamPredicate<Person>(){
                @Override
                public boolean test(Person p) {
                    return p.getAge() >= 20 && p.getAge() <= 25;
                }
            }
        );
    }
}
```

```

    });
    System.out.println(subList1);

    System.out.println("person sublist with lastName starting with letter p");
    List<Person> subList2 = extractSubListBySamPredicate(pList,
        new SamPredicate<Person>(){
            @Override
            public boolean test(Person p) {
                return p.getLastName().startsWith("p");
            }
        });
    System.out.println(subList2);
}
...
}

```

Syntaxe (très allégée) avec lambda expression depuis de jdk 1.8 :

```

package tp.langage.v8.lambda;
....
import java.util.function.Predicate;

public class FilterSortListJava8TestApp {
    ...
    public static List<Person> extractSubListByJava8Predicate(List<Person> pList, final
    Predicate<Person> predicate) {
        final List<Person> sublist = new ArrayList<Person>();
        for (Person p : pList) {
            if (predicate.test(p)) {
                sublist.add(p);
            }
        }
        return sublist;
    }
    public static void mainWithLambdaExpressions() {
        List<Person> pList = StaticPersonList.initList();

        System.out.println("person sublist with age from 20 to 25:");
        List<Person> subList1 = extractSubListByJava8Predicate(pList,
            (person) -> person.getAge() >= 14 && person.getAge() <= 25 );
        System.out.println(subList1);

        System.out.println("person sublist with lastName starting with letter p");
        List<Person> subList2 = extractSubListByJava8Predicate(pList,
            (person) -> person.getLastName().startsWith("p") );
        System.out.println(subList2);
    }
}

```

2.2. Lambda expressions

La nouvelle syntaxe **(arguments) -> corps de la fonction** apportée par le jdk 1.8 est appelée "**lambda expression**".

Derrière cette syntaxe très concise se cache un gros travail de déduction / résolution de la part du compilateur :

- fabrication automatique d'une implémentation respectant l'interface uni-fonctionnelle précisée par le contexte (ex : fonction englobante invoquée)
- mise en rapprochement des arguments de la "lambda expression" avec les paramètres d'entrées de la méthode abstraite (de l'interface fonctionnelle)
- utilisation du corps de la fonction précisé au sein de la "lambda expression" avec éventuelle interprétation contextuelle (lexicale) du mot clef this .
- ...

→ les types des arguments peuvent ainsi être déduits (et leurs compatibilités vérifiées) par le compilateur .

Exemples (variations syntaxiques) :

```
Collections.sort(pList,
(Person p1, Person p2) -> { return Integer.compare(p1.getAge() , p2.getAge()) ; })
```

ou bien avec return implicite (si une seule instruction dans { }) :

```
Collections.sort(pList, (Person p1, Person p2) -> Integer.compare(p1.getAge() , p2.getAge()) )
```

ou bien (avec déduction/inférence des types de p1 et p2 selon le contexte) :

```
Collections.sort(pList, (p1, p2) -> Integer.compare(p1.getAge() , p2.getAge()) )
```

Exemple dans son contexte :

```
public static void sortListComparatorInnerAnonymousImplementation() {
    List<Person> pList = StaticPersonList.initList();
    Collections.sort(pList, new java.util.Comparator<Person>(){
        @Override
        public int compare(Person p1, Person p2) {
            if (p1.getAge() > p2.getAge()) { return 1; }
            else if (p1.getAge() < p2.getAge()) { return -1; }
            else { return 0; }
        }
    });
    System.out.println(pList);
}
```

simplifié en

```
public static void sortListWithLambdaExpression() {
    List<Person> pList = StaticPersonList.initList();
    Collections.sort(pList,
        /* (Person p1, Person p2) -> p1.getAge() == p2.getAge() ? 0 : (p1.getAge() < p2.getAge() ? -1 : 1) */
        (Person p1, Person p2) -> Integer.compare(p1.getAge(), p2.getAge())
        /* (p1,p2) -> p1.getLastname().compareTo(p2.getLastname()) */
    );
    System.out.println(pList);
}
```

NB:

- D'un point de vue "typage de données / signature" , une lambda expression peut être vue par le compilateur comme un équivalent d'une implémentation d'une classe anonyme imbriquée implémentant une interface unifonctionnelle.
- D'un point de vue "performance/implémentation réelle" , une lambda expression est une construction spécifique du compilateur (qui est plus légère et plus performante qu'une classe anonyme imbriquée) . Conceptuellement, on est pas très loin d'un code "inline" du c++ ou d'une référence de fonction fléchée de javascript.

2.3. Référence de fonctions

NB : la (nouvelle) syntaxe **NomClasse::nomFonction** correspond à une **référence de fonction** .
→ les arguments et le corps de la fonction référencée sont alors utilisés de la même façon que ceux d'une "lambda expression" .

NB2 : une référence de fonction peut éventuellement **référer un "constructeur"** tel que le montre cet exemple :

```
....map(person -> new Student(person));
```

pouvant être ré-écrit en

```
...map(Student::new);
```

NB :

A la place d'une lambda expression à n paramètres en entrée, on pourra placer :

- soit une référence de fonction statique avec n paramètres en entrée
- soit une référence de fonction ordinaire (pas statique) avec n-1 paramètres en entrée sachant que lorsque cette méthode sera exécutée , le mot clef **this** sera utilisé en faisant office de premier paramètre d'un équivalent en forme lambda .

Exemple :

```
...
public class Person {
    ...
    public static int comparePIP2ByAge(Person p1, Person p2) {
        return p1.getAge() - p2.getAge() ;
    }

    public int compareThisP2ByAge(Person p2) {
        return this.getAge() - p2.getAge() ;
    }
}
```

```
public static void sortListWithFunctionReference() {
    List<Person> pList = StaticPersonList.initList();
    Collections.sort(pList, Person::comparePIP2ByAge);
    //Person::comparePIP2ByAge() have same code as java.util.Comparator.compare()
    //but with different name and without explicit interface implementation
    System.out.println("via static function reference, pList=" + pList);

    pList = StaticPersonList.initList();
    Collections.sort(pList, Person::compareThisP2ByAge);
    System.out.println("via no static function reference, pList=" + pList);

    System.out.println("liste triée par nom puis par prénom (with function reference:");
    pList.sort(Comparator.comparing(Person::getLastName)
                .thenComparing(Person::getFirstName));
    System.out.println(pList);
}
```

Bien qu'un peu limite d'un point de vue conceptuel , **une référence de méthode peut quelquefois être préfixée par une instance existante :**

nomInstance::nomMethode

est une syntaxe possible (à utiliser si besoin d'un point de vue "pratique" dans certains contextes).

2.4. Scope visibility in lambda expression :

```
public class Java8TestApp {
    private String message="Hello World (V8)";
    public static void main(String[] args) {
        (new Java8TestApp()).testRun();
    }
    public void testRun(){
        //lambda expression (compatible run()) utilisant "message" du scope parent:
        /* Runnable r1 = () -> System.out.println(this.message); */
        Runnable r1 = () -> System.out.println(message);
        Thread t1 = new Thread(r1); t1.start();
    }
}
```

NB : En langage **java** , une lambda expression est formulée par $(p) \rightarrow p.getXy() \geq this.seuil$

en langage **javascript** , une fonction fléchée est formulée par $(p) \Rightarrow p.xy \geq this.seuil$

Dans les 2 cas le mot clef **this** (utilisé dans une lambda) fait référence au contexte objet englobant.

2.5. Function, Predicate, Supplier, Consumer, ...

Avec un peu plus de recul et plus formellement :

Une **interface fonctionnelle** correspond à un **type (de traitement fonctionnel) abstrait**.

Une **"lambda expression"** (ou bien une référence de méthode) correspond à une **implémentation concrète d'une interface fonctionnelle**.

→ on peut écrire

```
Runnable r1 = () -> System.out.println("message");
```

D'autre part, le package **java.util.function** comporte un paquet d'**interfaces uni-fonctionnelles génériques** :

Function<T,R> - takes an object of type T and returns R.

Supplier<T> - just returns an object of type T.

Predicate<T> - returns a boolean value based on input of type T.

Consumer<T> - performs an action with given object of type T (no return)

BiFunction - like Function but with two parameters.

BiConsumer - like Consumer but with two parameters

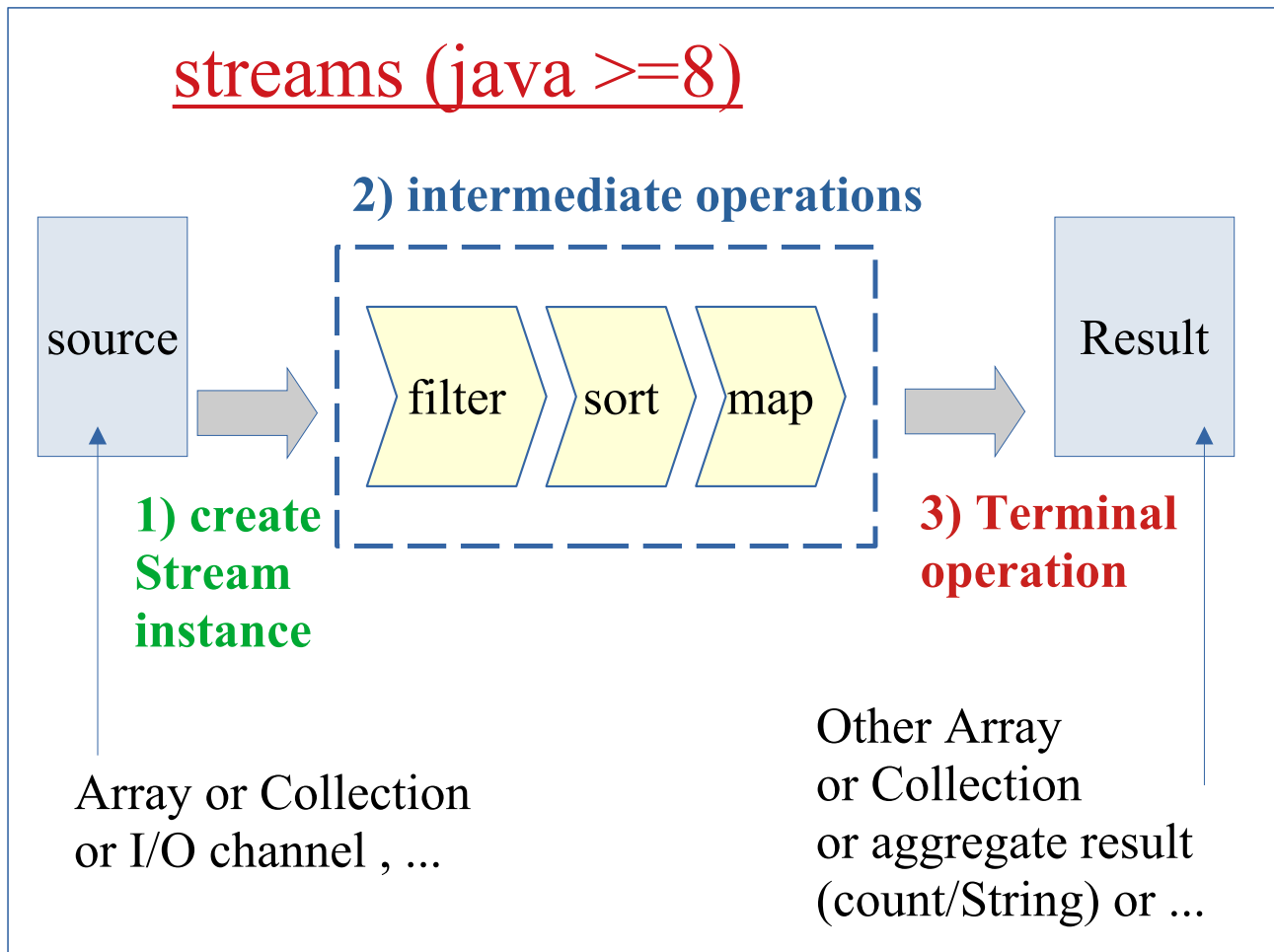
Interface Fonctionnelle	signature de la fonction	exemple
UnaryOperator <T>	T apply(T t)	String::toLowerCase
BinaryOperator <T>	T apply(T t1, T, t2)	BigInteger::add
Predicate <T>	boolean test(T t)	Collection::isEmpty
Function <T, R>	R apply(T t)	Arrays::asList
BiFunction <T, U, R>	R apply(T t, U u)	(x, y) -> x * y
Supplier <T>	T get()	Instant::now
Consumer <T>	void accept(T t)	System.out::println
autres (variantes , ...)	...	

Exemple d'utilisation (explicite et directe) :

```
List<String> liste = initListOfString();
Function<String, String> atrFct = (name) -> {return "@" + name;};
Function<String, Integer> lengthFct = (name) -> name.length() ;
//ou bien Function<String, Integer> lengthFct = String::length ;
for (String s : liste) {
    System.out.println( atrFct.apply(s) + " , length=" + lengthFct.apply(s));
}
```

L'utilisation des types fonctionnels abstraits et génériques (de **java.util.function**) est souvent effectuée indirectement en tant que paramètres des opérations (sort, filter, map, ...) sur les "Streams".

3. Streams (java 8 et plus)



NB: l'opération terminale `.collect(Collectors.toList())` disponible depuis java 8 peut syntaxiquement est simplifiée en `.toList()` depuis java 17 .

3.1. Stream et opérations (sort , filter , map , reduce , collect , ...)

`java.util.stream.Stream` (depuis le jdk 1.8) permet d'effectuer **des opérations de tri , filtrage , ... avec une syntaxique très concises sur des flux de données en vrac** (*bulk data operations in english*).

Exemple:

```
public static void mapWithStream(){ // map() is for "transformation"

    List<Person> persons = StaticPersonList.initList();
    Stream<Student> studentsStream = null;
    /*
    //v1 (explicit):
    studentsStream = persons.stream().filter(p -> p.getAge() <= 30)
        .map(new Function<Person, Student>() {
            @Override
            public Student apply(Person person) {
                return new Student(person);
            }
        });

    //v2 (with lambda expression):
    studentsStream = persons.stream().filter(p -> p.getAge() <=30 )
        .map(person -> new Student(person));
    */

    //v3 (with function/constructor reference):
    studentsStream = persons.stream().filter(p -> p.getAge() <=30 )
        .map(Student::new);

    System.out.println("liste of students:");
    studentsStream.forEach(System.out::println);

    //with collect() terminal operation (at end of operation stream) :
    List<Student> students = persons.stream().filter(p -> p.getAge() <=25 )
        .map(Student::new)
        .collect(Collectors.toList()); //or .collect(Collectors.toCollection(ArrayList::new));

    System.out.println("liste of (youngs) students:" + students);
}
```

```
List<Person> listePersonnesFiltreesTrieesEtTransformees =
    persons.stream()
    .filter( (p)->p.getAge()>=18 )
    .sorted( (p1,p2)->Integer.compare(p1.getAge(), p2.getAge()))
    .map( (p) -> { p.setLastname(p.getLastname().toUpperCase()); return p; } )
    .collect(Collectors.toList()); // ou bien directement toList() depuis java 17
```

```
public static void skipAndLimitOnStream(){
    final List<Integer> demoValues = Arrays.asList(1,2,3,4,5,6,7,8,9,10);
    //limit the input -> [1, 2, 3, 4] :
    System.out.println(demoValues.stream().limit(4).collect(Collectors.toList()));
    //jumping over the first 4 elements -> [5, 6, 7, 8, 9, 10] :
    System.out.println(demoValues.stream().skip(4).collect(Collectors.toList()));
}
```

```
public static void reduceStreamToASingleResult(){
    final List<String> demoValues = Arrays.asList("1_2_3","-4_5_6","-7_8_9");
    System.out.println("stream reduced to as single optional result: "
        + demoValues.stream().reduce(String::concat));

    final List<Integer> demoValues2 = Arrays.asList(5,8,12);
    System.out.println("stream reduced to as single result (somme des valeurs): "
        + demoValues2.stream().reduce(0 , (x,y) -> x+y));
    //.reduce(identity, accumulator) where identity is initialValue
    // (or default result if stream is empty)
}
```

```
public static void filteringWithStreamFilterAndLambda(){
    List<Person> pList = StaticPersonList.initList();
    pList.stream().filter( (p) -> p.getAge() >= 32 ).forEach( (p) -> System.out.println(p) );

    List<String> liste = initListOfString();
    //liste.stream().map((s)->s.toUpperCase()).forEach(System.out::println);
    liste.stream().map(String::toUpperCase).forEach(System.out::println);

    //.stream() for sequential operations , .parallelStream() for parallel operations
    //source of stream can be:
    //Stream.of(val1,val2,val3...) , Stream.of(array) and list.stream().
    //or final Stream<String> splitOf = Stream.of("A,B,C".split(", "));
```

```
}
```

Autres exemples :

```
String s = IntStream.iterate(1, i -> i * 2) //generation d'une sequence a priori infinie 1, 2, 4, ...
    .limit(10) //limitation a 10 iterations
    .mapToObj(String::valueOf) //transformation de chaque élément en String
    .collect(Collectors.joining(" ")); //collector rassemblant tout en une seule grande chaine
    //via des concatenations (en boucle) avec le separateur/jointeur spécifié
System.out.println(s); //affiche 1 ; 2 ; 4 ; 8 ; 16 ; 32 ; 64 ; 128 ; 256 ; 512
```

```
List<Product> listProd = ProductUtil.initSampleProductList();
Optional<Product> ofp=listProd.stream().filter(p -> p.getPrice() >= 200).findFirst();
System.out.println("premier produit trouve avec prix >= 200 : " + ofp.orElse(null));

boolean auMoinsUnProduitExistantQuiCommenceParPrinter =
    listProd.stream().anyMatch(p -> p.getLabel().startsWith("printer"));
if(auMoinsUnProduitExistantQuiCommenceParPrinter)
    System.out.println("au moins un des produits existants commence par printer");
```

```
double prixMaxi =listProd.stream()
    .map(p -> p.getPrice())
    .reduce(Double::max).orElse(0.0);
System.out.println("prixMaxi="+prixMaxi);
```

```
List<String> liste1 = Arrays.asList("rouge", "vert", "bleu");
List<String> liste2 = Arrays.asList("jaune", "orange", "bleu");
List<String> listeCouleurs = Stream.concat(liste1.stream(), liste2.stream())
    .distinct()
    .collect(Collectors.toList());
System.out.println("liste des couleurs (sans doublon)="+listeCouleurs);
```

→ liste des couleurs (sans doublon)=[rouge, vert, bleu, jaune, orange]

```
List <Point> points = Arrays.asList(new Point(1, 2),new Point(2, 4));
System.out.println("liste de points =" + points);
points.stream().forEach(p -> p.translate(10, 5));//forEach() = operation terminale (void)
    // p.translate() : void/ne retourne rien et modifie p
System.out.println("Apres translation(10,5) , liste de points =" + points);
```

liste de points =[java.awt.Point[x=1, y=2], java.awt.Point[x=2, y=4]]
 Apres translation(10,5) , liste de points =[java.awt.Point[x=11,y=7], java.awt.Point[x=12,y=9]]

VII - Classes utilitaires , aspects divers

1. Quelques anciennes classes utilitaires

1.1. Générateur de nombre aléatoire – classe Random

```
Random generateur = new Random( ); //initialisé avec le temps courant par défaut.
entierAleatoire = generateur.nextInt(); // variante : generateur.nextInt(max_exclusive_value);
reelAleatoire = generateur.nextDouble(); // de 0.0d (inclusive) à 1.0d (exclusive)
...
```

1.2. Gestion des instants (TimeStamp) – classe Date

```
Date d = new Date(); // Constructeur par défaut → Date d'aujourd'hui.
Date d2 = new Date(nbMsEcouleesDepuis01_01_1970);
//Date d3 = new Date(annee,mois,jour); /*deprecated */
//Date d4 = new Date(annee,mois,jour,heure,min,sec); /*deprecated */
long nbMsEcouleesDepuis01_01_1970 = d.getTime();
d.setTime(nbMsEcouleesDepuis01_01_1970 );

System.out.println("date = " + d.toString() );
//System.out.println("date GMT = " + d.toGMTString() ); /*deprecated */
//System.out.println("jour du mois = " + d.getDay() ); /*deprecated */ // getX() ↔ setX(x)
//System.out.println("mois = " + d.getMonth() ); // de 0 à 11 /*deprecated */
//System.out.println("année = " + d.getYear() ); /*deprecated */
//System.out.println("heure=" + d.getHours() + "/" + d.getMinutes() + "/" +
d.getSeconds());/*deprecated */
```

Remarque: une grande partie des méthodes de la classe Date est aujourd'hui obsolète (pas d'internationalisation, problème sur le calendrier). Il est donc fortement conseillé d'utiliser conjointement les classes annexes java.util.Calendar et java.text.DateFormat.

La classe **Date** devrait normalement n'être utilisée que pour stocker une valeur de type "Date + Heure (TimeStamp)" (exprimée en **ms**).

D'autre part , les classes **LocalDate** , **LocalTime** , **LocalDateTime** apparues depuis la version 8 de java sont des alternatives très conseillées vis à vis de **java.util.Date** considérée de plus en plus comme obsolète .

1.3. gestion des dates du calendrier - classe Calendar

`Calendar cal = Calendar.getInstance();` //pas de new (*Calendar* = classe abstraite)
 // la méthode de classe `getInstance()` retourne généralement une instance de la sous classe
 // concrète *GregorianCalendar*.

L'instance créée par `Calendar.getInstance()` comporte la date d'aujourd'hui et l'heure actuelle comme valeurs par défaut.
 Pour changer certaines valeurs, il faut utiliser une des méthodes `set(...)`.

```
cal.setTime(date /* de type Date et comportant nbMsEcouleesDepuis01_01_1970 */)
cal.set(int annee,int mois,int jourDuMois);
cal.set(int annee,int mois,int jourDuMois,int heure,int minute,int second);
```

Remarque importante: chaque variante d'utilisation de la méthode `get(int field)` ci-dessous est associée à une variante `set(int field,int value)` pour fixer la valeur du champ.

```
int seconde = cal.get(Calendar.SECOND); // de 0 à 59
```

```
int minute = cal.get(Calendar.MINUTE); // de 0 à 59
```

```
int heure = cal.get(Calendar.HOUR_OF_DAY); // de 0 à 23
```

```
int jourDuMois = cal.get(Calendar.DAY_OF_MONTH); // de 1 à 31
```

```
int mois = cal.get(Calendar.MONTH);
// Le numéro du mois va de 0 (Calendar.JANUARY) à 11 (Calendar.DECEMBER)
```

```
int annee = cal.get(Calendar.YEAR);
```

```
int jourDeLaSemaine = cal.get(Calendar.DAY_OF_WEEK);
// le jour de semaine va de 1(Calendar.SUNDAY) à 7(Calendar.SATURDAY)
```

```
int semaineDansAnnee = cal.get(Calendar.WEEK_OF_YEAR); // de 1 à 52
```

```
Date date /*instant*/ = cal.getTime();
```

Remarque:

Pour afficher une date sous forme de chaîne de caractères , il faut utiliser conjointement la classe **DateFormat** du package **java.text** (voir paragraphe suivant)

2. Internationalisation

La classe **Locale** permet la récupération automatique des éléments spécifiques à l'aspect "régional" du poste utilisateur (langue, symbole monétaire ,).

Cette classe est bien souvent automatiquement utilisée par beaucoup d'autres classes (exposées ci-après):

Un programme devant fonctionner dans différentes langues devrait récupérer les libellés de la façon suivante:

```
ResourceBundle myResources = null;
myResources = ResourceBundle.getBundle("mypackage.MyResources");
String libelle = myResources.getString("msg.welcome");
```

La méthode statique **ResourceBundle.getBundle("mypackage.MyResources")** va rechercher une classe vérifiant les points suivants:

- de nom *mypackage.MyResources_xx* où xx est le code du pays local (ex: *MyResources_fr* , *MyResources_de* , ...)
- héritant de **ResourceBundle** (ou d'une de ses sous classes **ListResourceBundle** , ...).

version par défaut (utilisée si MyResources_fr n'est pas trouvée):

```
public class MyResources extends java.util.ListResourceBundle {
    public Object[][] getContents() {
        return contents;
    }
    static final Object[][] contents = {
        {"msg.welcome", "welcome"},
        {"msg.goodbye", "goodbye"},
    };
}
```

version française:

```
public class MyResources_fr extends java.util.ListResourceBundle {
    public Object[][] getContents() {
        return contents;
    }
    static final Object[][] contents = {
        {"msg.welcome", "bienvenue"},
        {"msg.goodbye", "au revoir"},
    };
}
```

NB: Si la classe *mypackage.MyResources* n'est pas trouvée dans le CLASSPATH , la méthode **ResourceBundle.getBundle("mypackage.MyResources")** recherche alors un **fichier texte** de nom **MyResources.properties** (ou *MyResources_fr.properties* ou ...).

Ces fichiers **".properties"** sont également à placer au sein du sous répertoire "mypackage" situé au niveau du CLASSPATH.

Exemple: MyResources.properties

```
msg.welcome=welcome
msg.goodbye=goodbye
```

MyResources_fr.properties

```
msg.welcome=bienvenue
msg.goodbye=au revoir
```

3. Mise en forme du texte (java.text)

Le package **java.text** comporte des classes permettant de mettre en forme textuelle des choses diverses (nombres , dates ,) .

3.1. DateFormat et SimpleDateFormat

```
DateFormat df = DateFormat.getDateInstance(DateFormat.LONG);
String chDate = df.format(date); Date d = df.parse(chaineDate);
DateFormat tf = DateFormat.getTimeInstance(DateFormat.LONG);
String chHeure = tf.format(date);
DateFormat dtf = DateFormat.getInstance(DateFormat.LONG);
String chDateEtHeure = dtf.format(date);

SimpleDateFormat simpleDateFormat = new SimpleDateFormat("yyyy-MM-dd");
String sdate = simpleDateFormat.format(new Date());
```

3.2. MessageFormat

La classe MessageFormat comporte une fonction statique essentielle dénommée format qui permet de générer une chaîne de caractères à partir d'un format générique au sein duquel certains éléments spéciaux ({0} , {1} , {2} , ...) sont automatiquement remplacés par les éléments d'un tableau de valeurs passé en paramètre. Chaque élément du tableau doit être de type Object et comporte donc une méthode toString() qui va bien.

Remarque importante : dans le format générique , il faut doubler les caractères ' pour qu'il soient interprétés comme de véritables simples quotes.

Exemples:

```
Object tabVal[] = new Object[3];

tabVal[0]=new Integer(2);
tabVal[1]= "oeufs"; tabVal[2]="poule";
String chRes = java.text.MessageFormat.format("La {2} a pondue {0} {1}.\n", tabVal);

tabVal[0]="Dupond";
tabVal[1]= new Integer(2);
String chRes2 = java.text.MessageFormat.format(
    "Select * from tableX where nom='{0}' AND num={1}", tabVal);
```

double simple quote
(deux fois ')

3.3. classe abstraite NumberFormat et sous classe DecimalFormat

Exemple : Mise en forme d'un nombre réel avec seulement deux chiffres après la virgule:

```
NumberFormat nfmt = NumberFormat.getInstance();
nfmt.setMaximumFractionDigits(2);
nfmt.setMinimumFractionDigits(2);
double x=246.78934;
System.out.println("La valeur de x est : " + nfmt.format(x));
```

4. Optional , DateTime , ...

4.1. Optional<T>

`java.util.Optional` (depuis le jdk 1.8) correspond à un **conteneur (jamais "null") de valeur "objet" optionnelle** . Ceci un sémantiquement plus précis qu'une valeur "null" .

Depuis java 8 , le **type de retour** `Optional<T>` plutôt que T éventuellement null ne fait qu'**expliquer clairement** (via le type , sans autre commentaire ou documentation supplémentaire) **l'aspect optionnel d'une référence vers un objet en valeur de retour**

Cela pourrait correspondre à la **multiplicité 0..1** en UML mais malheureusement `Optional<T>` n'est prévu que pour n'être utilisé qu'au niveau des types de retour (pas sur un type d'attribut/propriété).
Principale raison : `Optional<T>` n'est pas bien gérée par la sérialisation JSON (valeur null préférée) ni par la persistance automatique de JPA/Hibernate (valeur null préférée).

Autrement dit, **`Optional<T>` est surtout utile pour éviter un `NullPointerException` en plein milieu d'un enchaînement de traitements via les streams de java >=8 .**

Exemple :

```
...
public static List<String> initWinnerList(){
    String[] winnerArray = {"bob", "anna", "alice"};
    List<String> winnerList = Arrays.asList(winnerArray);
    return winnerList;
}
public static Optional<String> getWinnerByName(String name){
    for(String s : initWinnerList()){
        if(s.equals(name))
            return Optional.of(s);
    } /*else*/
    return Optional.empty(); //instead of return null
}
public static void testOptional(){
    Optional<String> opStr = getWinnerByName("alice");
    System.out.println("Optional<String> opStr = " + opStr );
    if(opStr.isPresent())
        System.out.println(opStr.get());
    //with lambda expression:
    opStr.ifPresent( (s) -> System.out.println(s) );
}
```

```

opStr = getWinnerByName("looser");
System.out.println("Optional<String> opStr = " + opStr );
//System.out.println(opStr.get());
//java.util.NoSuchElementException instead of NullPointerException
opStr.ifPresent((s)->System.out.println(s));

//String str = "abc";
String str = null;
Optional<String> opS = Optional.ofNullable(str); //build .empty() if null
System.out.println("Optional<String> opS = " + opS );

String msg = opS.map((notNullStr)-> "not null string value : " + notNullStr)
                .orElse("empty optional");
System.out.println(msg);
}

```

NB :

optional.get(); retourne la valeur interne (non nulle) ou bien retourne une **exception** si vide/empty.

optional.orElse(null); retourne la valeur interne (non nulle) ou bien retourne **null** si vide/empty.

```

public static void testDiversAvecNouvellesMethodesDeOptionalDepuisJava9() {
    Optional<String> opS1 = Optional.of("s1");
    Optional<String> opS2 = /*Optional.ofNullable(null); */ Optional.empty();

    //op.or(...) permet de construire et fournir un autre optional (valeur par défaut
    // ou plan B) si l'optionnel original est null/empty :
    System.out.println(opS1.or(()->Optional.of("default_string"))); //affiche Optional[s1]
    System.out.println(opS2.or(()->Optional.of("default_string")));
    //affiche Optional[default_string]

    //op.ifPresentOrElse(nomEmptyConsumer_as_lambda , emptyActionLambda)
    //permet de déclencher alternativement une lambda ou une autre en fonction d'un
    // contenu vide ou pas , pas d'enchaînement après (opération terminale en "void")
    opS1.ifPresentOrElse( (value) -> System.out.println("opS1="+value),
                        () -> System.out.println("value of opS1 is empty"));
    //affiche opS1=s1
    opS2.ifPresentOrElse( (value) -> System.out.println("opS2="+value),
                        () -> System.out.println("value of opS2 is empty"));
    //affiche value of opS2 is empty
}

```

4.2. LocalTime , LocalDate (de java.time)

java.time.**LocalTime** , **LocalDate** sont plus simples à utiliser que **Date** et **Calendar**

Ces classes (rangées dans le package *java.time*) **comportent également beaucoup plus de fonctionnalités** (comparaisons de dates , calculs d'écarts , ...).

Exemple :

```
public static void testLocalTimeAndLocalDate(){
LocalTime now = LocalTime.now();      System.out.println("now is " + now);
LocalTime later = now.plus(8, ChronoUnit.HOURS);
System.out.println("later (now+8h) is " + later);

LocalDate today = LocalDate.now();      System.out.println("today is " + today);
LocalDate thirtyDaysFromNow = today.plusDays(30);
System.out.println("thirtyDaysFromNow is " + thirtyDaysFromNow);

LocalDate nextMonth = today.plusMonths(1); System.out.println("nextMonth is " + nextMonth);
LocalDate aMonthAgo = today.minusMonths(1); System.out.println("aMonthAgo is " + aMonthAgo);

String dateString = "25/02/2022" ; //éventuel résultat d'une saisie au clavier
DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd/MM/yyyy");
LocalDate localDate = LocalDate.parse(dateString, formatter);

//LocalDate date14July2015 = LocalDate.of(2015, 7, 14);
LocalDate date14July2015 = LocalDate.of(2015, Month.JULY, 14);

LocalTime time = LocalTime.of(14 /*h*/, 15 /*m*/, 0 /*s*/);
LocalDateTime datetime = date14July2015.atTime(time);
System.out.println("le 14 juillet 2015 à 14h15 : " + datetime);

LocalDate date1=today , date2=nextMonth;
Period p1 = Period.between(date1, date2) ;
System.out.println("periode p1:" + p1); //ex : P1M6D pour une Période de 1 Month et 6 Days

long periodeEnNombreDeJours = ChronoUnit.DAYS.between(localDate1, localDate2);

LocalTime time1= time;
LocalTime time2= LocalTime.of(14 /*h*/, 30 /*m*/, 0 /*s*/);
Duration d = Duration.between(time1, time2);      System.out.println("durée d:" + d);

Duration twoHours = Duration.ofHours(2); System.out.println("durée de 2 heures:" + twoHours);
Duration tenMinutes = Duration.ofMinutes(10); System.out.println("durée de 10 minutes:" + tenMinutes);
Duration thirtySecs = Duration.ofSeconds(30); System.out.println("durée de 30 secondes:" + thirtySecs)
```

```

LocalTime t2 = time.plus(twoHours);      System.out.println("14h15 plus 2 heures:" + t2);

//with(temporalAdjuster) :
LocalDate premierJourDeCetteAnnee=
    LocalDate.now().with(TemporalAdjusters.firstDayOfYear());
System.out.println("premierJourDeCetteAnnee="+premierJourDeCetteAnnee);
    LocalDate dernierJourDuMois=  LocalDate.now().with(TemporalAdjusters.lastDayOfMonth());
System.out.println("dernierJourDuMois="+dernierJourDuMois);

ZoneId myZone = ZoneId.systemDefault();
System.out.println("my (local) zoneId is:" + myZone);

//lien entre java.util.Date et java.time... :
Date date = new Date();
Instant nowInstant = date.toInstant();
LocalDateTime dateTime = LocalDateTime.ofInstant(nowInstant, myZone);
System.out.println("today/now from Date:" + dateTime);
}

```

```

LocalDateTime now = LocalDateTime.now();
System.out.println("basic/default display of LocalDateTime.now() :" + now);
//ex :2021-11-23T09:28:12.289915600
Instant instantT = now.atZone(ZoneId.systemDefault()).toInstant();
long nbMsSinceFirstJanuary1970GMT = instantT.toEpochMilli();
System.out.println("instantT.toEpochMilli() , timestamp , nb ms since 1970-01-01 00:00:00
GMT=" + nbMsSinceFirstJanuary1970GMT); //ex : 1637656092289

```

```

LocalDate nowDate = LocalDate.now();
System.out.println("basic/default display : today (nowDate local) is " + nowDate);//ex :2021-11-23
String sdate_fr_2=nowDate.format(DateTimeFormatter.ofPattern("EEEE, dd MMMM
yyyy",Locale.FRENCH)); // EEEE means fullname of weekday
System.out.println("sdate_fr_2="+sdate_fr_2);//exemple: dimanche, 13 septembre 2020

```

4.3. Encodage base64 (java.util.Base64)

```

public static void testBase64(){
    try {
        // Encode using basic encoder :
        String base64encodedString =
            Base64.getEncoder().encodeToString("Myjava8String".getBytes("utf-8"));
        System.out.println("Base64 Encoded String (Basic) : " + base64encodedString);

        // Decode :
        byte[] base64decodedBytes = Base64.getDecoder().decode(base64encodedString);
        System.out.println("Original String: " + new String(base64decodedBytes, "utf-8"));
    } catch (UnsupportedEncodingException e) {
        e.printStackTrace();
    }
}

```

4.4. Divers autres apports de java 8

Amélioration de l'introspection → `parameter.getName()` retourne véritable nom du paramètre (stocké dans `byteCode`) plutôt que "arg0" , "arg1" , ...

`@Repeatable` ([java.lang.annotation](#)) , ...

VIII - Entrées/sorties (io) – fichiers

Pour manipuler , les entrées/sorties fichiers et le FileSystem , le langage java a apporté progressivement des librairies de classes au fur et à mesure des évolutions de versions.

java.io date de java 1 et fonctionne en mode synchrone bloquant.

java.nio de java 4 a apporté de nouvelles classes et abstractions.

java.nio2 de java 7 a encore un peu amélioré les choses avec plus d'asynchronismes.

1. Essentiel de java.io

NB: Bien que le package **java.io** soit très ancien , il a le mérite de fonctionner avec toutes les versions de java (même sur android) et ses fonctionnalités (bien que réduites) peuvent suffirent pour des besoins simples.

1.1. Lecture / écriture dans un fichier et à l'écran.

Les classes abstraites **InputStream** et **OutputStream** définissent les méthodes de bases permettant d'effectuer des entrées/sorties : **.read()** , **.write()**, **.flush()**, **.close()**, ...

Ces classes génériques sont spécialisées en fonction du support :

FileInputStream / **FileOutputStream** pour lire ou écrire dans un fichier

ByteArrayInputStream / **ByteArrayOutputStream** ---> tableau d'octets en mémoire

PipedInputStream / **PipedOutputStream** ---> tuyau de communication entre 2 threads

Avec les classes ci-dessus, on peut effectuer des entrées/sorties de bas niveau (flux d'octets brut sans notion de format). Pour contrôler plus finement le format des données à lire ou à écrire, on sera généralement amené à instancier à partir de ces flux bruts, l'une des classes suivantes:

DataOutputStream	Pour écrire en binaire des entiers, réels, ...
PrintStream	Pour écrire sous forme de chaînes ASCII des String,int,...
BufferedReader & InputStreamReader	pour lire des chaînes de caractères que l'on récupérera sous forme de String,int,...

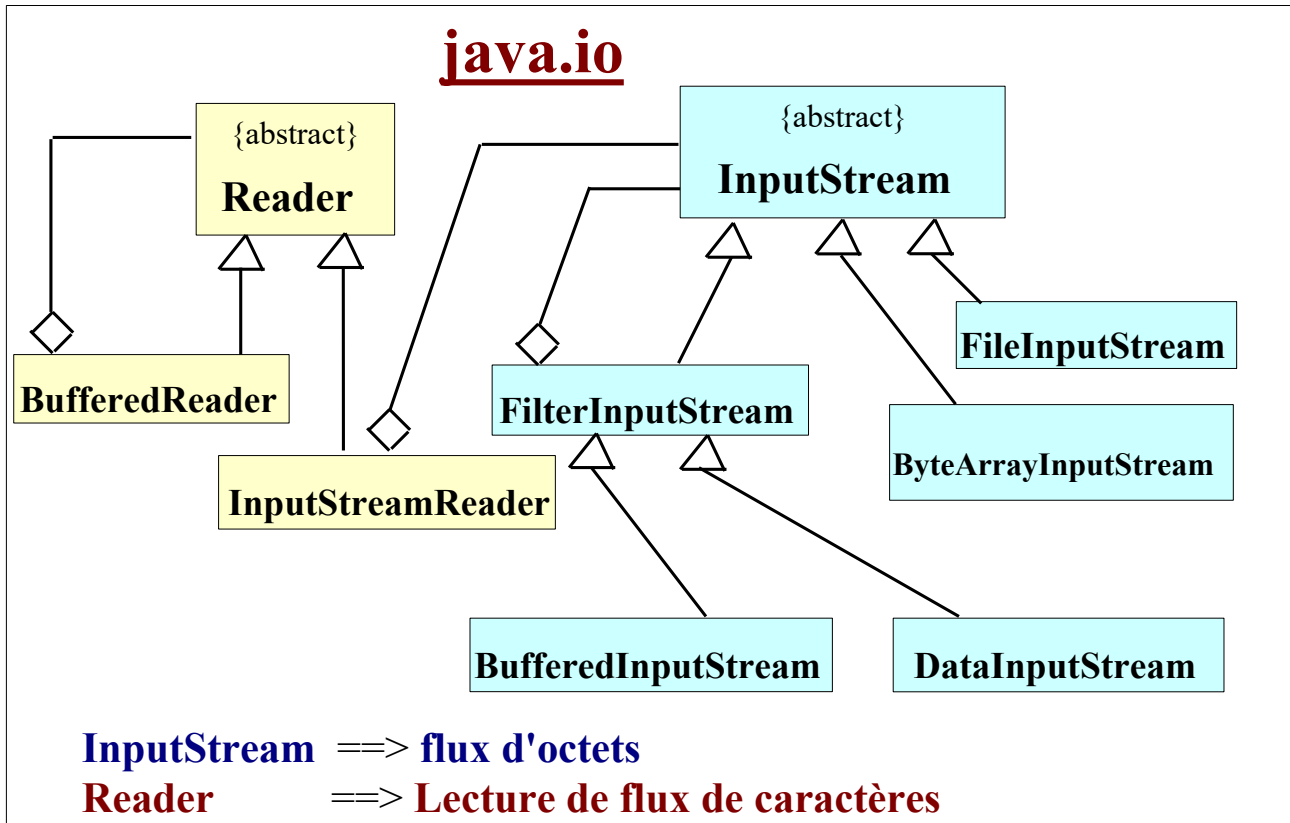
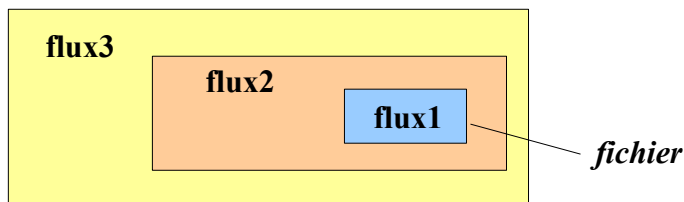
Les entrées/sorties standards (descripteurs linux 0,1,2) sont depuis JAVA manipulées au moyen de **System.in** , **System.out** et **System.err** qui sont des instances statiques (variables de classe) intégrées au sein de la classe System .

IX - Principe fondamental d'imbrication des flux

Le package **java.io** est organisé en suivant le principe du **modèle de conception** (Design Pattern) "**Décorateur**" qui associe *héritage* et *aggrégation* .

Quelque soit le type de flux d'octets manipulé, celui-ci peut être vu comme étant une chose de type **OutputStream** ou bien **InputStream**.

Pour obtenir des fonctionnalités spécifiques , il faut imbriquer un flux de bas niveau dans un flux de haut niveau:



Exemple: pour lire des lignes dans un fichier texte, il faut commencer par récupérer un flux de bas niveau en lecture sur le fichier:

```
FileInputStream flux1 = new FileInputStream("c:\\repx\\fl.txt");
```

Ce flux ne comporte qu'une simple méthode **read()** permettant de **lire un paquet d'octets en binaire**.

On imbrique donc ce flux dans un autre de plus haut niveau qui va apporter des fonctionnalités supplémentaires (Reconnaissance des caractères et **lecture caractère par caractère**):

```
InputStreamReader flux2 = new InputStreamReader(flux1);
```

Ceci ne nous permet toujours pas de lire facilement des lignes entières.

On imbrique donc ce flux intermédiaire dans un troisième flux d'encore plus haut niveau et qui sera capable de "bufferiser" les caractères lus, reconnaître des fins de lignes et retourner des **lignes entières** via la méthode supplémentaire **readLine()**:

```
BufferedReader flux3 = new BufferedReader(flux2);
```

X - Vieil exemple de code (possible depuis java 1)

Cet exemple permet de saisir une phrase à partir de l'entrée standard. Cette ligne est alors introduite dans un fichier fl.txt.

Ce fichier est ensuite intégralement relu puis ré affiché à l'écran.

```
import java.io.*;

class AppliModeTexte {

public static void main(java.lang.String[] args) {
try {
    System.out.println("Entrez une valeur :");
    BufferedReader dts = new BufferedReader(
        new InputStreamReader(System.in /*flux d'entrée brut*/));

    String s = dts.readLine();
    System.out.println("-----> Valeur saisie :" + s);

    // Ecriture de quelques lignes dans le fichier "fl.txt"

    FileOutputStream of = new FileOutputStream("fl.txt");
    PrintStream ps = new PrintStream(of);

    ps.println("Ligne 1 (debut)");
    ps.println("Valeur saisie : " + s );
    ps.println("Ligne 3 (fin)");
    // fermetures dans l'ordre inverse des ouvertures:
    ps.close(); of.close();

    //Relecture de ce fichier et affichage des lignes
    //sur la sortie standard

    FileInputStream ifile = new FileInputStream("fl.txt");
    BufferedReader dis =
        new BufferedReader(new InputStreamReader(ifile));

    while (true)
    {
        s = dis.readLine(); // lecture d'une ligne dans le fichier
        if(s!=null) // Réécriture de celle si sur la sortie standard
            System.out.println(s);
        else break;
    }
    dis.close(); ifile.close();

    } /* end of try */

catch(java.io.IOException e)
    {
        System.err.println("Exception I/O"); e.printStackTrace();
    }
    }/* end of main */
}
```

NB : pour écrire des fichiers textes en contrôlant bien le jeu de caractères utilisé (ex : UTF8 ou autres) , on peut utiliser la classe **OutputStreamWriter** avec méthode **.write(String str)** en tant qu'alternative à **PrintStream**

1.1. Exemple 2 (lecture simple d'un petit fichier .csv)

produits.csv

```
numero;label;prix
1;cahier;5.5
2;stylo;2.1
3;classeur;6
4;gomme;3.3
```

```
public class Catalogue {
    private List<Produit> listeProduits = new ArrayList<>();

    public void lireFichier(String fileName) {
        try {
            FileInputStream fis = new FileInputStream(fileName);
            BufferedReader br = new BufferedReader(new InputStreamReader(fis));
            br.readLine(); //lecture de la première ligne du fichier .csv avec titres colonnes

            String ligne = br.readLine();
            while (ligne != null) {
                String[] t = ligne.split(";");
                Produit p = new Produit(Integer.parseInt(t[0]), t[1], Double.parseDouble(t[2]));
                this.listeProduits.add(p);
                ligne = br.readLine();
            }
            br.close(); fis.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

//...
}
```

version simplifiée (via Scanner) :

```
public void lireFichier(String fileName) {
    try {
        Scanner scanner = new Scanner(fileName);
        scanner.nextLine(); //lecture de la première ligne du fichier .csv avec titres colonnes
        while (scanner.hasNext()) {
            String ligne = scanner.nextLine();
            String[] t = ligne.split(";");
            Produit p = new Produit(Integer.parseInt(t[0]), t[1],
                                    Double.parseDouble(t[2]));
            this.listeProduits.add(p);
        }
        scanner.close();
    } catch (IOException e) { e.printStackTrace(); }
}
```

```
Catalogue c = new Catalogue();
c.lireFichier("produits.csv");
c.afficherProduits();
```

2. Spécificités à connaître et détails intéressants

2.1. java.util.zip

Les classes (filtres) **GZIPOutputStream** et **GZIPInputStream** du package **java.util.zip** permettent de gérer le **format de compression gzip** .

exemple:

```
...
GZIPOutputStream outputGZipStream = new GZIPOutputStream(fileStream);
outputGZipStream.write(...);
outputGZipStream.close();
```

2.2. Fichier de données accompagnant le code

Lorsqu'une application java a besoin de charger en mémoire un fichier de données qui est déployé avec son code (au même endroit que le code compilé : répertoire ou fichier ".jar"), il est conseillé de procéder de la façon suivante:

```
InputStream fluxLecture = getClass().getClassLoader().getResourceAsStream("ficData.txt");
```

car le chemin relatif "*ficData.txt*" est exprimé par rapport à la racine du code et ne devrait pas changer (même si tout est recopié sur une autre machine).

3. try with resources (AutoClosable)

Ce code opérationnel

```
public void lireFichierWithScanner(String fileName) {
    Scanner scanner = null;
    try {
        //scanner = new Scanner(new File(fileName));
        scanner = new Scanner(Paths.get(fileName));
        while(scanner.hasNext()) {
            String ligne = scanner.nextLine();
            System.out.println(ligne) ;
        }
    } catch (IOException e) {
        e.printStackTrace();
    }
    finally {
        if (scanner != null) {
            scanner.close();
        }
    }
}
```

peut se simplifier en

```
public void lireFichierWithScannerWithTryWithResources(String fileName) {
    //try (Scanner scanner = new Scanner(new File(fileName)) ){
    try (Scanner scanner = new Scanner(Path.of(fileName)) ){
        while(scanner.hasNext()) {
            String ligne = scanner.nextLine();
            System.out.println(ligne) ;
        }
    } catch (IOException e) {
        e.printStackTrace();
    }
    //Automatic finally with call
    //of .close() of AutoCloseable interface implementation in Scanner class
}
```

Variante possible depuis java9 :

```
final Scanner scanner = new Scanner(new File(fileName)) ;
try (scanner){ ...}
catch(...) { ... }
```

try/catch avec plusieurs ressources :

```
try (Scanner scanner = new Scanner(new File("testRead.txt")) ;
     PrintWriter writer = new PrintWriter(new File("testWrite.txt"))) {
    while (scanner.hasNext()) {
        writer.print(scanner.nextLine());
    }
}
```

Avec ordre d'appels des .close() = inverse de l'ordre dans try(...)

Ressource personnalisée compatible avec "try with resources" :

```
public class MyResource implements AutoCloseable {
    @Override
    public void close() throws Exception {
        System.out.println("Close MyResource");
    }
}
```

Principales classes ou Interfaces Java ayant déjà implémenté l'interface AutoCloseable :

- Scanner
- PrintWriter et beaucoup d'autres Writer
- Reader , InputStreamReader , BufferedReader,
- InputStream , FileInputStream , ObjectInputStream, ...
- OutputStream , FileOutputStream , DataOutputStream , ObjectOutputStream,
- Pipe, Socket , ...
- Connection, Statement , ResultSet , RowSet de jdbc

4. NIO , NIO2

⇒ plein de classes intéressantes pour gérer :

- chemins menant à des fichiers (**Path**)
- répertoires , FileSystem (déplacement , renommage de fichier)
- lectures/écritures synchrones et asynchrones

Utile ou pas selon les besoins spécifiques à tel ou tel projet.

⇒ **NIO,NIO2 est une des annexes de ce cours** (à approfondir si besoin).

XI - Introspection et Sérialisation

1. Introspection (java.lang.reflect)

1.1. Potentiel de l'intropection

Introspection

Le package **java.lang.reflect** permet d'effectuer une **introspection** des classes java. L'introspection consiste à *demander aux classes de dresser la liste de leurs attributs et méthodes*.

Cette **faculté d'auto-analyse** qu'offre les mécanismes internes du langage Java est pour l'instant inexistante en C++ .

➔ L'introspection est un **gros point fort de Java** .

➔ Ceci *permet d'automatiser entièrement des opérations de bas niveaux (sauvegarde / restauration de valeurs, ...)*.

```
import java.lang.reflect.*;
Class c = Class.forName(nomClasse);
Field[] tabChamps = c.getDeclaredFields();
Method[] tabMethods = c.getDeclaredMethods();
+ boucle "for(i=0; i < tabXxxx.length; i++) ..."
```

Deux grandes variantes du point de départ :

```
Class<?> c = Class.forName("nomPackage.NomClasseJava") ;
```

ou bien

```
Class<?> c = objectInstance.getClass() ;
```

java.lang.**Class**<T> est une **méta-classe** : une classe très spéciale dont une instance décrit la structure d'une classe ordinaire.

1.2. Exemple d'introspection

Cet exemple montre comment générer automatiquement des lignes de fichier .csv depuis un objet java quelconque (Devise, Produit, Strat, Personne, ...) :

```
package util;
import java.io.FileOutputStream; import java.io.IOException;
import java.io.PrintStream; import java.lang.reflect.Field;
import java.util.List;

public class MyCsvUtil {

    public static String writeValuesAsCsvString(Object obj){
        String csvString=null;
        Class<?> c = obj.getClass(); //meta description de la classe de l'objet obj
        try {
            Field[] tabChamps = c.getDeclaredFields();
            for(Field f : tabChamps) {
                f.setAccessible(true); //pour accéder aux parties privées
                String fieldValue= f.get(obj)==null?null:f.get(obj).toString();
                csvString=(csvString==null)?fieldValue : csvString + ";" +fieldValue;
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
        return csvString;
    }

    public static String writeFieldNamesAsCsvString(Object obj){
        String csvString=null;
        Class<?> c = obj.getClass(); //meta description de la classe de l'objet obj
        try {
            Field[] tabChamps = c.getDeclaredFields();
            for(Field f : tabChamps) {
                String fieldName= f.getName();
                csvString=(csvString==null)?fieldName:csvString + ";" +fieldName;
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
        return csvString;
    }

    public static void writeValuesAsCsvFile(List<?> col,String fileName){
        if(col.isEmpty()) return;//fin fonction sans rien faire
        try (PrintStream ps = new PrintStream(new FileOutputStream(fileName))){
            Object firstObj = col.get(0);
            String ligneEnteteAuFormatCsv= writeFieldNamesAsCsvString(firstObj);
            ps.println(ligneEnteteAuFormatCsv);
            for(Object obj:col) {
                String ligneValeursAuFormatCsv= writeValuesAsCsvString(obj);
                ps.println(ligneValeursAuFormatCsv);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
        //try_with_autoCloseable resource
    }
}
```

Le code ci dessus analyse automatiquement les structures des classes java *Produit* et *Stat* héritant toutes les deux de *Object* .

```
MyCsvUtil.writeValuesAsCsvFile(listeProduits,"produits.csv");
```

génère automatiquement

```
id;label;prixHt;tauxTva  
1;stylo bille rouge;1.6;20  
2;grand classeur;4.6;20  
3;produitXyz;4.4;10  
4;gomme;2.1;20
```

```
MyCsvUtil.writeValuesAsCsvFile(listeStats,"stats.csv");
```

génère automatiquement

```
label;somme;moyenne;ecartType  
statHt;12.7;3.175;1.3386093530227552  
statTtc;14.8;3.7;1.5143315356948757
```

1.3. Introspection avec annotations

L'annexe sur les annotations permettra de mieux comprendre l'exemple ci-dessous.

CsvIgnore.java

```
package util;
import java.lang.annotation.Documented; import java.lang.annotation.ElementType;
import java.lang.annotation.Retention; import java.lang.annotation.RetentionPolicy; import java.lang.annotation.Target;

@Documented
@Target(ElementType.FIELD) //utilisable au dessus d'un field/attribut
@Retention(RetentionPolicy.RUNTIME) //conservée dans code compile et accessible au runtime
public @interface CsvIgnore {
    //...
}
```

```
...
import com.fasterxml.jackson.annotation.JsonIgnore;
import util.CsvIgnore;
```

```
public class Stat {
    private String label;
    private Double somme;
    private Double moyenne;

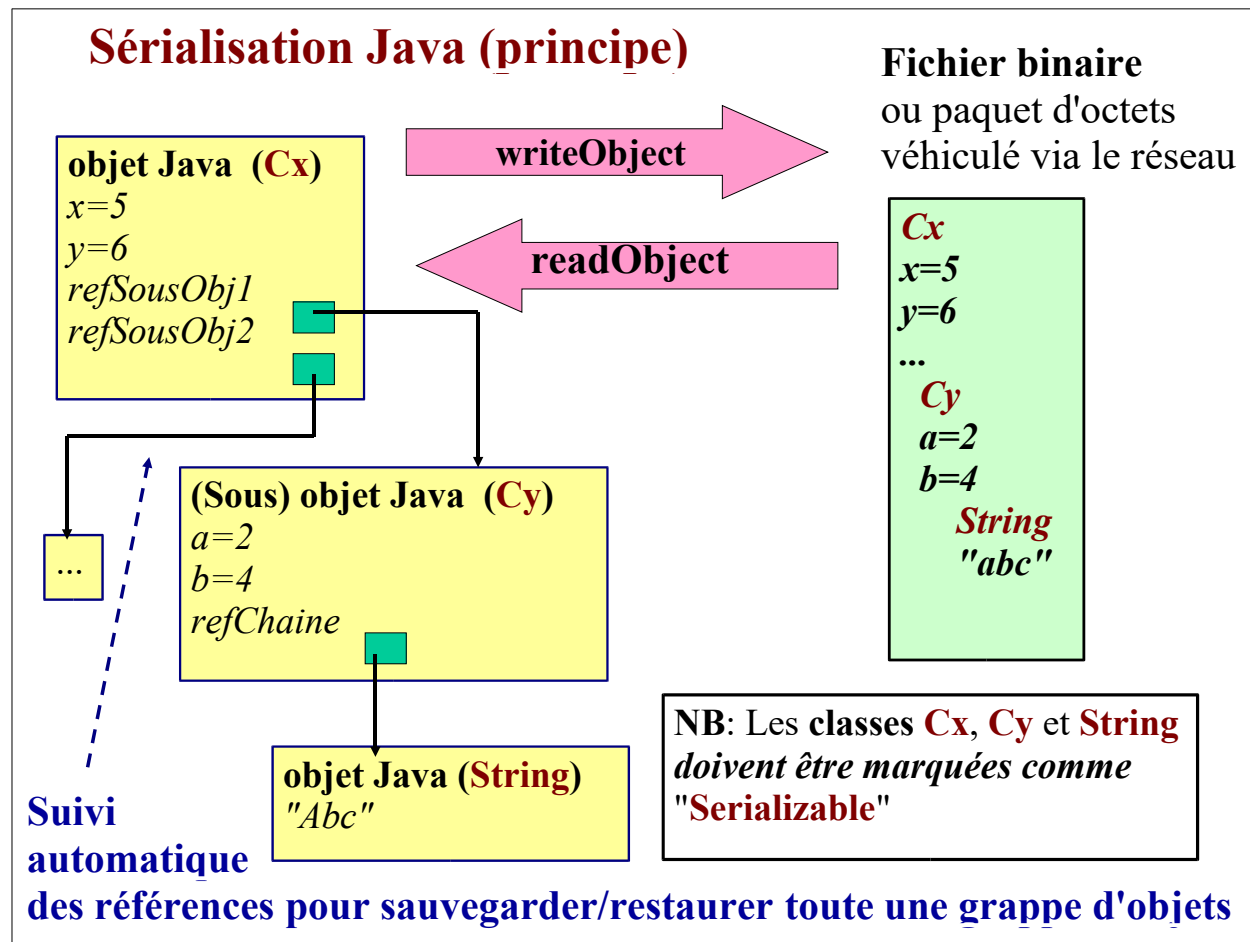
    @CsvIgnore
    @JsonIgnore
    private Double ecartType;
    ...
}
```

```
...
Field[] tabChamps = c.getDeclaredFields();
for(Field f : tabChamps) {
    CsvIgnore annotCsvIgnore = f.getAnnotation(CsvIgnore.class);
    //si annotCsvIgnore==null , @CsvIgnore n'a pas été placé au dessus de f
    if(annotCsvIgnore==null) {
        String fieldName= f.getName();
        csvString=(csvString==null)?fieldName:csvString + ";" +fieldName;
    }
}
....
```

Fichier **stats.csv** généré en ignorant la partie ecartType :

```
label;somme;moyenne
statHt;12.7;3.175
statTtc;14.8;3.7
```

2. Sérialisation (et persistance élémentaire)



2.1. Problématique de la persistance :

Un objet (instance) Java vit en mémoire et sa durée de vie est éphémère (le temps de l'exécution de l'application au maximum).

En programmation orienté objet, la **persistance** désigne le fait de pouvoir **stocker les valeurs d'un objet entier dans un flux** (fichier local, base de données, ...) **de façon à pouvoir , plus tard , restaurer les valeurs de cet objet dans une nouvelle instance en tous points identique à l'instance d'origine.**

D'un point de vue technique, la persistance est un peu plus compliquée qu'un simple dump binaire puisqu'il faut **suivre les références et sauvegarder tous les objets liés à l'instance courante (objet principal).**

D'autre part, lorsque l'on doit recréer une instance à partir des données que l'on extrait du flux, **il faut tenir compte des types (classes) exacts des objets liés qu'il faut reconstruire, restaurer et rattacher à l'objet principal.**

Lorsque l'on utilise les mécanismes prédéfinis du langage ou d'une API pour gérer la persistance, on a généralement affaire à un **code très simple** mais en contre partie, **on ne contrôle pas vraiment le**

format de ce qui est sauvegardé et l'on doit faire attention aux problèmes liés à des décalages d'octets dus à des versions différentes de la classe.

2.2. Mécanisme de persistance élémentaire du langage JAVA

Les classes **ObjectOutputStream** et **ObjectInputStream** comportent respectivement les méthodes **readObject** et **writeObject** permettant de gérer automatiquement la persistance des objets java.

NB: pour pouvoir appeler **readObject()** ou **writeObject()** sur un objet JAVA, il faut que la classe (dont est issue l'instance) implémente l'interface **java.io.Serializable**.

Le mécanisme de persistance ainsi déclenché fera en sorte que :

- Les attributs marqués via le mot clef "**transient**" ne soient pas sauvegardés, ni restaurés.
- Les attributs "**static**" (liés à la classe et non pas aux instances) ne soient pas traités.

Sérialisation (exemple de code)

```
import java.io.*;

-----
FileOutputStream ofStream = new FileOutputStream(pathName);
ObjectOutputStream ofluxObj = new ObjectOutputStream(ofStream);
ofluxObj.writeObject(monObjet);

-----
FileInputStream ifileStream = new FileInputStream(pathName);
ObjectInputStream ifluxObj = new ObjectInputStream(ifileStream);
monObjet = (MaClasse) ifluxObj.readObject();

-----
public class MaClasse implements java.io.Serializable
{
    private int x,y;
    private String nom;
    ...
}
```

Remarque:

- L'interface **Serializable** ne comporte aucune fonction imposée.
C'est une interface de marquage (uniquement utilisée pour tester le type).
- La notion de sérialisation n'est pas limitée au support "fichier".
On peut stocker un ensemble d'objets Java dans un flux quelconque.
- L'api **RMI** (Remote Method Invocation) **utilise de façon transparente la sérialisation** pour **transférer** un ensemble d'objets du serveur vers le client ou vice-versa .

2.3. Gestion des versions

Les mécanismes internes du langage Java génère automatiquement une valeur par défaut (selon un algorithme lié à la compilation) pour le champ spécial **serialVersionUID** (**ajouté automatiquement à la classe s'il n'est pas présent au sein du code source**) . Ceci permet de lever automatiquement une exception de type *InvalidClassException* en cas d'incohérence sur le numéro de version .

NB: Il est grandement conseillé d'introduire explicitement la variable de classe **serialVersionUID** (de type **long**) au sein d'une classe sérialisable pour bien contrôler le numéro de version des objets qui seront lus ou écrits:

```
class Cx implements java.io.Serializable
{
    static final long serialVersionUID = 1L;
    ....
}
```

2.4. Autres mécanismes de persistance

L'interface **java.io.Serializable** (et les méthodes *readObject()* / *writeObject()* de la classe **java.io.ObjectOutputStream**) correspondent au mécanisme de **persistance élémentaire** du langage JAVA.

Il existe d'autres mécanismes de persistance dans le monde Java:

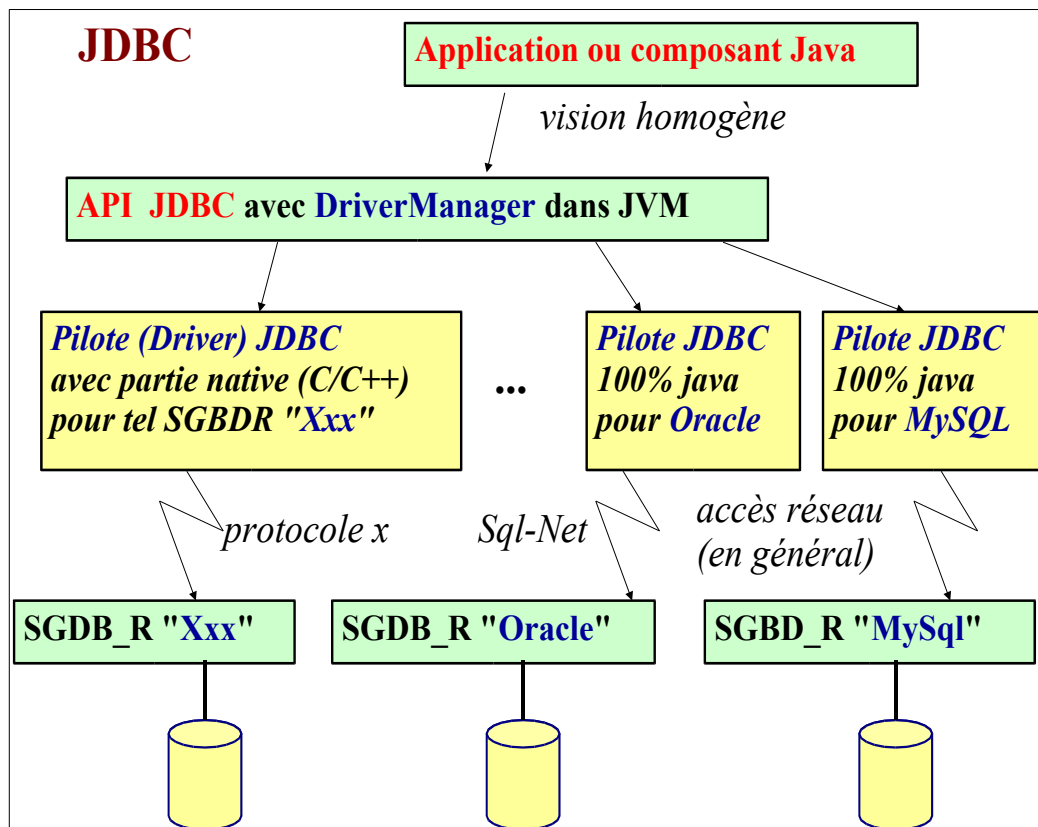
- L'API "**JAXB**" (*Java Api for Xml Binding*) permet de gérer une persistance au format XML
- L'API "**JPA/Hibernate**" permet de gérer une persistance au sein d'une base de données relationnelle (MySQL , Oracle , ...)
- L'API "**Jackson-databind**" permet de gérer une persistance au format JSON

XII - JDBC (accès aux bases de données)

1. JDBC : Présentation et structure

Présentation de JDBC

- **JDBC** signifie *Java DataBase Connectivity* .
- Il s'agit d'une **API standard** et de bas niveau *permettant à un programme Java de s'interfacer* avec **une base de données relationnelle** quelconque (Oracle, DB2, MySQL, Postgresql ...) .
- L'api **JDBC** correspond au package **java.sql** .
- **javax.sql.DataSource** est une extension standard permettant de gérer les *Pools de connexions* .
- Principales fonctionnalités de l'api JDBC :
 - * Effectuer une connexions vers une base de données
 - * Lancer des ordres SQL "Insert into, Delete From , Update, ..."
 - * Récupérer des enregistrements suite à une requête "Select ... From"
 - * Récupérer des informations sur la structure d'une base (MetaData)
 - * Déclencher des procédures stockées
 - * ...



2.1. Connexion via DriverManager.getConnection()

En utilisant simplement le JDK standard (et l'api JDBC incorporée) , n'importe quel programme java peut établir une connexion JDBC de la façon suivante:

```
import java.sql.*;
....
// chargement de la classe de pilote:
Class.forName("com.mysql.cj.jdbc.Driver"); // version mysql (mysql.jar)

// Création de l'URL identifiant la base de données
// format --> jdbc:nom_du_pilote:nom_source_de_donnees;param1=val1;...
String chUrl_2 = "jdbc:mysql://localhost/test"; // version MySql

// Ouverture de la connexion:
Connection connexion = DriverManager.getConnection(chUrl_2,
                                                    /* "user" */, /* "passwd" */);
connexion.close(); // Déconnexion (très important ==> dans finally {} )
```

NB: en cas d'erreur (échec au niveau de la connexion) , on se retrouve dans le bloc

```
catch( SQLException /* ou Exception */ ex ) {
...}
```

NB: Pour éviter d'utiliser des paramètres fixés "en dur" dans le code java , on utilise généralement un fichier de propriétés (à placer dans le CLASSPATH):

paramDB.properties

```
#driver JDBC , necessite mysql-connector-...jar ou autre dans le CLASSPATH
driver=com.mysql.cj.jdbc.Driver
#url de la base de donnée
url=jdbc:mysql://localhost:3306/mydb?serverTimezone=UTC
username=root
password=root
```

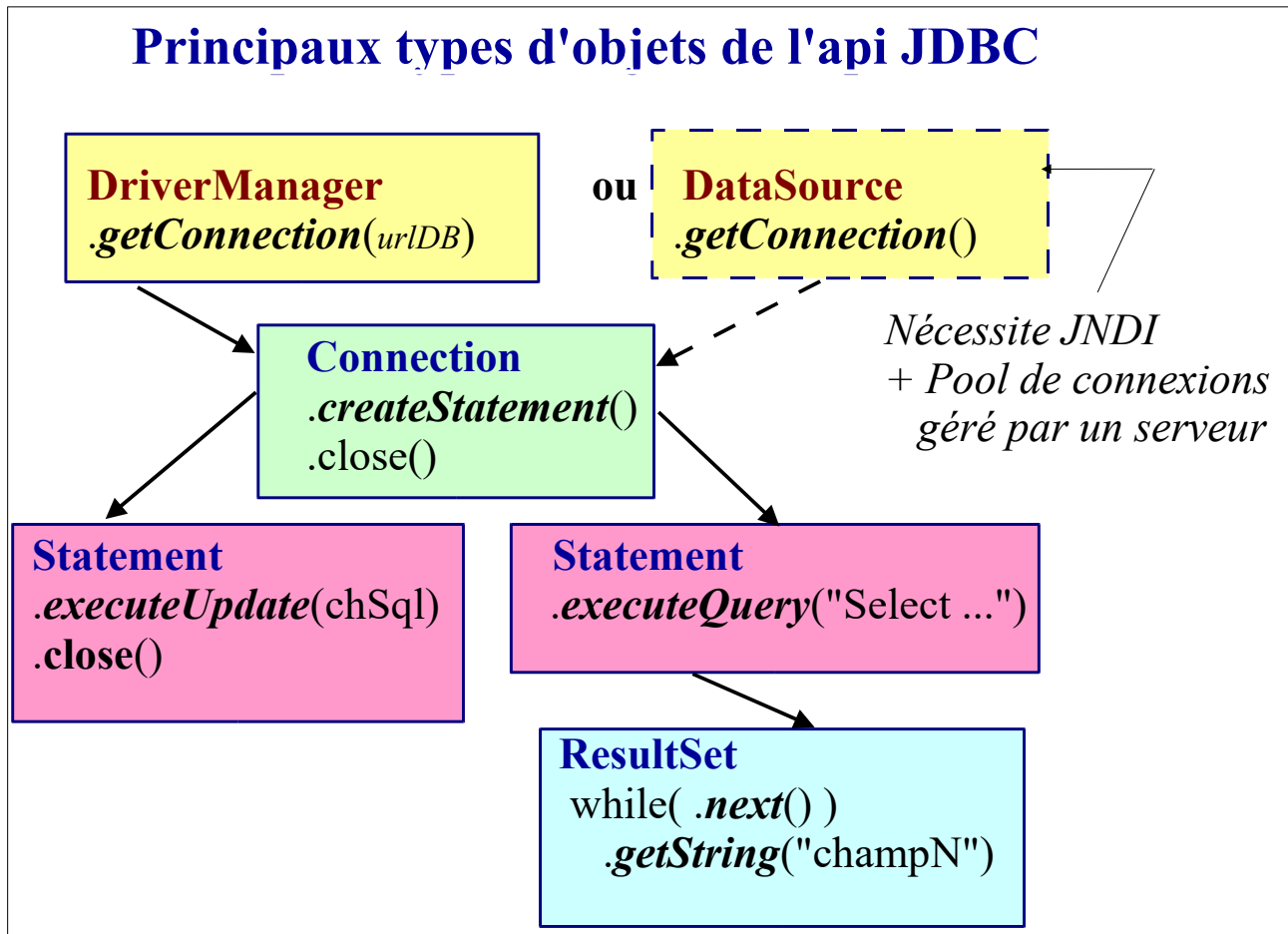
```
ResourceBundle ressources = ResourceBundle.getBundle("paramDB") ; //
paramDB.properties
String driver = ressources.getString("driver"); String chUrl = ressources.getString("url");
String username = ressources.getString("username"); String password =
ressources.getString("password");
Class.forName(driver); cn = DriverManager.getConnection(chUrl,username,password) ; ...
```

2.2. via un pool de connexions (DataSource)

Le package additionnel `javax.sql` comporte le type `DataSource` qui correspond à un accès à un **pool de connexions** pris en charge par un serveur d'applications JEE (ex: Tomcat , WebSphere , JBoss, WebLogic , Jonas , ...) ou bien par des api tierces parties (c3p0, hikari , ...).

...

3. Principaux objets de l'api JDBC



4. Lancer un ordre sql

```
Statement statement=connexion.createStatement();
```

```
String chOrdreSql = "DELETE FROM telephone WHERE nom='dupond'";
```

```
int nbLignesAffectees= statement.executeUpdate(chOrdreSql);
```

```
...
```

```
statement.close(); // à ne pas oublier pour ne pas perturber les traitements ultérieurs
```

NB: Ceci doit être englobé au sein d'un traitement d'exceptions : `try/ catch(SQLException)`

5. Effectuer une requête (select)

```
Statement statement=connexion.createStatement();

String chRequeteSql = "SELECT * FROM telephone WHERE numero LIKE '01%'";

try {
    ResultSet rs = statement.executeQuery(chRequeteSql);
}
catch( SQLException ex) { ... }
```

6. Balayer les lignes du résultat

```
while(rs.next())
{
    String nom = rs.getString("NOM"); // récupérer la valeur du champ "NOM"
    String numero = rs.getString("NUMERO"); ...
}
rs.close(); // à ne pas oublier ou bien à gérer avec "try with resource"
```

NB:

- suivant le type du champ , on utilisera une des fonctions **getXxx()** disponible dans la classe **ResultSet** .
- La fonction **getObject()** permet de récupérer la valeur d'un champ de type quelconque sous la forme d'un objet (classe **Integer**, **Double**, **String** , ...) sur lequel on peut toujours invoquer la fonction **toString()**.
- Si la valeur d'un champ n'est pas renseignée , sa valeur nulle (null sql) sera vue comme une valeur en retour de type 0 ou null(JAVA).
D'autre part, la méthode booléenne **wasNull()** permet de savoir si la valeur d'un champ est le null sql.
- Les versions 2, 3 et 4 de JDBC offre un balayage bi-directionnel via les méthodes **first()** , **last()** , **previous()** , **absolute(pos)** , **relative(offset)** . De plus les méthodes **isFirst()**, **isLast()**, **isBeforeFirst()** et **isAfterLast()** permettent de tester la position courante.

7. Accès à la structure de la base (MetaData)

L'api JDBC permet de s'enquérir de la structure d'une base quelconque.

On peut récupérer la liste des tables d'une base et la liste des champs d'une certaine table:

```
DatabaseMetaData meta = cn.getMetaData();           // cn = objet de type Connection
System.out.println("\n Liste des tables de la base:");
String tabOfTableType[] = {"TABLE","VIEW"};
ResultSet rs = meta.getTables(null,null,"%",tabOfTableType);
while( rs.next())
{
    System.out.print("Nom: "+ rs.getObject(3) );
    System.out.println(" - Type: " + rs.getObject(4));
}
rs.close(); // indispensable
```

```
System.out.println("\n Liste des champs de la table TableAdr:");
rs = meta.getColumns(null,null,"tableAdr", "%");
while( rs.next())
    System.out.println(rs.getObject(4));
rs.close(); // indispensable
```

```
System.out.println("\n Liste des procedures :");
rs = meta.getProcedures(null,null,"%");
while( rs.next())
    System.out.println( rs.getObject(3));
rs.close(); // indispensable
```

Soit **rs** le fruit d'une requête SQL, on peut alors connaître la liste des champs grâce aux instructions suivantes:

```
ResultSetMetaData rsMeta = rs.getMetaData();
int nbChamps = rsMeta.getColumnCount();
for(int i=1;i<=nbChamps;i++)
{
    String chNomChamp = rsMeta.getColumnLabel(i);
    int dataType = rsMeta.getColumnType(i);
    switch(dataType)
    {
        case Types.VARCHAR:
        case Types.CHAR:
            chValChamp = rs.getString(i); ... break;
        case Types.INTEGER:
            chValChamp = String.valueOf( rs.getInt(i) ); ... break;
        ...
    }
}
```

Attention: Les **numéros des champs** vont de **1 à n** (et pas de 0 à n-1).

8. Gestion des transactions (tout ou rien)

```

connexion.setAutoCommit(false); // true par défaut

// quelques ordres SQL (Mises à jour , Insertions , Suppressions)

if(...)
    connexion.commit(); // pour valider Toutes les mäj.
else
    connexion.rollback(); // pour annuler toutes les mäj depuis le dernier commit/rollback

```

9. Préparer et lancer n fois un ordre Sql paramétrable

```

PreparedStatement pstmt =
    cn.prepareStatement( "Insert Into Table2 Values (?,?)" );
for(int i=0;i<3;i++)
{
    pstmt.setString(1,"Valeur"+i /*valeur du champ1 (texte)*/);
    pstmt.setInt(2,i /*valeur du champ2 (numérique)*/);
    pstmt.executeUpdate();
}
...

```

10. Appels de procédures stockées

```

CallableStatement cstmt = cn.prepareCall( "{call fromNom(?)}" );

cstmt.setString(1/*numéro du param*/, "Power User" /*valeur*/);

rs = cstmt.executeQuery();
while( rs.next())
{
    System.out.print( rs.getObject(1) + "," );
    System.out.print( rs.getObject(2) + "," );
    System.out.print( rs.getObject(3) + "," );
    System.out.print( rs.getObject(4) + "," );
    System.out.println( rs.getObject(5));
}
rs.close();

```

11. fermer proprement les connexions

Vers 2005 (sans try with resource) :

```
static void closeCn(Connection cn){
    try { cn.close(); } catch (SQLException e) {e.printStackTrace();}
}
//à appeler au sein d'un bloc finally()
```

Maintenant avec "try with resources" éventuellement imbriqués :

```
...
try(Connection cn = initConnection()) {
    try(Statement st= cn.createStatement()) {
        try (ResultSet rs= st.executeQuery("select * from Compte")) {
            while(rs.next()) { ...
            }
        }
    }
} catch (SQLException e) {
    e.printStackTrace();
}
//automatic close (no need of finally bloc)
...
```

Il est important de bien fermer les connexions si elles proviennent d'un datasource/pool car sinon les connexions ne sont pas libérées et tout se bloque.

12. Récupérer la valeur d'une clef auto-incrémentée

```
/*Integer*/ Long recupValeurAutoIncrPk(PreparedStatement pst){
    /*Integer*/ Long pk=null;
    try {
        ResultSet rsKeys = pst.getGeneratedKeys();
        if(rsKeys.next()){ pk= rsKeys.getLong(1) ; /*rsKeys.getInt(1);*/ }
    } catch (SQLException e) { e.printStackTrace(); }
    return pk;
}

public Long insertNewCompte(Compte cpt){
    Long pk=null; Connection cn = initConnection();
    PreparedStatement pst = null;
    String reqSql = "insert into Compte(label,solde) values(?,?)" ;
    try { pst = cn.prepareStatement( reqSql, Statement.RETURN_GENERATED_KEYS );
        pst.setString(1, cpt.getLabel()); pst.setDouble(2, cpt.getSolde());
        pst.executeUpdate(); //avec auto_increment mysql sur colonne numCpt
        pk = recupValeurAutoIncrPk (pst) ;
    } catch (SQLException e) { e.printStackTrace(); }
    finally { closeRs(rsKeys);closeSt(pst);closeCn(cn); }
    return pk;
}
```

13. Fonctionnalités à partir de la version 2 de JDBC

La version 2 de JDBC (accompagnant le JDK 1.2) , offre les fonctionnalités suivantes:

- Gestion des champs binaires (**blob**) et nouveaux types .
- Gestion de **curseur à plusieurs lignes** (comme ODBC)
- Gestion directe des **mise à jour** depuis l'objet **ResultSet** (méthodes UpdateXXX())
- ...

13.1. Très rares mises à jour directes à partir d'un objet ResultSet

13.1.a. Ajout d'un nouvel enregistrement:

```
rs.moveToInsertRow(); // préparer un nouvel enregistrement vierge en mémoire
rs.updateString("nom","dupond"); // donner une valeur au champ 1
...
rs.updateInt("age",30); // donner une valeur au champ n
rs.insertRow(); // ajouter le nouvel enregistrement dans la base
```

13.1.b. Mise à jour de l'enregistrement courant:

```
int age = rs.getInt("age");
rs.updateInt("age",age+1); // modifier 1 ou plusieurs champ en mémoire
rs.updateRow(); // enregistrer les modifications dans la base de données.
```

13.1.c. Suppression de l'enregistrement courant:

```
rs.deleteRow();
```

14. Mémento SQL + Mise en oeuvre MySQL

14.1. Exemples (simples) de requêtes SQL

SELECT fieldlist FROM tablenames [WHERE searchcondition] [GROUP BY fieldlist [HAVING searchconditions]] [ORDER BY fieldlist]

- **SELECT DISTINCT** Last_Name **FROM** Employees **WHERE** Last_Name = 'Smith'
- **SELECT Count(*)**, **Avg**(Salary) , **Max**(Salary) **FROM** Employees
- **SELECT** Department, **Count**(Department) **FROM** Employees **GROUP BY** Department **HAVING Count**(Department) > 100
- **DELETE FROM** Employees **WHERE** Title = 'Trainee'
- **UPDATE** Orders **SET** Freight = Freight * 1.03 **WHERE** Ship_Country = 'UK'
- **UPDATE** Orders **SET** Amount = Amount * 1.1, Freight = Freight * 1.03 **WHERE** Country = 'UK'
- **INSERT INTO** T_Rubrique (id,label) **VALUES** (2 , "Automobile")

14.2. Exemples de scripts pour base "MySQL/MariaDB"

NB: la version "MariaDB" est complètement openSource et plus facile à installer que MySQL sur une machine windows.

- *télécharger le serveur MariaDB depuis l'url <https://mariadb.org/>*
- *installer le logiciel en choisissant un mot de passe pour root (ex : root) et en activant éventuellement l'option "connect as root from other host"*

numéro de port par défaut = 3306 et choisir de démarrer comme un service (automatic ou manuel) . Si proposition d'installer le complément graphique HeidiSQL , répondre plutôt "oui"

- *Lancer ensuite des scripts de ce genre:*

init_db.bat

```
REM lancer si besoin le service mariadb ou mysql (net start mysql ou mariadb)
set MYSQL_HOME=C:\Program Files\MariaDB 11.8
cd /d %~dp0
REM mot de passe souvent root ou formation ou rien
"%MYSQL_HOME%\bin\mysql" -u root -p < init_db.sql
pause
```

init_db.sql

```
CREATE DATABASE IF NOT EXISTS devise_db;
USE devise_db;
DROP TABLE IF EXISTS devise;
CREATE TABLE devise( code VARCHAR(3) ,
                      monnaie VARCHAR(64) NOT NULL,
                      d_change DOUBLE,
                      PRIMARY KEY (code)) ;

INSERT INTO devise(code,monnaie,d_change) VALUES('USD' ,'Dollar',1.0);
INSERT INTO devise(code,monnaie,d_change) VALUES('EUR' , 'Euro',1.05);
INSERT INTO devise(code,monnaie,d_change) VALUES('GBP' , 'Livre',0.7);
show tables;
describe devise;
SELECT * FROM devise;
```

ou bien (assez souvent)

```
...
CREATE TABLE xxx (id integer auto_increment, ... , PRIMARY KEY(id));
...
```

et si nécessaire : grant_priv_on_mydb.sql

```
# GRANT ALL PRIVILEGES
GRANT SELECT,INSERT,UPDATE,DELETE
ON mydb.*
TO mydbuser@'%'
IDENTIFIED BY 'mypwd';
FLUSH PRIVILEGES;
```

XIII - Threads (java)

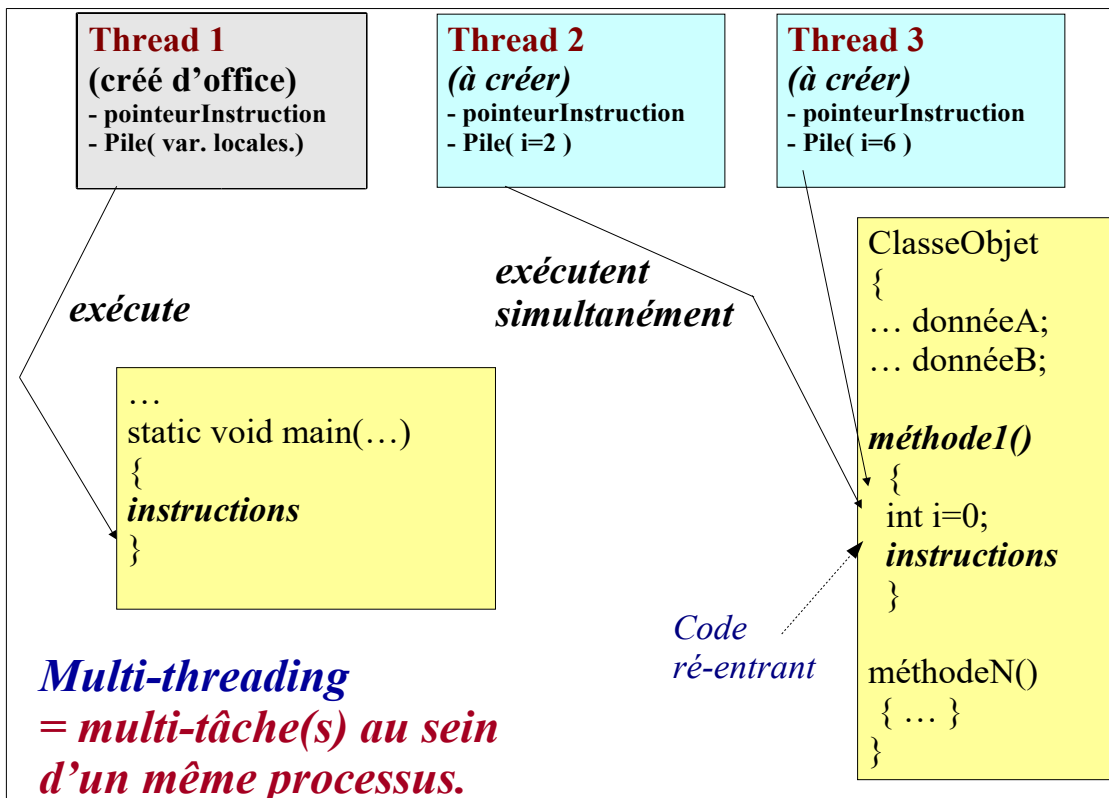
1. Concept de threads

Concept de Thread

Un **thread** est un **fil d'exécution**. Il s'agit d'une **entité dynamique** qui peut exécuter un ensemble d'instructions. L'association de *plusieurs threads* permet de mettre en œuvre **des traitements parallèles au sein d'un même processus**.

Sous Java, la gestion des threads fait intervenir trois entités fondamentales:

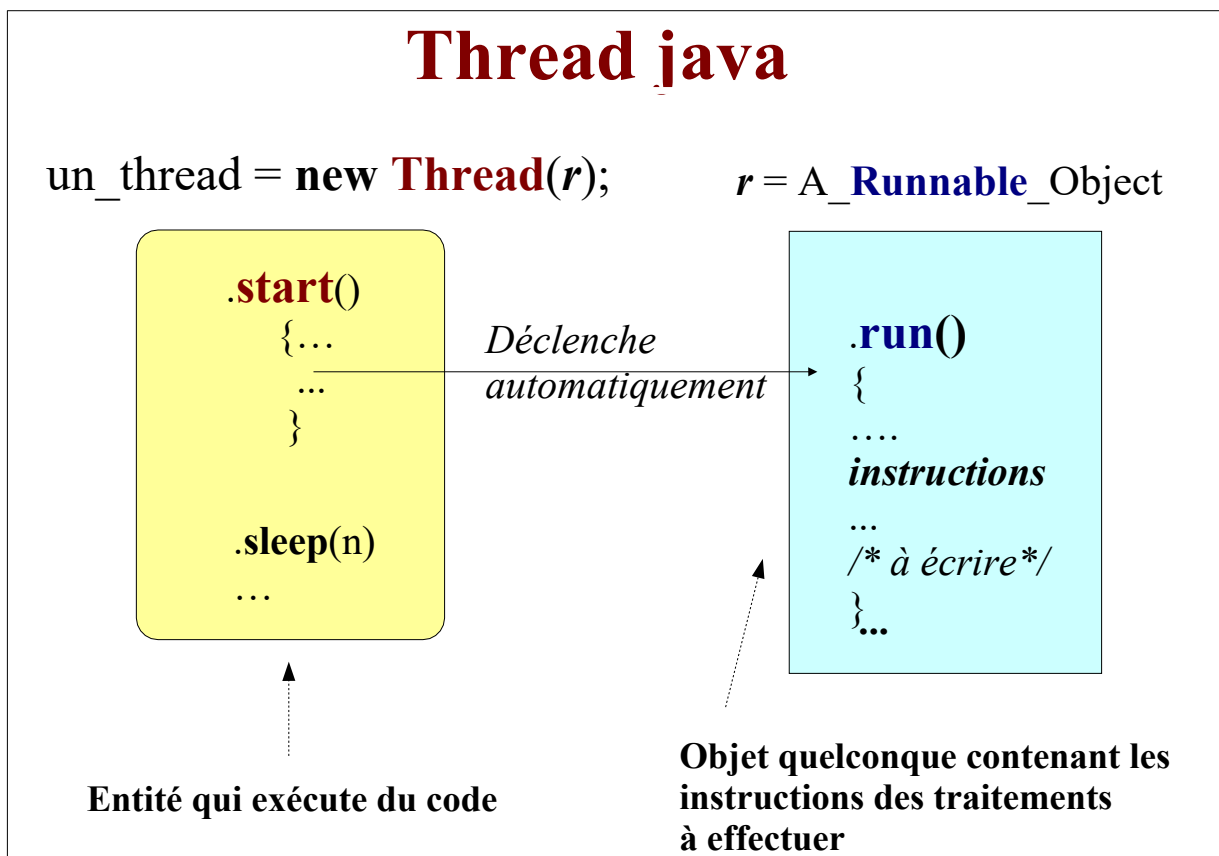
- Le **moniteur de threads** (gestionnaire central et caché dans les API de JAVA qui gère les *exécutions concurrentes des différents threads*).
- Une instance de la classe **Thread** (*entité qui exécute du code* et que l'on peut lancer, arrêter,...)
- Un objet implémentant l'interface **Runnable** et dont la méthode **run** correspond au point d'entrée de *l'ensemble des instructions à exécuter* par un nouveau thread.



Remarque: Il existe toujours au moins un thread au sein d'une application ou applet JAVA. Ce thread principal est créé automatiquement, il s'occupe de la gestion des événements associés à l'interface graphique ou bien de l'enchaînement des instructions partant de la méthode statique `main()` en mode texte. Si on fait exécuter par ce thread principal et implicite des traitements longs (boucle infinie, accès réseau, calculs, ...), on risque alors de bloquer la gestion de l'interface graphique et l'utilisateur aura alors l'impression que le programme ne répond plus.

Il est donc nécessaire de créer un ou plusieurs threads supplémentaires dès que l'on veut effectuer des opérations qui peuvent être longues ou bloquantes (Animations, Accès réseau, ...).

2. Gestion des threads avec java



La méthode statique `sleep()` est souvent employée dans les animations, elle permet d'insérer des temporisations entre deux affichages d'image par exemple.

```
try{
    Thread.sleep(n); //méthode statique qui endort Thread.currentThread()
} catch (InterruptedException ex) { ex.printStackTrace(); }
```

NB (Alternative) : Au lieu de créer une classe implémentant l'interface Runnable, on peut également sous classer la classe Thread et programmer directement la méthode `run()` dans cette sous classe (Thread spécifique).

Exemple de code: (comptage croissant et/ou décroissant)

```

package tp.basic;

public class CountTask implements Runnable {
    int from;
    int to;

    CountTask(int from,int to){
        this.from=from; this.to=to;
    }
    CountTask(){
        this(1,5);
    }

    public void display_with_pause(int i) {
        System.out.println(">>> i=" + i + " - " + Thread.currentThread().getName());
        try {
            Thread.sleep(1000); //1000ms de pause
        } catch (InterruptedException e) {
            throw new RuntimeException(e);
        }
    }

    @Override
    public void run() {
        if(this.from <= this.to){
            for(int i=from;i<=to;i++){
                display_with_pause(i);
            }
        } else {
            for(int i=from;i>=to;i--){
                display_with_pause(i);
            }
        }
    }
}

```

```

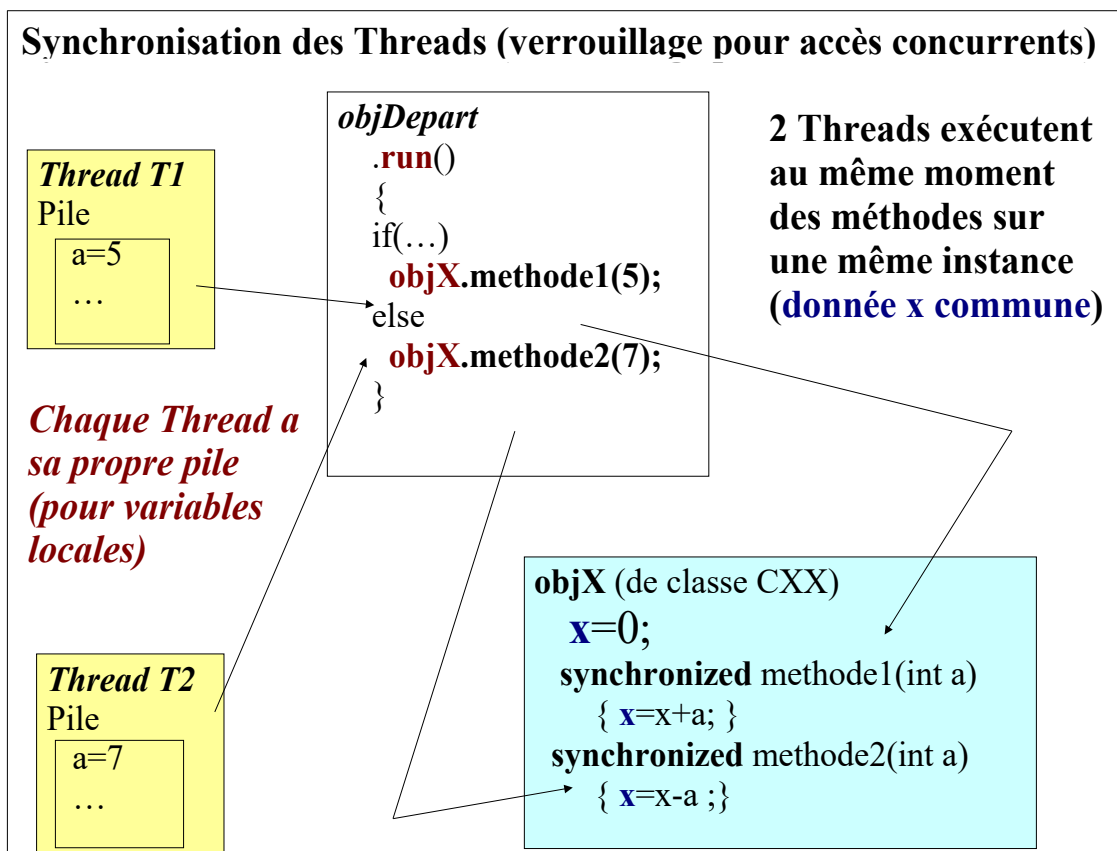
package tp.basic;
public class BasicThreadExample {
    public static void main(String[] args) {
        System.out.println("start of main() executed by " + Thread.currentThread().getName());
        CountTask countTask = new CountTask(0,5);
        CountTask countDownTask = new CountTask(5,0);
        Thread t1= new Thread(countTask); t1.setName("countUpThread");
        Thread t2= new Thread(countDownTask); t2.setName("countDownThread");
        t1.start(); t2.start();
        System.out.println("end of main() executed by " + Thread.currentThread().getName());
    }
}

```

}

⇒ résultats :*start of main() executed by main**end of main() executed by main*>>> *i=5 - countDownThread*>>> *i=0 - countUpThread*>>> *i=4 - countDownThread*>>> *i=1 - countUpThread*>>> *i=3 - countDownThread*>>> *i=2 - countUpThread*>>> *i=3 - countUpThread*>>> *i=2 - countDownThread*>>> *i=4 - countUpThread*>>> *i=1 - countDownThread*>>> *i=0 - countDownThread*>>> *i=5 - countUpThread*

3. Synchronisation des threads



Accès concurrents / synchronisation des threads

Dans certains cas , il se peut que *plusieurs threads* puissent *exécuter simultanément différentes méthodes sur une même instance et donc manipuler les mêmes données internes de l'objet*.

Pour que toutes ces opérations puissent s'effectuer sans interférence, il faut temporairement verrouiller ces données pour qu'un seul thread puisse les modifier à la fois.

Le langage java a choisi de gérer cette synchronisation via un mot clef **synchronized** qui *permet au moniteur central de la machine virtuelle de gérer la concurrence entre les threads* :

Dès qu'un thread exécute une méthode synchronisée sur un objet java, tous les autres threads qui souhaitent exécuter une (même ou autre) méthode synchronisée sur la même instance seront automatiquement bloqués (mis en attente).

Le mot clef **synchronized** place donc une sorte de **verrou** sur un **objet complet** (pour protéger l'accès aux données internes).

Syntaxe:

Lorsque le mot clef **synchronized** est utilisé pour préfixer une méthode , l'objet (this) de la classe courante est alors automatiquement verrouillé durant tout le temps d'exécution de la méthode:

```
public class Cx {
protected int x=0;
synchronized public void methode1(int a)
{
    x=x+a;
    ...
}
...
}
```

Cas particulier des méthodes statiques:

Lorsque le mot clef **synchronized** est placé devant une méthode "**static**" , le verrou porte sur les membres statiques de la classe .

Lorsque la syntaxe **synchronized(Object) { ... }** est utilisée à l'intérieur d'une méthode , seul l'objet (ou le sous objet) précisé est verrouillé et la durée du verrou correspond au temps nécessaire à un thread pour exécuter le bloc d'instructions délimité par les accolades :

```
public class Cy {
protected CPartie objPartX = new CPartie();
```

```

protected int y=0;

public void methodeOrdinaire(...)
{
    ...
    synchronized( objPartX )
    {
        objPartX.x++;
        ...
    }
    ...
    synchronized( this )
    {
        this.y++;
        ...
    }
}

```

3.1. Autres aspects avancés sur les threads

- On peut donner un nom explicite (chaîne de caractères) à un Thread. Celui-ci peut être passé au niveau du constructeur.
- **join()** de la classe *Thread* permet d'attendre la fin de l'exécution d'un thread.
- Les Threads peuvent être organisés en groupe(s) et sous groupe(s).
- On peut modifier la **priorité** d'un thread : `un_thread.setPriority(n);`
- La classe **ThreadLocal** permet de gérer des variables qui sont accessibles globalement par une grande partie du code source et qui sont locales vis à vis des Threads (chaque Thread a sa propre valeur).

Le **jdk 1.5** a apporté un bon nombre de nouvelles fonctionnalités pointues sur les threads (ex: **sémaphore** du package *java.util.concurrent*).

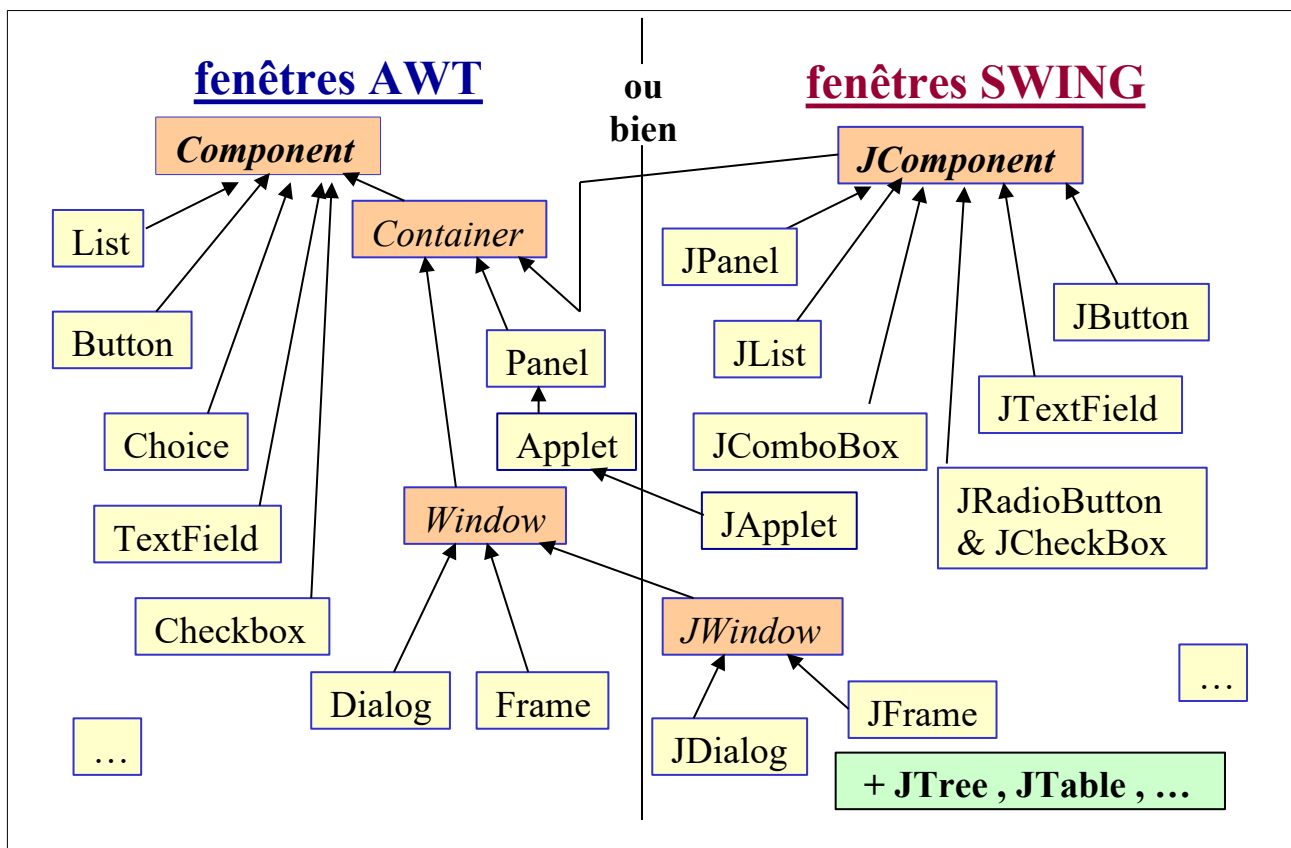
XIV - Annexe - Api pour IHM/GUI (swing, ...)

1. Éléments de base sur AWT/SWING et événements

1.1. Gestion des fenêtres et des contrôles

Pour gérer les fenêtres , il existe (au choix) deux api (relativement proches et parallèles):

- Les classes (de java.awt) héritant de **Component**. Il s'agit d'une encapsulation JAVA des api natives du système d'exploitation.
- Les classes (de javax.swing) héritant de **JComponent** (fenêtre 100 % java).



1.1.a. Composant de l'interface graphique (Component)

La classe abstraite java.awt.**Component** hérite directement de **Object** et correspond à un composant quelconque de l'interface graphique au sens large (fenêtre, contrôle,...). Parmi les méthodes de cette classe on retiendra principalement les suivantes:

getParent()	retourne le Container contenant l'objet
setBackground(color)	Fixe la couleur de fond
setForeground(color)	Fixe la couleur d'avant plan (texte, traits,...)
setFont(fonte)	Fixe la police de caractères
setVisible(booléen)	cache ou rend visible l'objet graphique
setEnabled(booléen)	active ou inactive (grise) l'objet
setLocation(x,y), setSize(w,h)	déplace , redimensionne l'objet
requestFocus()	demande le focus sur cet objet

bounds()	retourne la position et dimension de l'objet sous la forme d'un objet Rectangle (coordonnée)
-----------------	--

Dans l'arbre d'héritage on trouvera directement sous la classe Component une série de classes correspondant aux contrôles (boutons poussoirs, zones d'éditations , ...) plus une classe abstraite nommée java.awt.Container .

1.1.b. Fenêtres "Container" et sous fenêtres

La classe java.awt.**Container** correspond à un Conteneur (fenêtre pouvant contenir d'autres sous fenêtres). Ses principales méthodes sont les suivantes:

add(component)	Ajoute un composant (contrôle,panel, ...)
remove(component)	Enlève un sous objet
countComponents()	Retourne le nombre de composants
setLayout(layoutMgr)	Associe un gestionnaire de répartition au conteneur
locate(x,y)	Retourne une référence sur le Component en (x,y)

- Les classes java.awt.**Panel** et javax.swing.**JPanel** correspondent à un "sous-paneau" . Il s'agit d'un élément généralement invisible qui permet de gérer finement la disposition des contrôles dans une fenêtre en jouant le rôle de **Conteneur intermédiaire**: un objet Panel est toujours contenu dans un autre conteneur et contient à son tour d'autres contrôles ou sous conteneurs.
- Les classes "Fenêtre" (java.awt.**Window** et javax.swing.**JWindow**) correspond à une fenêtre applicative (sans bordure , ni menu) qui comporte deux sous classes importantes: java.awt.*Dialog* (boîte de dialogue) et java.awt.*Frame* (fenêtre cadre ou fenêtre principale).
- Les classes java.awt.**Frame** et javax.swing.**JFrame** correspondent à une fenêtre principale qui peut avoir un titre, une barre de menu, une bordure permettant de la redimensionner , un icône et un curseur spécifique. Cette classe est très souvent sous classée car elle correspond au point central d'une application graphique. Les principales méthodes de **Frame** sont:

setTitle(titre)	Change le titre
setMenuBar(barreDeMenu)	Associe un menu à la fenêtre
setResizable(monBooleen)	true / false

1.1.c. Les Contrôles (composants graphiques élémentaires)

Libellé (étiquette)

```
lbl = new javax.swing.JLabel("Nom :");
```

Champ de saisie simple (une seule ligne)

```
champ = new javax.swing.JTextField();
champ.setText("Valeur par défaut");
String texteSaisi = champ.getText();
champ.select(2 /*start*/,6 /*end*/);
String texteSelectionne = champ.getSelectedText();
```

Bouton poussoir

```
jbtnOk = new javax.swing.JButton("Ok");  
jbtnOk.setText("OK");
```

"Case à cocher" ou "bouton radio" accompagné de son libellé.

En version swing:

2 classes différentes:

- **JCheckBox** pour les cases à cocher (non exclusives).
- **JRadioButton** pour les boutons "radio" (exclusifs entre eux).

Paramétrage de l'exclusivité entre différentes options:

```
ButtonGroup bg = new ButtonGroup(); // objet invisible  
bg.add(jRadioButtonMarie);  
bg.add(jRadioButtonCelibataire);
```

Liste déroulante pour choisir un élément parmi n.

En version swing:

```
combo = new javax.swing.JComboBox();  
combo.addItem("rouge");  
combo.addItem("vert"); combo.addItem("bleu");  
  
combo.setSelectedItem("vert"); //ou combo.setSelectedIndex(1);  
  
nbElts = combo.getItemCount();  
int selIndex = combo.getSelectedIndex();  
String chaineChoisie = combo.getSelectedItem();
```

Liste d'éléments (avec sélection multiple possible)

En version swing:

```
Vector vectSel = new Vector();  
JList jListSel = new JList();  
...  
vectSel.addElement(uneChose);  
...  
...  
jListSel.setListData(vectSel);  
...  
int tabSelIndex[] = jListSel.getSelectedIndices();
```

Zone de saisie multi-lignes

En version swing:

<i>classe</i>	<i>fonctionnalités</i>
JTextArea	zone de texte simple à plusieurs lignes (light).
JTextPane	éditeur de texte sophistiqué (mise en forme possible des caractères).
JEditorPane	Zone de texte sophistiquée (text/plain ou text/html)

Boite de dialogue

- `javax.swing.JDialog` est la classe générique des boites de dialogue en version *SWING*.
- Un choix de nom de fichier s'effectue avec la classe **JFileChooser** :

```
JFileChooser chooser = new JFileChooser();

int returnVal = chooser.showOpenDialog(parent);
if(returnVal == JFileChooser.APPROVE_OPTION)
{
    fileName= chooser.getSelectedFile().getName();
}
```

Menu

JMenuBar, JMenu, JMenuItem (à imbriquer et à attacher à la fenêtre principale)
JPopupMenu ==> menu contextuel (sur click droit)

1.1.d. Fonctionnalités générales des composants SWING élémentaires:

<code>.setToolTipText("Chaine InfoBulle")</code>	précise le texte de la bulle d'aide
<code>.setVisible(true/false)</code>	montre ou cache le composant
<code>.setEnabled(false/true)</code>	grise ou dégrise le composant
<code>.setOpaque(false)</code>	précise la "non transparence" du fond

Nb: ces méthodes proviennent de **JComponent**

1.1.e. Gestion simple du scrolling (JScrollPane)

```
JScrollPane jScrollPaneXXX = new JScrollPane();
...
jScrollPaneXXX.getViewPort().setView(jUneChose); // chose = liste, panel, arbre, ...
```

1.1.f. Boîte de message et Prompt

La classe **JOptionPane** comporte quelques **méthodes statiques** qu'il suffit d'appeler directement (en préfixant par le nom de la classe) et qui permettent d'afficher des boîtes de messages et des invites pour saisir des valeurs:

(NB: p=this ou null ou fenêtre parente)

méthode statique	fonctionnalité
JOptionPane. showMessageDialog (p,"Bienvenue");	Affiche une boîte de message
name= JOptionPane. showInputDialog (p,"Quel est votre nom");	Prompt
JOptionPane. showConfirmDialog (p,"voulez vous ...?") => renvoie une valeur du genre OK_OPTION ou YES_OPTION	Demande de confirmation (Yes,No,Cancel)

Paramètres optionnels de ces méthodes statiques:

title : titre de la mini boîte de dialogue.

optionType: YES_NO_OPTION ou OK_CANCEL_OPTION ou ...

messageType: WARNING_MESSAGE ou ERROR_MESSAGE ou QUESTION_MESSAGE ou INFORMATION_MESSAGE.

1.1.g. Onglets (JTabbedPane)

La classe **JTabbedPane** correspond à un conteneur de type "Série d'onglets".

Chaque onglet sera programmé sous la forme d'un élément héritant de **JPanel**.



Le simple fait d'incorporer plusieurs panneaux (JPanel) dans un conteneur de type "JTabbedPane" permet d'obtenir le fameux Look à onglets.

Chaque onglet est associé à un nom ou libellé (sous forme de chaîne de caractères):

```
JTabbedPane jSerieOnglets = new JTabbedPane();
...
jSerieOnglets.addTab("Onglet1", jPanelOnglet1);
jSerieOnglets.addTab("Onglet2", jPanelOnglet2);
```

1.2. Gestionnaire de répartition (LayoutManager)

Un gestionnaire de répartition (défini par l'interface **LayoutManager**) est un **objet invisible** qui sera **associé à un conteneur** et qui servira à disposer au mieux les contrôles qui seront ultérieurement placés dans le conteneur.

Le programmeur a simplement besoin d'installer (de façon facultative) le gestionnaire de répartition.

→ conteneur.**setLayout**(layoutManager);

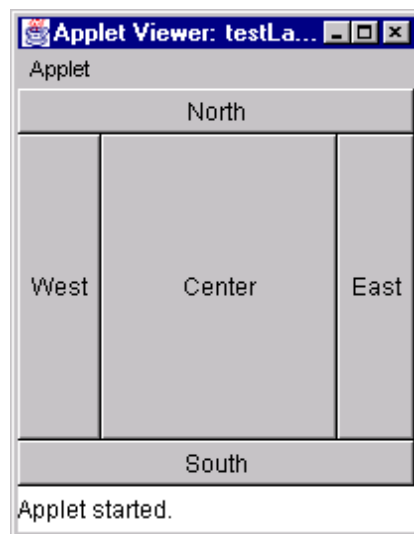
Celui-ci sera alors automatiquement utilisé par la suite.

Il faudra tout de même préciser certains paramètres lors de l'ajout d'un contrôle dans le conteneur → exemple: conteneur.**add**("South",contrôle);

JAVA dispose de quelques gestionnaires de répartition prédéfinis:

BorderLayout	Arrange les contrôles sur les bords et au centre du conteneur en fonction des indications "South", "North", "East", "West", "Center" .
---------------------	--

	Les paramètres facultatifs <i>hgap</i> et <i>vgap</i> du constructeur permettent de préciser l'espacement entre deux composants adjacents.
FlowLayout (par défaut)	Arrange les composants en ligne de gauche à droite. Fait entrer autant de composants que possible sur la ligne courante avant de passer à la prochaine. Au sein d'une ligne les choses sont alignées en fonction du paramètre facultatif <i>align</i> du constructeur. FlowLayout.CENTER , .LEFT , .RIGHT
GridLayout, CardLayout, GridBagLayout, ...	... autres gestionnaires (complexes)...



1.2.a. Positionnement absolu (sans *LayoutManager*)

```
this.*getContentPane().*/setLayout( null );
```

```
textField1.setBounds( new Rectangle(126, 8, 63, 21) );
```

1.3. Classe Graphics – pour Dessiner (lignes, rectangles, ...)

Pour dessiner quoi que ce soit dans une fenêtre ou dans un autre périphérique graphique (imprimante, mémoire), il faut passer par la classe Graphics. Cette classe est à mettre en parallèle avec la notion de "Graphic Context (gc)" de X-Window ou bien encore avec le "Device Context (DC)" de Win32.

La classe java.awt.**Graphics** permet de préciser comment les choses seront dessinées (Coordonnées, fontes, couleurs, modes des tracés, styles des hachures, ...). C'est aussi au travers d'elle que seront invoquées les méthodes permettant de dessiner telles que `drawLine()` ou `fillRect()`.

Dans la pratique, on ne peut pas directement créer une nouvelle instance de la classe Graphics qui est d'ailleurs abstraite mais on procédera de la façon suivante:

- En manipulant l'instance "graphics" qui nous est fournie par la méthode **paint()** permettant de (re)dessiner le contenu de la fenêtre qui a besoin d'être rafraîchie.
- En obtenant l'instance en invoquant la méthode **getGraphics()** des classes Component ou Image.

```
public abstract class java.awt.Graphics extends java.lang.Object {
    // Constructors: protected Graphics();
    public abstract void clearRect(int x, int y, int width, int height);
    public abstract void clipRect(int x, int y, int width, int height);
    public abstract void drawArc(int x, int y, int width, int height, int startAngle, int arcAngle);
    public abstract boolean drawImage(Image img, int x, int y, ImageObserver observer);
    ...
    public abstract void drawLine(int x1, int y1, int x2, int y2);
    public abstract void drawOval(int x, int y, int width, int height);
    public abstract void drawPolygon(int xPoints[], int yPoints[], int nPoints);
    public void drawRect(int x, int y, int width, int height);
    public abstract void drawRoundRect(int x, int y, int width, int height,
                                         int arcWidth, int arcHeight);
    public abstract void drawString(String str, int x, int y);
    public abstract void fillArc(int x, int y, int width, int height, int startAngle, int arcAngle);
    public abstract void fillOval(int x, int y, int width, int height);
    public abstract void fillPolygon(int xPoints[], int yPoints[], int nPoints);
    public abstract void fillRect(int x, int y, int width, int height);
    public abstract void fillRoundRect(int x, int y, int width, int height,
                                         int arcWidth, int arcHeight);
    ...
    public abstract void setColor(Color c);
    public abstract void setFont(Font font);
    public abstract void setPaintMode();
    public abstract void setXORMode(Color c1);
    public abstract void translate(int x, int y);
}
```

2. Gestion des événements

Le principe de fonctionnement du modèle événementiel en vigueur depuis le jdk 1.1 **est basé sur le modèle "fournisseur / abonnés"**:

L'événement sera envoyé à tous les objets qui auront préalablement marqué leurs intérêts vis à vis de celui-ci via une procédure d'enregistrement.

Au sein de ce modèle événementiel, il faut considérer plusieurs entités :

- L'objet qui génère l'événement (fenêtre, contrôle, ...) (*source d'événement*)
- Un (ou plusieurs) objet(s) qui vont traiter (gérer) l'événement (*listener*).
+ un éventuel objet *intermédiaire* appelé *adaptateur*
- Un objet (*event*) qui va véhiculer les détails (données) de l'événement.

Dans le cas le plus simple une fenêtre parente (englobante) pourra gérer le rôle de listener.

2.1.a. Notion d'abonnement

Le fait de s'abonner se code de la façon suivante:

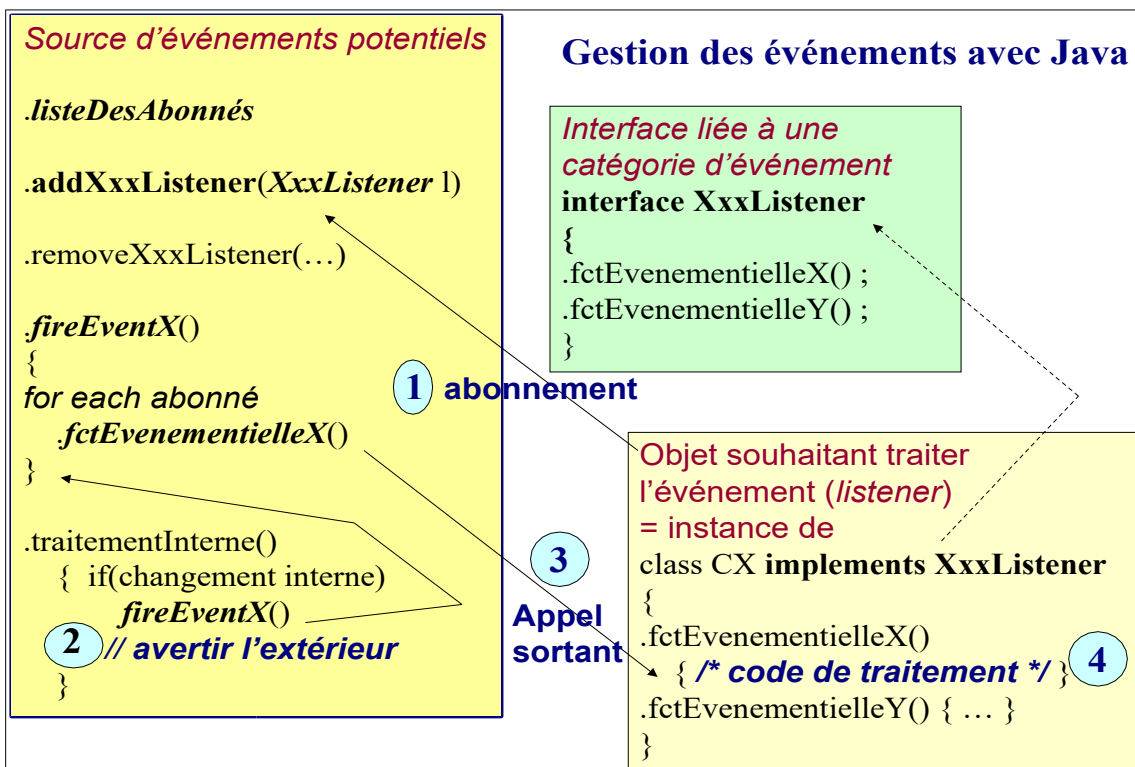
```
composantSource.addXXXListener(objetQuiSouhaiteGererEvenement);
```

```
composantSource.addXXXListener(deuxièmeAbonné);
```

>>>> *XXX correspond ici à une catégorie d'événements.* <<<<

Pour chacune de ces catégories (XXX) , il existe:

- une méthode pour s'abonner: *addXXXListener()*
- une **interface XXXListener** que doit implémenter l'objet qui s'abonne et qui souhaite traiter l'événement.
- une sous classe d'événement *XXXEvent* héritant de la classe abstraite *java.awt.AWTEvent* dérivant elle même de *java.util.EventObject*.



Exemple:

Si l'on veut traiter au niveau d'une instance de la classe Delegate le click sur un bouton poussoir, il faudra alors coder les choses de la façon suivante:

```
import java.awt.*; import javax.swing.*;
import java.awt.event.*; // pour accéder à l'interface XXXListener

class PetiteFenetre extends JFrame{
    public PetiteFenetre()
    {
        Delegate unDelegate = new Delegate();
        JButton monBouton = new JButton("Touch Me");
        monBouton.addActionListener(unDelegate); // abonnement
        this.getContentPane().add(monBouton);
    }
    public static void main(String [] argv)
    {
        JFrame f = new PetiteFenetre();
        f.pack();
        f.setVisible(true);
    }
}
```

```
class Delegate implements ActionListener
{
    ...    /* il faut coder toutes les méthodes de l'interface ActionListener
           → soit une seule méthode ici (cas le plus simple) */
    public void actionPerformed(ActionEvent e)
    {
        // traitement (gestion) du message:
        System.exit(0);
    }
}
```

Dans la plupart des interfaces graphiques(IHM) , l'entité délégué qui traite l'événement est très fortement lié à l'objet fenêtre , Panneau, ou boîte de dialogue qui contient directement le composant source qui sera à la source de l'événement (ex: Bouton poussoir).

NB1: Un composant graphique peut très bien gérer lui même l'événement reçu:
(implements XXXListener et addXXXListener(this);)

NB2: Si c'est la fenêtre parent (qui s'abonne et) qui gère les événements provenant de ses fenêtres filles, il faudra alors distinguer celles-ci via la source de l'événement:

```
Object origine=e.getSource();
    if(origine==sousComposant1)
    { ... }
    else if(origine== sousComposant2) ...
```

La plupart des générateurs de code ont généralement recours à l'utilisation des classes imbriquées:

```
class MaFenetre .... // classe "conteneur graphique"
{
...
public void initialiser()
{
...
jButtonOK.addActionListener(new
    /*début du code de la classe imbriquée*/ java.awt.event.ActionListener()
    {
        public void actionPerformed(ActionEvent e)
        {
            jButtonOK_actionPerformed(e);
        } /* fin du code de la classe imbriquée */
    });
...
}
...
void jButtonOK_actionPerformed(ActionEvent e)
{ ... /* traitement de l'événement */ }
}
```

Interfaces liées aux catégories de messages (ayant chacune une méthode d'abonnement addXXXListener et un type d'événement XXXEvent qui leur correspondent) :

<i>Interface / catégorie d'événements</i>	<i>types de composants sources</i>
ActionListener	Button (éventuellement JRadioButton, JCheckBox, ...)
MouseListener	Panel, Canvas , ...(éventuellement JList, ...)
KeyListener	Zone de saisie (==> codez en priorité <i>keyReleased</i>)
ItemListener	Boutons radios , cases à cocher , zones de listes, ...
...	...

<i>Interface</i>	<i>méthodes à implémenter (void)</i>
ActionListener	actionPerformed (ActionEvent)
AdjustmentListener	ajustmentValueChanged(AjustmentEvent)
ComponentListener	componentHidden(ComponentEvent) componentShown(ComponentEvent) componentMoved(ComponentEvent) componentResized (ComponentEvent)
ContainerListener	componentAdded(ContainerEvent) componentRemoved(ContainerEvent)
FocusListener	focusGained(FocusEvent) focusLost(FocusEvent)
ItemListener	itemStateChanged (ItemEvent)
KeyListener	keyPressed(KeyEvent) keyReleased (KeyEvent) keyTyped(KeyEvent)
MouseListener	mouseClicked (MouseEvent) mouseEntered(MouseEvent) mouseExited(MouseEvent)

	mousePressed (MouseEvent) mouseReleased (MouseEvent)
MouseMotionListener	mouseDragged(MouseEvent) mouseMoved(MouseEvent)
TextListener	textValueChanged(TextEvent)
WindowListener	windowActivated(WindowEvent) windowClosed (WindowEvent) windowClosing(WindowEvent) windowDeactivated(WindowEvent) windowDeiconified(WindowEvent) windowIconified(WindowEvent) windowOpened(WindowEvent)

Exemple très classique (nouvelle sélection dans une liste déroulante):

```
jComboBoxTypeBase.addItemListener(new java.awt.event.ItemListener()
{
    public void itemStateChanged(ItemEvent e) {
        jComboBoxTypeBase_itemStateChanged(e);
    }
});
...
void jComboBoxTypeBase_itemStateChanged(ItemEvent e) {
    int index=jComboBoxTypeBase.getSelectedIndex();
    ... }
...
```

2.1.b. Adaptateurs

NB: L'API de JAVA fournit des *adaptateurs par défaut* qui ne font rien (code vide) sous la forme de *classes abstraites* (**MouseAdapter**, **KeyAdapter**, **WindowAdapter**, ...). Il suffit alors de dériver une ces classes pour ensuite avoir la possibilité de ne coder (redéfinir) qu'une seule des méthodes de l'interface XXXListener correspondante:

```
class MonAdaptateur extends MouseAdapter {
    public void mouseEntered(MouseEvent e) { ... }
}
```

Exemples très classiques:

```
jTreeDep.addMouseListener(new java.awt.event.MouseAdapter() {
    public void mouseClicked(MouseEvent e) {
        jTreeDep_mouseClicked(e);
    } });
...
```

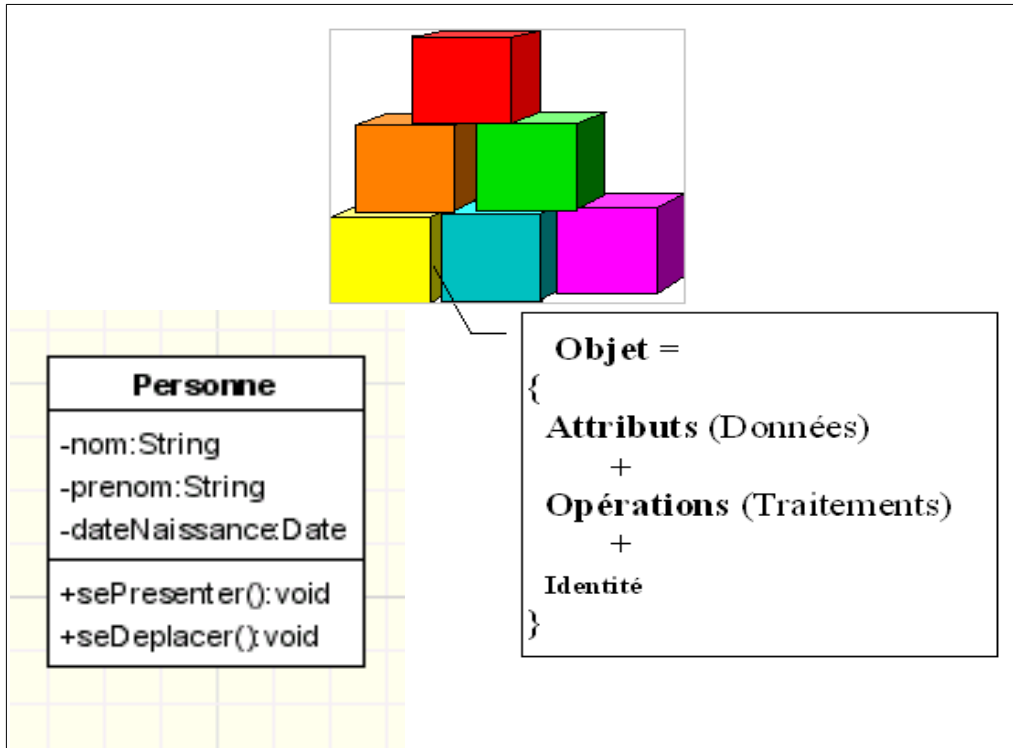
```
jTextFieldAnneeMini.addKeyListener(new java.awt.event.KeyAdapter() {
    public void keyReleased(KeyEvent e) {
        jTextFieldAnneeMini_keyReleased(e);
    } });
...
```

ANNEXES

XV - Annexe – Prog Objet et UML (aperçu)

1. Concepts objets fondamentaux

1.1. Objets, classes et instances

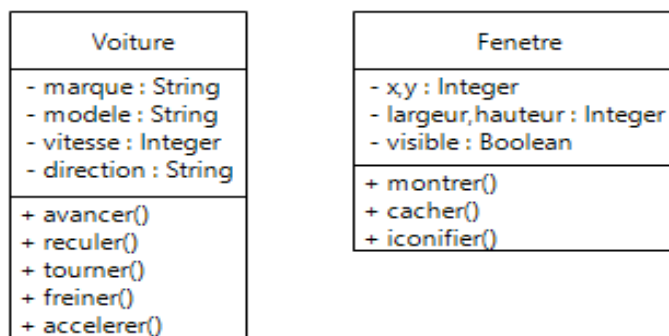


Notion de classe

Pour **regrouper** les objets aux caractéristiques et comportements identiques, on fait appel à la notion de **classe**.

Une **classe** peut être vue comme un **type** (*abstrait ou concret*) d'objets.

Une **classe** est une sorte de **moule** à partir duquel seront générés les objets que l'on appelle **instances** de la classe. [*Un objet est créé à l'image de sa classe*]



Toutes les instances d'une même classe ont une même structure commune et ont un même comportement.

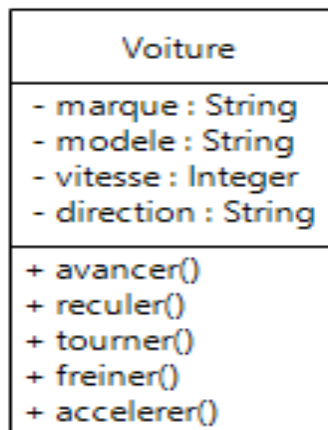
Instances

Rappel: un **exemplaire** (objet) issu à partir d'une certaine classe est appelé une **instance**.

Les différentes instances d'une même classe se distinguent par les **valeurs** (*assez souvent différentes*) de leurs **attributs** ==> **Chaque instance a ses propres valeurs** et ainsi son propre **état**.

instance 1

Classe



V_AZ45BV456:Voiture
 marque=Renault
 modèle=Clio
 vitesse=50 , direction=nord

...

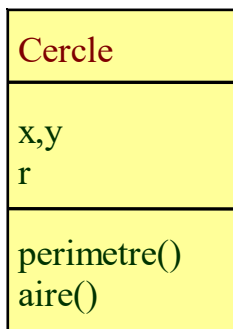
instance N

V_BN78BV936:Voiture
 marque=Peugeot
 modèle=308
 vitesse=90 , direction=sud

Classes & Instances

Modélisation UML

Classe java (Cercle.java → Cercle.class)



```
public class Cercle {
  // Attributs (données):
  private double x,y; // coordonnées du centre
  private double r; // rayon
  // Méthodes:
  public double perimetre() { return 2 * Math.PI * r; }
  public double aire() { return Math.PI * r * r; }
}
```

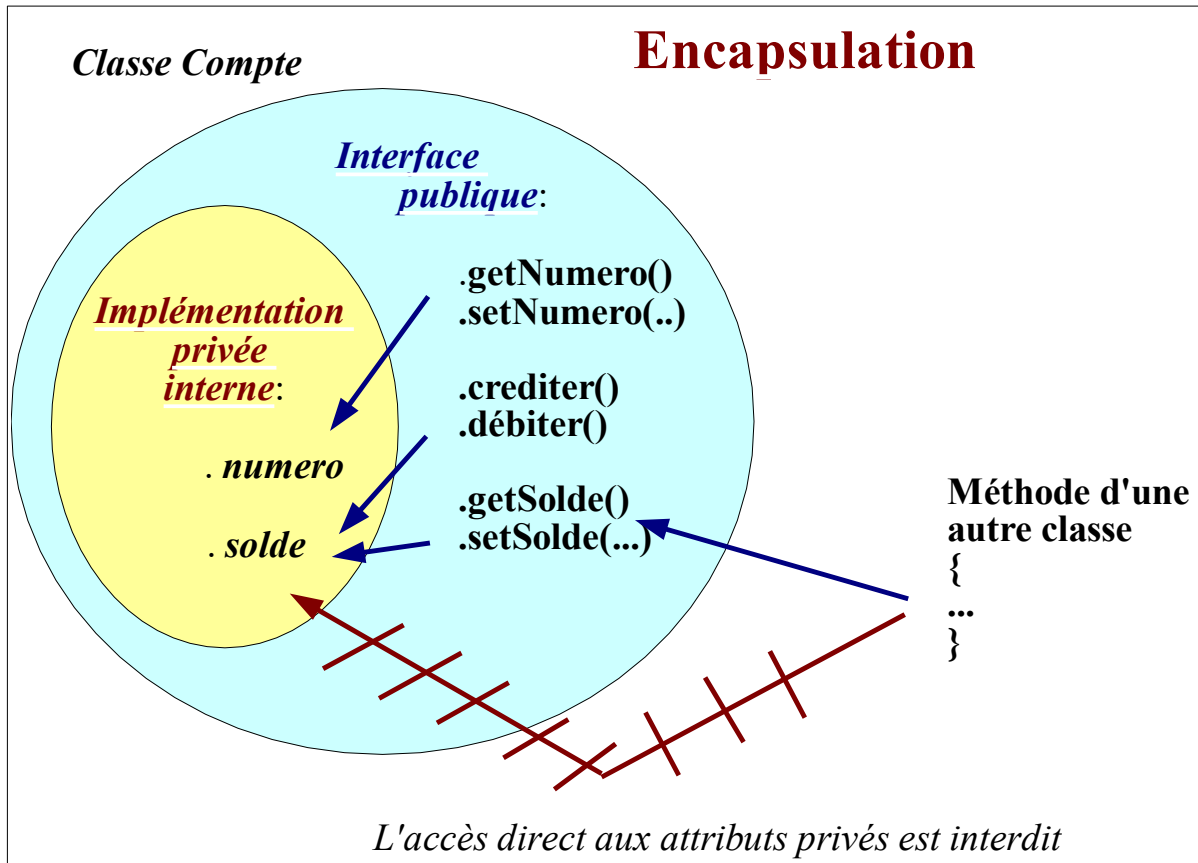
Pour utiliser cette classe, il nous faut maintenant :

- déclarer une variable de type "référence sur un objet de classe Cercle"
- créer un objet ou une instance (exemplaire) de la classe via l'opérateur **new**.

Cercle c1; //c1 est ici une référence sur un objet de type Cercle qui n'existe pas encore.

c1 = new Cercle(); //c1 référence maintenant un nouvel exemplaire de la classe Cercle.

1.2. Encapsulation et communication par messages



Communication par messages



La partie dynamique des objets est assurée par la notion d'envoi de message.
Envoyer un message à un objet c'est lui dire ce qu'il doit faire.

Lorsqu'un objet reçoit un message, il active la méthode de même nom définie au niveau de sa classe.

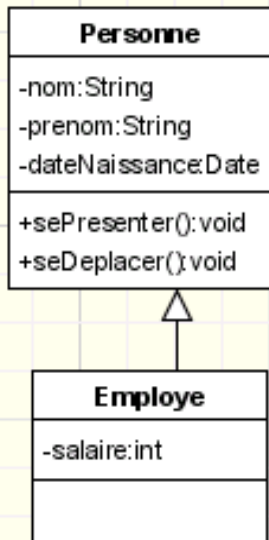
Lorsque O1 envoie le message Msg à O2, il ne fait que demander à O2 d'effectuer l'opération qui est en relation avec le message .
message = requête.

Tout message comporte trois éléments:

- * le destinataire (objet receveur)
- * le sélecteur (Nom du message ou de la méthode)
- * une liste d'arguments éventuellement vide

1.3. Héritage et polymorphisme

Héritage / Généralisation / Spécialisation

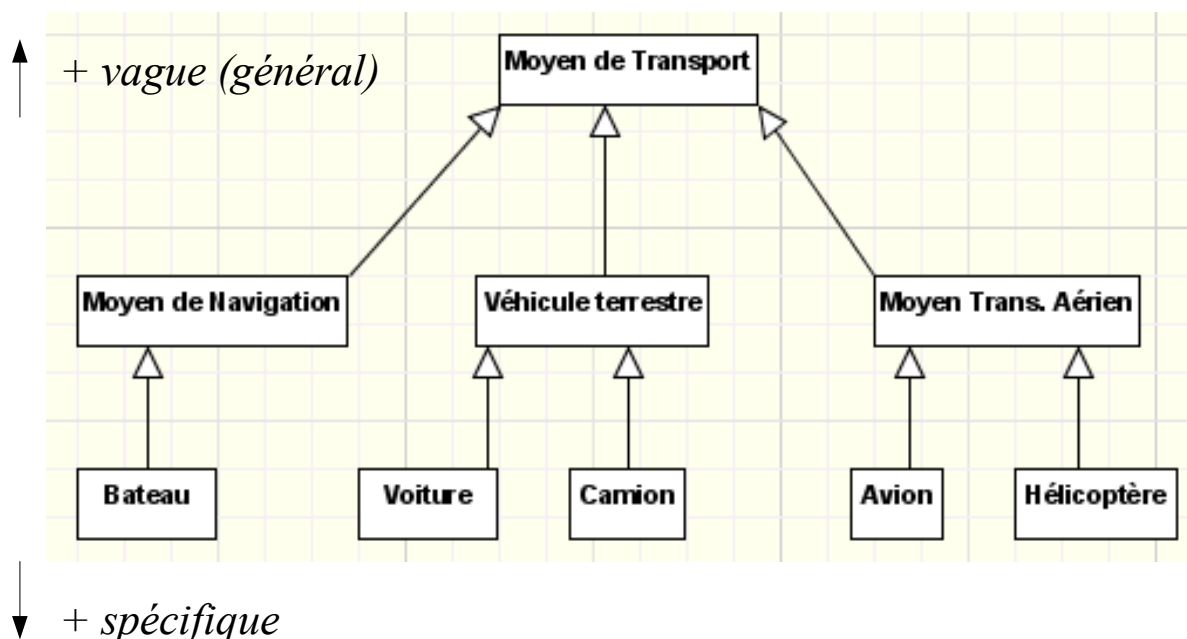


* La notion d'héritage consiste à définir une nouvelle classe à partir d'une classe existante en spécialisant certaines choses.

* La classe dérivée (ou sous classe) reprendra tous les attributs et toutes les opérations de la surclasse .

* La structure de données est conservée au niveau des sous classes; elle peut néanmoins être enrichie via l'ajout de nouveaux attributs.

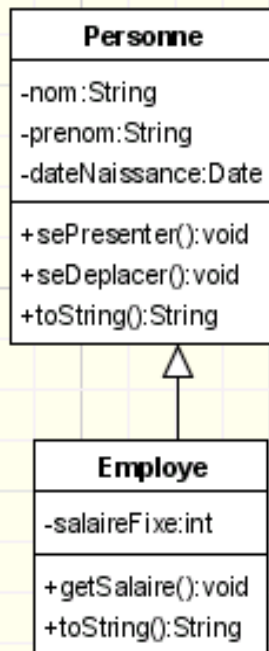
Arbre (ou graphe) d'héritage



Une relation d'héritage est représentée par une grosse flèche triangulaire allant de la sous-classe vers la sur-classe et signifiant

"is kind of / est une sorte de"

Héritage et lien entre types de données



En héritant de *Personne* la classe *Employé* est alors implicitement considérée comme un *type de donné* spécifique mais néanmoins compatible avec le type générique *Personne*.

Un employé est un **cas particulier** de personne.

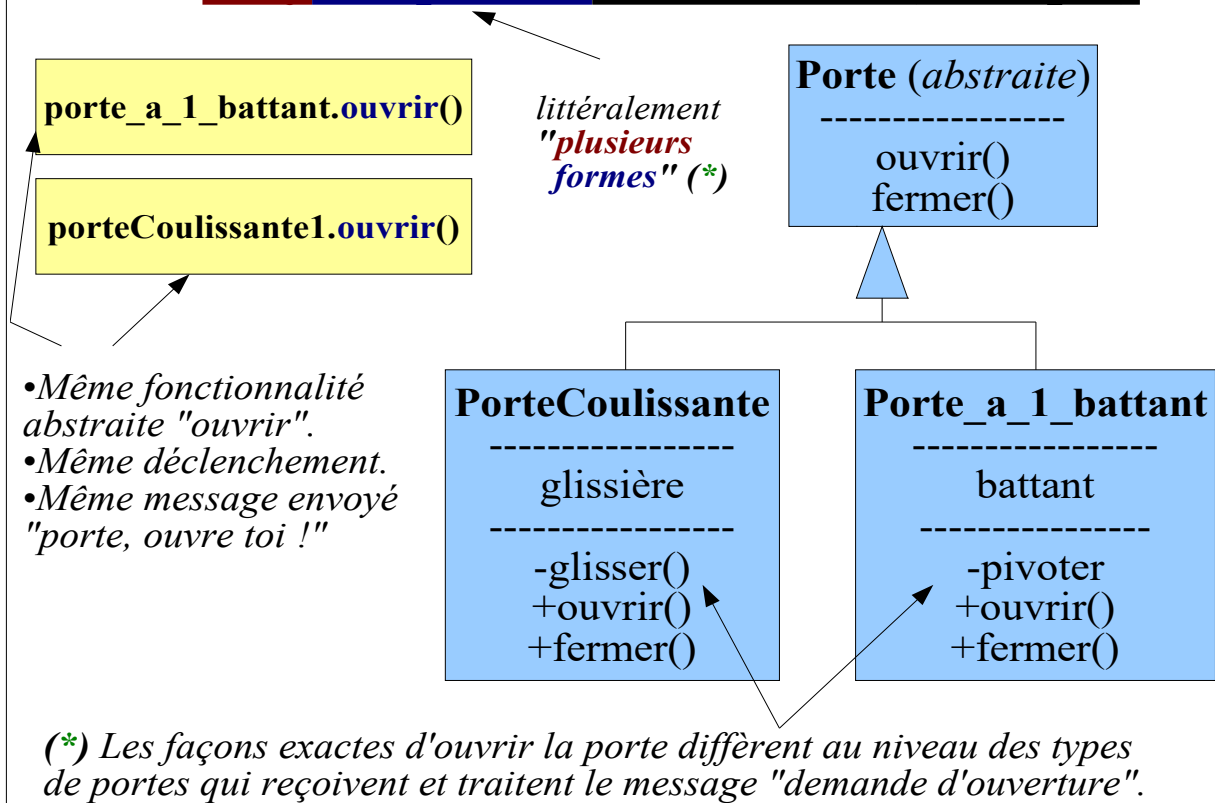
```

Personne p = null;
Employe emp = new Employe("dupond" , ...);

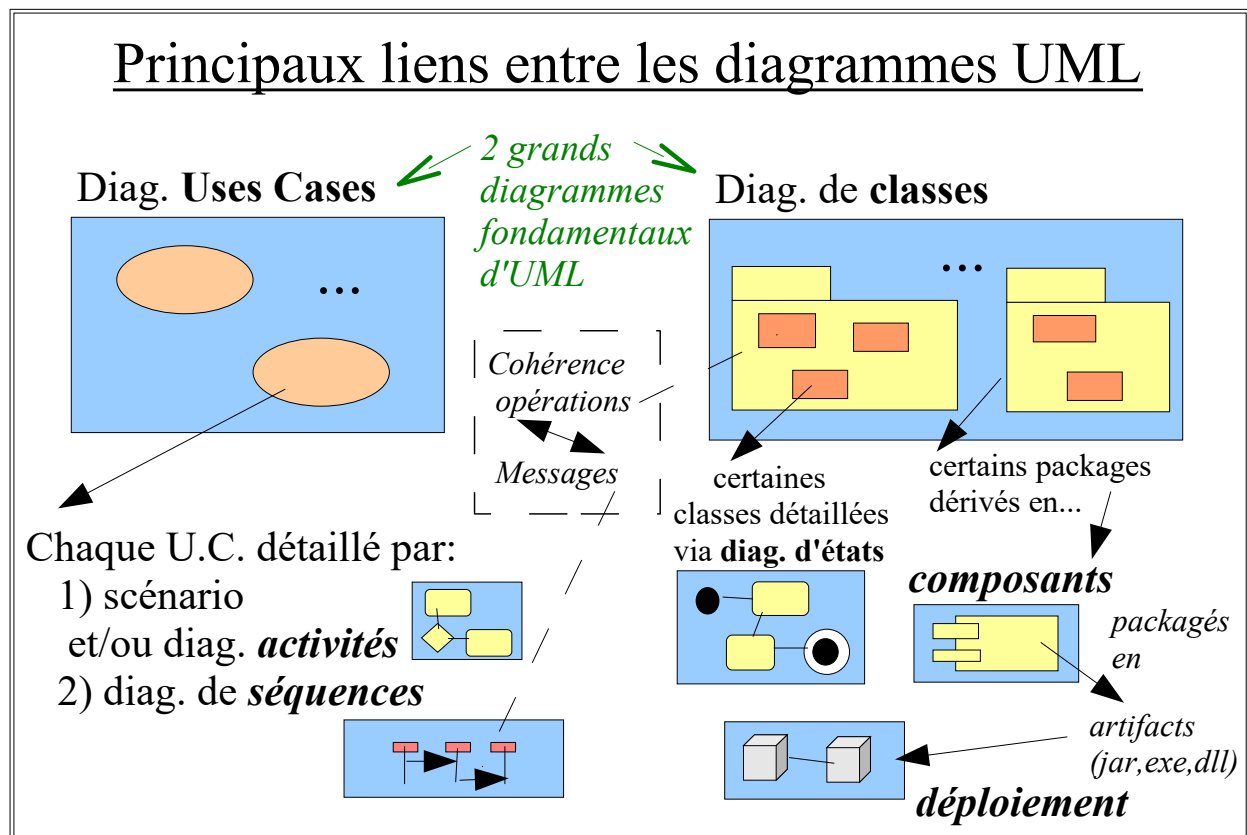
p = emp;

p.sePresenter(); ...
  
```

Polymorphisme fonctionnel (exemple)

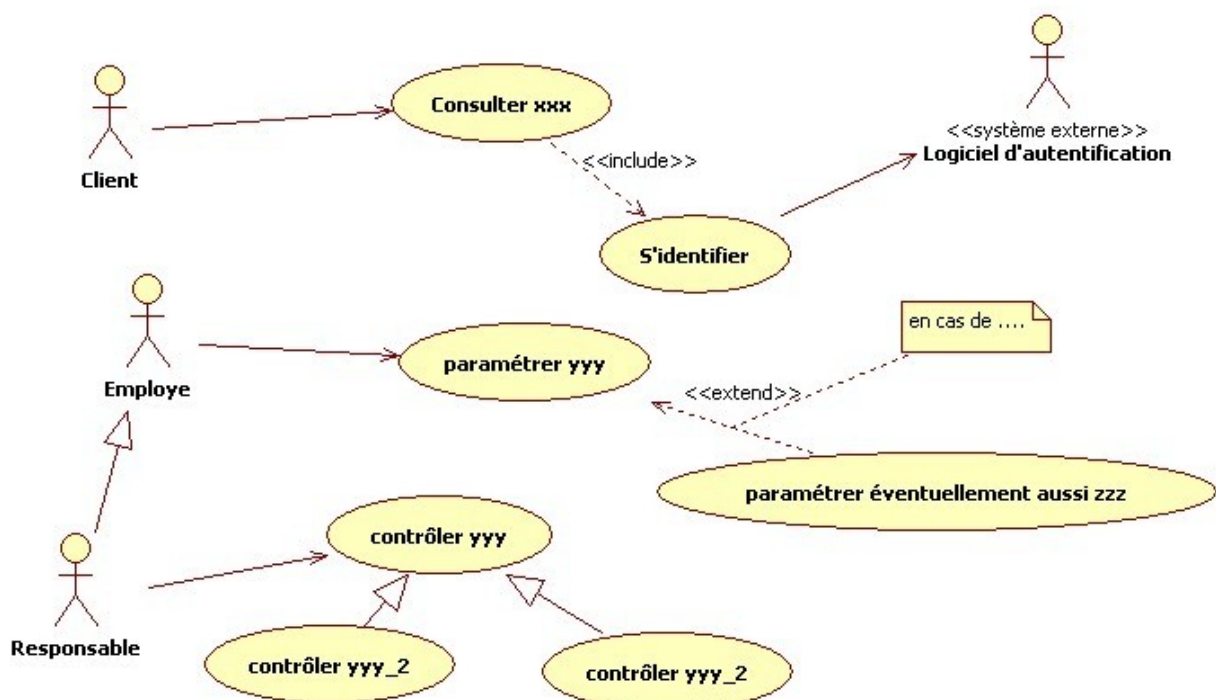


2. Aperçu sur UML (Unified Modeling Language)

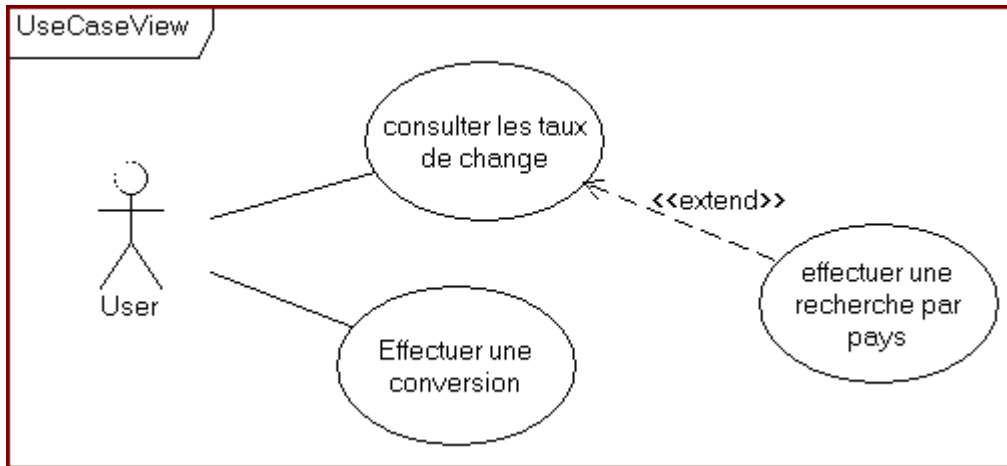


2.1. Diagramme des cas d'utilisations (Uses Cases)

exemple:



sur micro étude cas "*conversion de devises*":



2.2. Diagramme de classes

exemples:

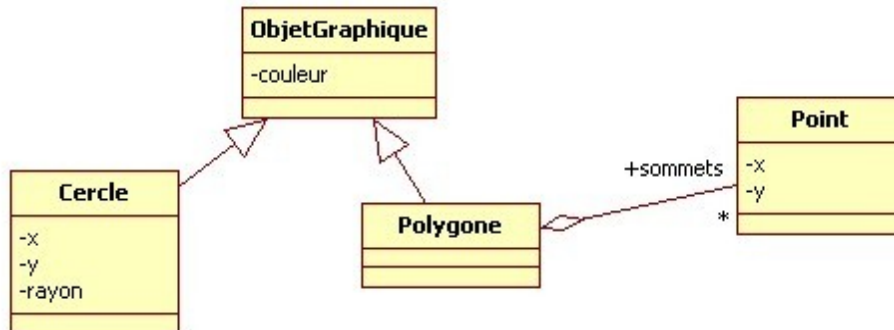
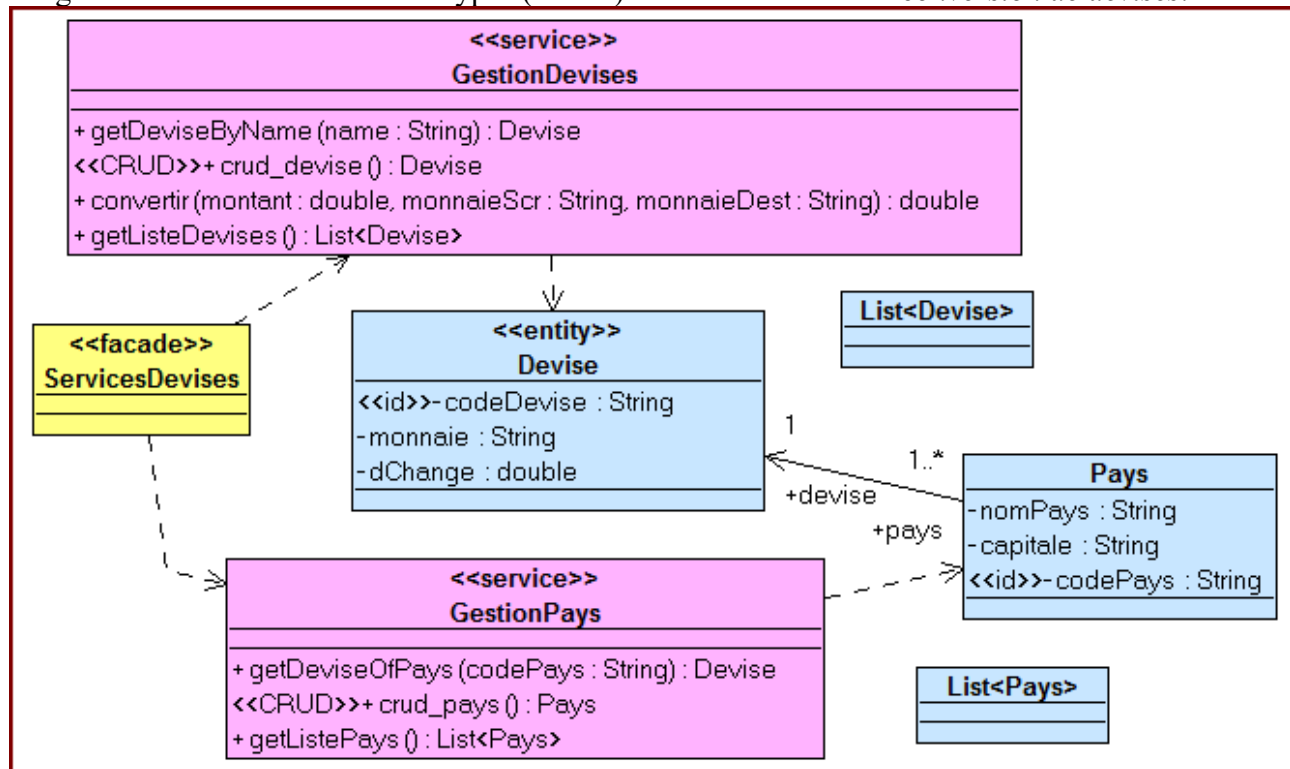
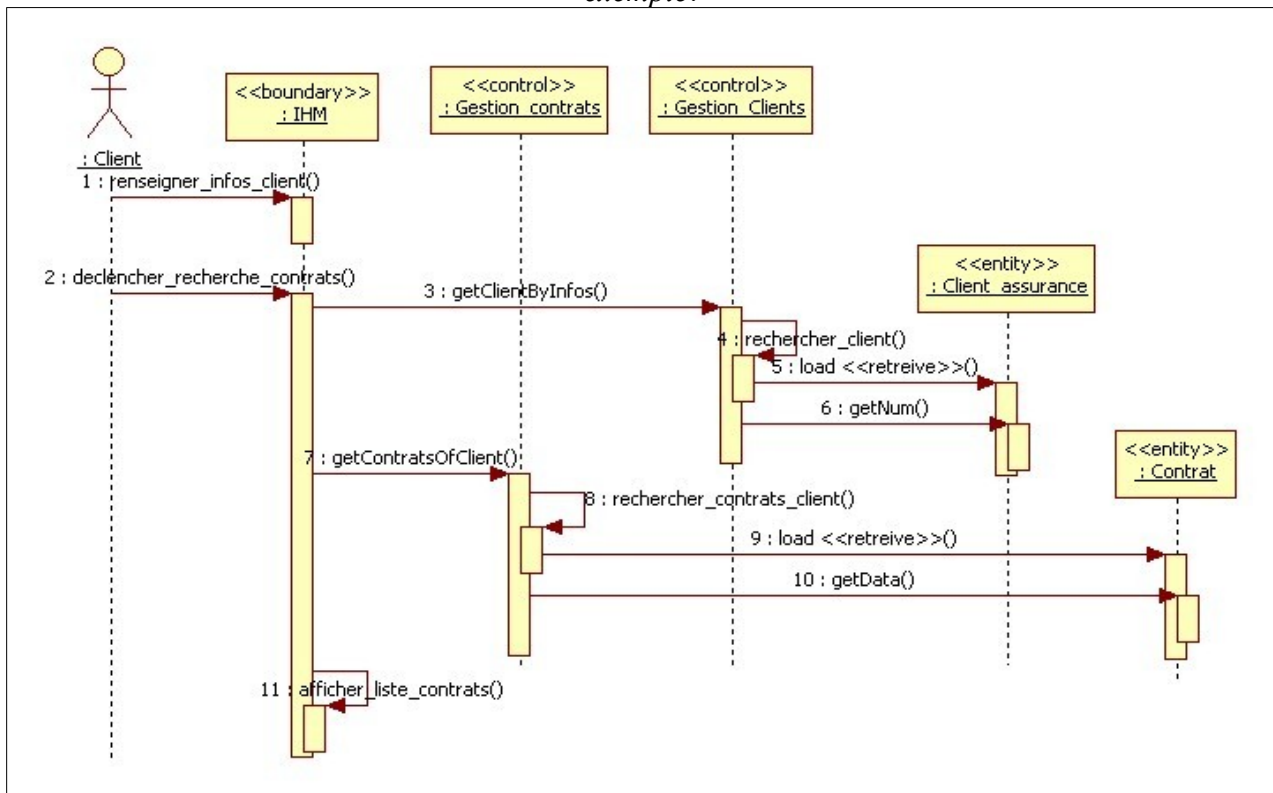


Diagramme de classes avec stéréotypes (n-tiers) sur micro étude cas "conversion de devises":



2.3. Diagramme de séquence

exemple:



XVI - Annexe – nouveautés v12-v17 et v18-v21

1. Switch expressions

La nouvelle forme du **switch()** avec des "**lambda**" était apparue dès les jdk 1.12 et 1.13 en mode "preview". C'est enfin devenu une nouvelle fonctionnalité "standard" à partir de la version **1.14**

1.1. expression d'un switch/case avec des "lambda"

```
int dayOfWeek = 4;
System.out.print("dayOfWeek="+dayOfWeek + " -> ");
switch(dayOfWeek) {
    case 1 -> System.out.println("monday");
    case 2 -> System.out.println("tuesday");
    case 3 -> System.out.println("wednesday");
    case 4 -> System.out.println("thursday");
    case 5 -> System.out.println("friday");
    case 6 -> System.out.println("saturday");
    case 0 , 7 -> System.out.println("sunday");
    default -> System.out.println("unknown");
}
```

Résultat:

```
dayOfWeek=4 -> thursday
```

1.2. switch en tant qu'expression retournant un résultat

```
String dayName="vendredi";
System.out.print("dayName="+dayName + " -> ");
int dayNumber =
    switch(dayName) {
        case "lundi" , "monday" -> 1;
        case "mardi" , "tuesday" -> 2;
        case "mercredi" , "wednesday" -> 3;
        case "jeudi" , "thursday" -> 4;
        case "vendredi" , "friday" -> 5;
        case "samedi" , "saturday" -> 6;
        case "dimanche" , "sunday" -> 7;
        default -> 0;
    };
System.out.println("dayNumber="+dayNumber);
```

Résultat :

```
dayName=vendredi -> dayNumber=5
```

1.3. switch avec expressions complexes et mot clef yield

Lorsqu'une expression doit nécessiter plusieurs instructions pour être calculée, on peut englober celles-ci par un bloc d'accolades et préciser la valeur calculée/retournée via le mot clef "yield" (ressemblant à un "return" mais de niveau switch et pas de niveau fonction) .

```
int value = (int) (Math.random() * 14);
String result = switch( value ) {
    case 0, 2, 4, 6, 8 -> {
        double racine = Math.sqrt( value );
        yield "chiffre pair dont la racine carré vaut " + racine;
    }
    case 1, 3, 5, 7, 9 -> {
        double carre = value * value;
        yield "chiffre impair dont le carré vaut " + carre;
    }
    default -> "ce n'est plus un chiffre, mais un nombre";
};
System.out.println("value=" + value + "->" + result );
```

Résultats :

value=2->chiffre pair dont la racine carré vaut 1.4142135623730951

value=13->ce n'est plus un chiffre, mais un nombre

value=3->chiffre impair dont le carré vaut 9.0

...

2. Pattern Matching instanceof

En preview en version 14, 15 et standardisé en version 16 et 17/LTS .

```
public class TestPatternMatchingInstanceOfApp {

    public static void main(String[] args) {
        String s = "abc";
        oldStyleWithoutPatternMatchingInstanceOf(s);
        newStyleWithPatternMatchingInstanceOf(s);
    }

    public static void oldStyleWithoutPatternMatchingInstanceOf(Object obj) {
        if (obj instanceof String) {
        String str = (String) obj;
        int len = str.length();
        System.out.println("obj is as string of length="+len);
        }
    }

    public static void newStyleWithPatternMatchingInstanceOf(Object obj) {
        if (obj instanceof String str) {
        int len = str.length();
        System.out.println("obj is as string of length="+len);
        }
    }
}
```

C'est un **sucre syntaxique** (simplification de syntaxe) **bien pratique** .

3. TextBloc

Un "TextBloc" (depuis java 15) est une chaîne de caractère formulée comme un **bloc de texte sur plusieurs lignes**. Encadré par des `"""` (triple guillemet), un *textBloc* peut comporter des caractères quelconques qui n'ont plus besoin d'échappement.

Ceci est surtout utile pour le format JSON (nécessitant des `"` et pas de `'`) à l'intérieur d'une chaîne java délimitée par des `"`.

écriture sans textbloc et très peu lisible :

```
public static void oldStyleWithoutTextBloc() {
    String myJsonString
        = "{\r\n" + "\"username\" : \"JeanBon\", \r\n" + "\"country\" : \"France\" \r\n" + "}";
    System.out.println("myJsonString="+myJsonString);
}
```

équivalent bien plus lisible avec textBloc :

```
public static void newStyleWithTextBloc() {
    String myJsonStringAsTextBloc = """
        {
            "username" : "AlexTherieur",
            "country" : "Belgique"
        }
        """;
    System.out.println("myJsonStringAsTextBloc="+myJsonStringAsTextBloc);
}
```

→ affiche

```
myJsonStringAsTextBloc={
"username" : "AlexTherieur",
"country" : "Belgique"
}
```

```
public static void someTextBlocs() {
    //au minimum 2 lignes de code :
    //  """ suivi de newligne pour commencer
    //  texte quelconque au milieu
    //  """; pour finir
    String simpleTextBloc0 = """
        blabla""";
    System.out.println("simpleTextBloc0="+simpleTextBloc0);
    //affiche simpleTextBloc0=blabla
}
```

```
//pour bien controller l'intentation , il faut savoir que lorsque le compilateur va analyser
//le code source, il va supprimer le nombre minimum de tabulations qu'il y a
//au niveau de toutes les lignes de code (du textBloc sans tenir compte du tout début = "")
//Bonne pratique : au sens IDE (eclipse/IntelliJ/...) placer des tabulations pour indenter le code
//          au sens contenu final du textBloc , utiliser plutôt des espaces et sauts de lignes
//          placer la fin "" sur une ligne séparée
//          (comportant le nombre de tabulations qui seront explicitement supprimées du texte)
String textBloc3AvecIndentationControlee = ""
    {
        "username" : "AlexTherieur",
        "country" : "Belgique"
    }
    """;

System.out.println("textBloc3AvecIndentationControlee="+textBloc3AvecIndentationControlee);

//par défaut les espaces en fin de ligne sont considérés inutiles et ne sont pas conservés
//le caractère d'échappement \s permet d'explicitement demander à conserver les espaces en fin de ligne
String textBloc4AvecEspacesConserveEnFinDeLignes = ""
    blabla  \s""";

System.out.println("textBloc4AvecEspacesConserveEnFinDeLignes="
    +textBloc4AvecEspacesConserveEnFinDeLignes + "suiteApres");

}
```

```
public static void textBlocsWithFormattedParams() {
    String username="toto";
    int age=40;
    double height = 1.73;
    boolean ok=true;
    String jsonString = ""
        {
            "username" : "%s",
            "age" : %d,
            "height" : %s,
            "ok" : %b
        }
        """.formatted(username,age,String.valueOf(height),ok);
        //same syntax of String.format
    System.out.println("jsonString="+jsonString);
}
```

affiche

```
{ "username" : "toto",
  "age" : 40,
  "height" : 1.73,
  "ok" : true
}
```

4. Record (classe de données simplifiée pour DTO)

"Preview" en version 14 et 15, les "record" sont standardisés depuis java 16 (et 17 LTS) .

//exemple de classe POJO/JavaBean classique (pour comparaison) :

```
public class OldCustomer{

    private Integer id;
    private String firstName;
    private String lastName;

    public OldCustomer() {        super();
    }

    public OldCustomer(Integer id, String firstName, String lastName) {
        super();
        this.id = id;
        this.firstName = firstName;
        this.lastName = lastName;
    }

    @Override
    public String toString() {
        return "OldCustomer [id=" + id + ", firstName=" + firstName
            + ", lastName=" + lastName + "]";
    }

    public Integer getId() {
        return id;
    }
    public void setId(Integer id) {
        this.id = id;
    }

    //+ autres get/set
    //+ hashCode()
    //+ equals()
}
```

classe équivalente avec annotations de lombok :

```
import lombok.*;

@Getter @Setter @ToString
@NoArgsConstructor @AllArgsConstructor
@EqualsAndHashCode
public class LombokCustomer {
    private Integer id;
    private String firstName;
    private String lastName;
}
```

Utilisation classique d'un javaBean codé avec ou sans lombok:

```
OldCustomer oldC1 = new OldCustomer(1,"jean","Bon");
System.out.println("oldC1 as POJO =" + oldC1.toString());
System.out.println("id of oldC1 =" + oldC1.getId());
oldC1.setFirstName("luc");
System.out.println("firstName of oldC1 =" + oldC1.getFirstName());
```

```
OldCustomer oldC2 = new OldCustomer();
System.out.println("oldC2 =" + oldC2.toString());
```

```
System.out.println("-----");
```

```
LombokCustomer lC1 = new LombokCustomer(1,"jean","Bon");
System.out.println("olC1 as Lombok POJO =" + lC1.toString());
System.out.println("id of lC1 =" + lC1.getId());
lC1.setFirstName("luc");
System.out.println("firstName of lC1 =" + lC1.getFirstName());
```

```
LombokCustomer lC2 = new LombokCustomer();
System.out.println("lC2 =" + lC2.toString());
```

Nouveau mot clef **record**

Le nouveau mot clef **record** permet de demander au compilateur de construire automatiquement une sorte de pseudo **POJO IMMUTABLE** (sans Setter possible !) et avec des accesseurs de type .xxx() ne respectant même pas les conventions .getXxx() !!!

Dans les détails : classe construite "final" héritant de **java.lang.Record** et avec des champs privés "final" et avec .equals()/.hashCode()/.toString() et des accesseurs de type .xxx() et allArgsConstructor .

NB :

- record ne peut pas être vu comme un remplacement de lombok car IMMUTABLE/final/read-only et car .xxx() plutôt que .getXxx()
- record doit plutôt être vu comme une sorte de petit DTO/VO très léger

Minimalist v1 (with no default constructor):

```
public record CustomerRecord(Integer id,String firstName,String lastName) {
}
```

Utilisation :

```
CustomerRecord c1 = new CustomerRecord(1,"jean","Bon");
System.out.println("c1 as record =" + c1.toString());
System.out.println("id of c1 =" + c1.id()); //mais pas .getId() ni .id
c1.setFirstName("luc");
System.out.println("firstName of c1 =" + c1.firstName()); //mais pas .getFirstName() ni .firstName
```

public void **setFirstName**(String firstName) { this.firstName = firstName; }
 est **interdit/impossible** car **this.firstName** est **final/immutable** !!!

// V2 (with explicit default constructor) :

```
public record CustomerRecord(Integer id,String firstName,String lastName) {
    public CustomerRecord() { this(0,null,null);}
    //possible mais pas souvent utile si final/immutable !!!
}
```

Utilisation :

```
CustomerRecord c2 = new CustomerRecord();
System.out.println("c2="+c2.toString());
//affiche ici CustomerRecord[id=0, firstName=null, lastName=null]
```

V3 (V2 + quelques redefinitions & ajout de methodes)

```
public record CustomerRecord(Integer id,String firstName,String lastName) {
    public CustomerRecord() { this(0,null,null);}
    public String fullName() { return firstName + " " + lastName;}
    public String lastName() { return lastName.toUpperCase(); }
}
```

V4 (V3 + new record constructor checking syntax)

```
public record CustomerRecord(Integer id,String firstName,String lastName) {
    //not useful default constructor:
    public CustomerRecord() { this(0,"unknown","unknown");}

    //new method:
    public String fullName() { return firstName + " " + lastName;}

    //old pojo style getter as a new complementary method :
    public String getFirstName() { return firstName; }

    //accessor redefinition :
    public String lastName() { return lastName.toUpperCase(); }

    //specific record new syntax for allArgsConstructor
    //partial redefinition with value(s) checking:
    public CustomerRecord {
        if (lastName == null || lastName.isBlank()) {
            throw new IllegalArgumentException("lastName is required");
        }
    }
}
```

```

public class TestRecordApp {

    public static void main(String[] args) {
        testUsefulRecordV1();
        testUsefulRecordV2();
        testUsefulRecordV3();
    }

    //private or public locally RECORD seems to be a good use case for "record" new concept :

    private record AddressV1(Integer number,String street,String zipCode,String town) {
        public String toJsonString() {
            return ""
                {
                    "number" : %d,
                    "street" : "%s",
                    "zipCode" : "%s",
                    "town" : "%s"
                }
                ""formatted(number,street,zipCode,town);
        }
    };

    public static void testUsefulRecordV1() {
        AddressV1 a1 = new AddressV1(12,"rueElle","75000" ,"Paris");
        System.out.println("a1="+a1.toString());
        System.out.println("a1="+a1.toJsonString());
    }

    //V2 avec compatibilité avec api jackson-databind (souvent utilisé par JEE , Spring, ...)
    //pratique pour définition de DTO/VO (à la volée)
    public record AddressV2(Integer number,String street,String zipCode,String town) {
    };

    public static void testUsefulRecordV2() {
        AddressV2 a1 = new AddressV2(12,"rueElle","75000" ,"Paris");
        System.out.println("a1="+a1.toString());
        try {
            ObjectMapper jacksonObjectMapper = new ObjectMapper();
            String a1JsonString = jacksonObjectMapper.writeValueAsString(a1);
            System.out.println("via jackson, a1JsonString="+a1JsonString);
        } catch (JsonProcessingException e) {
            e.printStackTrace();
        }
    }
}

```

Résultats :

```

a1=AddressV1[number=12, street=rueElle, zipCode=75000, town=Paris]
a1={

```

```
"number" : 12,
"street" : "rueElle",
"zipCode" : "75000",
"town" : "Paris"
}
```

```
a1=AddressV2[number=12, street=rueElle, zipCode=75000, town=Paris]
via jackson, a1JsonString={"number":12,"street":"rueElle","zipCode":"75000","town":"Paris"}
```

*//NB : Les records de java ne sont bien gérés qu'avec des versions récentes de jackson-databind .
 // la version 2.12.5 de **jackson-databind** supporte bien les records de java 17 et est compatible
 //avec une version récente de SpringBoot et SpringMVC (sur partie @RestController)*

```
public class Dto {
    public record Address(Integer number,String street,String zipCode,String town) {
    };

    public record Person(Integer id,String firstName,String lastName,Address address) {
        public Person(Integer id,String firstName,String lastName) {
            this(id,firstName,lastName,null);
        }
    };
}
```

```
public static void testUsefulRecordV3() {
    Dto.Person p1Dto = new Dto.Person(1,"jean","Bon" ,
                                     new Dto.Address(12,"rueElle","75000" ,"Paris"));
    System.out.println("\np1Dto="+p1Dto.toString());

    try {
        ObjectMapper jacksonObjectMapper = new ObjectMapper();
        String p1JsonString = jacksonObjectMapper.writeValueAsString(p1Dto);
        System.out.println("via jackson, p1JsonString="+p1JsonString);
        //-----
        Dto.Person p1BisDto=
            jacksonObjectMapper.readValue(p1JsonString,Dto.Person.class);
        System.out.println("via jackson, p1BisDto=clone de p1Dto="+p1BisDto.toString());
    } catch (JsonProcessingException e) {
        e.printStackTrace();
    }
}
```

Résultats :

```
p1Dto=Person[id=1, firstName=jean, lastName=Bon, address=Address[number=12,
street=rueElle, zipCode=75000, town=Paris]]
```

```
via jackson, p1JsonString={"id":1,"firstName":"jean","lastName":"Bon","address":
{"number":12,"street":"rueElle","zipCode":"75000","town":"Paris"}}
```

```
via jackson, p1BisDto=clone de p1Dto=Person[id=1, firstName=jean, lastName=Bon,
address=Address[number=12, street=rueElle, zipCode=75000, town=Paris]]
```

Attention (restriction importante)

En interne les "record" sont codés comme des classes spéciales héritant de la classe abstraite *java.lang.Record* .

Etant donné que java ne supporte que l'héritage simple (via mot clef extends), **il n'est pas possible d'utiliser le mot clef extends de manière à ce qu'un record hérite d'un autre record** .

On peut cependant utiliser le mot clef **implements** entre un **record** et une (ou plusieurs) **interface(s)**.

Exemple :

```
public class MyRecords {  
    public interface Product {Long id(); String label(); Double price();}  
    public record ProductL0(Long id,String label,Double price) implements Product {  
    }  
  
    public interface ProductI1 extends Product {String features();}  
    public record ProductL1(Long id,String label,Double price,String features)  
        implements ProductI1{  
    }  
}
```

```
MyRecords.ProductL0 p0 = new MyRecords.ProductL0(1L,"productA", 12.5);
```

```
MyRecords.Product pRef = p0;
```

```
System.out.println(pRef.toString());
```

```
System.out.println("id=" + pRef.id() + " label=" + pRef.label() + " price=" + pRef.price());
```

Autre détails :

La méta classe **java.lang.Class<T>** comporte maintenant une nouvelle méthode booléenne **isRecord()** .

5. Sealed classes

"Preview" en versions 15,16 et standardisé en version 17 .

```
//NB: "SEALED" signifiant scellé en francais
//est une nouveauté du langage java permettant d'indiquer que seules certaines classes
//(précisées par permits ... , ... )
//auront le droit d'hériter de la classe ou interface actuelle
//cela ressemble à final class qui interdit carrément tout héritage

//sealed keyword can be used if option "enabled preview features for java 16"
public sealed interface AnimalDomestique permits Chat, Chien {
    public void sayHello();
    //...
}
// effet : seules les classes Chat et Chien auront le droit d'hériter de AnimalDomestique .

/*
//HERITAGE via extends ou implements INTERDIT via sealed sur AnimalDomestique :
final class Ce implements AnimalDomestique{
//...
}
*/
```

```
//déclarée final , la classe Chat ne peut pas avoir de sous classe
public final class Chat implements AnimalDomestique{

    @Override
    public void sayHello() {
        System.out.print("chat miaou - ");
    }

    public void ronronner() {
        System.out.println(" ronron ");
    }

}
```

```
//déclarée non-sealed , la classe Chien peut pas avoir des sous classes
public non-sealed class Chien implements AnimalDomestique{
    @Override
    public void sayHello() {
        System.out.print("chien wouf wouf - ");
    }

    public void aLaNiche() {
        System.out.println(" dodo niche ");
    }

}
```

```
public class ChienFou extends Chien{
    @Override
    public void sayHello() {
        System.out.print("chienFou wouf wouf grrr wouf grrr - ");
    }

    public void tournerEnRond() {
        System.out.println(" tourne pas rond dans sa tete ");
    }
}
```

```
public static void operationOnSealed(AnimalDomestique a) {
    if(a instanceof Chien chien) {
        chien.aLaNiche();
        if(chien instanceof ChienFou chienFou) {
            chienFou.tournerEnRond();
        }
    }
    if(a instanceof Chat chat) {
        chat.ronronner();
    }
}

public static void testSealed() {

    var animauxDomestiques = new ArrayList<AnimalDomestique>();
    animauxDomestiques.add(new Chat());
    animauxDomestiques.add(new Chien());
    animauxDomestiques.add(new ChienFou());

    animauxDomestiques.stream().forEach(a-> { a.sayHello(); operationOnSealed(a);});
}
```

affiche

```
chat miaou - ronron
chien wouf wouf - dodo niche
chienFou wouf wouf grrr wouf grrr - dodo niche
tourne pas rond dans sa tete
```

NB : Une interface scellée est implémentable comme un paquets de "record" .

Exemple :

```
public sealed interface Vivant {
    record Vertebre(String name, String p1 , String p2) implements Vivant {}
    record Invertebre(String name, String p3) implements Vivant {}
}
```

NB: Le principal intérêt des classes scellées de java 16/17 tient dans la possibilité d'utiliser un pattern matching au sein d'un switch/case de type d'objet sans partie "default" .

6. Pattern matching sur switch/case of object

//NB: still in "preview mode" in java 17-LTS !!!

//java --enable-preview --source 17

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-compiler-plugin</artifactId>
  <version>3.8.1</version>
  <configuration>
    <release>${java.version}</release>
    <source>${java.version}</source>
    <target>${java.version}</target>
    <compilerArgs>--enable-preview</compilerArgs>
  </configuration>
</plugin>
```

Exemple :

```
package tp.j15_16_17;

public class TestPatternMatchingSwitchPreviewApp {

    static double getDoubleUsingSwitch(Object o) {
        return switch (o) {
            case Integer i -> i.doubleValue();
            case Float f -> f.doubleValue();
            case Double d -> d.doubleValue();
            case String s -> Double.parseDouble(s);
            case null -> 0d;
            default -> 0d;
        };
    }

    static String getTypeAnimalDomesticAsString(AnimalDomestique a) {
        return switch (a) {
            case Chat chat -> "chat";
            case Chien chien -> "chien";
            //default -> "ni chat , ni chien";
        };
    }

    /*
```

Remarque importante :

*Ce switch/case avec pattern-matching de type nécessite absolument une partie "default" si AnimalDomestique n'est pas scellé (sans le mot clef sealed et ...)
et ne nécessite pas de partie "default" si AnimalDomestique est scellé*

Autrement dit l'intérêt principal des classes scellées tient dans la possibilité d'utiliser directement les différents types de classes concrètes (dérivant d'un même type abstrait scellé) au sein d'un switch/case sans default sans avoir besoin de gérer en parallèle une énumération à valeurs possibles fixes/finies.

```

*/
}

static String getTypeVivantFromRecord(Vivant v) {
    return switch (v) {
        case Vivant.Invertebre ive -> "invertebre";
        case Vivant.Vertebre ve -> "vertebre";
        //with default if Vivant is not sealed
    };
}

public static void main(String[] args) {
    System.out.println(getDoubleUsingSwitch("12.5")); //12.5
    System.out.println(getDoubleUsingSwitch(12.6)); //12.6
    System.out.println(getDoubleUsingSwitch(12.7f)); //12.699999809265137
    System.out.println(getDoubleUsingSwitch(12)); //12.0
    System.out.println(getDoubleUsingSwitch(null)); //0.0

    System.out.println(getTypeAnimalDomesticAsString(new Chat())); //chat
    System.out.println(getTypeAnimalDomesticAsString(new Chien())); //chien
    System.out.println(getTypeAnimalDomesticAsString(new ChienFou())); //chien

    System.out.println(getTypeVivantFromRecord(
        new Vivant.Invertebre("limace", "pas rapide"))); // invertebre
    System.out.println(getTypeVivantFromRecord(
        new Vivant.Vertebre("homme", "intelligent", "pas toujours sage"))); // vertebre
    }
}

```

7. utilitaire jpackage

L'utilitaire **jpackage** sert à construire des installeurs d'applications pour windows, linux ou mac.

Ça peut construire des fichier ".msi" pour windows.

package_with_jpackage.bat

```

cd /d "%~dp0"
REM this script should be run after mvn package or ...

set JAVA_HOME=C:\Program Files\Java\jdk-17
REM NB: on windows10 , with jdk17 , wix tools set is a required dependency of jpackage
REM wix311-binaries.zip can be download from https://github.com/wixtoolset
set WIX_HOME=C:\Prog\wix311
set PATH=%PATH%;%JAVA_HOME%\bin;%WIX_HOME%
REM type=msi or exe for windows , deb or rpm for windows , dmg or pkg on mac
set TYPE=msi
set NAME=JPackagedemoApp

```

```

set MAIN_CLASS=tp.MainClassForJPackageTestApp
set INPUT_JAR_DIR=./target
set MAIN_JAR=tp_java_8_14-0.0.1-SNAPSHOT.jar
set OPTIONS=--win-console --java-options '--enable-preview'

jpackage --input %INPUT_JAR_DIR% --name %NAME% --main-jar %MAIN_JAR%
--main-class %MAIN_CLASS% --type %TYPE% %OPTIONS%
REM fichier construit : JPackageDemoApp.msi

REM apres installation de JPackageDemoApp.msi
REM C:\Program Files\JPackageDemoApp\JPackageDemoApp.exe et runtime java

REM https://www.baeldung.com/java14-jpackage
REM https://www.devdungeon.com/content/use-jpackage-create-native-java-app-installers

```

8. Autres apports des versions récentes (12,...,17)

8.1. Helpful *NullPointerException* (JEP 358)

Depuis le java 14, les traces générées autour de *NullPointerException* sont beaucoup plus claires/explicites .

Exemple de code avec bug:

```

int[] arr = null;
arr[0] = 1;

```

Avec les jdk précédent (ex : v11) :

```

Exception in thread "main" java.lang.NullPointerException
at tp.MyClass.main(MyClass.java:27)

```

Maintenant, la log d'erreur indique plus clairement la source du problème:

```

java.lang.NullPointerException: Cannot store to int array because "arr" is null

```

9. Nouveau type de licence depuis java 17

Le JDK 17 D'Oracle De Nouveau Gratuit Pour Une Utilisation Commerciale

Le JDK Oracle est à nouveau disponible gratuitement pour une utilisation en production - sous la nouvelle "[Oracle No-Fee Terms et conditions](#)" (NFTC). Cette décision annule une [décision de 2018](#) de facturer l'utilisation en production du JDK d'Oracle et n'affecte pas la [distribution OpenJDK d'Oracle](#). La NFTC s'applique à la version 17 récemment publiée d'Oracle JDK et aux versions futures.

Autre raison de passer à java 17 : **java 17 est plus rapide/performant que java 11 .**

10. Pattern and record matching

Rappel du pattern matching "if/instanceof" (déjà possible en java 16,17)

```
if (obj instanceof String str) {
    ...
}
```

10.1. Pattern matching switch (preview java 17 , finalisé java 21)

```
static double getDoubleUsingSwitch(Object o) {
    return switch (o) {
        case Integer i -> i.doubleValue();
        case Float f -> f.doubleValue();
        case Double d -> d.doubleValue();
        case String s -> Double.parseDouble(s);
        case null -> 0d;
        default -> 0d;
    };
}
```

```
static String getTypeAnimalDomesticAsString(AnimalDomestique a) {
    return switch (a) {
        case Chat chat -> "chat";
        case Chien chien -> "chien";
        //default -> "ni chat , ni chien"; //default not mandatory if AnimalDomestique is "sealed"
    };
}
```

10.2. Destructuration via "record pattern"

```
public class MyRecords {
    public record Point(int x,int y) {};
    public record GPS_Point(double latitude,double longitude) {};
    public record Location(String name,GPS_Point position) {};
}
```

```

import tp.java_new_21plus.record_pattern.MyRecords.*;

public class RecordDeconstructApp {

    public static void main(String[] args) {
        Double d1=23.5;
        print(d1);

        Point pt1 = new Point(20,30);
        print(pt1);
        print_destructuring(pt1);
        switch_destructuring(pt1);

        Location loc1 = new Location("Paris" , new GPS_Point(48.866667, 2.333333));
        print(loc1);
        print_destructuring(loc1);
        switch_destructuring(loc1);
    }

    public static void print(Object o) {
        //Pattern matching for instance of (since java 16/17):
        if(o instanceof Double d) {System.out.println("double d="+d);}
        if(o instanceof Point pt) { System.out.println("Point pt="+pt);}
        if(o instanceof Location loc) { System.out.println("Location loc="+loc);}
    }

    public static void print_destructuring(Object o) {
        //record pattern = destructureur (like javascript) in Pattern matching for instance of
        if(o instanceof Point (int x,int y)) {
            System.out.println("Destructuring Point x="+x + " y="+y);
        }
        if(o instanceof Location (String name,GPS_Point(double latitude,double longitude))) {
            System.out.println("Destructuring Location name="+name
                               + " latitude="+latitude + " longitude="+longitude );
        }
    }

    public static void switch_destructuring(Object o) {
        //record pattern = destructureur (like javascript) in switch expression
        double maxi = switch(o) {
            case Point (int x,int y) -> (double) Math.max(x, y);
            case Location (String name,GPS_Point(double latitude,double longitude))
                -> Math.max(latitude, longitude);
            default -> 0;
        };
        System.out.println("switch_destructuring, maxi="+maxi);
    }
}

```

11. Virtual Thread

11.1. Threads natifs et virtuels

Un **thread natif** est géré par le système d'exploitation (ex : linux ou windows)

Un **thread virtuel** n'est géré que par la machine virtuelle java . C'est possible depuis les versions 18,19,20,21 de java. En étant plus léger, un thread virtuel peut démarrer plus rapidement et le changement de contexte d'un thread à l'autre est également plus rapide.

11.2. Démarrage d'un thread virtuel

La classe **java.lang.Thread** comporte maintenant une méthode **.ofVirtual()** permettant de créer un nouveau thread virtuel .

Thread.ofVirtual()

```
.name(virtualThreadName)
//.start(runnable); //code automatiquement démarré
.unstarted(runnable); //code à démarrer via appel à .start() ou équivalent
```

Exemple :

```
import java.time.Duration;

public class MyThreadUtil {

    static void log(String message) {
        System.out.println(message + " by " + Thread.currentThread());
    }

    static void sleep(Duration duration) {
        try { Thread.sleep(duration);} catch (InterruptedException e) { e.printStackTrace(); }
    }

    static void sleep(long nbMs) {
        try { Thread.sleep(nbMs);} catch (InterruptedException e) { e.printStackTrace();}
    }
}
```

```
public class MyRunnableUtil {
    public static Runnable prepareCoffeeRunnable() {
        return ()->{
            MyThreadUtil.log("prepare coffee");
            MyThreadUtil.sleep(Duration.ofMillis(1000));
            MyThreadUtil.log("coffee is ready");
        };
    }
}
```

```

public static void testSimpleVirtualThread() throws Exception {
    Thread vt1 = Thread.ofVirtual().name("vt1")
                .unstarted(MyRunnableUtil.prepareCoffeeRunnable());
    vt1.start();// démarrage de vt1
    vt1.join();//attente de la fin de vt1 depuis main Thread
}

```

==>

prepare coffee by VirtualThread[#21,vt1]/runnable@ForkJoinPool-1-worker-1

coffee is ready by VirtualThread[#21,vt1]/[runnable@ForkJoinPool-1-worker-1](#)

11.3. Threads virtuels via Executors

```

//one unnamed virtualThread for each concurrent task submitted to the executor
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    ....
}

```

```

public static void testViaExecutors() throws Exception {
    //one named virtualThread (generated by factory)
    //for each concurrent task submitted to the executor
    final ThreadFactory virtualThreadFactory =
        Thread.ofVirtual().name("routine-", 0).factory();
    //names of virtual threads will be "routine-0", "routine-1", ...
    try (var executor = Executors.newThreadPerTaskExecutor(virtualThreadFactory)) {
        Future<?> f1 = executor.submit(MyRunnableUtil.prepareJokeRunnable());
        Future<?> f2 = executor.submit(MyRunnableUtil.prepareCoffeeRunnable());

        Future<Integer> f3 = executor.submit(MyRunnableUtil.longComputationCallable());
        f1.get();// attente fin de f1
        f2.get();// attente fin de
        Integer resF3 = f3.get();// attente resultat de f3
        System.out.println("resF3="+resF3);
    }
}

```

Résultat :

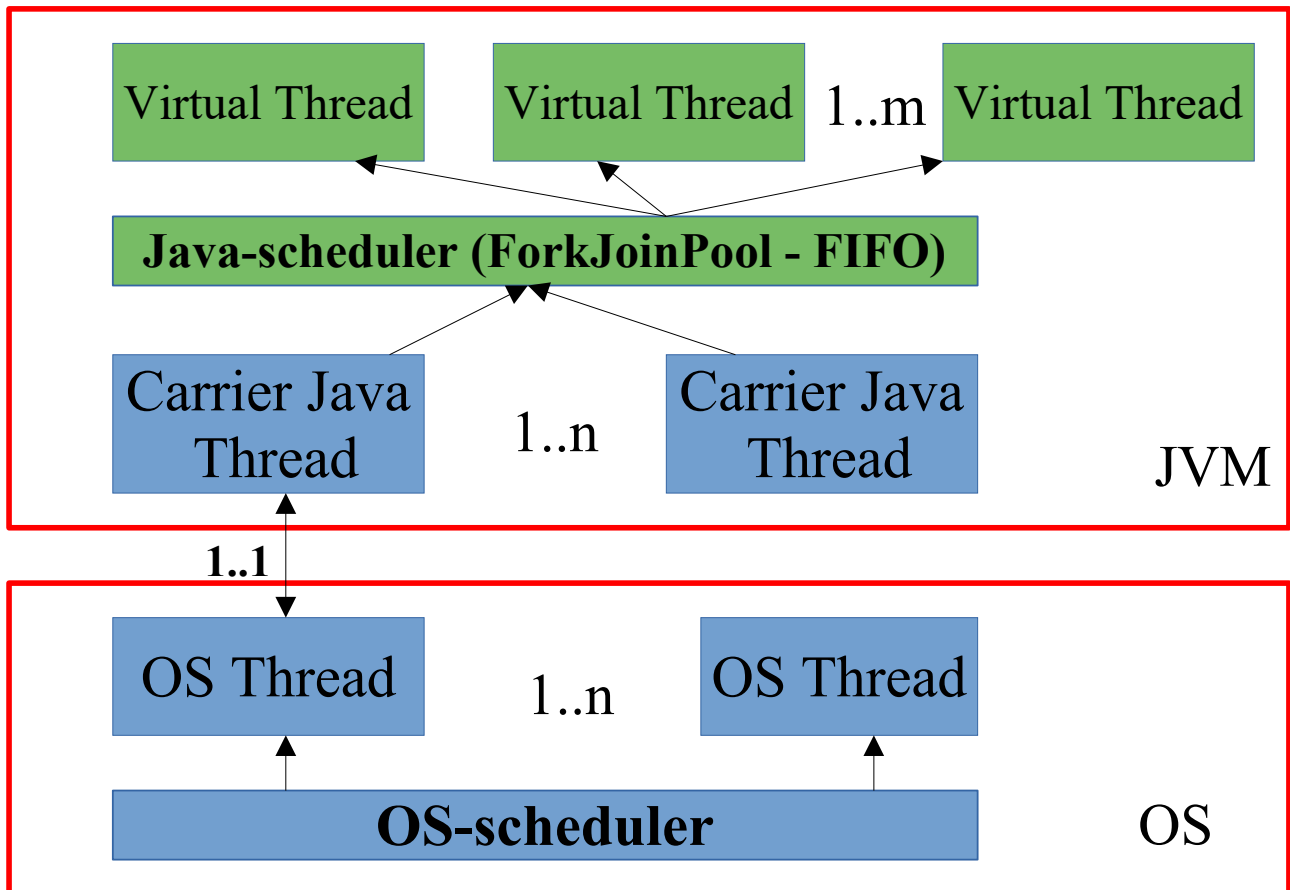
prepare a joke by VirtualThread[#21,routine-0]/runnable@ForkJoinPool-1-worker-1
 prepare coffee by VirtualThread[#23,routine-1]/runnable@ForkJoinPool-1-worker-2
 start computing by VirtualThread[#25,routine-2]/runnable@ForkJoinPool-1-worker-3
 joke is ready by VirtualThread[#21,routine-0]/runnable@ForkJoinPool-1-worker-1
 coffee is ready by VirtualThread[#23,routine-1]/runnable@ForkJoinPool-1-worker-1
 computing is ready by VirtualThread[#25,routine-2]/runnable@ForkJoinPool-1-worker-3
 resF3=5

11.4. CarrierPlatformThread

//[#virtualThreadId,virtualThreadName]/... carrierPlatformThread

//exemple : VirtualThread[#37,routine-8]/runnable@ForkJoinPool-1-worker-4

VirtualThreads and CarrierPlatformThread



La machine virtuelle java s'appuie en interne sur un pool de "carrier java thread" (mappé chacun avec un thread natif de l'OS) pour prendre en charge un paquet de threads virtuels.

Lorsqu'un thread virtuel est bloqué (en attente, en sommeil, ...), celui-ci peut être temporairement dissocié du "carrier java thread" porteur de manière à ce que ce "carrier java thread" puisse exécuter d'autres tâches plus utiles (autre "virtual thread").

Le lien entre un "virtual_thread" et un "carrier java thread" peut évoluer dans le temps.

Exemple :

VirtualThread[#24,routine-2]/runnable@ForkJoinPool-1-worker-7

VirtualThread[#24,routine-2]/runnable@ForkJoinPool-1-worker-6

Par défaut sur une machine à n processeurs/coeurs, le "ForkJoinPool" comportera n "carrier java

thread" (ex : *ForkJoinPool-1-worker-1* , ... , *ForkJoinPool-1-worker-8*)

Cas d'utilisation :

Les threads virtuels apportent un plus dans le cas où beaucoup d'opérations concurrentes sont potentiellement bloquées (en attentes , exemple = appel synchrone vers api REST).

Par contre , les `VirtualThreads` n'apportent rien pour un usage intensif du CPU . Pour cela l'algorithme du forkjoin et les `ParallelStream` sont déjà assez bien optimisés.

12. Structured Concurrency (preview java 21)

L'api "**Structured Concurrency**" (toujours en mode "preview" en java 21) sert à déclencher des sous tâches en parallèle (qui seront exécutées par des threads virtuels) tout en ayant la possibilité d'attendre (en mode "bloquant avec timeout") selon les besoins :

- soit la fin de toutes les sous-tâches
- soit le résultat de la sous-tâches la plus rapide

Exemple :

```
public class FakeLongTaskService {
    public static FakeLongTaskService INSTANCE = new FakeLongTaskService();

    public double slowSquareRoot(double x,int nbMs) throws InterruptedException{
        try {
            Thread.currentThread().sleep(nbMs);
        } catch (InterruptedException e) {
            System.err.println(Thread.currentThread()+"was interrupted before end of sleep="+nbMs);
            throw e;
        }
        if(nbMs<50) throw new RuntimeException("lunatic exception because nbMs<50");
        double res = Math.sqrt(x);
        System.out.println("slowSquareRoot(x="+x + ") returning res="+res
            + " after sleep="+ nbMs + " "+ Thread.currentThread());
        return res;
    }
}
```

En mode attente du résultat de la sous-tâches la plus rapide :

```
...
import java.time.Instant;
import java.util.concurrent.StructuredTaskScope;
import java.util.concurrent.TimeoutException;

public class WithFirstFinishedSubTask {

    public static WithFirstFinishedSubTask INSTANCE = new WithFirstFinishedSubTask();
    /*
     ShutdownOnSuccess captures the first result and shuts down the task scope
     to interrupt unfinished threads and wakeup the owner.
     This class is intended for cases where the result of any subtask will do ("invoke any")
     and where there is no need to wait for results of other unfinished tasks.
     It defines methods to get the first result or throw an exception if all subtasks fail.
     */

    public Double quickerOfSlowSquareRoot(double x) {
        Double firstFinishedResult=0.0;
        try (var scope = new StructuredTaskScope.ShutdownOnSuccess<Double>()) {
            //int extra = -200; //causing all subtaskToFail because xxx + extra < 50
            //int extra = 1000; //causing timeoutException
            int extra = 10; //no global error
            StructuredTaskScope.Subtask<Double> subtask1 =
                scope.fork() -> FakeLongTaskService.INSTANCE.slowSquareRoot(x, 200+extra);//200ms
            StructuredTaskScope.Subtask<Double> subtask2 =
                scope.fork() -> FakeLongTaskService.INSTANCE.slowSquareRoot(x, 100+extra);
            StructuredTaskScope.Subtask<Double> subtask3 =
                scope.fork() -> FakeLongTaskService.INSTANCE.slowSquareRoot(x, 50+extra);//50ms
```

```

StructuredTaskScope.Subtask<Double> subtask4 =
    scope.fork(() -> FakeLongTaskService.INSTANCE.slowSquareRoot(x, 40));
    //if less than 50 with negative extra ->lunatic Exception
//NB: sometimes futurRes4 throws a RuntimeException , it fails ,
//but "no problem" if at least another // subtask succeed

//scope.join();
scope.joinUntil(Instant.now().plusSeconds(1)); //with TimeoutException if all is not finish Is later now

//firstFinishedResult = scope.result(); //without specific exception
firstFinishedResult = scope.result(e -> new RuntimeException("cannot compute squareRoot"));
    //with specific exception if all failed (but not timeout)
System.out.println("firstFinishedResult="+firstFinishedResult);
} catch (InterruptedException e) {
    throw new RuntimeException(e);
} catch (TimeoutException e) {
    throw new RuntimeException(e);
} catch (/*ExecutionException*/ Exception e) {
    throw new RuntimeException(e);
}
}
return firstFinishedResult;
}
}

public static void testQuickerSubTask() {
    try {
        double res = WithFirstFinishedSubTask.INSTANCE.quickerOfSlowSquareRoot(9.0);
        System.out.println("quickerOfSlowSquareRoot() returning "+res);
    } catch (Exception e) {
        System.err.println("quickerOfSlowSquareRoot() throwing "+e.getMessage());
    }
}
}

```

Résultats :

slowSquareRoot(x=9.0) returning res=3.0 after sleep=60

VirtualThread[#200]/runnable@ForkJoinPool-1-worker-1

VirtualThread[#199]/runnable@ForkJoinPool-1-worker-8was interrupted before end of sleep=110

VirtualThread[#198]/runnable@ForkJoinPool-1-worker-7was interrupted before end of sleep=210

firstFinishedResult=3.0

quickerOfSlowSquareRoot() returning 3.0

En mode "attente de la fin de toutes les sous-tâches parallèles"

```

...
import java.util.concurrent.StructuredTaskScope;
import java.util.concurrent.TimeoutException;

public class AllFinishedSubTasks {

    public static AllFinishedSubTasks INSTANCE = new AllFinishedSubTasks();
    /*
    ShutdownOnSuccess captures the first result and shuts down the task scope
    to interrupt unfinished threads and wakeup the owner.
    This class is intended for cases where the result of any subtask will do ("invoke any")
    and where there is no need to wait for results of other unfinished tasks.

```

It defines methods to get the first result or throw an exception if all subtasks fail.
 */

```
public List<Double> doubleListSlowSquareRoot(List<Double> doubleList) {
    List<Double> doubleResultList = new ArrayList<>();
    try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
        List<StructuredTaskScope.Subtask<Double>> subtaskList =
            new ArrayList<StructuredTaskScope.Subtask<Double>>();
        //int extra = 1000; //causing timeoutException
        int extra = 10; //no global error
        for(Double val : doubleList) {
            StructuredTaskScope.Subtask<Double> subtask = scope.fork(
                () -> (Double) FakeLongTaskService.INSTANCE.slowSquareRoot(val, 100+extra)
            );
            subtaskList.add(subtask);
        }
        //scope.join();
        scope.joinUntil(Instant.now().plusSeconds(1)); //with timeout

        for(StructuredTaskScope.Subtask<Double> subtask : subtaskList ) {
            doubleResultList.add(subtask.get());
        }
    } catch (InterruptedException e) { throw new RuntimeException(e);
    } catch (TimeoutException e) { throw new RuntimeException(e);
    } catch (/*ExecutionException*/ Exception e) { throw new RuntimeException(e); }
    return doubleResultList;
}

public static void testAllFinishedParallelSubTask() {
    try {
        List<Double> doubleList = Arrays.asList(9.0 , 4.0 , 25.0 , 16.0);
        System.out.println("doubleList="+doubleList);
        List<Double> resultList =
            AllFinishedSubTasks.INSTANCE.doubleListSlowSquareRoot(doubleList);
        System.out.println("doubleListSlowSquareRoot() returning "+resultList);
    } catch (Exception e) {
        System.err.println("doubleListSlowSquareRoot() throwing "+e.getMessage());
    }
}
}
```

Résultats :

```
doubleList=[9.0, 4.0, 25.0, 16.0]
slowSquareRoot(x=4.0) returning res=2.0 after sleep=110
VirtualThread[#203]/runnable@ForkJoinPool-1-worker-5
slowSquareRoot(x=16.0) returning res=4.0 after sleep=110
VirtualThread[#205]/runnable@ForkJoinPool-1-worker-1
slowSquareRoot(x=25.0) returning res=5.0 after sleep=110
VirtualThread[#204]/runnable@ForkJoinPool-1-worker-8
slowSquareRoot(x=9.0) returning res=3.0 after sleep=110
VirtualThread[#202]/runnable@ForkJoinPool-1-worker-7
doubleListSlowSquareRoot() returning [3.0, 2.0, 5.0, 4.0]
```

13. Foreign Function and Memory (preview java 21)

L'api FFM (Foreign Function and Memory) est toujours en mode **preview** en java 21.

Cette api permet d'accéder à des fonctions et blocs mémoires externes à la JVM (ex : fonction native du langage "C") de manière efficace et rigoureuse (mieux que JNI ou JNA).

Attention:

sans arrêt , de petits changements au niveau de l'api FFM (Foreign Function and Memory) entre les previews des versions 18,19,20,21 (PAS STABLE DU TOUT)

Le code ci-dessous est pour le jdk 21

<https://openjdk.org/jeps/442> pour java 21 (third preview of FFM)

```
...
import java.lang.foreign.Arena;
import java.lang.foreign.FunctionDescriptor;
import java.lang.foreign.Linker;
import java.lang.foreign.MemorySegment;
import java.lang.foreign.SymbolLookup;
import java.lang.foreign.ValueLayout;
import java.lang.invoke.MethodHandle;

public class FFMTestApp {
    public static void main(String[] args) throws Throwable {
        // 1. Get a lookup object for commonly used libraries
        SymbolLookup stdlib = Linker.nativeLinker().defaultLookup();

        // 2. Get a handle to the "strlen" function in the C standard library
        MethodHandle strlen = Linker.nativeLinker().downcallHandle(
            stdlib.find("strlen").orElseThrow(),
            FunctionDescriptor.of(ValueLayout.JAVA_LONG, ValueLayout.ADDRESS));

        try (Arena offHeap = Arena.ofConfined()) { //version java 21

            // 3. Convert Java String to C string and store it in off-heap memory
            MemorySegment str = offHeap.allocateUtf8String("Happy Coding!");
                                                    //13 caracteres

            // 4. Invoke the foreign function
            long len = (long) strlen.invoke(str);

            System.out.println("len = " + len); //len=13 (ok)
        }
    }
}
```

14. Vector api (incubator java 21)

"**Vector api**" (toujours en mode "incubator" en java 21) sert à effectuer d'un seul coup des opérations mathématiques (ex : addition, multiplication,...) sur des tableaux de valeurs en utilisant certaines fonctionnalités pointues de certains micro-processeurs récents.

```
package tp.java_new_21plus.vector_api;
import jdk.incubator.vector.IntVector;

public class TestVectorApi {
    //in run configuration : VM arguments
    //--add-modules=jdk.incubator.vector

    public static void main(String[] args) {
        smallVectorAddition();
        largeVectorAdditionV1(); //with IntVector.SPECIES_512
        largeVectorAdditionV2(); //with IntVector.SPECIES_128 , 4 iterations
    }

    public static void display_intTab(String tabName, int[] values) {
        System.out.print(tabName+"={");
        for(int v : values)
            System.out.print(" " + v);
        System.out.print("}\n");
    }
}
//...
}
```

```
public static void smallVectorAddition() {
    int[] a = {1, 2, 3, 4};
    int[] b = {17, 18, 19, 20};
    int[] c = new int[4]; //4fois 32bits = 128bits

    // Scalar (ordinary) computation:
    //c[0] = a[0] + b[0];
    //...
    //c[15] = a[15] + b[15];
    for (int i = 0; i < c.length;i++)
        c[i] = a[i] + b[i];
    display_intTab("c",c);

    // Vector computation:
    IntVector va = IntVector.fromArray(IntVector.SPECIES_128, a, 0);
    IntVector vb = IntVector.fromArray(IntVector.SPECIES_128, b, 0);
    IntVector vc = va.add(vb); //add operation between 2 vector

    int[] c2 = new int[4];
    vc.toArray(c2, 0); //store vc in ordinary array of scalar
    display_intTab("c2",c2);
}
```

variante sans boucle :

```
public static void largeVectorAdditionV1() {
    int[] a = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16};
    int[] b = {17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32};

    // Vector computation:
    int[] c2 = new int[16];
    IntVector va = IntVector.fromArray(IntVector.SPECIES_512, a, 0);
    IntVector vb = IntVector.fromArray(IntVector.SPECIES_512, b, 0);
    IntVector vc = va.add(vb); //add operation between 2 vector
    vc.toArray(c2, 0); //store vc in ordinary array of scalar
    display_intTab("c2", c2);
}
```

16 fois 32 = 512 (tableaux de 16 cases de int (32bits))

variante avec boucle et offset:

```
public static void largeVectorAdditionV2() {
    int[] a = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16};
    int[] b = {17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32};

    // Vector computation:
    int[] c2 = new int[16];
    for (int offset = 0; offset < c.length; offset += IntVector.SPECIES_128.length()) {
        IntVector va = IntVector.fromArray(IntVector.SPECIES_128, a, offset);
        IntVector vb = IntVector.fromArray(IntVector.SPECIES_128, b, offset);
        IntVector vc = va.add(vb); //add operation between 2 vector
        vc.toArray(c2, offset); //store vc in ordinary array of scalar
    }
    display_intTab("c2", c2);
}
```

Résultats :

WARNING: Using incubator modules: jdk.incubator.vector

c={ 18 20 22 24}

c2={ 18 20 22 24}

c={ 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46 48}

c2={ 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46 48}

XVII - Entrées/sorties (nio,nio2) – Path, Files, ...

Pour manipuler , les entrées/sorties fichiers et le FileSystem , le langage java a apporté progressivement des librairies de classes au fur et à mesure des évolutions de versions.

java.io date de java 1 et fonctionne en mode synchrone bloquant.

java.nio de java 4 a apporté de nouvelles classes et abstractions.

java.nio2 de java 7 a encore un peu amélioré les choses avec plus d'asynchronismes.

1. NIO2 (new IO)

1.1. Introduction à nio , nio2

Dès l'origine du langage java , la classe File du package java.io était disponible pour accéder aux éléments du "file system" (fichier , répertoire, ...) et pour effectuer des opérations dessus (tester l'existence d'un fichier , supprimer des fichiers , ...).

Bien qu'opérationnel , le package java.io (et la classe File) ont quelques limitations importantes :

- La classe File manque de fonctionnalité importante telle qu'une méthode "copy".
- Beaucoup de méthode retourne des booléens plutôt que des exceptions . Ce qui rend assez délicate l'analyse des problèmes (raison exacte de l'erreur?).
- Pas de bonne gestion des liens symboliques .
- Un ensemble très limité d'attributs sur les fichiers sont disponibles sur java.io.File.

Pour surmonter certaines lacunes , un nouveau package "java.nio" a été introduit dès java4 de façon à apporter quelques améliorations très techniques telles que les suivantes :

- *Channels et Selectors*: un "channel" est une abstraction d'une caractéristique de bas niveau telle qu'un fichier mappé en mémoire .
- *Buffers*: Buffering pour toutes les classes wrapper/primitives (sauf pour Boolean).
- *Jeu de caractères*: **Charset** (*java.nio.charset*), avec encodeurs et décodeurs entre `bytes[]` et symboles "Unicode".

La version 7 de java a introduit un nouveau (gros) package "**java.nio.file**" (alias nio2) .

Ce package apporte à la fois de nouvelles fonctionnalités techniques telles que :

- gestion des liens symboliques
- bonne gestion des "Path" (aspect bien séparé).
- meilleure gestion des "File attributes"

et de nouvelles syntaxes (plus concises , plus "orienté objet") telles que les remontées d'exceptions.

Attention : Entre java1, java4 , java7 , des ajouts , mais pas de suppression (pour garder une compatibilité avec les anciennes versions).

Bien que la classe "java.io.File" n'est pas officiellement considérée comme obsolète / "deprecated" , il est très conseillé d'utiliser les nouvelles classes de nio2 (sachant qu'il existe des passerelles entre "java.io" et "java.nio.file") .

2. Principales classes et interfaces de nio2

NIO 2 (associé aux packages "**java.nio.file**" et "**java.nio.file.attribute**") repose sur plusieurs classes et interfaces dont les principales sont :

- **Path** : encapsule un chemin dans le système de fichiers
- **Files** : avec méthodes statiques pour manipuler les éléments du système de fichiers
- **FileSystemProvider** : fournisseur technique (implémentation) de FS
- **FileSystem** : encapsule un système de fichiers
- **FileSystems** : fabrique permettant entre autres de créer une instance de FileSystem
- **FileStorage** : système de stockage (ex : partition / volume logique ,)

Quelques équivalences/transpositions de fonctionnalités basiques entre "java.io" et "nio2" :

Fonctionnalité	java.io	NIO 2
Encapsuler un chemin	java.io.File	java.nio.file.Path
Vérifier les permissions	File.canRead(), File.canCrite() et File.canExecute()	Files.isReadable(), Files.isWritable() et Files.isExecutable().
Vérifier le type d'élément	File.isDirectory(), File.isFile()	Files.isDirectory(Path, LinkOption...), Files.isRegularFile(Path, LinkOption...),
Taille d'un fichier	File.length()	Files.size(Path)
Obtenir ou modifier la date de dernière mise à jour	File.lastModified() , File.setLastModified(long)	Files.getLastModifiedTime(Path, LinkOption...), Files.setLastModifiedTime(Path, FileTime)
Modifier les attributs	File.setExecutable(), File.setReadable(), File.setReadOnly(), File.setWritable()	Files.setAttribute(Path, String, Object, LinkOption...)
Déplacer un fichier	File.renameTo()	Files.move()
Supprimer un fichier	File.delete()	Files.delete()
Créer un fichier	File.createNewFile()	Files.createFile()
Créer un fichier temporaire	File.createTempFile()	Files.createTempFile(Path, String, FileAttributes<?>), Files.createTempFile(Path, String, String, FileAttributes<?>)
Tester l'existence d'un fichier	File.exists	Files.exists() ou Files.notExists()
Obtenir le chemin absolu	File.getAbsolutePath() ou File.getAbsolutePath()	Path.toAbsolutePath()
Chemin canonique (sans ".", "..")	File.getCanonicalPath() ou File.getCanonicalFile()	Path.toRealPath() ou Path.normalize()
Convertir en URI	File.toURI()	Path.toURI()
L'élément est-il caché	File.isHidden()	Files.isHidden()
Obtenir le contenu	File.list() ou File.listFiles()	Path.newDirectoryStream()

d'un répertoire		
Créer un répertoire	File.mkdir() ou File.mkdirs()	Path.createDirectory()
Obtenir le contenu du répertoire racine	File.listRoots()	FileSystem.getRootDirectories()
Place totale , libre , ... sur FS	File.getTotalSpace() File.getFreeSpace()	FileStore.getTotalSpace() FileStore.getUnallocatedSpace()

3. Gestion des chemins (Path)

L'interface "java.nio.file.**Path**" est une représentation abstraite d'un chemin (relatif ou absolu) au sein d'un système de fichiers.

Un chemin peut référencer un fichier , un répertoire , un lien symbolique , un sous-chemin , ...

Les instances de "Path" sont "immutables" et peuvent être utilisées dans un contexte multi-threads.

3.1. Obtention d'une instance de "Path"

Récupération d'un chemin en appelant explicitement **getPath()** sur une **instance de "FileSystem"** :

```
Path chemin = FileSystems.getDefault().getPath(
    "C:/Users/powerUser/Temp/monfichier.txt");
```

équivalent indirect via la méthode (utilitaire / "helper") statique **Paths.get()** :

```
Path chemin = Paths.get("C:/Users/powerUser/Temp/monfichier.txt");
Path chemin2 = Paths.get(URI.create("file:///C:/Users/powerUser/Temp/monfichier.txt"));
Path chemin3 = Paths.get(System.getProperty("java.io.tmpdir"), "monfichier.txt");
```

NB : **Path.of()** existe depuis java 11 et se comporte comme **Paths.get()**

NB : bien que (sous windows) on puisse utiliser le séparateur "\\" (ex : [C:\\RepXy\\monfichier.txt](#)) , il vaut mieux utiliser le séparateur "/" (plus portable et plus simple) .

Passerelle "io / nio2" (depuis java 7) :

En partant d'une instance de l'ancienne classe **java.io.File** , on peut appeler la nouvelle méthode **".toPath()"** pour récupérer une instance de "java.nio.file.Path".

3.2. Obtention des éléments d'un chemin

Méthode	Rôle
String getFileName()	Retourne le nom du dernier élément du chemin. Si le chemin concerne un fichier alors c'est le nom du fichier qui est retourné
Path getName(int index)	Retourne l'élément du chemin dont l'index est fourni en paramètre. Le premier élément (hors racine) possède l'index 0 et correspond à la première partie sous la racine (ex : "windows" sous racine " c:/ ").

<code>int getNameCount()</code>	Retourne le nombre d'éléments du chemin (hors racine)
<code>Path getParent()</code>	Retourne le chemin parent ou null s'il n'existe pas
<code>Path getRoot()</code>	Retourne la racine d'un chemin absolu (par exemple C:\ sous windows ou / sous Unix) ou null pour un chemin relatif
<code>String toString()</code>	Retourne le chemin sous la forme d'une chaîne de caractères
<code>Path subPath(int <i>beginInclusiveIndex</i>, int <i>endExclusiveIndex</i>)</code>	Retourne un sous-chemin (hors racine) [<i>beginIndex</i> , <i>endIndex</i> [

Exemple avec un chemin absolu :	Exemple avec un chemin relatif :
path=C:\Windows\Fonts\arial.ttf <code>toString()</code> = C:\Windows\Fonts\arial.ttf <code>getFileName()</code> = arial.ttf <code>getRoot()</code> = C:\ <code>getName(0)</code> = Windows <code>getNameCount()</code> = 3 <code>getParent()</code> = C:\Windows\Fonts <code>subpath(0,2)</code> = Windows\Fonts	path=Fonts\arial.ttf <code>toString()</code> = Fonts\arial.ttf <code>getFileName()</code> = arial.ttf <code>getRoot()</code> = null <code>getName(0)</code> = Fonts <code>getNameCount()</code> = 2 <code>getParent()</code> = Fonts <code>subpath(0,2)</code> = Fonts\arial.ttf

3.3. Manipulation, combinaison et conversions de chemins

Méthode	Rôle
<code>Path normalize()</code>	Normaliser (ou rendre "canonique") un chemin en supprimant les éléments non indispensables « . » et « .. » qu'il contient.
<code>Path relativize(Path other)</code>	Retourner le chemin relatif permettant d'aller du "path courant" vers celui fourni en paramètres .
<code>Path resolve(Path relative)</code>	Combiner deux chemins (courant + relatif_en_arg) pour former global
<code>Path toAbsolutePath()</code>	Retourne un chemin absolu (en <i>tenant compte du répertoire courant du contexte d'exécution</i> (idem à "pwd")).
<code>Path toRealPath(LinkOption...)</code>	Retourner le chemin physique du chemin notamment en résolvant les liens symboliques selon les options fournies . Peut lever une exception si le fichier au bout du chemin courant (absolu ou relatif) n'existe pas .
<code>URI toUri()</code>	Retourner le chemin sous la forme d'une URI (file:///)

Exemple1 :

```
path.toString() = C:\Utilisateurs\..\Windows\..\Fonts\arial.ttf
path.normalize() = C:\Windows\Fonts\arial.ttf
```

```
path.toString() = ..\..\Fonts\..\arial.ttf
path.normalize() = ..\..\Fonts\arial.ttf (seul le ../ inutile supprimé)
```

Exemple2 :

```
Path windowsFontPath = Paths.get("c:/Windows/Fonts");
Path usersPath = Paths.get("c:/Utilisateurs");
//relative path from current to arg
Path relativePathFromUsersToWindowsFonts= usersPath.relativeTo(windowsFontPath);
System.out.println("relative path=" + relativePathFromUsersToWindowsFonts) ;

relative path=..\Windows\Fonts
```

Exemple3 :

```
Path windowsPath = Paths.get("c:/Windows");
Path arialRelativePath = Paths.get("Fonts/arial.ttf");
Path globalPath = windowsPath.resolve(arialRelativePath);
System.out.println("globalPath="+globalPath);

globalPath=c:\Windows\Fonts\arial.ttf
```

```
Path windowsFontsPath = Paths.get("C:/Windows/Fonts");
System.out.println("uri=" + windowsFontsPath.toUri()); //java.net.URI

uri=file:///C:/Windows/Fonts/
```

```
Path relativePath = Paths.get(".");
Path absolutePath = relativePath.toAbsolutePath(); //selon rep courant (pwd)
System.out.println("absolutePath=" + absolutePath);

absolutePath=C:\Users\didier\workspace\test_j7_j8\.
```

Rappel : sous linux , un lien symbolique se construit via la commande
\$ ln -s /nom_du_dossier_source nom_du_lien

Attention, sous windows , un lien symbolique ne doit pas être confondu avec un raccourci .
A l'époque de windows XP , un lien symbolique se construisait avec la commande "*junction.exe*" (à installer).
Depuis "Vista" , et encore aujourd'hui avec windows 7, 8 et 10, un lien symbolique se construit avec la commande **mklink** suivante :

MKLINK [[/D] | [/H] | [/J]] Lien Cible

/D : Crée un lien symbolique vers un répertoire. Par défaut, il s'agit d'un lien symbolique vers un fichier.

/H : Crée un lien réel à la place d'un lien symbolique.

/J : Crée une jonction de répertoires.

Lien : Spécifie le nom du nouveau lien symbolique.

Cible : Spécifie le chemin d'accès (relatif ou absolu) auquel le nouveau lien fait référence.

Exemple (à lancer avec des droits "administrateur"):

```
cd c:\tmp\bb
```

```
mklink /J lien_vers_aa c:\tmp\aa
```

jonction créée pour lien_vers_aa <==> c:\tmp\aa

```
Path path = Paths.get("C:/tmp/bb/raccourci_vers_aa/f1.txt");
→ java.nio.file.NoSuchFileException: C:\tmp\bb\raccourci_vers_aa\f1.txt
```

```
Path path = Paths.get("C:/tmp/bb/lien_vers_aa/f1.txt");
try {
    System.out.println("chemin indirect sans suivre résolution lien symbolique="
```

```

        + path.toRealPath(LinkOption.NOFOLLOW_LINKS));
System.out.println("chemin direct suivant résolution lien symbolique="
        + path.toRealPath());
    } catch (IOException e) {
        e.printStackTrace();
    }
chemin indirect sans suivre résol lien symbolique=C:\tmp\bb\lien_vers_aa\f1.txt
chemin direct suivant résolution lien symbolique=C:\tmp\aa\f1.txt

```

3.4. Comparaisons de chemins

La méthode `.equals()` permet de tester si deux chemins ont des valeurs identiques.

L'interface **Path** hérite de **Comparable**. Les chemins peuvent donc être automatiquement **triés**.

L'interface **Path** comporte en outre les méthodes de comparaison spécifiques suivantes :

Méthode	Rôle
int compareTo (Path other) (Comparable<Path>)	Comparer le chemin avec celui fourni en paramètre et retourne 0 si identiques, <0 avant, >0 si après
boolean endsWith (Path other) boolean endsWith (String other)	Comparer la fin du chemin avec celui fourni en paramètre
boolean startsWith (Path other) boolean startsWith (String other)	Comparer le début du chemin avec celui fourni en paramètre

3.5. interface PathMatcher avec méthode matches(path) et "glob"

Un "glob" est une expression basée sur certains méta-caractères qui seront comparés à des parties de "path" .

```

Path path = Paths.get("C:/tmp/aa/f1.txt");
PathMatcher txtMatcher = FileSystems.getDefault().getPathMatcher("glob:*.txt");
if (txtMatcher.matches(path.getFileName())) {
    System.out.println(path + " reference un fichier texte");
}

```

Motif (dans glob)	Rôle associé / correspondance
*	Aucun ou plusieurs caractères
**	Aucun ou plusieurs sous-répertoires
?	Un caractère quelconque
{ }	Un ensemble de motifs exemple : {htm, html}
[]	Un ensemble de caractères. Exemple : [A-Z] : toutes les lettres majuscules

	[0-9] : tous les chiffres [a-z,A-Z] : toutes les lettres indépendamment de la casse Chaque élément de l'ensemble est séparé par un caractère virgule Le caractère - permet de définir une plage de caractères A l'intérieur des crochets, les caractères *, ? et / ne sont pas interprétés
\	Il permet d'échapper des caractères pour éviter qu'ils ne soient interprétés. Il sert notamment à échapper le caractère \ lui-même
Les autres caractères	Ils se représentent eux-mêmes sans être interprétés

Quelques exemples :

<i>Glob</i>	<i>Correspondances</i>
*.html	tous les fichiers ayant l'extension .html
???	trois caractères quelconques
[0-9]	tous les fichiers qui contiennent au moins un chiffre
*.{htm, html}	tous les fichiers dont l'extension est htm ou html
Test*.java	tous les fichiers dont le nom commence par un Test et possède une extension .java

NB : L'interface **PathMatcher** (existant depuis java 7) comporte une unique méthode "*Boolean matches(Path path)*". Elle peut donc être utilisée au sein de "lambda expression" depuis java 8.

Exemple (fonctionnant depuis java8) :

...
....

4. Classe utilitaire Files ("*helper*" avec 50 méthodes statiques)

4.1. Vérifications sur fichiers ou répertoires

Méthode	Rôle
boolean exists (Path)	vérifier l'existence sur le système de fichiers de l'élément dont le chemin est encapsulé dans le paramètre de type Path fourni
boolean notExists (Path)	
boolean isReadable (Path path)	peut être lu (droits en lecture) ?
boolean isWritable (Path path)	peut être modifié (droits en écriture) ?
boolean isHidden (Path path)	est caché ?
boolean isExecutable (Path path)	est exécutable ?
boolean isRegularFile (Path path)	est un fichier ?

boolean isDirectory (Path path)	est un répertoire ?
boolean isSymbolicLink (Path path)	est un lien symbolique ?
String probeContentType (Path path)	retourne type MIME d'un fichier (ex: "text/plain") (ou null si indéterminé). S'appuie par défaut sur l'OS sous jacent.

Exemple:

```
Path path = Paths.get("C:/tmp/aa/fl.txt");
if(Files.isRegularFile(path)){
    System.out.println(path + " est un chemin vers un fichier");
}
```

4.2. Création d'un fichier ou d'un répertoire

Méthode	Rôle ou bien exemple
Path createFile (Path path, FileAttribute<?>... attrs)	Créer un fichier dont le chemin est encapsulé par l'instance de type Path fournie en paramètre
Path createDirectory (Path dir, FileAttribute<?>... attrs)	Path monRepertoire = Paths.get("C:/temp/mon_repertoire"); Files.createDirectory (monRepertoire);
Path createDirectories (Path dir, FileAttribute<?>... attrs)	Créer d'un seul coup plusieurs niveaux de sous répertoire selon le chemin exprimé "C:/temp/ niveau1/niveau2/niv3 "
Path createTempDirectory (Path dir, String prefix , FileAttribute<?>... attrs) Path createTempDirectory (String prefix , FileAttribute<?>... attrs)	Créer un répertoire temporaire de nom " prefixe indiqué " + numéro_calculé_par_syst (ex : rep_1245643) au sein du répertoire indiqué ou (à défaut) au sein du répertoire système (par défaut) prévu pour les temporaires.
Path createTempFile (Path dir, String prefix , String suffix , FileAttribute<?>... attrs) Path createTempFile (String prefix , String suffix , FileAttribute<?>... attrs)	Créer un fichier temporaire de nom " prefixe indiqué " + numéro_calculé_par_syst + " suffixe indiqué " (ex : fic_1245643.txt) au sein du répertoire indiqué ou (à défaut) au sein du répertoire système (par défaut) prévu pour les temporaires. Le suffix peut éventuellement être à null .

Les **attributs** (de type *FileAttribute*) sont **facultatifs** (des valeurs par défaut existent).

Les attributs possibles seront étudiés ultérieurement .

NB : **createDirectory()** créer un seul niveau de sous répertoire à la fois (*contrairement à createDirectories*) .

Si le répertoire existe déjà , l'exception *FileAlreadyExists* est remontée.

Si le répertoire parent n'existe pas , *NoSuchFileException* est remontée.

4.3. Copie d'un fichier ou d'un répertoire

Méthode	Rôle / fonctionnalité
Path copy (Path source, Path target, CopyOption... options)	Copier un élément avec les options précisées (StandardCopyOption.REPLACE_EXISTING , StandardCopyOption.COPY_ATTRIBUTES)
long copy (InputStream in, Path target, CopyOption... options)	Copier tous les octets d'un flux de type InputStream vers un fichier
long copy (Path source, OutputStream out)	Copier tous les octets d'un fichier dans un flux de type OutputStream

NB : L'option pointue LinkOption.NOFOLLOW_LINKS permet de recopier si besoin un lien symbolique au bout du path indiqué (plutôt que de suivre le lien).

NB2: il est possible d'utiliser la méthode copy() sur un répertoire . Cependant, le répertoire sera créé sans que les fichiers contenus et .. ne soient eux aussi copiés .

Quoi que contienne le répertoire, la méthode copy ne crée qu'un répertoire vide. Pour copier le contenu du répertoire, il faut parcourir son contenu et copier chacun des éléments un par un.

4.4. Déplacement et suppression d'un fichier ou d'un répertoire

Méthode	Rôle
move (Path source, Path target, CopyOption... options)	Déplacer ou renommer un élément avec les options précisées (StandardCopyOption.REPLACE_EXISTING , StandardCopyOption.ATOMIC_MOVE)
void delete (Path path)	Supprimer un élément du système de fichiers (avec exception s'il n'existe pas ou si répertoire pas vide)
boolean deleteIfExists (Path path)	Supprimer un élément du système de fichiers s'il existe (sans exception s'il existe pas)

Si un déplacement efficace/performant demandé en mode "ATOMIC_MOVE" est impossible (par exemple déplacement de "[c:/repXy](#)" vers "[d:/repXy](#)") , une exception est alors remontée. On peut alors éventuellement ré-essayer sans l'option .

Des exceptions peuvent potentiellement remonter si un répertoire à déplacer n'est pas vide et que certains fichiers contenus sont en cours d'utilisation.

5. Parcours des éléments d'un répertoire

La solution de parcours proposée par NIO2 est plus performante que java.io.File.list(...) .

La méthode **newDirectoryStream()** de la classe utilitaire **Files** attend en paramètre un objet de type Path qui correspond au répertoire à parcourir et permet d'obtenir une instance de "**stream**" de type **DirectoryStream<Path>** (à parcourir avec un itérateur ou autre) .

Attention: il est très important d'invoquer la méthode **close()** de l'instance de type **DirectoryStream** pour libérer les ressources utilisées.

Exemple :

```

Path tmpAaPath = Paths.get("C:/tmp/aa");
DirectoryStream<Path> stream = null;
try {
    stream=Files.newDirectoryStream(tmpAaPath);
    Iterator<Path> iterator = stream.iterator();
    while(iterator.hasNext()) {
        Path p = iterator.next();
        System.out.println(p);
    }
} catch (IOException ex) { ex.printStackTrace(); }
finally {
    try { stream.close(); }
    catch (IOException e) { e.printStackTrace(); }
}

```

Exemple amélioré et simplifié :

```

Path tmpAaPath = Paths.get("C:/tmp/aa");

//NB1 : le second paramètre facultatif de Files.newDirectoryStream()
// sert à filtrer les éléments à parcourir (sans besoin de préfixe glob:)

//NB2: L'interface DirectoryStream implémente hérite de Closable et
//le try(avec _ressource_implémentant_interface Closable)
//sera automatiquement associé à un finally implicite déclenchant .close()

try (DirectoryStream<Path> stream = Files.newDirectoryStream(tmpAaPath, "*.txt")){
    for(Path p : stream){
        System.out.println(p);
    }
} catch (IOException ex){
    ex.printStackTrace();
}

```

On peut également paramétrer et utiliser un filtre spécifique lors du parcours :

```

Path tmpAaPath = Paths.get("C:/tmp/aa");
DirectoryStream.Filter<Path> littleSizeFilter = new DirectoryStream.Filter<Path>() {
    public static final long MEGABYTE = 1024*1024;
    @Override
    public boolean accept(Path element) throws IOException {
        return (Files.size(element) <= MEGABYTE );
    }
}; //fin de classe anonyme imbriquée implémentant DirectoryStream.Filter<Path>

try (DirectoryStream<Path> stream = Files.newDirectoryStream(tmpAaPath, littleSizeFilter)){
    for(Path p : stream){
        System.out.println(p);
    }
}

```

}

NB : ce code (java7) pourra être amélioré/simplifié via une lambda expression de java8 .

6. Parcours d'une hiérarchie de répertoires (visiteur)

La méthode **Files.walkFileTree()** permet de parcourir une (sous-)arborescence de répertoires en utilisant le **design pattern "visiteur"**. Ce type de parcours peut être utilisé pour rechercher, copier, déplacer, supprimer, ... des éléments de la hiérarchie parcourue.

Il faut écrire une classe qui implémente l'interface **java.nio.file.FileVisitor<T>**. Cette interface définit des méthodes qui seront des callbacks appelées lors du parcours de la hiérarchie.

Méthode	Rôle / fonctionnalité
FileVisitResult postVisitDirectory (T dir, IOException exc)	Le parcours sort d'un répertoire qui vient d'être parcouru ou une exception est survenue durant le parcours
FileVisitResult preVisitDirectory (T dir, BasicFileAttributes attrs)	Le parcours rencontre un répertoire, cette méthode est invoquée avant de parcourir son contenu
FileVisitResult visitFile (T file, BasicFileAttributes attrs)	Le parcours rencontre un fichier
FileVisitResult visitFileFailed (T file, IOException exc)	La visite d'un des fichiers durant le parcours n'est pas possible et une exception a été levée

Les méthodes de l'interface **FileVisitor** renvoient toutes une valeur qui appartient à l'énumération **FileVisitResult**. Cette valeur permet de contrôler le processus de parcours de l'arborescence :

- **CONTINUE** : poursuite du parcours
- **TERMINATE** : arrêt immédiat du parcours
- **SKIP_SUBTREE** : inhibe le parcours de la sous-arborescence.
- **SKIP_SIBLING** : inhibe le parcours des répertoires frères.

Exemple(s) à ajouter ici plus tard .

7. FileSystem (par défaut et "personnalisé")

7.1. Fabrique FileSystems

FileSystems est une **fabrique** (avec **méthodes statiques**) pour obtenir des objets **FileSystem**.

.getDefault() renvoie l'instance de type **FileSystem** qui encapsule le F.S. de la JVM.

.getFileSystem(fsUri) renvoie un **FileSystem** selon l'URI est fourni en paramètre.

.newFileSystem() surchargée permet de créer une instance spécifique de type **FileSystem** (cas pointu)

7.2. FileSystem

String separator = FileSystems.getDefault().**getSeparator()**; // / sous linux ou \ sous windows

```
Iterable<Path> dirs = FileSystems.getDefault().getRootDirectories();
for (Path name: dirs) {
    System.err.println(name); // C:\ , D:\ , ...
}
```

7.3. FileSystem spécifique

...

8. Lecture et écriture dans un fichier

8.1. Vue d'ensemble

Principaux apports de NIO et NIO2 :

IO	NIO	NIO2
Depuis Java 1.0 et 1.1	Depuis Java 1.4 (JSR 151)	Depuis Java 7 (JSR 203)
Synchrone bloquant	Synchrone non bloquant	Asynchrone non bloquant
File		Path
InputStream	FileChannel	AsynchronousFileChannel
OutputStream	SocketChannel	AsynchronousByteChannel
Reader (Java 1.1)	ServerSocketChannel	AsynchronousSocketChannel
Writer (Java 1.1)	(Charset, Selector,	AsynchronousServerSocketChannel
Socket	ByteBuffer)	1
RandomAccessFile		SeekableByteChannel

8.2. Options sur l'ouverture d'un fichier

L'énumération **StandardOpenOption** implémente l'interface *OpenOption* et définit les options d'ouverture standard d'un fichier :

Valeur	Signification
APPEND	Si le fichier est ouvert en écriture alors les données sont ajoutées au fichier. Cette option doit être utilisée avec les options CREATE ou WRITE

CREATE	Créer un nouveau fichier s'il n'existe pas sinon le fichier est ouvert
CREATE_NEW	Créer un nouveau fichier : si le fichier existe déjà alors une exception est levée
DELETE_ON_CLOSE	Supprimer le fichier lorsque son flux associé est fermé : cette option est utile pour des fichiers temporaires
DSYNC	Demander l'écriture synchronisée des données dans le système de stockage sous-jacent (pas d'utilisation des tampons du système) ???
READ	Ouvrir le fichier en lecture
SPARSE	Indiquer au système que le fichier est clairsemé ce qui peut lui permettre de réaliser certaines optimisations si l'option est supportée par le système de fichiers (c'est notamment le cas avec NTFS)
SYNC	Demander l'écriture synchronisée des données et des métadonnées dans le système de stockage sous-jacent
TRUNCATE_EXISTING	Si le fichier existe et qu'il est ouvert en écriture alors il est vidé. Cette option doit être utilisée avec l'option WRITE
WRITE	Ouvrir le fichier en écriture

8.3. Lecture de l'intégralité d'un fichier / Files.readAllLines()

Lecture (en boucle) de toutes les lignes d'un fichier texte :

```
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
...
List<String> lignes = Files.readAllLines(
    FileSystems.getDefault().getPath("c:/tmp/aa/fl.txt"), StandardCharsets.UTF_8);
for (String ligne : lignes)
    System.out.println(ligne);
```

Lecture d'un bloc de tout un (petit) fichier binaire :

```
byte[] binaryContent = Files.readAllBytes(binaryFilePath);
```

XVIII - Annexe – Quelques API de java

1. Process Api

1.1. Lancement d'un processus depuis une application java

```
public static void startNodePad() {
    ProcessBuilder builder = new ProcessBuilder("notepad.exe"); //de java.lang depuis java 1.5
    try {
        Process process = builder.start();
        System.out.println("pid of process=" + process.pid());
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

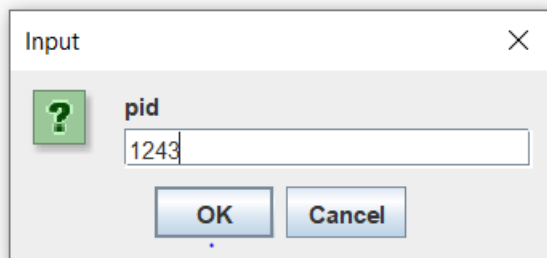
1.2. L'interface ProcessHandle (java.lang) depuis java 9

return type	method	fonctionnalités
static Stream < ProcessHandle >	<u>allProcesses()</u>	Returns a snapshot of all processes visible to the current process.
Stream < ProcessHandle >	<u>children()</u>	Returns a snapshot of the current direct children of the process.
int	<u>compareTo</u> (ProcessHandle other)	Compares this ProcessHandle with the specified ProcessHandle for order.
static ProcessHandle	<u>current()</u>	Returns a ProcessHandle for the current process.
Stream < ProcessHandle >	<u>descendants()</u>	Returns a snapshot of the descendants of the process.
boolean	<u>destroy()</u>	Requests the process to be killed.
boolean	<u>destroyForcibly()</u>	Requests the process to be killed forcibly.
boolean	<u>equals</u> (Object other)	Returns true if other object is non-null, is of the same implementation, and represents the same system process; otherwise it returns false.
int	<u>hashCode()</u>	Returns a hash code value for this ProcessHandle.
ProcessHandle.Info	<u>info()</u>	Returns a snapshot of information about the process.
boolean	<u>isAlive()</u>	Tests whether the process represented by this ProcessHandle is alive.
static Optional < ProcessHandle >	<u>of</u> (long pid)	Returns an Optional < ProcessHandle > for an existing native process.

CompletableFuture < ProcessHandle >	onExit()	Returns a CompletableFuture < ProcessHandle > for the termination of the process.
Optional < ProcessHandle >	parent()	Returns an Optional < ProcessHandle > for the parent process.
long	pid()	Returns the native process ID of the process.
boolean	supportsNormalTermination()	Returns true if the implementation of destroy() normally terminates the process.

1.3. Exemples de gestion de processus

```
import java.io.IOException;
import java.util.Optional;
import javax.swing.JOptionPane;
....
public static void destroyProcessById() {
    long pid=Long.parseLong(JOptionPane.showInputDialog(null, "pid"));
    Optional<ProcessHandle> optionalProcessHandle = ProcessHandle.of(pid);
    optionalProcessHandle.ifPresent(processHandle -> processHandle.destroy());
}
```



```
public static void startAndDestroyProcess() {
    //ProcessBuilder builder = new ProcessBuilder("C:\\Program Files\\Mozilla Firefox\\firefox.exe");
    ProcessBuilder builder = new ProcessBuilder("notepad.exe");
    try {
        Process process = builder.start();
        System.out.println("pid of process=" + process.pid());
        Thread.sleep(5000); //5000ms
        process.destroy();
        if (process.isAlive()) {
            System.out.println("process still alive ...");
            process.destroyForcibly(); //a utiliser que si process.destroy() ne suffit pas
        } else {
            System.out.println("process is no more alive ...");
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

```
}
}
```

```
...
public static String processHandleAsString(ProcessHandle ph) {
    return "pid="+ph.pid()+" - "+ph.info().toString();
}

public static void displayingAllProcess() {
    ProcessHandle.allProcesses().forEach(ph -> System.out.println(processHandleAsString(ph)));
}
-->
```

```
...
pid=21964 - [user: Optional[LAPTOP-DDC\didier], cmd: C:\Program Files (x86)\Google\Chrome\Application\chrome.exe,
startTime: Optional[2020-07-02T14:04:20.296Z], totalTime: Optional[PT0.078125S]]
pid=4840 - []
pid=5696 - [user: Optional[LAPTOP-DDC\didier], cmd: C:\Program Files\Java\jdk-11.0.4\bin\javaw.exe, startTime: Optional[2020-
07-02T14:47:38.376Z], totalTime: Optional[PT0.34375S]]
```

```
package org.mycontrib.tp;
public class MySubProcess {
    public static void main(String[] args) {
        System.out.println("MySubProcess ...");
        try { Thread.sleep(5000); /*5000ms*/ } catch (InterruptedException e) { e.printStackTrace(); }
        System.exit(0);
    }
}
```

```
public static void startAndWaitingProcess() {
    ProcessBuilder builder = new ProcessBuilder();
    /*
    final String javaHome = "C:\\Program Files\\Java\\jdk-11.0.4"; // " " need around directory name with space
    final String javaCmdeWithQuote = "\"" + javaHome+"\\bin\\java.exe" + "\"";
    //final String cmde = javaCmdeWithQuote + " -version";
    final String cmde = javaCmdeWithQuote+" -cp .\\target\\myProcessApp.jar org.mycontrib.tp.MySubProcess";
    System.out.println("cmde="+cmde);
    //builder.command("cmd.exe", "/c", cmde); // ok, "/c" = terminate after this run
    */
    //NB: .\\target\\myProcessApp.jar will be ready after mvn install or ...
    builder.command("java","-cp",".\\target\\myProcessApp.jar",
        "org.mycontrib.tp.MySubProcess"); //ok with java in PATH
    boolean isStopped = false;
    try {
        Process process = builder.start();
        System.out.println("pid of process=" + process.pid());
        BufferedReader outputReaderOfSubProcess = new BufferedReader(new
            InputStreamReader(process.getInputStream()));
        System.out.println("output of process (first line) =" +
            outputReaderOfSubProcess.readLine());
        isStopped=process.waitFor(3, TimeUnit.SECONDS);
        if(isStopped) {
```

```
        System.out.println("process is terminated");
    } else {
        System.out.println("process not terminated after 3s");
        isStopped=process.waitFor(3, TimeUnit.SECONDS);//re-wait
    }
    if (isStopped) {
        System.out.println("process is stopped with exit value="
            +process.exitValue());
    }
} catch (Exception e) {
    e.printStackTrace();
}
}
```

XIX - Annexe – Tests unitaires (essentiel JUnit 5)

1. Tests unitaires avec JUnit 5

1.1. Présentation de JUnit 3,4,5

JUnit est un *framework* simple permettant d'effectuer des **tests** (unitaires , de non régression, ...) au cours d'un développement java . [Projet Open source ---> <http://junit.sourceforge.net/> , <http://junit.org>] . *JUnit est intégré au sein des IDE Eclipse et IntelliJ.*

JUnit existe en versions 3 , 4 et 5 avec des différences significatives d'une version à l'autre .

1.2. Présentation des anciennes versions 3 et 4

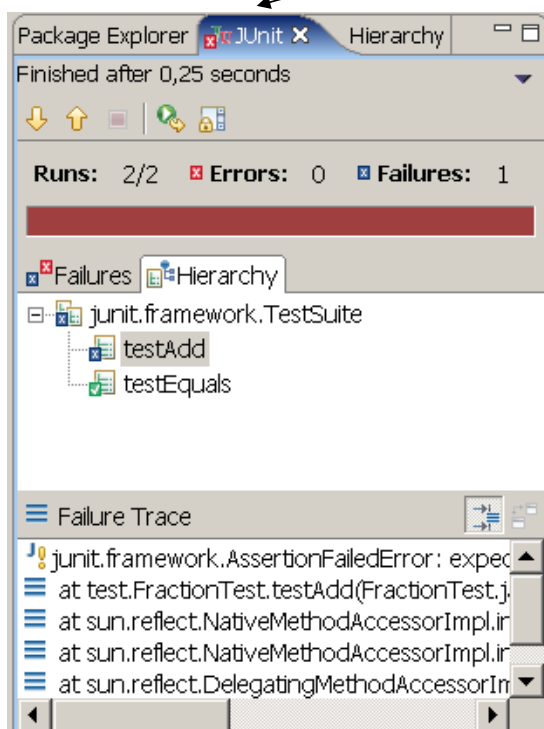
La très ancienne version 3 de JUnit n'utilisait pas d'annotations . Tout était basé sur un héritage (TestCase) et sur des conventions de nom sur les méthodes (setUp() , tearDown() , testXy()) . Depuis environ 2006/2007 , cette historique version 3 est devenue petit à petit obsolète (utilisée seulement dans les anciens projets).

La version 4 a été massivement utilisée dans la majorité des projets java de 2007 à 2019 environ . La version 4 était basée sur le package org.junit (Test, Assert, ...) et les principales annotations étaient @Test , @Before , @After , @BeforeClass , @AfterClass .

1.3. Lancement des tests unitaires

Lancement des tests unitaires

Depuis l'IDE eclipse:
Run as ... / JUnit test



TestSuite en JUnit3
et lancement après une sélection
de **package** en JUnit4 pour lancer
d'un coup toute une série de tests.

Comptabilisations :

Error(s) : exceptions java
non rattrapées.

Failure(s) : assertions
non vérifiées.

VERT si aucune erreur.

Depuis "maven" :

coder des classes nommées "**Test**Xy"
ou "XY**Test**" dans **src/test/java**
et lancement via **mvn test** ou autre.

1.4. Présentation de JUnit 5 ("jupiter")

La **version 5 de JUnit** a été entièrement restructurée et s'appuie sur certaines nouvelles fonctionnalités apportées par la **version 8 du langage java** (lambda expressions, ...).

Principales différences entre JUnit4 et JUnit5 :

JUnit 4	JUnit 5
package org.junit	package org.junit.jupiter.api
Assert.assertTrue() , Assert.assertEquals(...)	Assertions .assertTrue() , Assertions .assertEquals(...)
@Before , @After	@BeforeEach , @AfterEach
@BeforeClass , @AfterClass (avec static)	@BeforeAll , @AfterAll (avec static)
@Test(timeout="20")	@Timeout() ou bien Assertions.assertTimeout(Duration.ofMillis(200), () -> {}) ;
@Ignore	@Disabled
@Rule	@ExtendWith

@Category	@Tag
...	...

1.5. Configuration maven pour JUnit 5 ("jupiter")

```

...
<properties>
  <junit.jupiter.version>5.14.4</junit.jupiter.version>
</properties>
<dependencies>
<dependency>
  <groupId>org.junit.jupiter</groupId> <artifactId>junit-jupiter-api</artifactId>
  <version>${junit.jupiter.version}</version> <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.junit.jupiter</groupId> <artifactId>junit-jupiter-engine</artifactId>
  <version>${junit.jupiter.version}</version> <scope>test</scope>
</dependency>
</dependencies>
<build> <plugins>
  <plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-surefire-plugin</artifactId>
    <version>3.1.2</version> <!-- Need at least 2.22.0 to support JUnit 5 -->
  </plugin>
</plugins> </build> ...

```

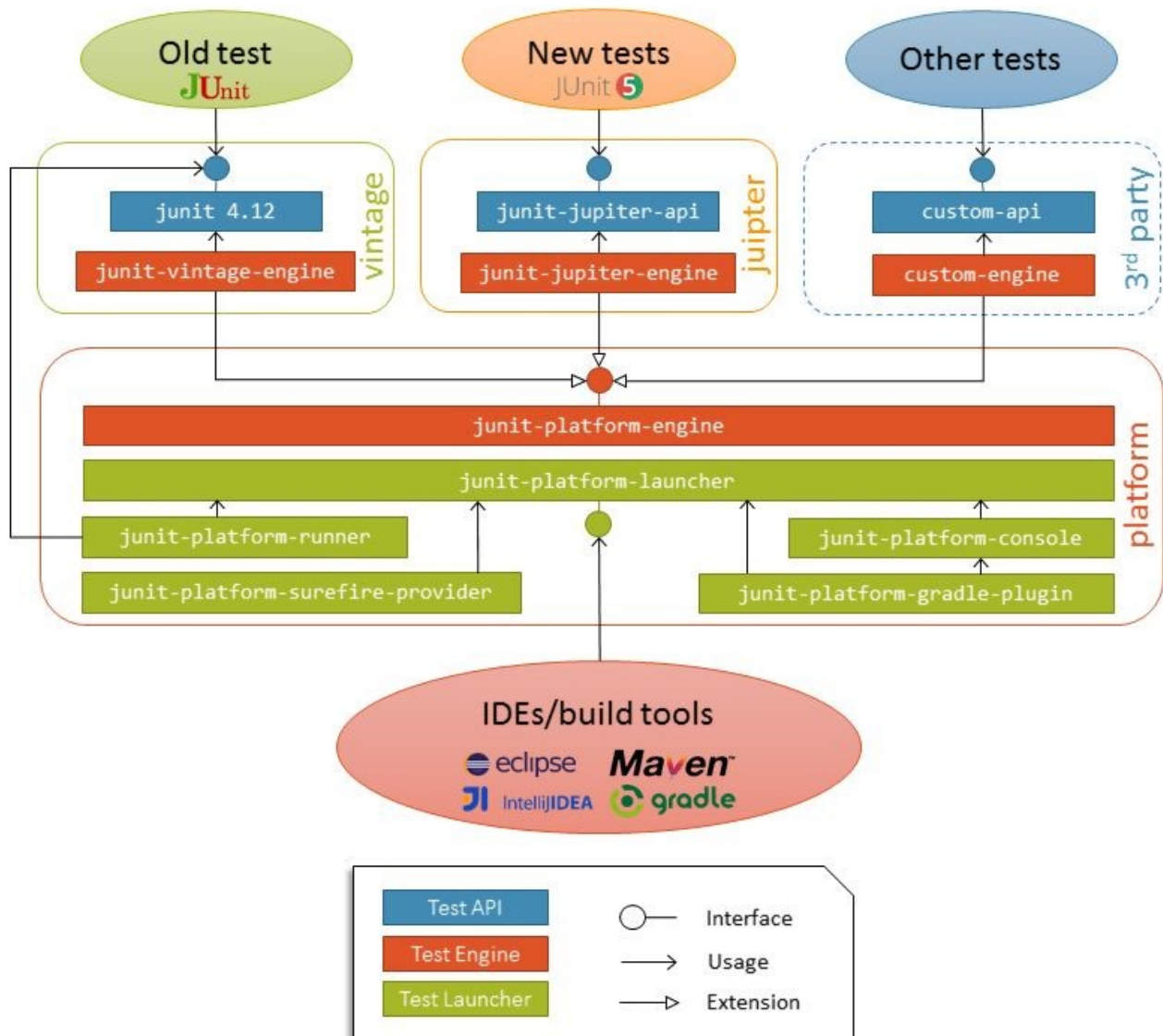
Eventuels compléments :

junit-jupiter-params support des *tests paramétrés* avec JUnit Jupiter.

junit-vintage-engine pour interpréter et exécuter des anciens tests codés en JUnit 3 ou 4

junit-platform-.... pour intégration et lancement (console , maven , gradle ,)

1.6. Structure de JUnit5



1.7. Test Unitaire avec JUnit 5 (@Test, @BeforeEach, @AfterEach)

```
import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
...
public class TestCalculatrice {
    private static Logger logger = LoggerFactory.getLogger(TestCalculatrice.class);
    private CalculatriceEx calculatriceEx; //à tester

    @BeforeEach // @BeforeEach in jUnit5 , @Before in jUnit4
    public void initCalculatriceEx() {
        this.calculatriceEx = new CalculatriceEx();
    }

    @Test
    public void testSum() {
        calculatriceEx.pushVal(1.1);
        calculatriceEx.pushVal(2.2);
        calculatriceEx.pushVal(3.3);
        double somme = calculatriceEx.sum();
        logger.trace("somme(1.1, 2.2, 3.3)="+somme);
        //Assert.assert...() avec jUNit 4 et Assertions.assert...() avec jUnit5/jupiter
        Assertions.assertEquals( 6.6 , somme, 0.00000001);
    }

    @Test
    public void testAverage() {
        calculatriceEx.pushVal(1.1);
        calculatriceEx.pushVal(1.5);
        double moyenne = calculatriceEx.average();
        logger.trace("moyenne(1.1, 1.5)="+moyenne);
        Assertions.assertEquals( 1.3 , moyenne, 0.00000001);
    }
}
```

NB: Comme avec JUnit 4, par défaut JUnit 5 crée une nouvelle instance de la classe de test pour exécuter chaque méthode de test. (*@TestInstance(Lifecycle.PER_METHOD)* par défaut)

```
Assertions.assertEquals(expectedValue, effectiveValue , delta ) ;
Assertions.assertTrue(booleanExpression) ;   Assertions.assertNotNull(...) ...
```

NB :

```
import static org.junit.jupiter.api.Assertions.assertTrue;
```

permet d'écrire **assertTrue** (res==5) au lieu de *Assertions.assertTrue*(res==5) et ça peut aider à masquer la différence de préfixe entre JUnit 4 et JUnit5 .

1.8. Test Unitaire JUnit5 avec static (@BeforeAll , @BeforeAll)

```
import org.junit.jupiter.api.BeforeAll;
...
public class TestCalculatrice {
    private static SimpleCalculatrice calculatrice; //à tester

    @BeforeAll // @BeforeAll in jUnit5 , @BeforeClass in jUnit4
    public static void initCalculatrice() {
        calculatrice = new SimpleCalculatrice();
    }

    @Test
    public void testAddition() {
        double resAdd=calculatrice.addition(5.5, 6.6);
        logger.trace("addition(5.5,6.6)="+resAdd);
        Assertions.assertTrue(resAdd >= (12.1 - 0.00000001) &&
                               resAdd <= (12.1 + 0.00000001));
    }

    @Test
    public void testMultiplication() {
        double resMult=calculatrice.multiplication(2.0, 3.3);
        logger.trace("multiplication(2.0, 3.3)="+resMult);
        Assertions.assertEquals( 6.6 , resMult, 0.00000001);
    }
}
```

Variante (sans static) :

```
import org.junit.jupiter.api.* ; ...
import org.junit.jupiter.api.TestInstance.Lifecycle;

@TestInstance(Lifecycle.PER_CLASS)
//a single reused and reinitialized instance of this test class for all test methods (ex:TestPerClassInstance@aeab9a1)
public class TestPerClassInstance {
    private static Logger logger = LoggerFactory.getLogger(TestPerClassInstance.class);

    @BeforeAll
    public void initBeforeAll() {
        logger.trace("TestPerMethodInstance.initBeforeAll() called (no static way) on this="+this );
    }

    @Test
    public void test1() {
        logger.trace("TestPerClassInstance.test1() called on this="+this );
    }
}
```

NB : En plaçant **@TestInstance(Lifecycle.PER_CLASS)** au dessus d'une classe de test Junit5 (imbriquée ou pas) , on a comme comportement le fait qu'une seule instance de la classe de test sera utilisée pour exécuter successivement toutes les méthodes de tests de la classe de test (contrairement au comportement par défaut PER_METHOD) .

1.9. Désactivation d'un test

@Ignore en V4 (org.junit.Ignore) , @Disabled en V5(org.junit.jupiter.api.Disabled)

WrongTests.java

```
package tp.converter;
import static org.junit.jupiter.api.Assertions.assertTrue;
import org.junit.jupiter.api.Disabled; import org.junit.jupiter.api.Test;

public class WrongTests {
    @Test
    @Disabled //test désactivé/ignoré
    public void testWithError() {
        int a=3; int b=0;
        assertTrue(a/b==0); //div by zero exception/error !!!
    }

    @Test
    @Disabled //test désactivé/ignoré
    public void testWithBadAssertion() {
        assertTrue(2+2==5); //2+2==4 !!!! not 5 !!!
    }

    @Test
    public void goodTrivialTest() { assertTrue(2+2==4);
    }
}

@Disabled("test temporairement désactivé")
sur classe ou méthode de test
```

1.10. Spécificités de JUnit5

```
@DisplayName("Ma classe de test JUnit5 que j'aime")
public class MyTest {
    @Test
    @DisplayName("Mon cas de test Xy qui va bien")
    void Testxy() { // ... }
}
```

NB: Les valeurs des @DisplayName seront affichées au sein des rapports d'exécution des tests

NB: contrairement à JUnit4, les **méthodes** préfixées par @BeforeEach , @Test , **n'ont plus absolument besoin d'être public** (protected ou bien *la visibilité implicite par défaut de niveau package suffit*) .

```
Iterable<Integer> attendu = new ArrayList<>(Arrays.asList(1, 2, 3));
Iterable<Integer> actuel = new ArrayList<>(Arrays.asList(1, 2)); //manque 3
Assertions.assertIterableEquals(attendu, actuel); //echec
```

vérifie que les collections ont mêmes tailles et mêmes valeurs.

@RepeatedTest(value = 3) est une variante de @Test permettant de lancer n fois un même test

1.11. Test avec timeout

```
@Test()
@Test(timeout = 200, unit = TimeUnit.MILLISECONDS)
public void testWithTimeout() {
    try {
        Thread.sleep(100); //ok
        //Thread.sleep(300); //not_ok
    } catch (InterruptedException e) { System.err.println(e.getMessage()); }
}
```

ou bien

```
@Test()
public void testWithTimeoutV2() {
    Assertions.assertTimeout(Duration.ofMillis(200),
        () -> { try {
            Thread.sleep(100); //ok
            //Thread.sleep(300); //not_ok
        } catch (InterruptedException e) {
            System.err.println(e.getMessage());
        }
    });
}
```

1.12. Assertions avec lambdas :

AssertAll() avec lambdas :

```
Assertions.assertAll("wrong Dimension",
    () -> Assertions.assertTrue(elt.getWidth() == 400, "wrong width"),
    () -> Assertions.assertTrue(elt.getHeight() == 500, "wrong height"));
```

vérifie que chacune des lambdas est ok (sans exception) .

AssertThrows avec lambda :

```
@Test
void verifierException() {
    String valeur = "123a";
    NumberFormatException nfe =
        Assertions.assertThrows(NumberFormatException.class,
            () -> { Integer.valueOf(valeur); }
        );
}
```

vérifie qu'une exception est bien levée (suite à erreur volontaire) et du bon type .

AssertTimeout avec lambda :

```

@Test
void verifierTimeout() {
    Assertions.assertTimeout(Duration.ofMillis(200),
        () -> { * traitement potentiellement trop lent * return "...";
        });
}

```

1.13. Suppositions (pour exécuter ou pas un test selon le contexte) :

```
assumeTrue( System.getenv("OS").startsWith("Windows") );
```

//la suite du test ne sera exécutée que si os courant est Windows

//si le test n'est pas exécuté --> même comportement que si test vide --> ok/réussi/vert

//variante avec lambda exécutée qui si supposition ok :

```

assumingThat(System.getenv("OS").startsWith("Windows"), () -> {
    assertTrue(new File("C:/Windows").exists(), "Repertoire Windows inexistant");
});

```

1.14. Tests imbriqués de JUnit5

```

public class MyTest {

    @BeforeEach
    void mainInit() {
        System.out.println("BeforeEach / first level");
    }

    @Nested
    class MonTestImbrique {
        @BeforeEach
        void subInit() {
            System.out.println("BeforeEach imbrique");
            valeur = 5;
        }

        @Test
        void simpleTestImbrique() {
            System.out.println("SimpleTest imbrique valeur=" + valeur);
            Assertions.assertEquals(5, valeur);
        }
    }
}

```

```
}
}
```

NB : On peut éventuellement utiliser le mode `PER_CLASS` au niveau de la classe comportant des sous classes annotées avec `@Nested` pour contrôler le nombre d'appels effectués (au niveau des méthodes préfixées par `@BeforeAll` et `@AfterAll`).

1.15. Tests paramétrés via source/série de valeurs

Après avoir ajouté la dépendance nécessaire *junit-jupiter-params* , on peut écrire des méthodes de tests avec un paramètre qui sera alimenté avec une source/série de valeurs .

Un test paramétré sera ainsi lancé plusieurs fois (avec des arguments différents pour obtenir une certaine variété)

@ParameterizedTest

```
@ValueSource(ints = { 1, 2, 3 })
void testParametreAvecValueSource(int valeur) {
    assertEquals(valeur + valeur, valeur * 2);
}
```

`@ValueSource(ints = { 1, 2, 3 })` ou `@ValueSource(strings = { "un", "deux" })` ou ...

import java.util.stream.Stream;

```
@ParameterizedTest
@MethodSource("fournirDonneesParametres")
void testTraiterSelonMethodSource(String element) {
    assertTrue(element.startsWith("elt"));
}

static Stream<String> fournirDonneesParametres() {
    return Stream.of("elt1", "elt2");
}
```

```
public enum Direction { NORD , SUD, EST , OUEST }
@ParameterizedTest
    @EnumSource(TestParameterizedWithSources.Direction.class)
    void testTraiterSelonEnumValues(Direction direction) {
        assertTrue(direction.toString().length()>=3);
        logger.trace("testTraiterSelonEnumValues direction="+direction);
    }
```

```
@ParameterizedTest
    @CsvSource(delimiter = ';',
        value= { "1;1;un" ,
                "2;4;quatre" ,
                "3;9;neuf"
            }
    )
    void testTraiterSelonCsvSource(int n,int expectedCarre,String comment) {
        assertTrue(n*n == expectedCarre);
        logger.trace("testTraiterSelonCsvSource n="+n
            + " expectedCarre="+expectedCarre
            + " comment="+comment);
    }
```

```
@ParameterizedTest
    @CsvFileSource( resources= "/carres.csv", delimiter = ';', numLinesToSkip = 1)
    //files= "carres.csv" found at eclipse project root
    //resources= "/carres.csv" found in src/test/resources
    void testTraiterSelonCsvFileSource(int n,int expectedCarre,String comment) {
        assertTrue(n*n == expectedCarre);
        logger.trace("testTraiterSelonCsvFileSource n="+n
            + " expectedCarre="+expectedCarre
            + " comment="+comment);
    }
```

src/test/resources/carres.csv

```
n;carre;comment
1;1;un
2;4;quatre
3;9;neuf
```

1.16. Tests dynamiques / @TestFactory

```
...
import static org.junit.jupiter.api.Assertions.assertEquals;

import java.util.ArrayList;
import java.util.Collection;
import org.junit.jupiter.api.DynamicTest;
import org.junit.jupiter.api.TestFactory;

class MonTestSimple {

    @TestFactory
    Collection<DynamicTest> dynamicTestsWithCollection() {
        Collection<DynamicTest> myDynamicTests = new ArrayList<>();
        for (int i = 1; i <= 5; i++) {
            int val = i;
            myDynamicTests.add(DynamicTest.dynamicTest("Ajout " + val + "+" + val,
                () -> assertEquals(val * 2, val + val));
        }
        return myDynamicTests;
    }
}
```

--> à priori intéressant que si couplé avec une introspection dynamique (framework de tests).

1.17. Suite de Tests (JUnit 5)

Durant la période de transition JUnit4 vers JUnit5 , une suite de Tests pouvait être codée de cette manière :

```
...
import org.junit.platform.runner.JUnitPlatform;
import org.junit.platform.suite.api.SelectClasses;
import org.junit.runner.RunWith;

//NB: JUnitPlatform nécessite org.junit.platform:junit-platform-runner:1.9.3 dans pom.xml
// @RunWith() nécessite org.junit.vintage:junit-vintage-engine:5.9.3 dans pom.xml

@RunWith(JUnitPlatform.class)
@SelectClasses{
    TestCalculsSimples.class,
    TestEmptySerie.class,
    TestSerie.class
}
public class CalculsSuiteV1 {
}
```

Au sein d'un projet récent (prise en charge au sein d'un IDE récent) , une suite de tests moderne junit5 sera plutôt codée de cette manière :

```
...
import org.junit.platform.suite.api.SelectClasses;
import org.junit.platform.suite.api.Suite;

//@RunWith(JUnitPlatform.class) will be deprecated , use @Suite instead
//but @Suite not already supported by some old IDE (ex: ok with eclipse 2023-03)

@Suite
@SelectClasses({
    TestCalculsSimples.class,
    TestEmptySerie.class,
    TestSerie.class
})
public class CalculsSuite {
}
```

```
@Suite
@SelectPackages({"tp.calculs", "tp.converter" })
public class AllTestsSuite {
}
```

1.18. Suite de Tests avec Tags

@Tag("nomDeTag") de JUnit5 peut être placé sur une classe ou bien une méthode de test

Ces noms de tags (préalablement appelés catégories en Junit4) permettront d'effectuer ultérieurement des filtrages sur les tests à lancer ou pas .

Il est éventuellement possible de placer plusieurs tags complémentaires au dessus d'un même test (ex : **@Tag("prod") @Tag("dev")**) .

Exemple :

```
...
import org.junit.jupiter.api.Tag;
import org.junit.jupiter.api.Test;

public class TestWithCategoryTag {
    private static Logger logger = LoggerFactory.getLogger(TestWithCategoryTag.class);

    @Test
    @Tag("SlowTests")
    public void testA() {
        logger.trace("slow testA");
    }
}
```

```

@Test
@Tag("FastTests")
public void testB() {
    logger.trace("fast testB");
}

@Test
@Tag("FastTests") @Tag("ImportantTests")
public void testC() {
    logger.trace("important fast testC");
}

@Test
@Tag("SlowTests") @Tag("SecondaryTests")
public void testD() {
    logger.trace("secondary slow testD");
}
}

```

```

...
import org.junit.platform.suite.api.ExcludeTags;
import org.junit.platform.suite.api.SelectClasses;
import org.junit.platform.suite.api.Suite;

@Suite
//@IncludeTags({"FastTests"})
@ExcludeTags({"SlowTests", "SecondaryTests"})
@SelectClasses({
    TestWithCategoryTag.class ,
    TestWithAssumptions.class
})
public class FastTestsSuite {
}

```

NB.:

- par défaut , @SelectPackages(...) sélectionne tous les sous-packages également .
- On peut éventuellement ajouter @IncludePackages(...) ou bien @ExcludePackages(...) pour filtrer les sous-packages à sélectionner.
- On peut également ajouter @IncludeClassNamePatterns({ "^.*Simple\$" }) pour filtrer les noms des classes de Tests à lancer .
- Possibilité de filter les category/tags au sein de la config maven:

```

<plugin>
  <artifactId>maven-surefire-plugin</artifactId>
  <configuration>
    <properties>
      <includeTags>FastTests</includeTags>
    </properties>
  </configuration>
</plugin>

```

1.19. Extensions JUnit5

L'ancien concept de règle (@Rule) JUnit4 a été remplacé par la notion d'extension (plus clair) en JUnit5 .

Exemple d'extension personnalisée :

```
package tp.extensions;
import org.junit.jupiter.api.extension.AfterEachCallback;
import org.junit.jupiter.api.extension.BeforeEachCallback;
import org.junit.jupiter.api.extension.ExtensionContext;
import org.junit.jupiter.api.extension.ExtensionContext.Namespace;
import org.slf4j.Logger; import org.slf4j.LoggerFactory;

//NB1: just one instance of this class
//((don't use attributes but context.getStore(Namespace.GLOBAL or SPECIFIC_NAMESPACE))
//NB2: JUnit5 Extensions seems to be rather slow (may be aop/weaver) ...
public class MyPerfCustomExtension implements AfterEachCallback, BeforeEachCallback {
    private static Logger logger = LoggerFactory.getLogger(MyPerfCustomExtension.class);
    private static final Namespace NAMESPACE =
        Namespace.create("tp", "extensions", "MyPerfCustomExtension");

    @Override
    public void beforeEach(ExtensionContext context) throws Exception {
        long startTime = System.nanoTime();
        logger.info("MyPerfCustomExtension, before execution of "
            + context.getDisplayName()
            + " at startTime=" + startTime );
        Namespace methodNamespace =
            NAMESPACE.append(context.getDisplayName());
        context.getStore(methodNamespace).put("startTime", startTime);
    }

    @Override
    public void afterEach(ExtensionContext context) throws Exception {
        long endTime = System.nanoTime();
        Namespace methodNamespace =
            NAMESPACE.append(context.getDisplayName());
        long startTime = (Long) context.getStore(methodNamespace).get("startTime");
        logger.info("MyPerfCustomExtension, after execution of "
            + context.getDisplayName()
            + " execDuration (ms) without good precision = "
            + ((double)(endTime - startTime)/1000000)
            + " startTime=" + startTime + " endTime=" + endTime );
    }
}
```

Utilisation d'une extension JUnit5 :

```

package tp.converter.other;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.slf4j.Logger; import org.slf4j.LoggerFactory;
import tp.extensions.MyPerfCustomExtension;

@ExtendWith(MyPerfCustomExtension.class)
public class TestWithExtension {
    private static Logger logger = LoggerFactory.getLogger(TestWithExtension.class);

    @Test
    public void fastTest() {
        try {
            Thread.sleep(20); //ok
        } catch (InterruptedException e) {
            System.err.println(e.getMessage());
        }
    }

    @Test
    public void slowTest() {
        try {
            Thread.sleep(60); //ok
        } catch (InterruptedException e) {
            System.err.println(e.getMessage());
        }
    }
}

```

Résultats :

MyPerfCustomExtension, before execution of slowTest() at startTime=510085534119300

*MyPerfCustomExtension, after execution of slowTest() execDuration (ms) without good precision = **92.514**
 startTime=510085534119300 endTime=510085626633300*

MyPerfCustomExtension, before execution of fastTest() at startTime=510085636236500

*MyPerfCustomExtension, after execution of fastTest() execDuration (ms) without good precision = **35.5131**
 startTime=510085636236500 endTime=510085671749600*

Possibilité d'utiliser conjointement plusieurs extensions complémentaires :

```

@ExtendWith({ MyPerfCustomExtension.class , MyBasicLogExtension.class })
public class TestWithExtension {
    ...
}

```

Principales extensions prédéfinies de JUnit5

Extensions	Caractéristiques
TempDirectory et @TempDir	Destruction automatique (en fin de tests) des fichiers et répertoires temporaires créés durant le test. NB : @TempDir peut être placé sur un attribut (static ou pas) d'une classe ou bien peut être placé sur un paramètre d'entrée d'une méthode de test
...	

Exemples :

```
import org.junit.jupiter.api.io.TempDir;
...

@TempDir
File tempDir;

@Test
void testWithTempDir() throws IOException {
    assertTrue(this.tempDir.isDirectory(), "Should be a directory ");

    File f1 = new File(tempDir, "f1.txt");
    List<String> lines = Arrays.asList("aaa", "bbb", "ccc");

    Files.write(f1.toPath(), lines);

    assertAll(
        () -> assertTrue(Files.exists(f1.toPath()), "File should exist"),
        () -> assertLinesMatch(lines, Files.readAllLines(f1.toPath())));
}
```

1.20. Test(s) dans interface (contrat d'interface)

Code à tester :

```
public interface IDecorate {
    //returned String must contains inputString
    //and may be decorated by any prefix or any suffix or both
    String decorate(String inputString);
}
```

```
public class DecorateWithPrefix implements IDecorate{
    @Override
```

```

    public String decorate(String inputString) {
        return ">>>" + inputString;
    }
}
//-----
public class DecorateWithSuffix implements IDecorate {
    @Override
    public String decorate(String inputString) {
        return inputString + "<<<";
    }
}

```

```

package tp.other;
import static org.assertj.core.api.Assertions.assertThat;
import org.junit.jupiter.api.Test;
import org.slf4j.Logger; import org.slf4j.LoggerFactory;

public interface IDecorateContract {

    static final Logger logger = LoggerFactory.getLogger(IDecorateContract.class);
    IDecorate create();

    @Test
    default void postcondition() {
        String parameter = "abc"; //or random string
        String res = create().decorate(parameter);
        logger.trace("res="+res);
        assertThat(res).isNotBlank()
            .contains(parameter);
    }
}

```

```

...
public class TestDecorateWithSuffix implements IDecorateContract {
    @Override
    public IDecorate create() {
        return new DecorateWithSuffix();
    }
    //AVEC DECLenchement AUTOMATIQUE
    //du @Test postcondition() defini dans l'interface IDecorateContract
}

```

... TRACE IDecorateContract:20 - res=>>>abc

XX - Annexe – Tests avancés (JaCoCo, Mockito,...)

1. Analyse de la couverture des tests

Toutes les parties importantes du code ont bien été testées ?

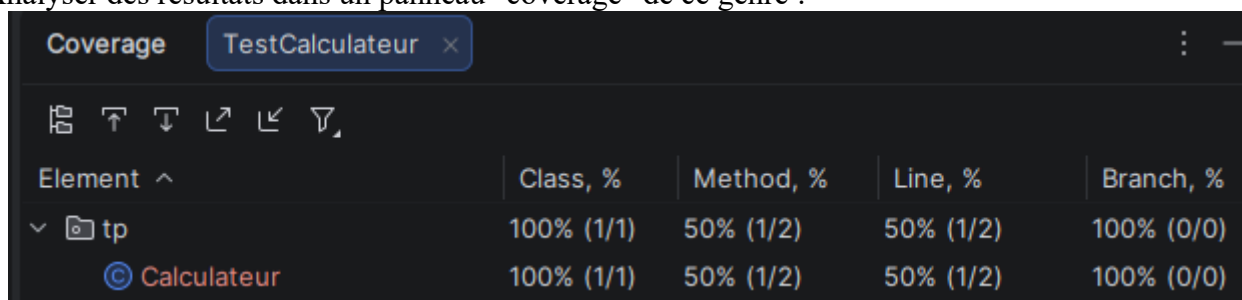
Rien n'a été oublié (packages, classes, méthodes, branches de git, ...)?

1.1. Couverture des tests dans IntelliJ

Installer si nécessaire (depuis Settings/plugins) le plugin "Code coverage for java".

Lancer un ou plusieurs tests via "Run With Coverage".

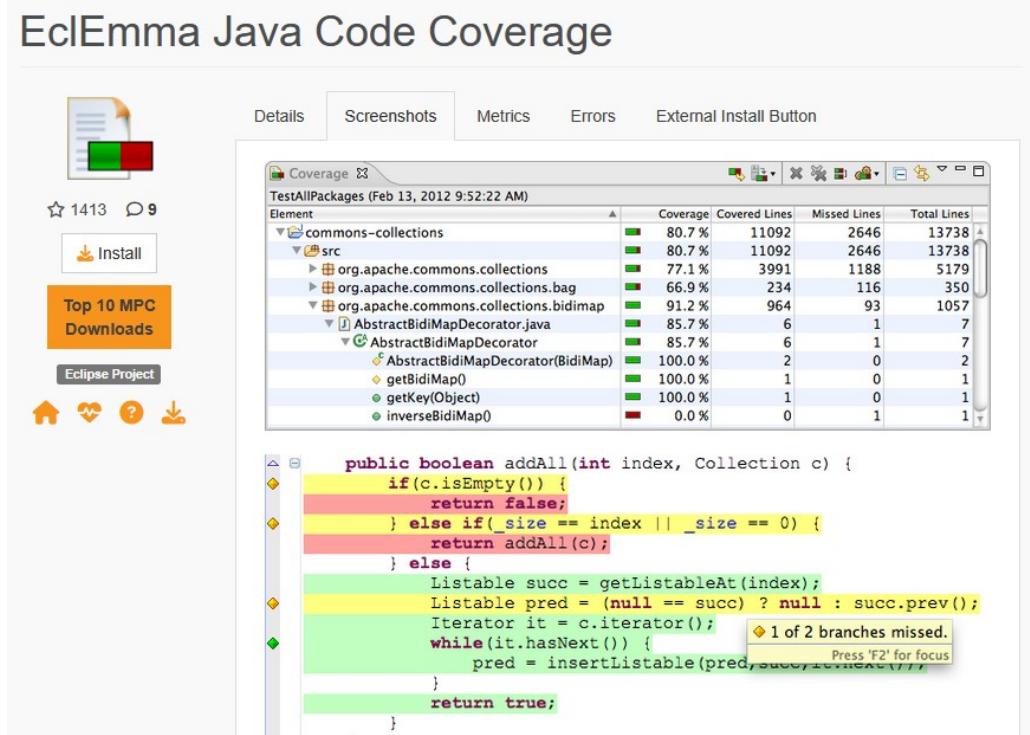
Analyser des résultats dans un panneau "coverage" de ce genre :



Element ^	Class, %	Method, %	Line, %	Branch, %
tp	100% (1/1)	50% (1/2)	50% (1/2)	100% (0/0)
Calculateur	100% (1/1)	50% (1/2)	50% (1/2)	100% (0/0)

1.2. Couverture des tests dans eclipse

Plugin de référence pour eclipse : *EclEmma Java Code Coverage*



EclEmma Java Code Coverage

Details | Screenshots | Metrics | Errors | External Install Button

Element	Coverage	Covered Lines	Missed Lines	Total Lines
commons-collections	80.7 %	11092	2646	13738
src	80.7 %	11092	2646	13738
org.apache.commons.collections	77.1 %	3991	1188	5179
org.apache.commons.collections.bag	66.9 %	234	116	350
org.apache.commons.collections.bidimap	91.2 %	964	93	1057
AbstractBidiMapDecorator.java	85.7 %	6	1	7
AbstractBidiMapDecorator	85.7 %	6	1	7
AbstractBidiMapDecorator(BidiMap)	100.0 %	2	0	2
getBidiMap()	100.0 %	1	0	1
getKey(Object)	100.0 %	1	0	1
inverseBidiMap()	0.0 %	0	1	1

```

public boolean addAll(int index, Collection c) {
    if(c.isEmpty()) {
        return false;
    } else if(_size == index || _size == 0) {
        return addAll(c);
    } else {
        Listable succ = getListableAt(index);
        Listable pred = (null == succ) ? null : succ.prev();
        Iterator it = c.iterator();
        while(it.hasNext()) {
            pred = insertListable(pred, succ, it.next());
        }
        return true;
    }
}

```

1 of 2 branches missed. Press 'F2' for focus

1.3. Configuration maven pour le plugin "jaCoCo"

pom.xml

```
...
<build> <plugins>  <plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <version>0.8.14</version>
  <executions>
    <execution>
      <goals> <goal>prepare-agent</goal>  </goals>
    </execution>
    <execution>
      <id>report</id>
      <phase>test</phase>
      <goals>  <goal>report</goal>  </goals>
    </execution>
  </executions>
</plugin></plugins></build></project>
```

1.4. Analyse des rapports de couverture des tests de jaCoco

après avoir lancer *mvn test* ou *mvn package* ou autre ,
 ouvrir **target/site/jacoco/index.html**
 pour visualiser les rapports de couverture des tests

with_jacoco

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
tp		63 %		n/a	1	3	1	3	1	3	0	1
Total	4 of 11	63 %	0 of 0	n/a	1	3	1	3	1	3	0	1

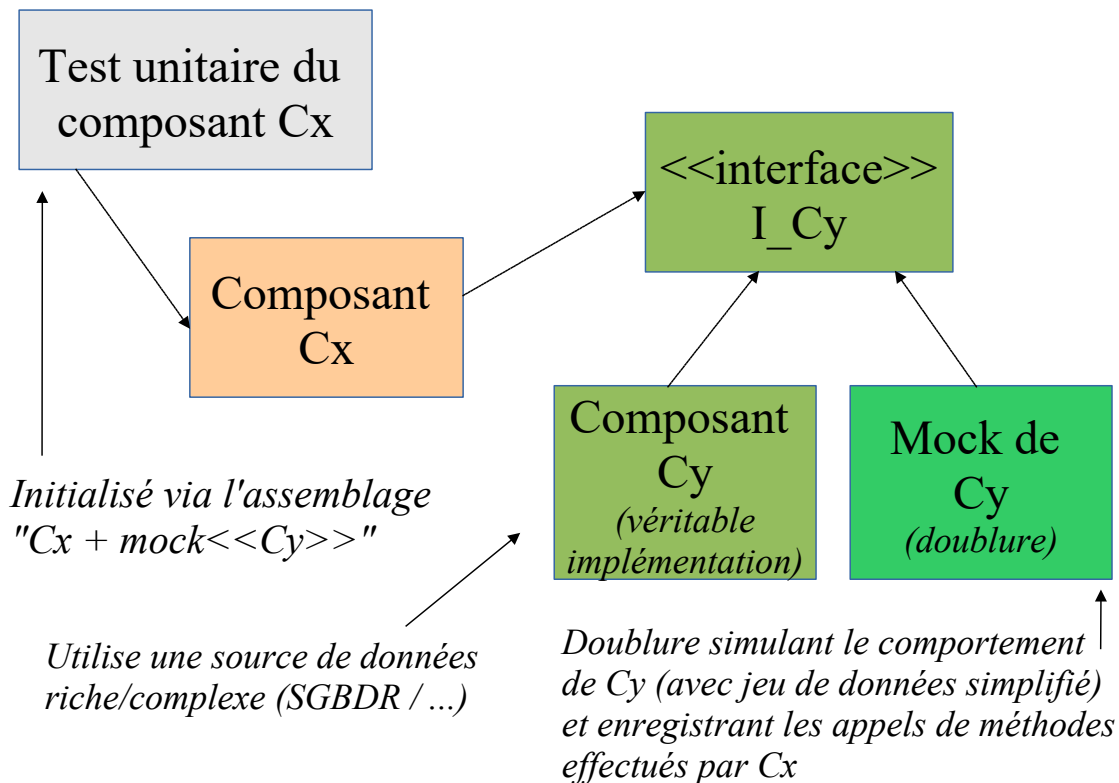
with_jacoco > tp > Calculateur

Calculateur

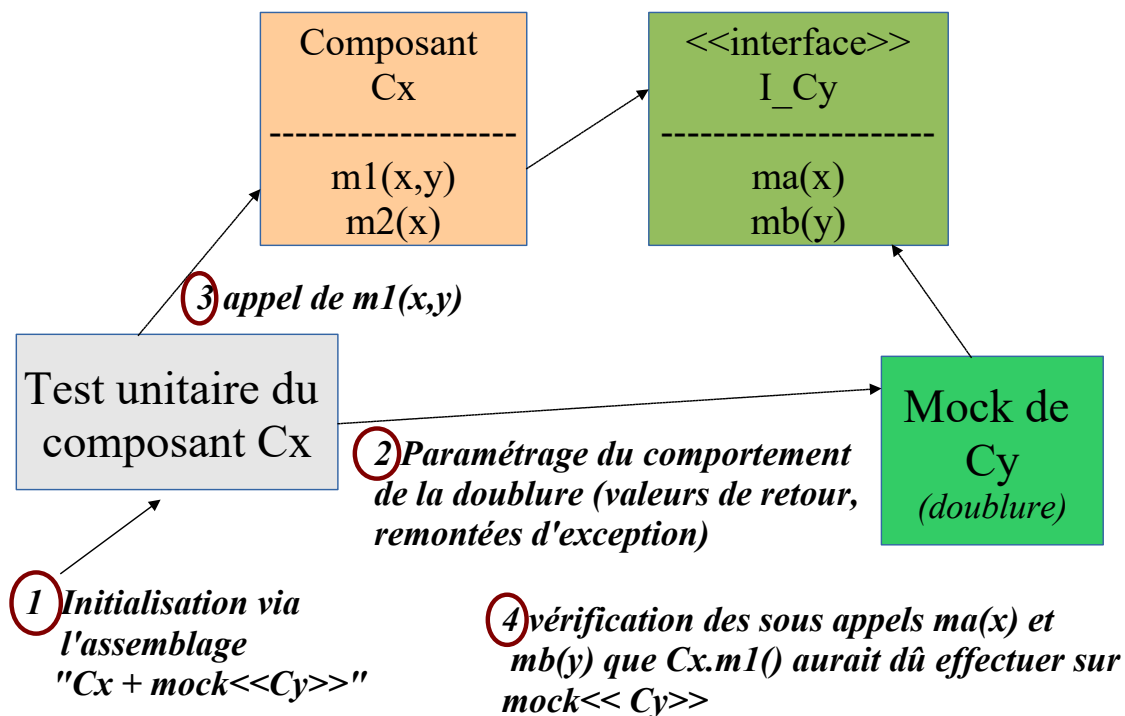
Element	Missed Instructions	Cov.
mult(double, double)		0 %
add(double, double)		100 %
Calculateur()		100 %
Total	4 of 11	63 %

2. Positionnement et intérêts des "mocks"

Positionnement des "mocks"



intérêts des "mocks"



3. Mockito

3.1. Présentation de Mockito

Mockito est une technologie java assez populaire pour mettre en œuvre des "Mocks" (simulacres).

3.2. Dépendance maven nécessaire

```
<dependency>
  <groupId>org.mockito</groupId>
  <artifactId>mockito-core</artifactId>
  <!-- <artifactId>mockito-junit-jupiter</artifactId> -->
  <version>5.10.0</version>
</dependency>
```

3.3. Initialisation du Mock au niveau d'un test Junit:

Solution1 (par annotation "**@Mock**" interprétée du fait de **@RunWith(MockitoJUnitRunner.class)**) :

```
import org.junit.runner.RunWith;
import org.mockito.Mock;
import org.mockito.runners.MockitoJUnitRunner;

@RunWith(MockitoJUnitRunner.class)
//@ExtendWith(MockitoExtension.class) //for JUnit 5
public class UserLoginMockTest {

    @Mock
    private static UserLogin userLogin;
    ....
}
```

Solution2 (par annotation "**@Mock**" interprétée du fait de l'appel à **MockitoAnnotations.initMocks(this)**; lors de l'initialisation) :

```
import org.mockito.Mock;
import org.mockito.MockitoAnnotations;

//rien ou @RunWith(SpringJUnit4ClassRunner.class)
public class UserLoginMockTest {

    @Mock
    private UserLogin userLogin;

    @Before
    public void init() /* or setUp or ... */ {
        MockitoAnnotations.initMocks(this);
    }
    ...
}
```

Solution3 (*sans annotation* Mockito , avec un appel explicite à **Mockito.mock()**) :

```
import org.mockito.Mockito;

public class UserLoginMockTest {
    private static UserLogin userLogin;

    @BeforeClass
    public static void init() {
        userLogin = Mockito.mock(UserLogin.class);
    }
    ....
}
```

Soit l'interface fonctionnelle suivante :

```
public interface UserLogin {

    public boolean verifyLogin(String username,String password);
    public String goodPasswordForUser(String username);
    public void setSize(int size);
    //pour tester valeur de retour par défaut:
    public String getAuteur();
    public int getSize(); //return nbAccount ( >=0)
    public double getDoubleValue(double x); //return x * 2
}
```

et soit une classe d'implémentation basique suivante :

```
public class UserLoginImpl implements UserLogin {
    private int size=10; //par défaut
    public boolean verifyLogin(String username, String password) {
        boolean res=false;
        if(password !=null && password.equals("pwd_"+username))
            return true;
        return res;
    }
    public String goodPasswordForUser(String username) {
        return "pwd_"+username;
    }
    public void setSize(int size) { this.size=size;
    }
}
```

```

    public String getAuteur() { return "didier";
    }
    public int getSize() { return size ;
    }
    public double getDoubleValue(double x) { return 2 *x;
    }
}

```

Le comportement de Mockito est alors le suivant :

3.4. Mockito as stub

Comportement par défaut d'un mock (géré par Mockito)

Avec aussi bien

userLogin = ***Mockito.mock***(***UserLogin.class***); //interface

que

userLogin = ***Mockito.mock***(***UserLoginImpl.class***); //classe d'implémentation

tout appel de méthode sur l'objet "mock" géré par mockito retourne une valeur par défaut de type "null" , false , 0 ou 0.0 selon le type de retour :

```

public void displayReturnValues(){
    boolean pwdOk= userLogin.verifyLogin("toto", "pwd_toto");
    System.out.println("pwdOk="+pwdOk);

    String goodPwd= userLogin.goodPasswordForUser("toto");
    System.out.println("goodPwd="+goodPwd);

    String auteur = userLogin.getAuteur();
    System.out.println("auteur="+auteur);

    int taille = userLogin.getSize();
    System.out.println("taille="+taille);

    double val = userLogin.getDoubleValue(3.2);
    System.out.println("val="+val);
}

```

Comportement vraie classe:

pwdOk=true
goodPwd=pwd_toto
auteur=didier
taille=10
val=6.4

Comportement du mock:

pwdOk=false
goodPwd=null
auteur=null
taille=0
val=0.0

Préciser (forcer) une valeur de retour via Mockito :

```
Mockito.when(userLogin.getSize()).thenReturn(5);
```

```
int taille = userLogin.getSize();
System.out.println("taille="+taille); → affiche toujours taille=5
userLogin.setSize(20);    taille = userLogin.getSize();
System.out.println("taille="+taille); → affiche toujours taille=5 (et pas 20 !)
```

On peut forcer un **retour d'exception** selon par exemple certaines valeurs en entrée :

```
Mockito.when(userLogin.setSize(Mockito.eq(-1)))
    .thenThrow(new IllegalArgumentException("message xy"));
```

Lorsque l'on "mock" une classe (et pas une interface), on peut explicitement demander à Mockito de rétablir le comportement de la véritable classe d'implémentation sur certaines méthodes :

```
Mockito.when(userLogin.getSize()).thenCallRealMethod();
Mockito.doCallRealMethod().when(userLogin).setSize(Mockito.anyInt());
// il existe aussi Mockito.anyString() , ...
```

```
userLogin.setSize(20) ;    taille = userLogin.getSize();
System.out.println("taille="+taille); → affiche taille=20
```

3.5. Mockito as "spy" et "mock = stub + spy"

Comportement par défaut d'un mock initialisé via Mockito.spy()

Fonctionnellement : stub = bouchon (comportement simulé) avec 0,null,0.0 par défaut
 .spy() = espionnage (enregistrement des appels pour vérifications ultérieures).
 .mock() = comportement "stub" + fonctionnalité de .spy()

userLogin = Mockito.spy(UserLogin.class); //interface
 ==> même comportement que via Mockito.mock() car pas de code par défaut

userLogin = Mockito.spy(new UserLoginImpl()); //classe d'implémentation

On obtient alors un **comportement normal** (identique à la classe d'origine) **sur toutes les méthodes sauf sur celles où l'on demande explicitement à redéfinir le comportement** :

Mockito.when(userLogin.getSize()).thenReturn(5);

Résultats (par défaut)
depuis code précédent :
 pwdOk=true
 goodPwd=pwd_toto
 auteur=didier
 taille=5 (à la place de taille = 10)
 val=6.4

On peut désactiver le comportement d'un setter (ou d'une méthode en void) :

Mockito.doNothing().when(userLogin).setSize(Mockito.anyInt());

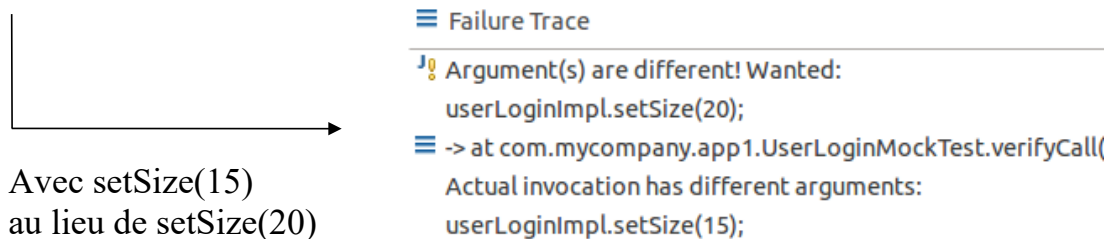
Autrement dit :

- sémantiquement **mock = "stub + spy"**
- pragmatiquement pas de différence notable entre Mockito.mock() et Mockito.spy() en partant d'une interface car si pas de code d'implémentation ---> toujours comportement "mock=stub+spy".
 Si par contre Mockito.spy() est appelé en partant d'une réelle instance de composant alors :
 - spy --> espionnage seulement pour ultérieur "verify"
 sans changer comportement de l'application
 - et mock (= stub + spy)--> verify possible ET implémentation de départ
 écrasée par Mockito.when().then...

Vérification des appels effectués sur un mock (spy) :

Mockito.spy(...) porte bien son nom lorsque l'on sait que l'on peut demander à Mockito d'espionner les appels effectués sur un "mock" et vérifier si certaines méthodes ont bien été appelées (avec certaines valeurs attendues de paramètres en entrée) :

```
@Test
public void verifyCall() {
    userLogin.setSize(15); //userLogin.setSize(20);
    //appel habituellement indirect effectué depuis
    // le code caché d'un composant à tester
    // vers le "spy" ou "stub+spy=mock" d'un sous composant
    Mockito.verify(userLogin).setSize(Mockito.eq(20));
}
```



Quelques exemples de vérifications via Mockito

```
// vérifie que la méthode m1 a été appelée sur obj,  
// avec une String strictement égale à "s1" :  
Mockito.verify(obj).m1(Mockito.eq("s1"));  
// note : ici, le matcher n'est pas indispensable, la ligne suivante est équivalente :  
Mockito.verify(obj).m1("s1");  
  
// vérifie que la méthode m2 n'a jamais été appelée sur l'objet obj :  
Mockito.verify(obj, Mockito.never()).m2();  
  
// vérifie que la méthode m3 a été appelée exactement 2 fois sur l'objet obj :  
Mockito.verify(obj, Mockito.times(2)).m3();  
  
// idem avec un nombre minimum et maximum d'appels :  
Mockito.verify(obj, Mockito.atLeast(3)).m3();  
Mockito.verify(obj, Mockito.atMost(10)).m3();  
  
// vérifie que la méthode m4 a été appelée sur obj,  
// avec un objet similaire à celui passé en argument :  
Mockito.verify(obj).m4(Mockito.refEq(obj2));
```

Quelques "**matchers**" pour vérifier ou paramétrer les valeurs des paramètres :

Mockito. eq (...)	Égal à ...
Mockito. refEq (obj2)	Égal à cet objet
Mockito. anyString() , anyInt() , anyFloat() ,	Chaîne quelconque , entier quelconque, ..
Mockito. anyObject()	Objet quelconque
Mockito. any (Class<T> c)	Objet d'un certain type
Mockito. anyList()	Toute implémentation de List
Mockito. argThat (new MyMatcher())	Vérifiant matcher spécifique
...	

On peut définir de nouveaux "matcher" via des classes qui héritent de **ArgumentMatcher<T>**

Attention :

L'ancienne méthode Mockito.**anyObject()** n'existe plus .

L'ancien ~~org.mockito.Matchers~~ est maintenant obsolète ou inexistant (selon les versions).

Il faut absolument utiliser **org.mockito.ArgumentMatcher** à la place .

NB : les matchers de Mockito sont différents de ceux de hamcrest !!!!

Exemple de matcher spécifique :

```
...
import org.mockito.ArgumentMatcher;

public class MyIntegerBetween implements ArgumentMatcher<Integer>{
    private int inclusiveMini;
    private int inclusiveMaxi;

    public MyIntegerBetween(int inclusiveMini, int inclusiveMaxi) {
        this.inclusiveMini = inclusiveMini;
        this.inclusiveMaxi = inclusiveMaxi;
    }
}
```

```

public MyIntegerBetween() {
    this(0,100); //default min max values
}

@Override
public boolean matches(Integer v) {
    return (v>= inclusiveMini && v <= inclusiveMaxi) ;
}
}

```

Utilisation :

```
//Mockito.when(serviceTauxInteretMock.tauxCourantAnnuelBce(120)).thenReturn(3.5);
```

```

Mockito.when(serviceTauxInteretMock.tauxCourantAnnuelBce(
    Mockito.intThat( new MyIntegerBetween(61,1000))
)).thenReturn(3.5);

```

NB : **Mockito.intThat()** , **Mockito.doubleThat()** , ... pour les arguments de types primitifs et **Mockito.argThat()** pour les arguments de type "Object" .

"Strict" vs "lenient" mode :

Depuis la version 2.0 de Mockito (qui fonctionne par défaut en mode "strict") , les "stubs" inutiles sont refusés de manière à éviter une écriture inutilement compliquée des tests :

org.mockito.exceptions.misusing.UnnecessaryStubbingException:

Unnecessary stubbings detected.

Clean & maintainable test code requires zero unnecessary code.

Please remove unnecessary stubbings or use 'lenient' strictness.

De manière à tout de même laisser du code un peu inutile (mais quelquefois un peu "explicatif") , on peut éventuellement utiliser **Mockito.lenient().when(...)**... plutôt que **Mockito.when(...)**...

Exemple :

```
Mockito.lenient().when(serviceTauxInteretMock.tauxCourantAnnuelBce(Mockito.intThat( new
MyIntegerBetween(25,60)))).thenReturn(3.0);
```

```
Mockito.when(serviceTauxInteretMock.tauxCourantAnnuelBce(Mockito.intThat( new
MyIntegerBetween(61,1000)))).thenReturn(3.5);
```

pour préparer une futur appel indirect à **serviceTauxInteretMock.tauxCourantAnnuelBce()** avec **taux=120** (pas compris entre 25 et 60) .

Eventuelle utilisation de hamcrest avec Mockito :

```

<dependency>
  <groupId>org.hamcrest</groupId>
  <artifactId>hamcrest</artifactId>
  <version>2.2</version>
  <scope>test</scope>
</dependency>

```

```

import org.mockito.hamcrest.MockitoHamcrest;
import org.hamcrest.Matchers;

...
Mockito.when(serviceTauxInteretMock.tauxCourantAnnuelBce(
    MockitoHamcrest.intThat( Matchers.greaterThanOrEqualTo(61))
)).thenReturn(3.5);
/*
Mockito.when(serviceTauxInteretMock.tauxCourantAnnuelBce(
    MockitoHamcrest.intThat(
        Matchers.allOf(Matchers.greaterThanOrEqualTo(61),Matchers.lessThanOrEqualTo(1000))
    )
)).thenReturn(3.5);
*/

```

4. Présentation de JMeter

JMeter est un produit **open source** de la communauté **Apache**.

JMeter est capable de **simuler une certaine charge** tout en **récoltant des mesures essentielles** (temps de réponses , débits, ...).

JMeter est développé en **java** et est donc:

- multiplateforme (windows , linux, ...)
- soumis à certains cycles de "GC/ramasse-miettes" qui ont tendance à introduire un léger niveau d'approximation dans les mesures recueillies (==> ordre de grandeur correct mais précision limitée) .
- Simple à utiliser (interface graphique perfectionnée).
-

Principales fonctionnalités du produit "JMeter":

- requêtes HTTP , SOAP , ...
- requêtes SQL (JDBC), ...
- requêtes LDAP , FTP , TCP, ...
- requêtes JMS,

- résultats en graphes , tableaux, rapports,
- vérifications via assertions, comparaisons, sauvegardes fichiers,

- Timer, threads(clients simultanés simulés)
- Pré et post-processeurs (extraction de valeurs via reg-expr, ...)
- Variables, boucles , ...

- Gestion automatique des cookies HTTP (utile entre autres pour identifier les sessions des clients simultanés simulés)
- ...

Les versions récentes de JMeter comportent maintenant un **mode "CLI"** (lignes de commandes en mode texte) permettant de déclencher et effectuer des mesures sans interface graphique. Ça améliore les performances et ça offre plus de souplesse.

Restriction importante :

JMeter n'est pas un navigateur internet et fonctionne d'un point de vue protocole réseau (ex : TCP , HTTP, ...) .

JMeter peut être très utile pour tester une montée en charge vers une api REST ou tout autre élément du coté backend .

JMeter ne peut pas être utiliser pour tester les performances d'un frondEnd javascript (ex : angular, react, vue, ...) s'exécutant au sein d'un navigateur .

5. Configuration et démarrage du produit JMeter

NB : au sein des versions récentes de JMeter on aura souvent intérêt à fixer la propriété `resultcollector.action_if_file_exists` à la valeur `APPEND` au sein de `apache-jmeter-5.6.3/bin/jmeter.properties` (pas très loin de la ligne 1050 ou autre):

```
resultcollector.action_if_file_exists=APPEND
```

car la valeur `DELETE` provoque souvent des erreurs/blocages et la valeur par défaut `ASK` déclenche trop souvent une popup assez pénible .

D'autre part , un lancement de jMeter depuis un autre répertoire que `apache-jmeter-5.6.3/bin` est conseillé pour éviter de mélanger les fichiers de jmeter à ceux de notre plan de test .

launch_jmeter.bat pas dans ~~*apache-jmeter-5.6.3/bin*~~ mais ***ailleurs*** !!!!

```
set JAVA_HOME=C:\Program Files\Java\jdk-21
set PATH=%JAVA_HOME%\bin;%PATH%
set JMeter_HOME=C:\Prog\apache-jmeter-5.6.3
%JMeter_HOME%\bin\jmeter
```

Une fois qu'un plan de test aura été mis au point (fichier `.jmx`) , on pourra lancer l'exécution de ce plan de test en ligne de commande (sans interface graphique) via :

launch_jmeter_cli.bat

```
set JAVA_HOME=C:\Program Files\Java\jdk-21
set PATH=%JAVA_HOME%\bin;%PATH%
set JMeter_HOME=C:\Prog\apache-jmeter-5.6.3
%JMeter_HOME%\bin\jmeter -n -t test_plan_xyz.jmx -l summary-report.jtl
REM -n for no_gui (cli mode)
REM -t test_plan.jmx
REM -l log_file.jtl
```

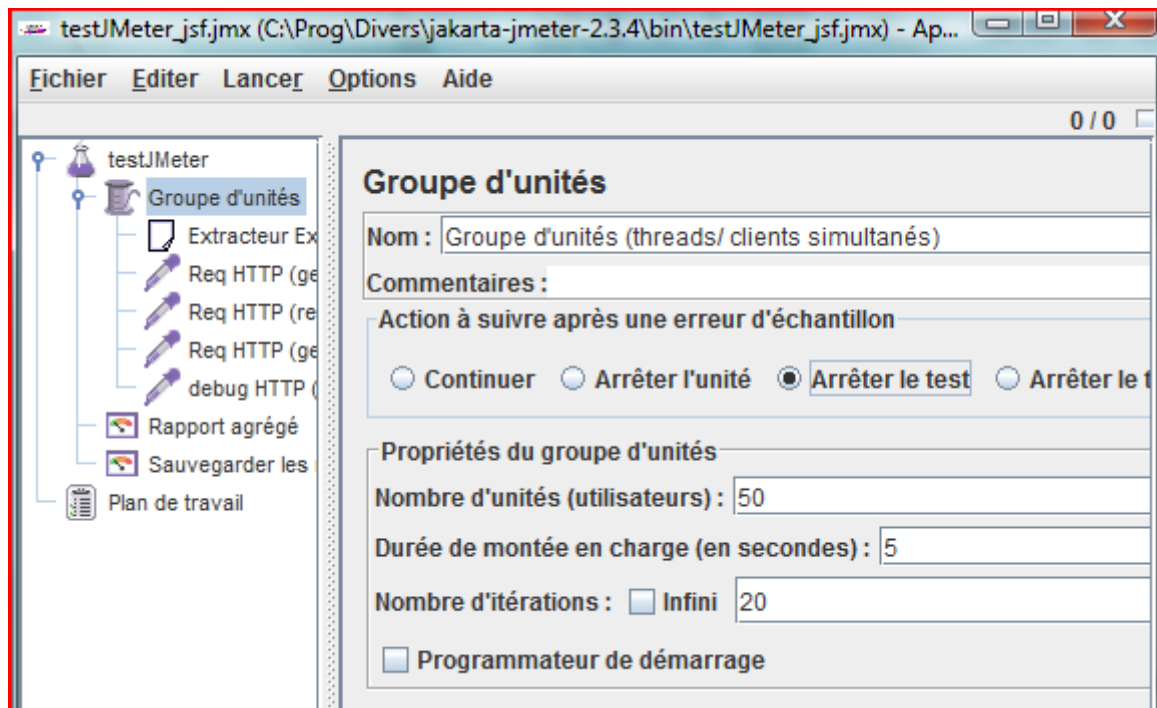
6. Configuration élémentaire d'un plan de test JMeter

Commencer par renommer le plan de test (ex: MyApp ou TestXY)

NB : les plans (configurations) de tests de jmeter sont stockés dans des fichiers XML avec l'extension **.jmx**

6.1. Configuration d'un groupe d'unités (clients simultanés)

Ajouter / Groupe d'unités [*Add / Thread group*]



10 Threads = 10 clients simultanés

Période de chauffe [*Ramp-up period*] = 5

==> les threads démarrent un par un toutes les $5 / 10 = 0,5$ s.

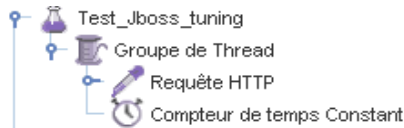
Une fois démarré, chaque thread enverra 20 requêtes (*Loop Count*) . Entre chaque requête , chacun des threads attendra par défaut **0** ms.

Pour paramétrer (si besoin) une pause éventuelle (ex: **1000 ms**) entre les requêtes consécutives envoyées par un même thread il faut alors incorporer (sous le groupe de Thread) un **Compteur de temps fixe**[*Timer/ConstantTimer*].

Compteur de temps Constant
 Nom:
 Temporisation Thread (en millisecondes):

6.2. Configuration d'une requête à lancer (plusieurs fois)

Sous ce "Groupe d'unités" ==> **Ajouter / Echantillon / Requête HTTP**
 "Thread Group" Add / Sampler / HTTP Request



Requête HTTP

Nom: Requête HTTP / Conversion

Serveur Web

Nom ou IP du Serveur: localhost

Numéro de Port: 8080

Requête HTTP

Protocole: Méthode: ☒ GET ☐ POST

Chemin: deviseWeb/convlt.jsp ☐ Rediriger Automatiquement ☒ Suivre

Envoyer les Paramètres Avec la Requête:

Nom:	Valeur
somme1	100
monnaie1	Euro
monnaie2	Yen

6.3. configuration de la récupération des résultats (synthèse, courbe, ...)

==> Sous la "Requête HTTP" ou bien sous "le groupe d'unités"

==> *Ajouter / Récepteur / Résultats Graphiques mélangés (pas très clair)*
Add / Listener / Graph Result

====>

*Add / Listener / **Response Time Graph** (bien en réglant interval = 100ms ou autre)*

====> **Ajouter / Récepteur / Rapport Agrégé (tableau récapitulatif)**
*Add / Listener / **Aggregate Report***

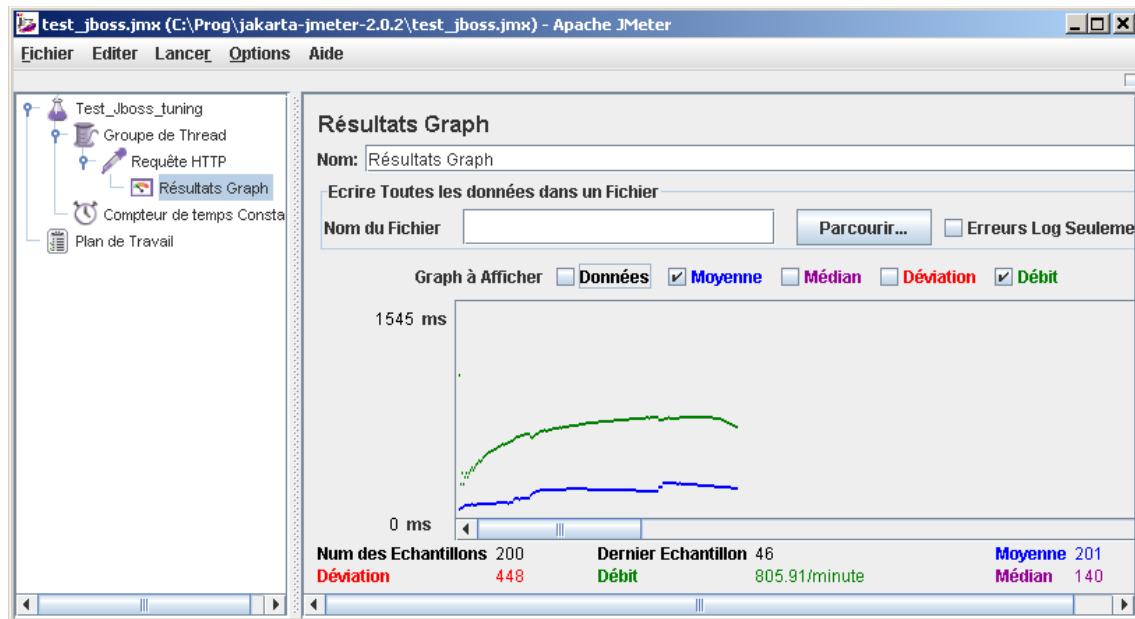
6.4. (re-)lancement du test et observation des résultats

Sauvegarder le "Test Plan" .

Ré-initialiser à zéro des courbes et résultats via "**Lancer / Nettoyer tout** [Run / Clear All]"

Activer la simulation de montée en charge via "**Lancer / Démarrer** [Run / Start]"

==>



Rapport Agrégé						
Nom: Rapport Agrégé						
Ecrire Toutes les données dans un Fichier						
Nom du Fichier		Parcourir...		☐	Erreurs Log Seulement	
URL	Count	Average	Min	Max	Error%	Rate
Requête HTTP	40	41	0	140	0,00%	4,6/sec
TOTAL	40	41	0	140	0,00%	4,6/sec

Quelques principes importants à respecter :

Si l'on souhaite obtenir des mesures/résultats corrects il est absolument indispensable de respecter cette règle fondamentale :

Faire fonctionner JMeter sur un autre ordinateur que celui où s'exécute le logiciel à tester

sauf peut être durant les toutes premières phases de mise au point .

Donc idéalement pas de **localhost** au sein des URLs à tester .

Si l'on ne respecte pas cette règle , JMeter se stresse lui même et les mesures ne seront pas fiables.

D'autre part , si en phase de "développement" ou "pré-prod" , certains logs du logiciel à tester sont mal réglés (ex : affichage de debug pour chaque requête sur la console texte) , les temps de réponse seront altérés et pas significatifs .

6.5. Response Time Graph

Add / Listener / Response time Graph

Réglage important :

interval (ms) entre 2 points du graphique

Response Time Graph

Name:

Comments:

Write results to file / Read from file

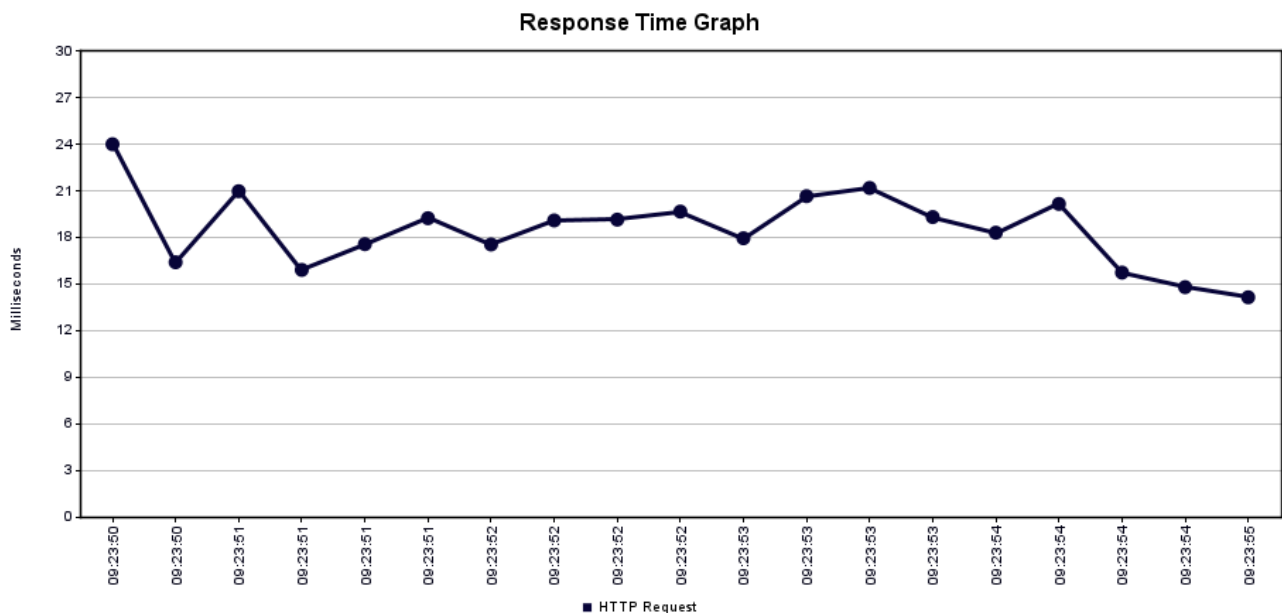
Filename

Settings **Graph**

Graph settings

Interval (ms):

Exemple de résultat :



7. Très important: tester tout d'abord le test !!!

Si l'on ne vérifie pas l'exactitude des paramétrages au niveau des tests, on peut alors confondre une réponse normale avec une réponse de type "message d'erreur" retournée par le serveur.

Des temps de réponses liés à des réponses négatives (messages d'erreurs) ne sont évidemment pas significatifs.

Pour être certain que la réponse retournée par le serveur est correcte/normale, il suffit de procéder de la manière suivante (durant la phase de mise au point):

- 1) diminuer (temporairement) au maximum le nombre de requêtes qui seront envoyées:

Groupe d'unités

Nom : Groupe d'unités (threads/ clients simultanés)

Commentaires :

Action à suivre après une erreur d'échantillon

☒ Continuer ☐ Arrêter l'unité

Propriétés du groupe d'unités

Nombre d'unités (utilisateurs) : 1

Durée de montée en charge (en secondes) : 1

Nombre d'itérations : ☐ Infini 1

- 2) ajouter un récepteur de type "*Sauvegarder les réponses vers un fichier*"

Sauvegarder les réponses vers un fichier

Nom : Sauvegarder les réponses vers un fichier

Commentaires :

Préfixe du nom de fichier : res_jsf

Nom de variable :

- 3) **lancer le test et vérifier le contenu du ou des fichier(s) généré(s)**
- à refaire à chaque nouveau re-paramétrage des tests (pour vérifier une non-régression)
- 4) une fois la phase de mise au point terminée, il suffit de **rendre inactif** le récepteur de type "*Sauvegarder les réponses vers un fichier*" pour ne pas ralentir inutilement les tests. En jouant ainsi sur le menu contextuel "activer/désactiver" ou "enable/disable", on peut basculer rapidement d'un mode "debug" vers un mode "test réaliste" et vice-versa. Il faudra également penser à ré-augmenter les nombres de boucles et d'utilisateurs simultanés au sein du groupe d'unité après avoir achever la phase de "debug".

8. Gestion correcte des sessions HTTP (cookies)

Si l'on souhaite tester une génération dynamique de pages web du côté serveur (ex : pages PHP ou JSP ou ASP ou autres), de façon à ce que JMeter se comporte exactement comme un navigateur internet au niveau de la gestion des sessions utilisateurs, il faut **absolument ajouter** la configuration "**Gestionnaire de cookies HTTP**" en dessous du niveau "groupe d'unités".

Rien d'important n'est à paramétrer à ce niveau, mais le "**Gestionnaire de cookies HTTP**" doit simplement être présent.



9. Successions de requêtes http liées entre elles

Un utilisateur utilise généralement une application web en passant par une suite logique de pages (accueil, menu1, menu2, sélection/recherche, résultats, action, statut, ...).

Ces différentes pages activées sont souvent liées entre elles par un ou plusieurs champs cachés (véhiculant un id "utilisateur" ou bien "un état de session" ou "...").

Ces "id" sont générés dynamiquement et ne sont donc pas constants (différents pour chaque utilisateur et pour chaque session).

JMeter offre heureusement un moyen pour "récupérer et ré-injecter" ces "id" véhiculés sous forme de champs cachés **via un processeur d'extraction d'expressions régulières** (ou bien Xpath).

9.1. Principe du "repost-hidden" (sur pages jsp)

repost_hidden.jsp

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
<title>repost_hidden</title>
</head>
<%
String chX=request.getParameter("x");
String chLast=request.getParameter("last");
int x=1;
int last=1;
if(chX!=null) x=Integer.parseInt(chX);
if(chLast!=null)
```

```

    try{ last=Integer.parseInt(chLast);}
    catch (NumberFormatException ex){ last=0; }
int y=last*x;
%>
<body>
    <form>
        x=<input type="text" name="x" /> <br/>
        <input type="hidden" name="last" value="<%=y%>" />
        <input type="submit" value="mult by last value"/>
    </form>
</body>
</html>
x=<%=chX%><br/>
last=<%=chLast%><br/>
y=last*x=<%=y%> (new last)
</body>
</html>

```

Cet exemple de page jsp montre l'utilisation d'un champ caché permettant de mémoriser une certaine valeur entre plusieurs appels consécutifs émis depuis le même utilisateur.

Résultats des ré-appels successifs:

x=3 mult by last value	x=2 mult by last value	x= mult by last value
x=null last=null y=last*x=1 (new last)	x=3 last=1 y=last*x=3 (new last)	x=2 last=3 y=last*x=6 (new last)

Nouvelle valeur de y = ancienne valeur multipliée par le x saisi .
L'ancienne valeur (last) est véhiculée via un champ caché (aller/retour "serveur-client-serveur").

```
<input type="hidden" name="last" value="1 , 3 puis 6"/>
```

Test JMeter associé:



Paramétrage de la première requête (en mode GET)

Requête HTTP

Nom : Req HTTP (get)

Commentaires :

Serveur web

Nom ou adresse IP : localhost Port : 8080

Requête HTTP

Protocole (défaut http) : Méthode : GET Encodage du c

Chemin : testJmeter/repost_hidden.jsp

☒ Rediriger automatiquement ☐ Suivre les redirections ☒ Connexions persistantes

Envoyer les paramètres avec la requête :

Nom :	Valeur :

Paramétrage de l'extracteur d'expression régulière (post response processor) :

Extracteur Expression régulière

Nom : Extracteur Expression régulière

Commentaires :

Champs réponse à cocher:

☒ Corps ☐ Corps (non échappé) ☐ Entêtes ☐ URL ☐ Code de

Nom de référence : last

Expression régulière : <input type="hidden" name="last" value="(.*")/>

Canevas : \$1\$

Correspond au num. (0 pour Aléatoire) : 1

Valeur par défaut : 0

==> la valeur du champ caché est ainsi récupérée dans la variable "last" ==> \${last}

Cette valeur est ensuite ré-injectée dans les requêtes ultérieures:

Requête HTTP

Nom : Req HTTP (repost)

Commentaires :

Serveur web

Nom ou adresse IP : localhost Port : 8080

Requête HTTP

Protocole (défaut http) : Méthode : POST Encodage :

Chemin : testJmeter/repost_hidden.jsp

☒ Rediriger automatiquement ☐ Suivre les redirections ☒ Connexions persistantes

Envoyer les paramètres avec la requête :

Nom :	Valeur :
x	2
last	\${last}

10. Configurations diverses

10.1. Test de charge (SQL) vers un serveur de base de données

NB: **JMeter** peut également effectuer des tests de montée en charge sur un SGBDR (MySQL , Oracle , ...) . Il faut penser à ajouter le driver JDBC adéquat au CLASSPATH de JMeter [*Archives ".jar" à déposer dans jakarta-jmeter-2.x.y\lib*]. Ce type de tests (**JDBC Request**) est très pratique pour effectuer des ajustements au niveau des pools de connexions JDBC .

10.2. Variété au niveau des requêtes pour plus de réalisme

Si l'on déclenche en boucle une requête avec toujours les mêmes valeurs de paramètres en entrée, le serveur peut éventuellement optimiser ses traitements internes (via des caches) et retourner extrêmement rapidement la même réponse. Pour mieux simuler des clients simultanés envoyant des requêtes un peu différentes on a alors besoin d'injecter des valeurs variables dans les requêtes.

L'introduction d'un **contrôleur logique de type forEach** (en tant que *parent d'une requête*) permet de répéter celle-ci avec une valeur de paramètre variable (exemple: `${forEachOutputVar}`).

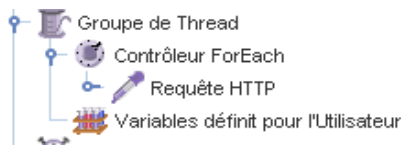
Requête HTTP

Protocole: Méthode: ☒ GET ☐ POST

Chemin : testTuningWeb/get_log_list ☐ Rediriger Automatiquement ☒ Suivre

Envoyer les Paramètres Avec la Requête:

Nom:	Valeur
log_level	\${forEachOutputVar}



Contrôleur ForEach

Nom:

Préfix du paramètre en entrée

Nom du paramètre en sortie

La boucle ForEach sera effectuée sur les variables suivantes (*_1, _2, _3, ..., _n*) :

Variables définit pour l'Utilisateur

Nom:

Variables définit pour l'Utilisateur	
Nom:	Valeur
forEachInputVar_1	TRACE
forEachInputVar_2	WARN

Le Listener "**voir l'arbre des résultats**" est dans ce cas très pratique pour contrôler les requêtes et les réponses .

10.3. Test JDBC

Copier le(s) driver(s) JDBC nécessaires (ex : mysql-connector-j....jar , postgresql-....jar) dans le sous répertoire **lib** de JMeter .

Thread group , JdbcConfiguration avec url jdbc et name of datasource/connectionPool
JdbcRequest avec same name of datasource/connectionPool ...

Ordre de grandeur pour mysql : min=5ms , average: 15ms

Ordre de grandeur pour postgresql :

10.4. Plugins utiles

JMeter peut être enrichi par certains plugins.

Le site officiel de JMeter comporte quelques paquets de plugins.

Le zip "**JmeterPlugins-Extras-1.2.0.zip**" (qu'il suffit d'extraire (en ajout) dans le répertoire de JMeter) comporte quelques plugins intéressants (dont "**TPS vs Threads**" et "**ResponseTime vs Threads**" .

NB : bien penser à cliquer sur "*configurer*" et cocher "*nombre d'unités actives*"

et dimensionner au mieux la période de chauffe (ramp up period) du groupe d'unités pour que les threads puissent démarrer progressivement (via "plugin "*Stepping Thread Group*" ou ...).

10.5. JMeter à la source:

Pour télécharger **JMeter** ou approfondir certains de ses aspects , l'url de départ est :

<http://jmeter.apache.org/>

XXI - Annexe – Monitoring JVM (strict essentiel)

1. JConsole

Le **jdk** est livré en standard avec certains utilitaires de surveillance :
Ces programmes (placés dans le sous répertoire **bin**)

jps (Jvm Process Status)	Liste les JVM en cours d'exécution sur la machine locale
jstat (Jvm Stat)	Affiche des statistiques sur une JVM
jstatd (jstat daemon)	Agent (daemon) à lancer en tâche de fond et servant d'intermédiaire pour obtenir les stats à distance via RMI
jconsole	console <i>JMX</i> dédiée à java (management & monitoring)

jps [-v]

⇒ liste les JVM et leurs *<vmid>* (PID)

jstat -options

⇒ liste les options possibles (ex: -gc , -class , ...)

jstat -gc <vmid> 5s 6

⇒ affiche 6 fois (toutes les 5s) des stats sur le GC de la machine virtuelle identifiée par tel ID .

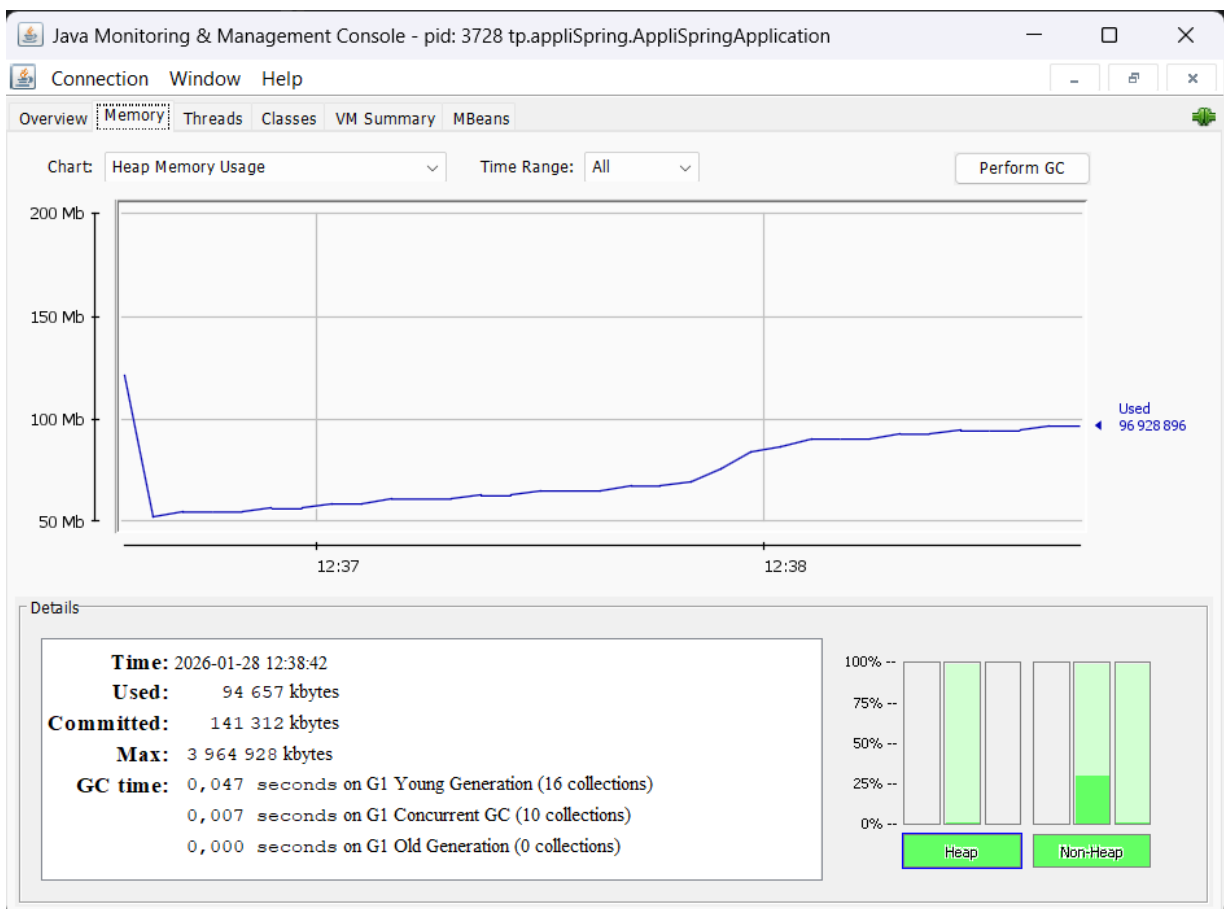
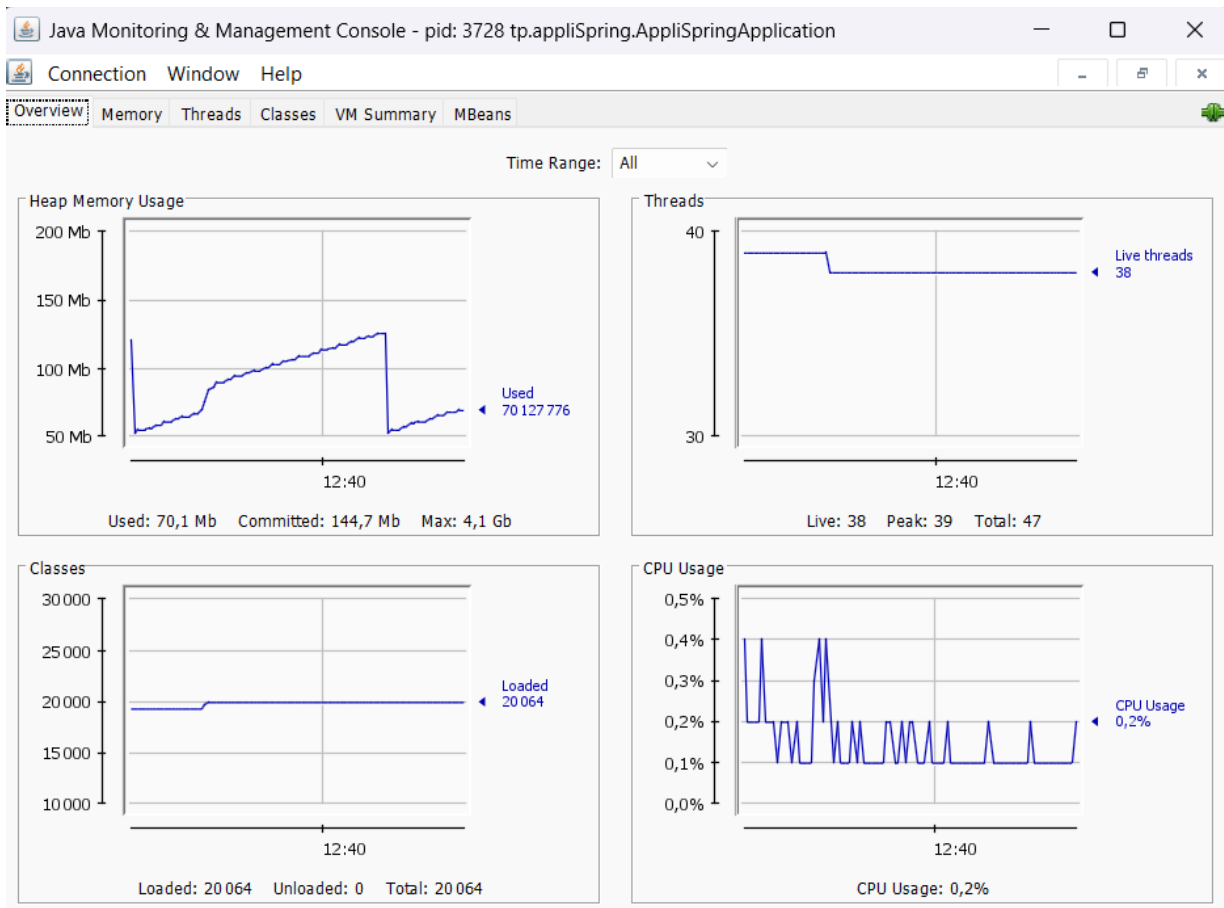
jconsole [*<vmid>*]

⇒ démarre la console graphique jconsole en lui demandant de se connecter à l'application ayant l' ID spécifié .

NB: **jconsole** ne fonctionne qu'avec des programme JAVA managés via JMX (lancés avec l'option **-Dcom.sun.management.jmxremote**)

.../...

NB : si on lance **jconsole** sans préciser de vmid , une boîte de dialogue est alors automatiquement proposée pour pouvoir choisir l'application java à surveiller .



Java Monitoring & Management Console - pid: 3728 tp.appliSpring.AppliSpringApplication

Connection Window Help

Overview Memory Threads Classes VM Summary MBeans

Virtual Machine: Java HotSpot(TM) 64-Bit Server VM version 21.0.9+7-LTS-338
Vendor: Oracle Corporation
Name: 3728@D2F2026

Process CPU time: 20,562 seconds
JIT compiler: HotSpot 64-Bit Tiered Compilers
Total compile time: 1,982 seconds

Live threads: 38
Peak: 39
Daemon threads: 32
Total threads started: 47

Current classes loaded: 20 061
Total classes loaded: 20 061
Total classes unloaded: 0

Current heap size: 115 137 kbytes
Maximum heap size: 3 964 928 kbytes
Committed memory: 141 312 kbytes
Pending finalization: 0 objects

Garbage collector: Name = 'G1 Young Generation', Collections = 16, Total time spent = 0,047 seconds
Garbage collector: Name = 'G1 Concurrent GC', Collections = 10, Total time spent = 0,007 seconds
Garbage collector: Name = 'G1 Old Generation', Collections = 0, Total time spent = 0,000 seconds

Operating System: Windows 11 10.0
Architecture: amd64
Number of processors: 16
Committed virtual memory: 408 108 kbytes

Total physical memory: 15 854 444 kbytes
Free physical memory: 1 821 940 kbytes
Total swap space: 25 781 140 kbytes
Free swap space: 771 832 kbytes

2. Processus d'optimisation (tuning)

Cycle

- Analyser
- Coder/paramétrer
- Mesurer pour vérifier

avec itérations à prévoir pour amélioration progressive.

Principaux objectifs de la phase d'intégration/optimisation :

- amélioration des temps de réponse
- amélioration de la consommation mémoire-mémoire
- amélioration éventuelle du temps de démarrage
- éventuelle détection et correction des fuites de mémoire

XXII - Annexe – Annotations Java

1. Annotations : Présentation et intérêts

Les **annotations** (disponibles depuis java 5) sont des *informations textuelles (ressemblant un peu à des commentaires)* qui sont *insérées au sein du code source (juste au dessus d'une classe, d'une méthode ou d'un attribut)*.

Avant java 5, dans les anciennes versions 1.3, 1.4 on utilisait des commentaires xdoclet.

Ces **annotations** sont quelquefois bien **utiles** car elles **pourront** (selon leurs portées) être **ultérieurement analysées par** :

- un **processeur** spécialisé (à écrire) et qui va **générer dynamiquement de nouvelles classes java durant la phase de compilation**
- l'**api de réflexion/introspection** de façon à *paramétrer certains mécanismes génériques* (persistance?, cryptage?, ...) au runtime.
-

On parle quelquefois en terme de **méta-programmation** lorsque l'on a recours aux annotations dans le cycle de développement.

Syntaxe des annotations:

NB: L'annotation porte toujours sur l'élément qui suit (sur la ligne suivante!).

@MonAnnotationSansParametre
public class Cxx { ...}

@AutreAnnotationSansParametre
public int méthodeX { ...}

@AnnotationAvecUneSeuleValeur("valeur_unique_parametre")
public class Cxx { ...}

@AnnotationAvecParamtres(param1="valeur_param1", param2=VALEUR2_CONTANTE)
public class Cxx { ...}

2. Annotations et méta-annotations prédéfinies

2.1. Annotations standards (interprétées par le compilateur):

<i>Annotations standards</i>	<i>Significations</i>
@Deprecated	Éléments (méthode , attribut, ...) devenu obsolète et qui ne devrait plus être utilisé. <u>NB</u> : l'utilisation conjointe du <i>commentaire javadoc @deprecated</i> permet en plus d'indiquer la raison et une suggestion de remplacement: /** * <i>@deprecated pas bien car ... , utiliser à la place</i> **/ @Deprecated
@Override	Pour que le compilateur vérifie que la méthode qui suit est bien une redéfinition d'une méthode héritée existante (et pas une nouvelle opération) ==> ceci permet de détecter des erreurs sur la signature.
@SuppressWarnings	Pour demander au compilateur d'ignorer certains Warnings sur l'élément (classe,...) qui suit sans pour autant ignorer les les warnings sur le reste de l'application --> ex: @SuppressWarnings({"deprecation","unchecked"})

2.2. Méta-annotations (Annotations sur annotations):

Nécessite =====> `import java.lang.annotation.*;`

exemple:

@Documented

public interface @MyDocumentedAnnotation { ...}

<i>Méta-Annotations</i>	<i>Significations</i>
@Documented	L'annotation doit apparaître dans la documentation générée par javadoc ou ...
@Inherit	L'annotation sera automatiquement héritée par les sous classes (@Inherit ne peut être utilisée que sur annotation de classe – ce n'est pas utilisable sur une annotation d'interface).
@Retention(...)	Pour indiquer la portée (ou durée de vie) de l'annotation. RetentionPolicy.SOURCE (code source uniquement) RetentionPolicy.CLASS (dans .class également mais pas pris en compte par la JVM , c'est la valeur par défaut). RetentionPolicy.RUNTIME (vu par la JVM et l'api "reflection").
@Target(...)	Pour que l'annotation ne soit utilisable que devant une ou plusieurs catégories d'éléments (ex: classe , méthode , ...). Si une annotation n'est pas bien utilisée (erreur de @Target) ==> Erreur du compilateur. Si la méta-annotation @Target n'est pas précisée , l'annotation peut être utilisée sur n'importe quel élément . ElementType.ANNOTATION_TYPE (devant annotation) ElementType.CONSTRUCTOR , ElementType.FIELD ,

Méta-Annotations	Significations
	ElementType.LOCAL_VARIABLE , ElementType.METHOD , ElementType.PACKAGE ElementType.PARAMETER ElementType.TYPE (devant classe , interface ou enum).

3. Création de nouvelles annotations

Une **nouvelle annotation** se code comme une **interface spéciale** ayant un **nom commençant par le caractère @** .

Une interface d'annotation est très souvent précédée par une ou plusieurs méta-annotations.

D'autre part, les éventuelles propriétés (implicitement "public") d'une annotation se codent comme des méthodes (implicitement abstraites) de l'interface d'annotation.

Le nom implicite d'une unique propriété est "**value**=" ==> **value()** ;

Exemples:

```
import java.lang.annotation.Documented;
import java.lang.annotation.Retention;
import static java.lang.annotation.RetentionPolicy.SOURCE;
```

```
@Documented
@Retention(SOURCE)
public interface @Important {
}
```

```
@Documented
@Retention(SOURCE)
public interface @TODO {
    /** Message décrivant la tâche à effectuer. */
    String value();
    /** Niveau de criticité de la tâche (défaut : NORMAL). */
    Level level() default Level.NORMAL;
    /** Enumération des différents niveaux de criticités. */
    public static enum Level { MINEUR, NORMAL, IMPORTANT };
}
```

Attention ! Les attributs d'une annotation n'acceptent que les éléments suivants :

- Un type primitif (**boolean**, **int**, **float**, etc.).
- Une chaîne de caractères (**java.lang.String**).
- Une référence de classe (**java.lang.Class**).
- Une Annotation (**java.lang.annotation.Annotation**).
- Un type énuméré (**enum**).
- Un tableau à une dimension d'un des types ci-dessus.

Toutes les annotations héritent implicitement de l'interface **java.lang.annotation.Annotation**.

4. Insertion d'annotations au sein d'un code source

```
...
@Important
public class Cxx {
    /* ... */
}
```

```
..
@TODO(value="La gestion des exceptions est ...", level=NORMAL)
public void methode1() {
    /* ... */
}

// étant donnée que Level.NORMAL est la valeur par défaut de la propriété level
// on peut également écrire:

@TODO(value="La gestion des exceptions est à améliorer ...")
public void methode1() {
    /* ... */
}

// étant donnée que value est le nom (par défaut) d'une unique propriété, on peut également écrire:

@TODO("La gestion des exceptions est à améliorer ...")
public void methode1() {
    /* ... */
}
```

5. Accès aux annotations (de rétention RUNTIME) via l'introspection de java >=5

L'API de Réflexion (java.lang.reflect) a été étendue en version 1.5 de façon à supporter les annotations. Pour cela, les classes **Package**, **Class**, **Constructor**, **Method** et **Field** possèdent quatre nouvelles méthodes décrites dans l'interface **AnnotatedElement** :

- **getAnnotation(Class<A>)** qui retourne l'annotation dont le type est passé en paramètre (ou *null* si cette annotation n'est pas présente).
- **getAnnotations()** qui retourne un tableau comportant une instance de toutes les annotations de l'élément.
- **getDeclaredAnnotations()** qui retourne un tableau comportant une instance de toutes les annotations directement présentes sur l'élément (c'est à dire sans les annotations héritées de la classe parente). Rappel: Seules les **Class** peuvent hériter d'une annotation.
- **isAnnotationPresent(Class<A>)** qui retourne un booléen indiquant si l'annotation dont le type est passé en paramètre est présente sur l'élément ou non. Cette méthode est surtout utile pour les annotations marqueurs (sans attribut).

Exemple:

```
// Instanciation de l'objet:  
Exemple objet = new Exemple();  
  
// On récupère la classe de l'objet :  
Class<Exemple> classInstance = objet.getClass();  
  
// On regarde si la classe possède une annotation :  
MonAnnotation annotation = classInstance.getAnnotation(MonAnnotation.class);  
  
if (annotation!=null) {  
    System.out.println ("MonAnnotation : " + annotation.value() );  
}
```

6. Analyse et traitement des annotations de rétention

« SOURCE »

NB : l'ancien outil **APT** (**A**nnotation **P**rocessing **T**ool) du **JDK 5.0** a été remplacé dès java 6, java 7 par JSR 269 et APT est maintenant **deprecated**.

Exemple (packagé comme 3 projets distincts / maven multi-modules) :
le code généré sera une classe héritant de **AbstractSimpleSingleton** permettant d'accéder à certains singletons (pour classe annotées via *@BasicSingleton*)

projet my_annot :

```
package tp.annotations;
import java.lang.annotation.ElementType; import java.lang.annotation.Target;
import java.lang.annotation.Retention;      import java.lang.annotation.RetentionPolicy;

@Target(ElementType.TYPE)
@Retention(RetentionPolicy.SOURCE)
public @interface BasicSingleton {
}
```

```
package tp.basic;
/* interface SimpleSingleton au sens SimpleSingleton_Access */
public interface SimpleSingleton {
    <T> T getInstance(Class<T> c) ; //accès à l'unique instance
    <T> int getCount(Class<T> c) ; // nombre d'accès demandés/effectués

    static SimpleSingleton implementation() {
        return AbstractSimpleSingleton.uniqueImplementationInstance();
    }
}
```

```
package tp.basic;
import java.util.HashMap; import java.util.Map;

public abstract class AbstractSimpleSingleton implements SimpleSingleton{

    protected Map<String,Object> mapClassInstance = new HashMap<>();
    protected Map<String,Integer> mapClassCount = new HashMap<>();

    public abstract void initMapClassInstance();
    /* //typical code in subclass:
    {
        //mapClassInstance.put("...Xxx",new Xxx());
        //mapClassInstance.put("...Yyy",new Yyy());
    } */
```

```

AbstractSimpleSingleton(){ this.initMapClassInstance(); }

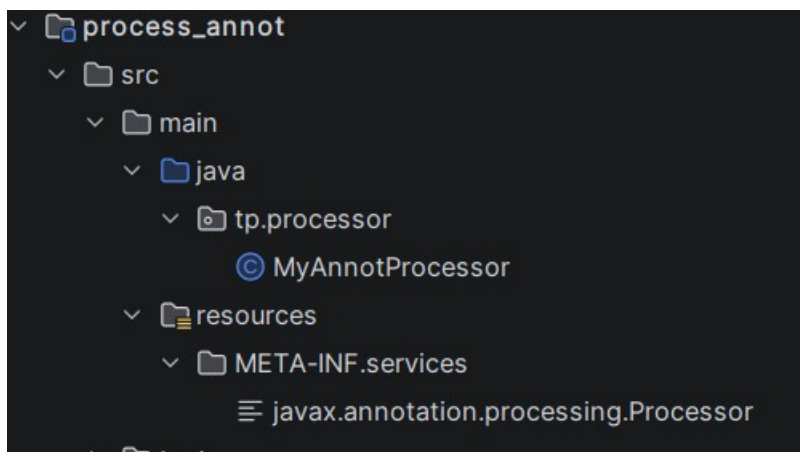
static SimpleSingleton uniqueSimpleSingleton=null;
static SimpleSingleton uniqueImplementationInstance() {
    if(uniqueSimpleSingleton==null) {
        try {
            //NB: tp.basic.MySimpleSingleton will be generated by annotations processor
            // at compile time
            Class<SimpleSingleton> c = (Class<SimpleSingleton>)
                Class.forName("tp.basic.MySimpleSingleton");
            uniqueSimpleSingleton = c.getDeclaredConstructor().newInstance();
        } catch (Exception e) {
            throw new RuntimeException(e);
        }
    }
    return uniqueSimpleSingleton;
}

@Override
public <T> T getInstance(Class<T> c) {
    String className = c.getName();
    Integer count = mapClassCount.get(className);
    if(count==null) count=new Integer(1);
    else count=new Integer(count+1);
    mapClassCount.put(className,count);

    return (T) mapClassInstance.get(className);
}

@Override
public <T> int getCount(Class<T> c) {
    Integer count = mapClassCount.get(c.getName());
    return (count!=null)?count.intValue():0;
}
}

```

projet process_annot :

Fichier indiquant la classe du projet (futur .jar) qui implémente un processeur d'annotations :

META-INF/services/javax.annotation.processing.Processor

tp.processor.[MyAnnotProcessor](#)

```
package tp.processor;

import javax.annotation.processing.*;
import javax.lang.model.SourceVersion;
import javax.lang.model.element.Element;
import javax.lang.model.element.TypeElement;
import javax.tools.JavaFileObject;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.ArrayList;
import java.util.List;
import java.util.Set;

/*
NB: this processor fully qualified class name must/may be register in
META-INF/services/javax.annotation.processing.Processor text file
(in src/main/resources)

NB: tp.basic.MySimpleSingleton will be generated by this annotations processor at compile time
*/

//@SupportedAnnotationTypes("tp.annotations.*")
@SupportedAnnotationTypes("tp.annotations.BasicSingleton")
@SupportedSourceVersion(SourceVersion.RELEASE_17)
//@SupportedOptions: to register allowed custom options that may be passed through the command line.
public class MyAnnotProcessor extends AbstractProcessor {

    public MyAnnotProcessor() {
        super();
    }

    public List<String> singletonClassNames = new ArrayList();

    @Override
    public boolean process(Set<? extends TypeElement> annotations,
        RoundEnvironment roundEnv) {

        //boucle sur annotations supportées (1 ou plusieurs) :
        for (TypeElement annotation : annotations) {

            //récupération de l'ensemble des éléments sur lesquels l'annotation a été placée dans le code source :
```

```

Set<? extends Element> annotatedElements
    = roundEnv.getElementsAnnotatedWith(annotation);

System.out.println("for annotation= " + annotation.toString()
    + " annotatedElements="+annotatedElements);
for(var e : annotatedElements){
    this.singletonClassNames.add(e.toString());
}
}

try {
    this.writeMySingletonImplementationJavaClass();
} catch (Exception e) {
    //throw new RuntimeException(e);
    //e.printStackTrace();
    System.err.println(e.getMessage());
}

return true;
}

private void writeMySingletonImplementationJavaClass() throws IOException {
    String mySingletonClassName = "tp.basic.MySimpleSingleton";
    File filer = this.processingEnv.getFiler();
    //javax.annotation.processing.Filer is useful for creating java source file
    JavaFileObject mySimpleSingletonFile = filer
        .createSourceFile(mySingletonClassName);
    try (PrintWriter out = new PrintWriter(mySimpleSingletonFile.openWriter())) {
        out.println("package tp.basic;\n");

        out.println("public class MySimpleSingleton extends AbstractSimpleSingleton {}");

        out.println("\t MySimpleSingleton() { super(); \n\t} \n\n");

        out.println("\t public void initMapClassInstance(){}");
        for(String singletonClassName : this.singletonClassNames) {
            out.println("\t\t mapClassInstance.put(\"\" + singletonClassName + "\",new " +
singletonClassName + "());");
        }
        out.println("\t");

        out.println("}");
    }
}
}

```

Projet using annot:***pom.xml***

```

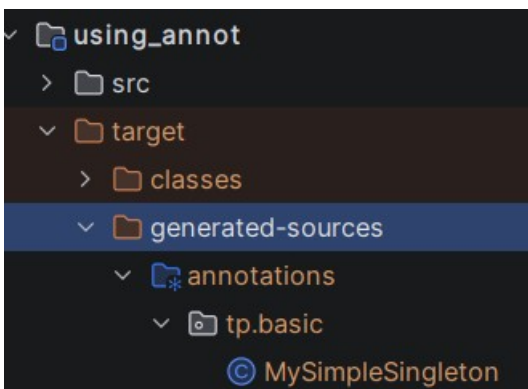
<?xml version="1.0" encoding="UTF-8"?>
<project ....>
  <dependencies>
    <dependency>
      <groupId>tp</groupId>
      <artifactId>my_annot</artifactId>
      <version>0.0.1-SNAPSHOT</version>
    </dependency>
  </dependencies>

  <build>
    <finalName>${project.artifactId}</finalName>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.14.1</version>
        <configuration>
          <annotationProcessorPaths>
            <path>
              <groupId>tp</groupId>
              <artifactId>process_annot</artifactId>
              <version>0.0.1-SNAPSHOT</version>
            </path>
          </annotationProcessorPaths>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>

```

après **mvn clean package**

→



code généré automatiquement durant la phase de compilation :

```
package tp.basic;

public class MySimpleSingleton extends AbstractSimpleSingleton {
    MySimpleSingleton(){ super(); }
    public void initMapClassInstance(){
        mapClassInstance.put("tp.computing.AverageComputing",new tp.computing.AverageComputing());
        mapClassInstance.put("tp.computing.SumComputing",new tp.computing.SumComputing());
    }
}
```

Utilisation :

@BasicSingleton

```
public class AverageComputing {
    public double average(double a , double b){ return (a+b)/2; }
}
```

```
public class MinComputing {
    public double min(double a , double b){ return Math.min(a,b); }
}
```

@BasicSingleton

```
public class SumComputing {
    public double add(double a , double b){ return a+b; }
}
```

```
public class MyApp {
    public static void main(String[] args) {
        SumComputing sc =
            SimpleSingleton.implementation().getInstance(SumComputing.class);
        System.out.println("5+6=" + sc.add(5,6));
        m2();    m3();
        System.out.println("nombre d'appels à getInstance() pour SumComputing: "
            +SimpleSingleton.implementation().getCount(SumComputing.class));
    }

    public static void m2(){
        SumComputing sc = SimpleSingleton.implementation().getInstance(SumComputing.class);
        System.out.println("3+4=" + sc.add(3,4));
    }
    public static void m3(){
        AverageComputing ac =
            SimpleSingleton.implementation().getInstance(AverageComputing.class);
        System.out.println("moyenne(3,5)=" + ac.average(3,5));
    }
}
```

=> 5+6=11.0 3+4=7.0 moyenne(3,5)=4.0

nombre d'appels à getInstance() pour SumComputing: 2

XXIII - Annexe – Structure globale appli.

1. Architecture généralement recommandée

Partie "Présentation" ==> Partie "Traitements métiers" ==> Partie "Accès aux données"

Appli mode texte		Calculs		Fichiers (txt,xml)
<i>ou</i>		<i>et/ou</i>		<i>et/ou</i>
AWT/SWING	==>	Assemblage/filtrage	==>	Base de données
<i>ou</i>		<i>et/ou</i>		<i>et/ou</i>
....		Règles de gestion , Mise à jour		Requêtes ...

Exemples d'organisation pour le code:

- Plusieurs *packages* (bien séparés) : (*ex*: presentation , business , data)
- Classes avec responsabilités bien séparées (MyApp , MainFrame , XXXData , ...)
- Design Pattern "Singleton" pour accéder aux "Fabriques" d'accès aux données , ... ou bien Framework "IOC" avec conteneur léger (ex: Spring).
- Pas de valeur en dur mais fichiers de configuration "xxx.properties" ou "xxx.xml"
- ...

XXIV - Annexe – Lombok (très utile)

XXV - Annexe – énoncés des TP

Les TP ci-après ne sont que des propositions (à caractère indicatif et non impératif).
Il ne faut pas hésiter à tester tout un tas de variantes (selon ses préférences personnelles).

NB: chaque nouvelle classe développée dans un TP devra être placée dans un **package** adéquat (dont le nom est à choisir) .

1. TP (prise en main du jdk)

Objectif : Edition, Compilation et Exécution sans IDE , avec un simple éditeur de texte et le jdk :

A faire : Hello World !!!

2. TP (première classe simple, conventions JavaBean)

Objectif : Ecrire une classe Java dans les règles de l'art

A faire:

- Créer un nouveau projet java (basé sur la technologie "**maven**") de nom "**tpInit**" sous eclipse ou bien intelliJ (choisir une version LTS de java , ex : 21 ou 25)
- Créer un package "**tp**" (dans src/main/java).
- Créer la classe "**MyApp1**" avec une méthode **main()** qui servira aux tests.
- Créer (et tester) une première version d'une classe "**Personne**" avec des attributs "**nom**", "**age**", "**poids**" déclarés provisoirement "public".
- Coder une méthode "public void **afficher()**" qui affiche à l'écran/console les valeurs des attributs.
(+ premiers tests)
- Rendre "**private**" les **attributs** de la classe "Personne"
- Générer les méthodes **getXxx()/setXxx(...)** (+tests)
- Coder quelques **constructeurs**. (+tests)
- Coder ou générer la méthode classique "public String **toString()**" (provenant de la classe Object) en y construisant une chaîne de caractères complète regroupant tous les attributs + return
- Reprogrammer la méthode **afficher()** de façon à ce quelle appelle **toString()** en interne.
- Créer deux instances p1 et p2 de la classe Personne avec les mêmes valeurs internes.
- Comparer ces 2 instances et afficher si (oui ou non) les valeurs internes sont identiques.
- Reprogrammer ou générer la méthode "public boolean **equals(Object obj)**" sur la classe Personne. Cette méthode doit renvoyer "true" si et seulement si toutes les valeurs internes de this et de obj sont identiques.

Suite facultative du TP :

Coder également une classe "**Bagage**" (avec label, poids, volume) . Avec des poids en grammes (Integer) et des volumes en litres (Double) avec constructeurs, toString() , getter/setter , ...
 Tester le bon comportement de la classe Bagage via une nouvelle méthode **testerBagage()** appelée par la méthode **main()** d'une nouvelle classe **MyApp2**.

Encore une fois la même chose !!! J'en ai marre de programmer des "get/set" pas passionnant disait le développeur "Jean Aimare" !!!

Heureusement, lombok est là pour vous simplifier la vie .

⇒ ajouter une dépendance lombok dans le pom.xml du projet

```
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <version>1.18.30</version> <!-- ou bien version plus récente -->
</dependency>
```

⇒ utiliser @Getter() @Setter() @ToString() @NoArgsConstructor() au dessus de la classe Bagage et commenter tous les équivalents générés par l'IDE eclipse ou intelliJ .

⇒ lancer un build maven via "mvn clean package" pour vérifier un bon fonctionnement maven.

⇒ les getters/setters générés par maven lors de la compilation sont normalement bien reconnus par l'IDE intelliJ qui propose d'installer automatiquement le plugin pour lombok.

⇒ avec l'IDE eclipse il faut effectuer une installation manuelle du plugin via le menu **Help / install new software** , sélectionner le chemin "<https://projectlombok.org/p2> " puis se laisser guider puis redémarrer eclipse , puis menu "**project clean**" .

3. TP (Tableaux, String, ...)

Code à placer dans **tp.MyApp2**

(Tous les tests liés aux personnes seront rangés dans tp.MyApp1 et **tout le reste** dans tp.MyApp2)

testTableauMoyenne() :

créer un tableau de 6 nombres réels
 et calculer la moyenne

Suite facultative du Tp :

Trouver et afficher la plus grande des valeurs du tableau:

testString() :

String s1 = "2023-01-17";

//extraire la partie mois de différentes façons et afficher cette valeur

String chaine="YTREZA" ;

//créer une nouvelle chaine inverse où tous les caractères sont dans l'ordre inverse

//afficher la chaine inverse

4. TP static – constante , ...

- Déclarer une constante `Personne.AGE_MAJORITE` avec une valeur de 18
- Ajouter les éléments suivants sur la classe "Personne" :
 - * variable de classe privée "esperanceVie"
 - * méthodes de classe `getEsperanceVie()` et `setEsperanceVie()`
 - * méthode ordinaire `estMajeur()` retournant un boolean
- Tester la nouvelle version de `Personne` dans `tp.MyApp1`
- Déclencher le calcul racine carré de 81 dans une méthode de la classe `tp.MyApp2`

5. TP (Collections & Generics)

Essais très progressifs sur des collections élémentaires (de "Integer" ou de "String")

`tp.MyApp2.testListeString()` :

- créer une liste (ArrayList) de String
- ajouter quelques valeurs (ex : "janvier" , "février" , ...)
- parcourir une première fois la liste pour afficher les valeurs
- parcourir une seconde fois la liste pour transformer les éléments (ex : en majuscules)
- parcourir une seconde fois la liste pour afficher les valeurs modifiées
- afficher la taille de la liste
- tester d'éventuels ajouts et suppressions

Suite facultative du TP :

- créer une autre liste de Double (avec quelques valeurs) et calculer la moyenne
- faire tourner l'exemple du support de cours (`MyStack<T>`)

TP facultatif (mais conseillé) : enum Genre sur Personne :

Ajouter une énumération **Genre** (avec HOMME , FEMME , INDETERMINE comme valeurs possibles) au niveau de la classe `Personne` (partie additionnelle `private Genre genre` à gérer/manipuler/afficher) .

La valeur par défaut sera `INDETERMINE` et la précision du genre pourra être effectuée par une nouvelle variante du constructeur sans remettre en question les constructeurs existants.

TP facultatif (mais conseillé) sur saisies via Scanner :

Dans une sous fonction de type `tp.MyApp2.testScanner()` :

//on va demander à l'utilisateur de saisir des nombres x et y

//on va les additionner ensemble et afficher le résultat

//Autre idée de TP classique : générer un nombre entre 0 et 100 de manière aléatoire

//puis faire deviner ce nombre avec à chaque essai une indication : "plus petit" ou "plus grand"

//et une fois le nombre à deviner trouvé afficher le nombre d'essais.

6. TP POO (Héritage, Polymorphisme, interface)

Programmation orientée objet bien structurée: Héritage, Polymorphisme, Interface sur mini étude de cas (ex : dessin) **Objectif principal de cette mini étude de cas:**

Programmer une petite application java qui générera au moins un fichier "dessin.svg" affichable via un navigateur internet .

6.1. création et paramétrage du projet

Créer (avec IntelliJ ou eclipse) un nouveau projet "maven" simple intitulé "**dessin**"

groupId : tp ou autre

ArtifactId : dessin

Version de java : 21 ou 25 ou ... selon disponibilités.

Exemple de "properties" dans **pom.xml** :

```
<properties>
  <java.version>21</java.version>
  <maven.compiler.source>${java.version}</maven.compiler.source>
  <maven.compiler.target>${java.version}</maven.compiler.target>
  <maven.compiler.release>${java.version}</maven.compiler.release>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
</properties>
```

6.2. classe principale tp.MyApp

- générer un nouveau package "**tp**" dans src/main/java
- générer une nouvelle classe "**MyApp**" contenant la méthode principale **main()** dans le package "tp" .
- Au sein de la méthode main() afficher à la console le message **"cette application va générer un fichier dessin.svg"**
- Lancer une première fois l'exécution de l'application java et vérifier le bon fonctionnement au sein de la console :



```
<terminated> MyApp (2) [Java Application] C:\Program Files\Java\jdk-11.0.4\bin\javaw.exe (23 juin 2020 à 17:54:50)
cette application va générer un fichier dessin.svg
```

6.3. exemple de fichier ".svg"

créer à coté de pom.xml un nouveau fichier vide **exemple.svg**

Ouvrir ce fichier et placer (via un copier/coller) le contenu suivant :

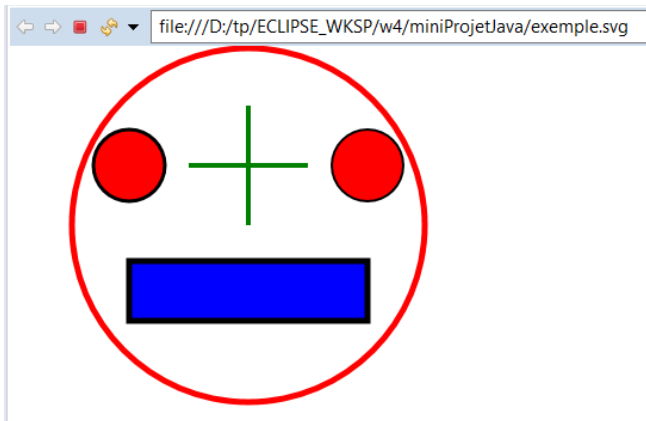
```
<svg xmlns='http://www.w3.org/2000/svg' height='400' width='500'>
  <circle cx='100' cy='100' r='30' stroke='black' stroke-width='3' fill='red' />
```

```

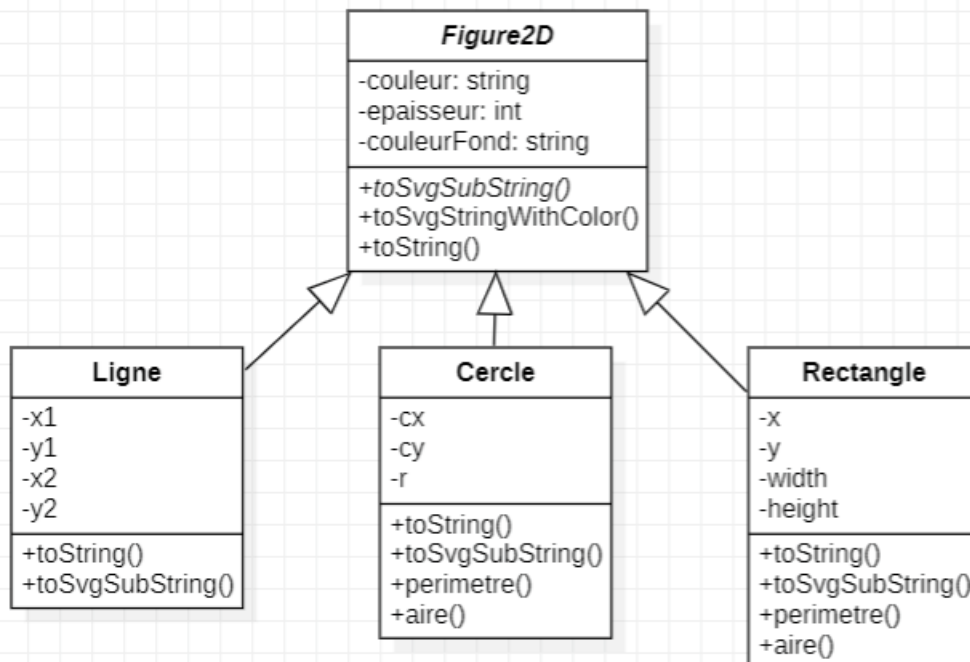
<line x1='200' y1='50' x2='200' y2='150' stroke='green' stroke-width='4' fill='none' />
<line x1='150' y1='100' x2='250' y2='100' stroke='green' stroke-width='4' fill='none' />
<circle cx='300' cy='100' r='30' stroke='black' stroke-width='2' fill='red' />
<rect x='100' y='180' width='200' height='50'
    fill='blue' stroke='black' stroke-width='5' />
<circle cx='200' cy='150' r='148' stroke='red' stroke-width='5' fill='none' />
</svg>

```

Visualiser ce dessin vectoriel au sein du navigateur web (ordinaire ou intégré à l'ide) :



6.4. première modélisation UML



6.5. première partie abstraite de tp.figure

créer (sous tp) le sous package **tp.figure**

créer (dans tp.figure) une *nouvelle classe abstraite* "Figure2D" avec le code initial suivant :

```
package tp.figure;

import java.io.ByteArrayOutputStream;
import java.io.PrintStream;

public abstract class Figure2D {
    protected String couleur="black"; //couleur du trait ou du contour ("black" par défaut)
    protected Integer epaisseur=1; //épaisseur du trait ou du contour (1 par défaut)
    protected String couleurFond; //null par défaut ("none" en svg)

    //design pattern "template method" avec polymorphisme sur sous tâche abstraite .
    public String toSvgStringWithColor() {
        //polymorphisme sur l'appel à this.toSvgSubString();
        //où this pourra référencer une instance de Cercle ou Ligne ou Rectangle
        String beginOfSvgString = this.toSvgSubString();

        ByteArrayOutputStream baos = new ByteArrayOutputStream();
        PrintStream ps = new PrintStream(baos);
        ps.printf("%s stroke='%s' stroke-width='%d' fill='%s'/>",
                beginOfSvgString ,
                this.couleur ,
                this.epaisseur ,
                this.couleurFond==null?"none":this.couleurFond);

        return baos.toString();
    }
    public abstract String toSvgSubString();
    //+get/set , +constructeurs , +toString()
}
```

Compléter ce code en générant via les assistants de l'IDE les éléments habituels
(+get/set , +constructeurs , +toString())

6.6. partie concrète de tp.figure

Créer au sein de tp.figure , 3 nouvelles classes concrètes :

- Ligne , Cercle , Rectangle héritant de Figure2D

NB :

- Chacune de ces classes devra comporter au minimum 2 constructeurs :
 - un constructeur par défaut (avec zéro argument)
 - un constructeur avec toutes les coordonnées comme premiers arguments importants

et avec les couleur, epaisseur et couleurFond comme derniers arguments secondaires

Coordonnées internes d'une ligne :

```
private int x1;
private int y1;
private int x2;
private int y2;
```

Coordonnées internes d'un cercle :

```
private int cx;//x du centre du cercle
private int cy;//y du centre du cercle
private int r; //rayon
```

Coordonnées internes d'un rectangle :

```
private int x;
private int y;
private int width;
private int height;
```

NB : On pourra coder les méthodes imposées petit à petit en les testant au fur et à mesure de leur mise en oeuvre.

Autrement dit , certaine méthodes pourront temporairement être laissées avec le code par défaut généré par les assistants de l'IDE .

Les méthodes délicates `toSvgSubString()` seront codées et testées ultérieurement .

6.7. premiers tests via la classe tp.MyApp

Coder au sein de tp.MyApp quelques tests de ce genre :

```
Rectangle r = new Rectangle(100,180,200,50,"black",5,"blue");
System.out.println(r.toString());
System.out.println("r.perimetre="+r.perimetre());
System.out.println("r.aire="+r.aire());
```

//idem pour Cercle

//idem pour Ligne (sans périmetre ni aire)

6.8. coder et tester les méthodes `toSvgSubString()`

La méthode `.toSvgSubString()` de la classe **Ligne** devra générer et retourner une chaîne de caractère de ce type : `<line x1='200' y1='50' x2='200' y2='150'`

La méthode `.toSvgSubString()` de la classe **Cercle** devra générer et retourner une chaîne de caractère de ce type : `<circle cx='300' cy='100' r='30'`

La méthode `.toSvgSubString()` de la classe **Rectangle** devra générer et retourner une chaîne de caractère de ce type : `<rect x='100' y='180' width='200' height='50'`

NB : étant donné que la méthode `toSvgStringWithColor()` héritée de la classe abstraite **Figure2D** ajoute une partie finale de de genre `stroke='black' stroke-width='3' fill='red' />`

on testera indirectement la méthode `.toSvgSubString()` en appelant `.toSvgStringWithColor()` sur des lignes , des cercles et des rectangles .

exemple :

```
System.out.println(r.toSvgStringWithColor());
```

6.9. Polymorphisme avec version préliminaire de MySvgUtil

- créer un nouveau package **tp.svg**
- créer et coder le début de la nouvelle classe **MySvgUtil** suivante :

```
package tp.svg;

import java.util.List;

import tp.figure.Figure2D;

public class MySvgUtil {

    public static String generateGlobalSvgContent(List<Figure2D> listeFig) {
        //utiliser un buffer sous forme d'une instance de la classe StringBuilder

        //ajouter au buffer la première ligne d'un bloc svg
        //<svg xmlns='http://www.w3.org/2000/svg' height='400' width='500'>

        //boucler sur chaque élément de la liste listeFig
        //et ajouter au buffer une ligne dont la valeur est construite
        //via la méthode .toSvgStringWithColor() appelant indirectement
        //la méthode polymorphe .toSvgSubString()

        //ajouter au buffer la dernière ligne d'un bloc svg
        //</svg>

        //retourner la transformation du buffer en String
    }
}
```

Pour tester cette classe , on ajoutera dans la méthode **main()** de **tp.MyApp** du code qui :

- créera une instance de `ArrayList<Figure2D>`
- y ajoutera quelques instances de `Cercle` , `Ligne` et `Rectangle`
- appellera la méthode statique . **generateGlobalSvgContent** de **MySvgUtil** .

On pourra enfin :

- recopier le texte généré à la console dans un fichier temporaire `mySvg.svg`
- visualiser ce fichier *mySvg.svg* au sein d'un navigateur internet

Une future suite de ce Tp permettra de compléter le code de la classe `MySvgUtil` de manière à directement générer un nouveau fichier SVG .

6.10. Seconde partie abstraite de tp.figure

Au sein du package `tp.figure` , coder les 2 interfaces suivantes :

```
public interface Surface {
    public double perimetre();
    public double aire();
}
```

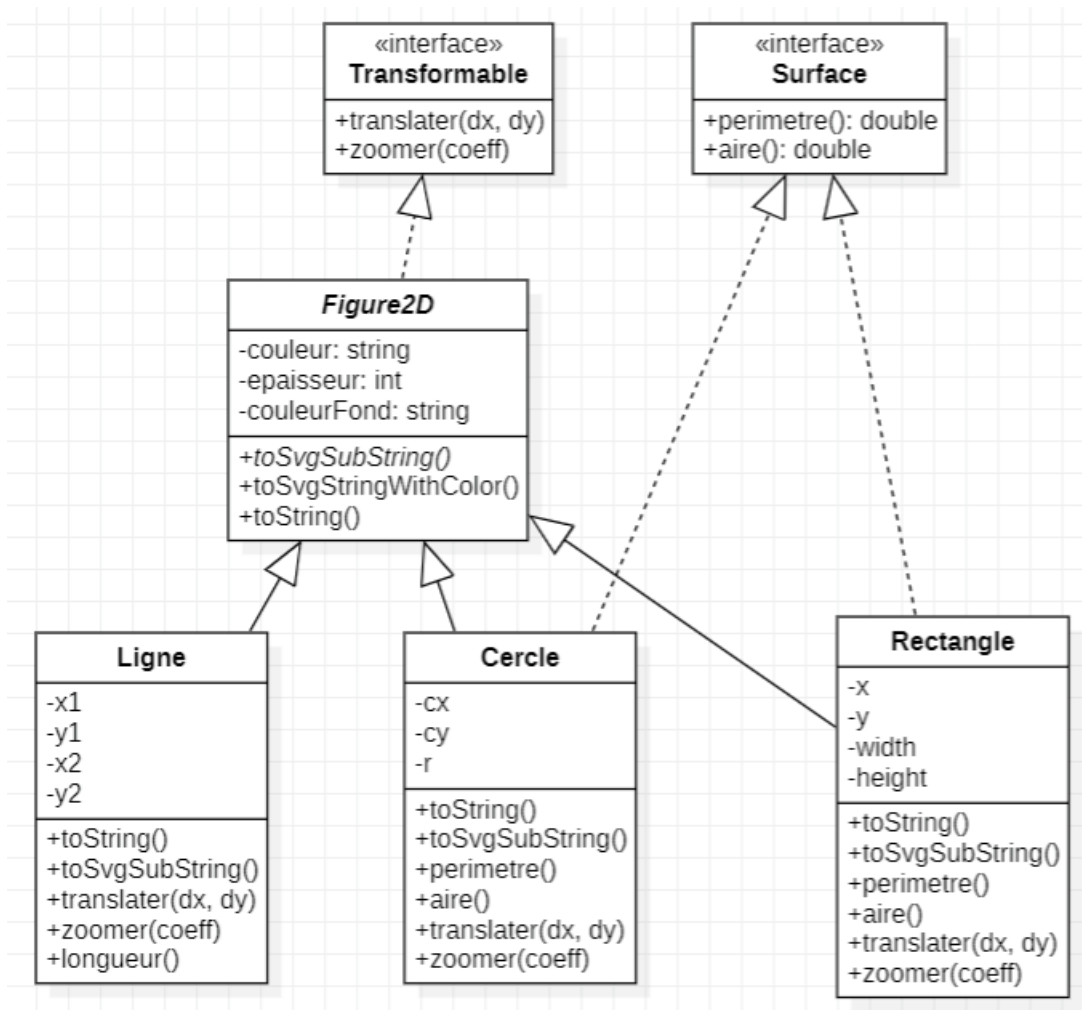
et

```
public interface Transformable {
    public void translater(int dx,int dy);
    public void zoomer(double coeff);
}
```

Restructurer *la classe abstraite "Figure2D"* de telle manière qu'elle implémente l'interface **Transformable** .

Les 3 classes concrètes `Cercle`,`Ligne` et `Rectangle` devront également coder les méthodes imposées par l'interface `Transformable` (indirectement récupérée par l'héritage de `Figure2D` déclarant implémenter l'interface `Transformable`) .

Restructurer les classes **Cercle** et **Rectangle** de manière à ce qu'elles implémentent explicitement l'interface **Surface** .



Suggestion de polymorphisme exploitant la nouvelle structure :

Au sein d'une nouvelle sous méthode statique `MyApp.changeFigures(List<Figure2D> listeFig)` appelée par `MyApp.main()`, ajouter quelques lignes de code de manière à :

- calculer une première fois la somme de tous les périmètres des surfaces et des longueurs des lignes
- déclencher en boucle sur `listeFig` une translation (`dx=30, dy=20`) de toutes les figures
- recalculer une seconde fois la somme de tous les périmètres des surfaces et des longueurs des lignes (et vérifier que la valeur totale n'a pas été changée)
- déclencher en boucle sur `listeFig` un zoom (`coeff=2`) de toutes les figures
- recalculer une seconde fois la somme de tous les périmètres des surfaces et des longueurs des lignes (et vérifier que la valeur totale est maintenant deux fois plus grande)

7. TP - Premier traitement d'exception

Ré-ouvrir (avec l'IDE IntelliJ ou Eclipse) le projet initial "**tpInit**" (celui où avait été codé la classe `Personne`).

améliorer le code de la méthode `.setAge()` de la classe **Personne** en soulevant une exception de type **IllegalArgumentException** (héritant de `RuntimeException`) via le mot clef **throw** en cas d'âge négatif invalide et tester le tout via au moins un **try/catch** au sein de `MyApp1.testPersonne()`.

8. TP - Second traitement d'exception

Ré-ouvrir (avec l'IDE IntelliJ ou eclipse) le projet "dessin" (celui de la mini étude de cas) .

8.1. Restructurer le code de tp.MyApp

```
public class MyApp {

    public static void main(String[] args) {
        double coeffZoom = 1;
        int dx=0;
        int dy=0;
        String typeFig=null;
        //premiersTests();
        //.....du code sera à ajouter ici ...
        enchaînerTransformationsEtGenerationFichierSvg(coeffZoom,dx,dy,typeFig);
    }

    public static List<Figure2D> buildListeFigures() {
        List<Figure2D> listeFigures = new ArrayList<>();
        listeFigures.add(new Rectangle(100,180,200,50,"black",5,"blue"));
        listeFigures.add(new Ligne(150,100,250,100,"green",4/*,null*/));
        listeFigures.add(new Cercle(100,100,30,"black",3,"red"));
        return listeFigures;
    }

    public static void enchaînerTransformationsEtGenerationFichierSvg(double coeffZoom ,
                                                                    int dx, int dy , String typeFig) {
        List<Figure2D> listeInitialeFigures = buildListeFigures();
        List<Figure2D> listeTransformeeFigures =
            listeInitialeFigures.stream()
                //effacter une premiere transformation de type zoomer(coeffZoom)
                //effacter une seconde transformation de type translation(dx,dy)
                //filtrer selon le type de figure (typeFig , ex : "Cercle") via un test de type instanceof ...
                .collect(Collectors.toList());
                //futur MySvgUtil.generateSvgFile(listeTransformeeFigures, "dessin2.svg");
                MySvgUtil.generateGlobalSvgContent(listeTransformeeFigures);
    }

    public static void premiersTests() {
        //déplacer ici les tests effectués au sein de la partie 1
    }
}
```

Lancer une première fois le code restructuré pour vérifier qu'il y a toujours une génération de lignes au format svg sur la console texte.

8.2. Récupération des éventuels arguments de la ligne de commande avec traitement d'exception

Ajouter du code dans la fonction principale **MyApp.main()** de manière à récupérer les valeurs des arguments facultatifs de la ligne de commande :

```
java tp.MyApp --coeffZoom=1.5 --dx=10 --dy=20 --typefig=Cercle
```

On se contentera dans un premier temps d'afficher les valeurs choisies (ou par défaut)

Dans un second temps , on affichera un message d'erreur de type "la valeur de dx est invalide (pas numérique) si Integer.parseInt(...) soulève une exception (idem pour dy et pour coeffZoom avec Double.parseDouble(...))

9. TP (logs , test unitaires) à éventuellement différer

A éventuellement mettre en place (en différé) une fois de JUnit5 et slf4j auront été abordés en cours :

- Réorganisation de certains packages (refactor)
- Génération d'un .jar et démarrage d'une application java en ligne de commande
- petit test unitaire élémentaire avec JUnit5 (@Test , @BeforeEach , assertTrue())
- génération de ligne de logs via slf4j et log4j2
- sensibilisation aux mocks (une démonstration "Mockito" sera idéalement effectuée par le formateur)

A faire dans au moins une des 2 applications des Tps ("tpInit" et/ou "dessin")

10. TP tris et lambda, stream

10.1. Tri et transformation de personnes

Ré-ouvrir (avec l'IDE IntelliJ ou eclipse) le projet initial "tpInit" (celui où avait été codé la classe Personne) .

Dans une sous fonction de type **MyApp1.testCollectionPersonne()** :

- créer une liste de Personne , y ajouter quelques valeurs
- trier cette liste par noms croissants puis afficher cette liste modifiée
- trier une nouvelle fois cette liste par ages décroissants puis afficher cette liste modifiée
- **Dans un second temps , une fois la partie 1 testée, refaire ça avec des lambda expressions**

Refaire cela avec des streams() et en enchaînant "filtrage des personnes majeures" , "tri selon age"

puis transformations avec noms en majuscules dans une méthode *MyApp1.testStream()* :

10.2. Tri et transformation de figure2D

Ré-ouvrir (avec l'IDE IntelliJ ou eclipse) **le projet "dessin"** (celui de la mini étude de cas) .

Coder les transformations et filtrages suggérés par les commentaires de
`enchaînerTransformationsEtGenerationFichierSvg()`

Tester cela via un affichage graphique au sein d'un navigateur d'un fichier temporaire *mySvg2.svg*
comportant une copie des lignes générées à la console

11. TP facultatif (LocalDate & LocalTime)

Ré-ouvrir (avec l'IDE IntelliJ ou eclipse) le projet initial "tpInit"

Ajouter dans tp.MyApp2 une sous méthode du main() , par exemple "testDates()" qui :

- demandera à saisir deux dates
- affichera l'écart en nombre de jours et/ou de mois

12. TP (Gestion des fichiers) :

12.1. Génération de fichier SVG sur application dessin

Ré-ouvrir (avec l'IDE IntelliJ ou eclipse) le projet "dessin" (celui de la mini étude de cas) .

- Ajouter la méthode `generateSvgFile(...)` au sein de la classe `svg.MySvgUtil` :

```
package tp.svg;

import java.util.List;

import tp.figure.Figure2D;

public class MySvgUtil {

    public static String generateGlobalSvgContent(List<Figure2D> listeFig) {
        //déjà codé et testé
    }

    public static void generateSvgFile(List<Figure2D> listeFig , String fileName) {
        //ex de fileName : "dessin.svg"

        //ouvrir le fichier en écriture (flux élémentaire + PrintStream)

        //écrire dans ce fichier le résultat de la sous méthode generateGlobalSvgContent()

        //fermer les flux ouverts
    }

}
```

Pour tester cette classe , on appellera la méthode `MySvgUtil.generateSvgFile` (plutôt que `MySvgUtil.generateGlobalSvgContent`) dans méthode adéquate de `tp.MyApp` (`main()` ou `enchaînerTransformationsEtGenerationFichierSvg()` ou `premiersTests()`)

On pourra enfin :

- vérifier l'existence d'un fichier directement généré (*dessin.svg* ou autre) via un **Refresh** IntelliJ ou eclipse

- visualiser le fichier généré via un navigateur internet

12.2. long TP facultatif: lecture et écriture de fichier CSV

Ré-ouvrir (avec l'IDE intelliJ ou eclipse) le projet initial "tpInit"

Coder (par ajout) :

- Classe Produit(numero, label, categorie, prix, poids)
- Classe Stat(nbProduits, moyennePrix, moyennePoids)
- Classes MyCsvApp et MyCsvUtil permettant de :
 - lire un fichier produits.csv et générer un fichiers stats_products.csv

produits.csv

```
num;label;categorie;prix;poids
1;cahier;papeterie;3.6;0.123
2;pommes;nourriture;2.6;1.0
3;poires;nourriture;3.69;1.2
4;stylo;papeterie;2.6;0.023
5;bananes;nourriture;1.79;1.3
6;gomme;papeterie;1.6;0.019
```

stats_products.csv

```
nbProduits;moyennePrix;moyennePoids
6;2.646666666666667;0.6108333333333333
```

Pleins d'extensions/suites facultatives pour ce TP :

- générer des fichiers triés
- générer un fichier .csv en utilisant de l'introspection / réflexion au sein de la classe MyCsvUtil
- coder une nouvelle annotation @CsvIgnore et l'utiliser au sein de MyCsvUtil pour ignorer certaines colonnes au sein d'un fichier .csv à générer

Encore une autre suite facultative à ce TP:

- ajouter la dépendance maven "jackson-databind" et générer un fichier .json à partir d'un objet java (de type "Produit") ou vice versa .

13. TP (Accès aux bases de données via JDBC):

Variantes proposées pour ce TP (en choisir une):

<i>Variante</i>	<i>1</i>	<i>2</i>
Application	tpInit	dessin
Nom de base de données	person_db	drawing_db
Nom table principale	personne (id,nom,age,poids)	dessin(id,titre,path,description)
Classe java (de données)	PersonEntity héritant de Personne (nom,age,poids)	Dessin(id,titre,path,description) où path sera soit un nom de

	et ajoutant un champ id de type Long qui sera considéré comme la clef primaire	fichier svg , soit une url d'un fichier téléchargeable en ligne
suggestion	Test JUnit	Test JUnit + éventuelle insertion d'un nouvel enregistrement lors d'une nouvelle génération de fichier svg

Utiliser l'api JDBC pour se connecter à une base de données relationnelles comportant une table de personnes ou de dessins . On codera au minimum une lecture et éventuellement les opérations standards "CRUD".

Indications :

- installer si besoin "**mariaDB**" (version "open source" de MySQL facile à installer sous windows ou linux) , préparer éventuellement une base de données via un script SQL **init_db.sql** (avec CREATE TABLE et quelques INSERT INTO ...)
- ajouter une dépendance maven permettant de se connecter à MySql (ou bien à MariaDB compatible).
- Structurer proprement le code d'accès aux données avec interface "PersonneOuDessinDao" et classe d'implémentation "PersonneOuDessinDaoJdbc"
- Utiliser un fichier **db.properties** (à idéalement placer dans src/main/resources) pour ne pas coder en dur les paramètres de connexions : (username, password, url, ...).
- Utiliser tout cela dans une classe de test (exemple : **TestJdbcDao**) en commençant par coder et lancer un test de lecture .

Exemple de dépendance maven :

```
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <version>26.7.0</version>
</dependency>
```

ou bien

```
<dependency>
  <groupId>org.mariadb.jdbc</groupId>
  <artifactId>mariadb-java-client</artifactId>
  <version>3.5.10</version>
</dependency>
```

14. TP (Gestion élémentaire des threads) :

Ré-ouvrir (avec l'IDE intelliJ ou eclipse) le projet initial "tpInit"

Tester le démarrage d'un nouveau Thread qui pourra :

- afficher un premier message "café en préparation"
- faire une pause de 5s
- afficher un second message "café prêt"

15. TP (syntaxes java récentes):

Sur application "**tpInit**", expérimenter (au sein d'une classe de test JUnit) différentes manières de générer un fichier **produit.json** :

- depuis une classe java **Produit(numero, label, categorie, prix, poids)** ou depuis un **record** équivalent .
- avec l'api jackson-databind ou bien avec des **text-blocs**

On pourra expérimenter une augmentation de prix paramétrée par une **switch expression** basée sur l'analyse de la catégorie .

On pourra éventuellement restructurer certains morceaux de codes des TP réalisés précédemment en ajoutant de type de code (ou d'améliorations) :

- mot clef "var"
- pattern matching , ...
- virtual thread

16. TP (debug et monitoring de JVM):

- Effectuer avec l'IDE un debug pas à pas (avec un point d'arrêt)
- Utiliser JConsole pour visualiser quelques éléments de la JVM (nb threads , mémoire consommée, ...) lorsque l'application sera en cours de fonctionnement (et pas encore arrêtée , par exemple en attente de saisie) .